ARDUINO

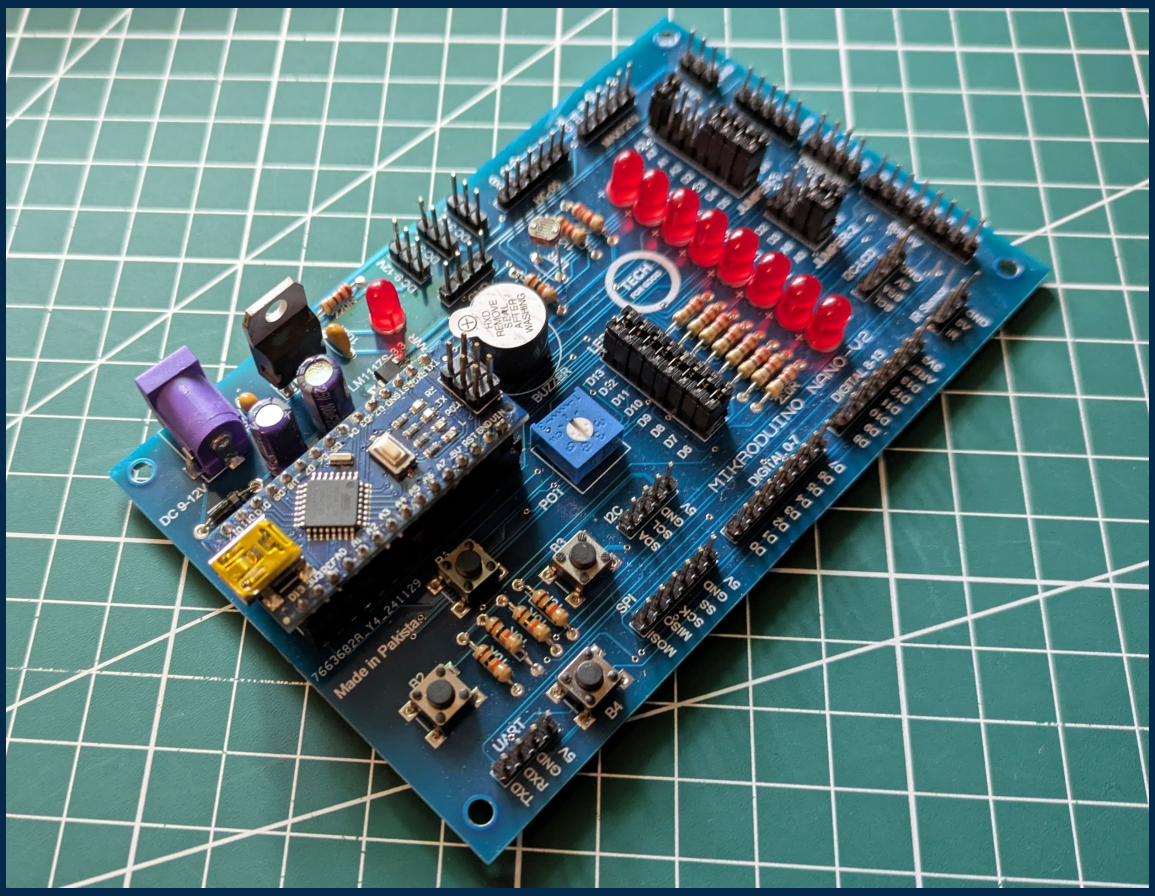
Getting Started With Arduino and C++

Programming With Projects

Advanced Methods to Learn Embedded Systems Programming

Using MikroDuino Nano V-2 Arduino Companion Board

Students | Hobbyists | Rapid Prototyping | Teaching



Amer Iqbal Qureshi

About Author



Writing about yourself is something I always find most difficult, yet its important for the reader to know as to whom they are interacting with and what's his background. I am professionally an Associate Professor of Cardiac Surgery. I worked at various tertiary care cardiac centers run by the government of Punjab, Pakistan.

Electronics and coding have been my hobbies ever since I was in school. I remember the first crystal radio that caught the local radio broadcast I made in Year 1978 perhaps. The joy of successful project, jubilated me to explore more. In later years I made many basic electronics projects in school lab, where my school teacher Mr. Majeed helped and encouraged me to a lot. In later years I made my own stereo music system and graphic equalizer. In high school I had the opportunity to experiment with digital electronics, I am thankful to my teacher Dr. Farkhand Shakeel who encouraged me to experiment with these components in his physics lab.

Then I was introduced to personal computers, almost at the same time, and used to write a lot of programs for games and fun using Sinclair spectrum and later commodore 64 computers. I met a nobel man, Mr. Khaleeq Mirza in a book shop, who was consultant with IBM systems in Pakistan and developing some software for their commercial clients, he introduced me to personal computers and gave access to his personal lab, where I learned the Basic language, but soon I turned to 8086 Assembly language. By 1984 I was programming in C and assembly mainly to re-write the DOS interrupt systems to incorporate security and encryption into the PC based database applications.

I got interested in microcontrollers in 2001 or so, when one of my cardiac surgery machines got out of order, and technician told me that we need to change the microcontroller board. At that time I got a chance to get a microchip PIC16F84 microcontroller, with little knowledge about how to use it and how to program it. Internet was rudimentary at that time, with some information available, eventually I made it work the first LED Blinked. And ever since then I kept on exploring microcontrollers from PIC microcontrollers to AVRs, then to ARM core. They are just wonderful toys.

I made several development boards for these microcontrollers primarily for myself and then for my friends and local community to help and explore embedded systems. I believe that electronics is a wonderful hobby with a multitude of disciplines and helps you enjoy research and development.

Amer Iqbal Qureshi

CHAPTER 1

Introduction

Arduino is an extremely popular single board computer that can be used to make a vast variety of intelligent devices. With this book you will learn various parts of the Arduino boards, its philosophy and the entire ecosystem that surrounds the microcontroller based project development. This includes planning electronics, hardware, resources, programming and debugging. All these are addressed in this book in different parts where need arises. This book focuses on programming and getting the most out of Arduino through programming. It is not intended to teach electronics and the hardware details, however a little description of hardware and basic electronics is given where it is related. I believe that only with a solid understanding of 'Basics' and 'Fundamentals' one cannot innovate. Innovation is nothing but a continuation of the existing knowledge.

To get most out of this book, I would recommend not only to write the programs yourself, but also think of new ways to achieve the same or even better results. There is only one thing that is most important in this journey, that is 'Passion' to achieve. No matter how many times your code fails, keep on trying, keep on asking yourself and your peers.

> *"When the desire and passion to achieve something is intense, the whole universe conspires for you"*

Why Learning to Program Arduino?

Before answering this question, I would like to ask yet another question, why we should learn to program? The world of electronics is changing fast, there were days when we used to make devices and projects using discreet components. Discreet and specific functionality components still exist, however they have largely been replaced by programmable microprocessors and microcontrollers. Even many discreet components like for example a chip to tune in an FM station needs external control signals to make it function properly. So in order to use this chip you need a programmable microprocessor that should send control instructions to this chip to properly perform its function.

Programming is not just about writing instructions, it is an art just like the writer of a book, a novel or a drama script writing. You have to make a plot as to how you would place your instructions, (which are quite atomic) to accomplish a task. Instructions of computer language and supported libraries (which are again collection of instructions, by someone

else) are just like vocabulary of your everyday language like English or Spanish or Arabic. The words have meanings, but you need to arrange them in a particular order to convey a message. Similarly computer language instructions, and its rules alone cannot do a complete solution. Of course you need to know what each instruction does, and then you combine your instructions in a particular order to achieve a working solution. This organized and properly ordered instructions, are called a 'Program'. So in a program not only choice of instructions are important but also their order is important.

Algorithm

When writing an essay or even a paragraph on some topic, you must have realized that different people would write it in different way, all using same or different sets of words, but end result is same. They convey the same message, some have achieved this with few words, some with more words, some have considered even minor details others have skipped those, and all that difference highlights the quality of essay.

An algorithm is similarly the route you plan in computer programming, to solve a particular problem. The choice of algorithm is sometimes a personal matter, some programmers use already described algorithms, while others tend to develop their own. In a real world situation it's a combination of both. Where applicable you use a previously described algorithm or algorithms and add your own logic to it to achieve a working solution.

Now that we know the programming is an art and there is always chances to improve your skills. Lets come back to our original question, "Why Using Arduino ?" There are many reasons to start programming electronic devices using Arduino. First we need to clarify what is Arduino actually, the board, like Arduino UNO or The programming language?

Does Arduino mean a 'Hardware Board' or Software environment?

Well different people will answer you this question in different ways. Most people think that Arduino is the hardware board. While others think it's the Integrated Development Environment (IDE), still others believe it's the compiler, yet others believe it's the community supported libraries and so on.

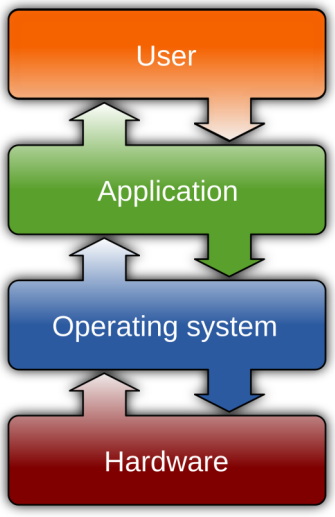
I believe that "Arduino" is an ecosystem, a chain of tools, boards (Only a few microcontrollers out of thousands support Arduino), an easy to use PC based software to write and compile programs, community supported huge collection of open source libraries of code and of course a huge community of professional, hobbyists and students ready to help and support.

Microcontroller Programming is a Low Level Programming

When it comes to programming a microprocessor the most native level is to program the internal registers and memory locations directly using commands that correspond to the instruction set of the microprocessor. This language is called assembly language. Its available for every type of microprocessor may it be the processor used in your PC, and industrial server computer or a small single board computer like an Arduino microcontroller. Programming in this language gives maximum control over the hardware but is time consuming and sometimes very complex.

It becomes a pain in neck when it comes to porting an application developed for one micro-

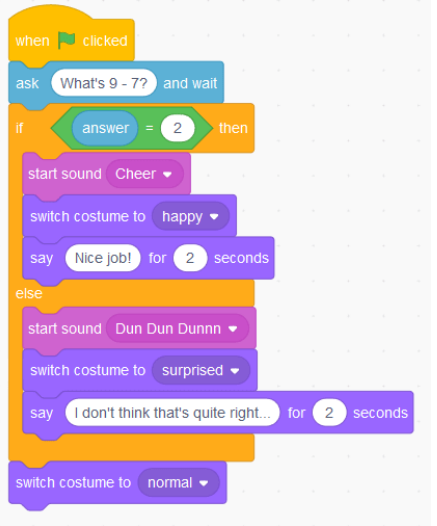
controller to another. It is here that some layers of software libraries come in handy. At a higher level these libraries that sit on top of the hardware are traditionally called an operating system. Just like LINUX or Microsoft Windows in case of PC. The operating system takes the responsibility of talking to the hardware as it understands the underlying registers and memory system very well, and frees up the application developer from these hardware details, instead he just orders the operating system to do a job and operating system invokes a set of commands to get the job done.



This looks cool, as if you change the underlying hardware, just install the appropriate operating system, and your application will remain essentially unchanged. This however comes at the cost of memory. The operating systems are usually extensive programs as they are made to cater the needs of a large number of different applications. So your system must provide adequate amount of memory to store and run the operating system software. This is usually not a big problem in modern day personal computers or even high end microcontrollers that have sufficient memory to host the operating systems.

However when it comes to the small low power microcontrollers that have a limited amount of memory and resources using an operating system is usually not feasible. Though theoretically speaking not impossible, but it leaves very little space for your application.

Microcontroller programmers usually adapt another strategy, that is intermediate level of abstraction. By using a compiler like C++ they are able to manage the things little better than assembly. The C++ language in embedded systems allows to use libraries that contain processor level hardware instructions, (Just like operating system) these libraries however are compiled directly into your application and deployed to the microcontroller as one application. This way you incorporate only the required code (From Libraries) into your application, conserving the memory and resources. Additionally embedded C++ language allows you to address the internal registers and memory locations of the microcontroller directly as well (Just like Assembly) provided you know the architecture of your particular microcontroller.



Like in this image where a ‘Scratch’ program is shown, the individual commands are used as ‘Building Blocks’ they are placed in a logical order to get a job done. When this designing is complete, the compiler converts these “Visual Blocks” into equivalent C++ commands and then in second phase C++ commands into microprocessor understandable machine code that can be executed.

When you need to change the code, you just modify the scratch blocks, and recompile the program. In native C++ language programming we take a similar approach but instead of

the “visual Blocks” we use various functions that are provided by the compiler. The beauty of these programming languages and compilers is that not only they provide built-in ready to use set of functions as part of the compiler but you can easily import and use the code provided by third parties as “libraries” to extend the functionality of your compiler thereby making application development even more simpler.

```
324   __weak void button_a_callback()
325   {
326
327       WIFI_LED_ON();
328       AZURE_LED_ON();
329       USER_LED_ON();
330       if (BUTTON_A_IS_PRESSED)
331       {     #define RGB_LED_SET_R(value) TIM3->CCR1 = value
332
333             Expands to:
334
335             TIM3->CCR1 = val
336           RGB_LED_SET_R(val);
337           RGB_LED_SET_G(val);
338           RGB_LED_SET_B(val);
339       }
340   }
```

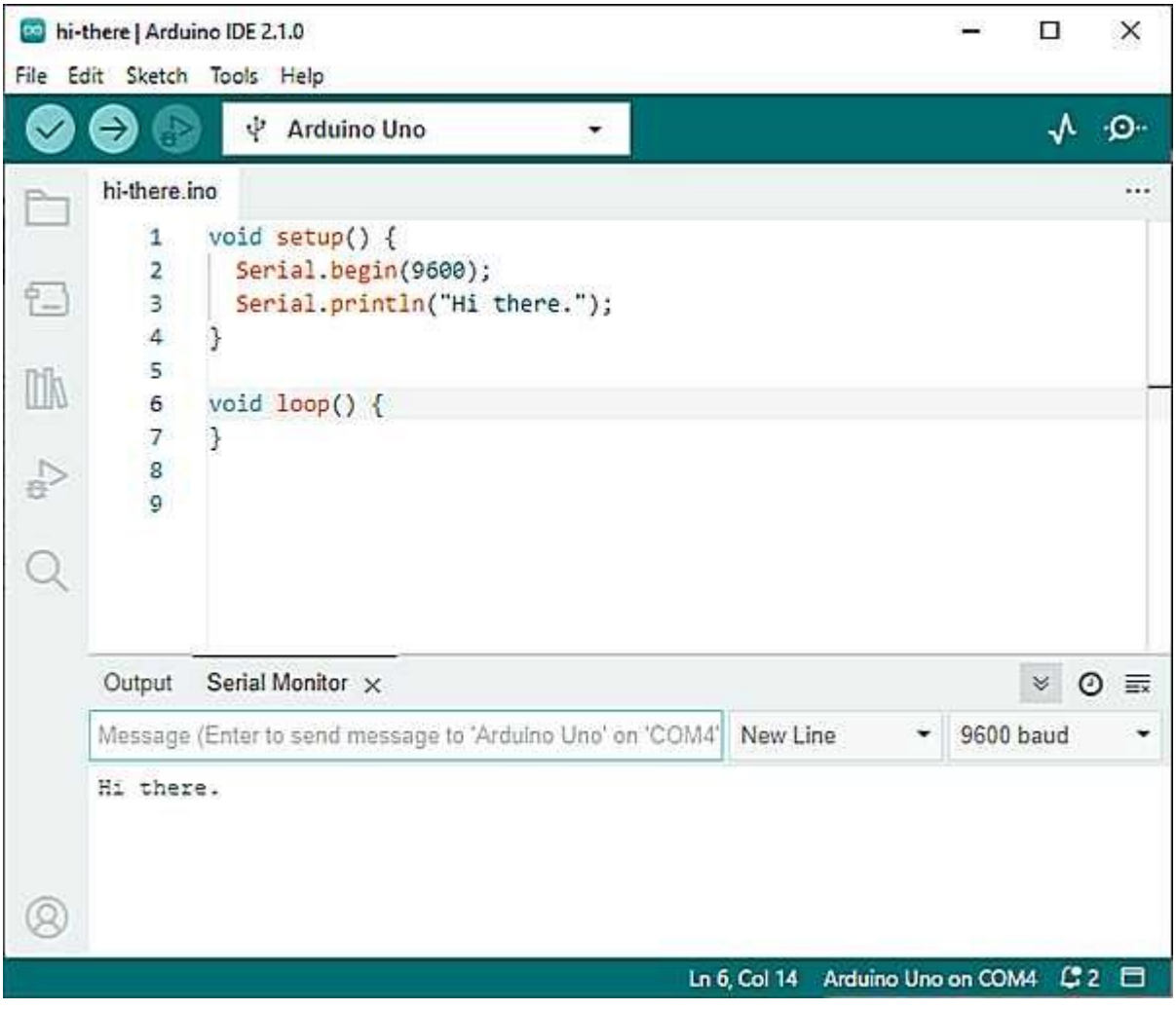
Here is an example C++ code where a number of functions with easy to remember textual names are called in a particular order to get the job done. These individual functions, actually contain many lines of code that is generated in background when we compile the program. However from an application developer perspective all we need to know is what action a particular function will perform, when it will be called in a program.

How Arduino Fits in all this Scenario?

So C++ is a cool fit , its partially low level language giving you access to all the resources of microcontroller as well as high level language giving you the flexibility of leveraging the powers of third party libraries. Arduino has made it a step forward, by issuing a framework based upon the same C++ language and compiler. The framework hides many unnecessary low level commands and makes programming more conceptual. Secondly it has used an object oriented structure of programming and forces the developers to write their libraries as object oriented classes. This makes these libraries adapt a uniform platform and uniform structure so that there is hardly any conflict among them. The objects can be duplicated in memory with separate data. Thirdly it has forced the community based development all to be open-source, this means the code is readily available for all to see, learn, modify (if they want) use and freely distribute.

The Integrated Development Environment of Arduino is simple, does not contain too many cluttered windows, options and settings. Above all they have integrated the process of transferring compiled code into the microcontroller very easy and simple. Previously you needed

a dedicated device called ‘programmer’ that was used to transfer the compiled file into the microcontroller and then run the application. Arduino has written a ‘Bootloader’, which is a special program that is first transferred into the microcontroller using standard programmer. After that the integrated development environment uses the USB communication to transfer new applications. All Arduino boards come with ‘Bootloader’ already programmed, so they can be plugged directly out of the box to your PC.



Learning Arduino programming is the beginning of fascinating journey into the rapidly evolving field of embedded systems. I accepts you as a newbie and makes the things simple and easy to achieve. When you achieve the results by simple commands, it gives you a satisfaction of achievement and motivates to dive more deep. The concepts of programming are same weather its Arduino, or a data server computer, only difference is the scale of resources available. Indeed I believe programming a device with limited resources is more challenging than a robust system with tons of resources.

The knowledge, experience and attitude to solve a problem that you gain while programming Arduino, will form the basis to accept the other advanced microcontrollers with ease. This is just like if you learn driving a small car, the same concepts apply to driving a bus or a truck.

This book is structured as a self-paced learning platform. You may be a student, hobbyist or a teacher you will find this book quite helpful in exploring this wonderful world. Each chapter builds upon the knowledge that you have gained from the previous chapters. Some theory like electronics, basic math's and related algorithms etc. are explained wherever their need arises. There are dedicated chapters where we use ready-made modules and publicly available libraries that will bring those modules to life in a matter of minutes. I would however emphasize on you to experiment with it and explore the native structure yourselves.

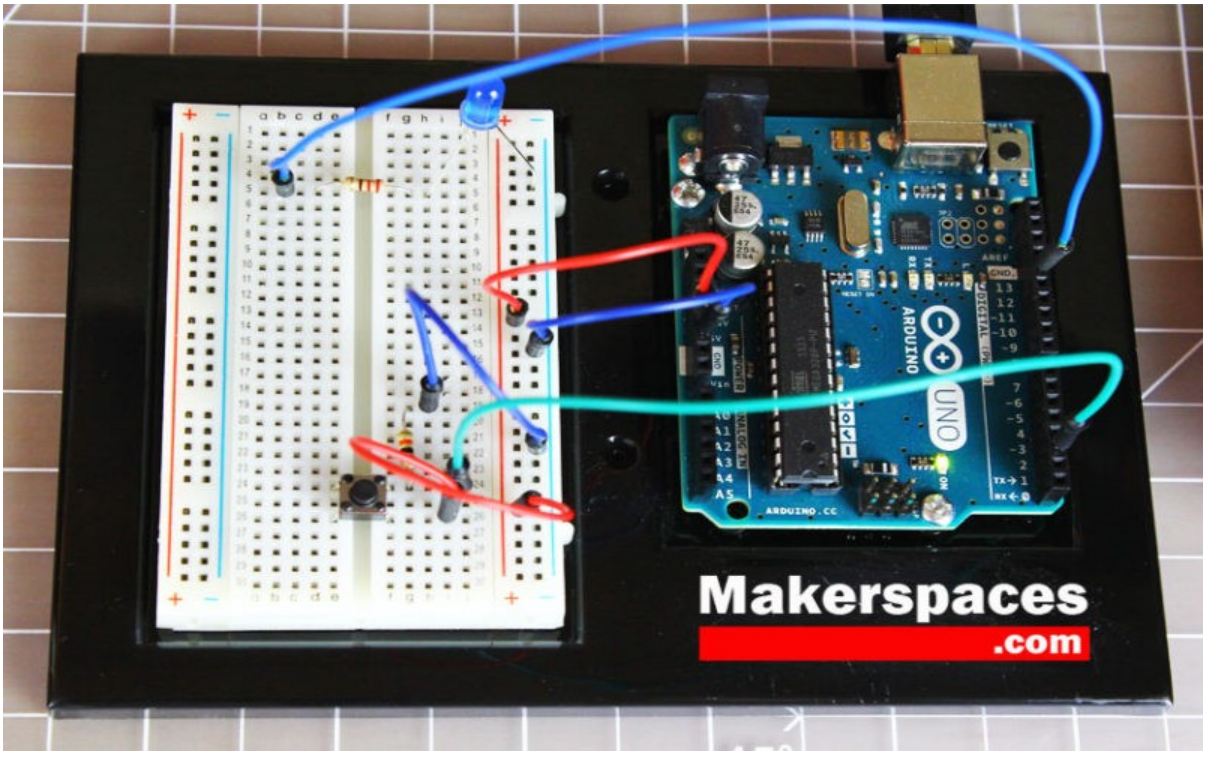
The lessons in this book have been classroom tested. Students have created projects of their own designs based on what they've learned with earlier versions of these lessons. hey have made model helicopters and airplanes, elaborate rolling robots, musical instruments, light panels, keyboards to drive synthesizers, "laser" tag games, hover boards, Segway-like vehicles, and more.

Whether you are exploring this book for yourself or to teach others, I hope you find the content engaging and useful. I invite you to share your thoughts, suggestions, and cool projects of your own.

CHAPTER 2

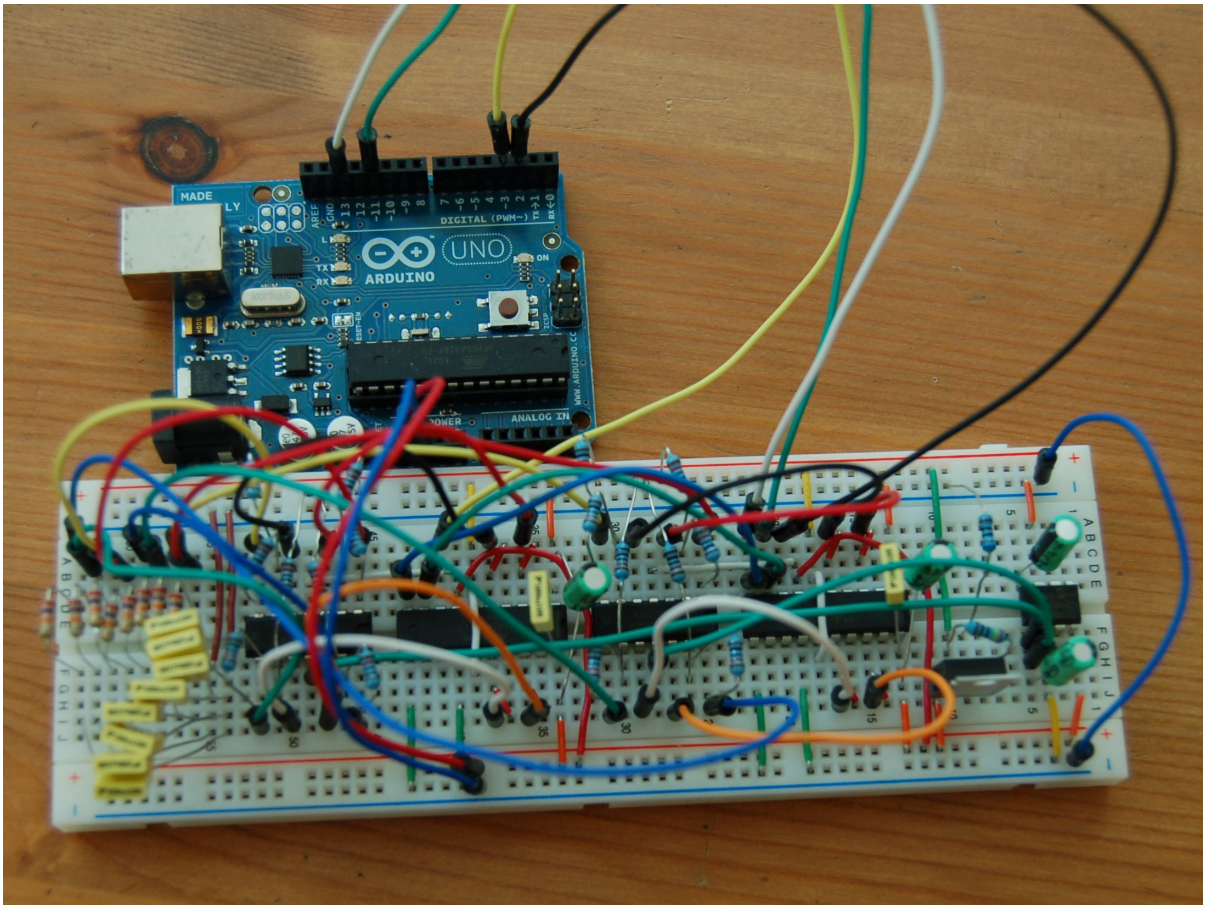
MikroDuino Nano Platform

The Arduino family has many development boards all sharing the same basic philosophy of easy to use and easy to program. The Arduino IDE and the modified C++ programming language that it implements, abstract many of the underlying hardware details. The boards provide necessary circuitry for the microcontroller, like power supply, access to its IO lines and various hardware modules, and programming interface. Apart from that the bare board does not do anything useful. In order to get it used in a project you need a supporting external circuit that should contain the required electronics, components and a plethora of devices. The external circuit is usually built rapidly (rapid prototyping) using breadboard. The circuit is first built on breadboard and then its controlling connections are made with the Arduino board. In this way a complete functional project is ready. Now the Arduino board is programmed and the project comes to life.

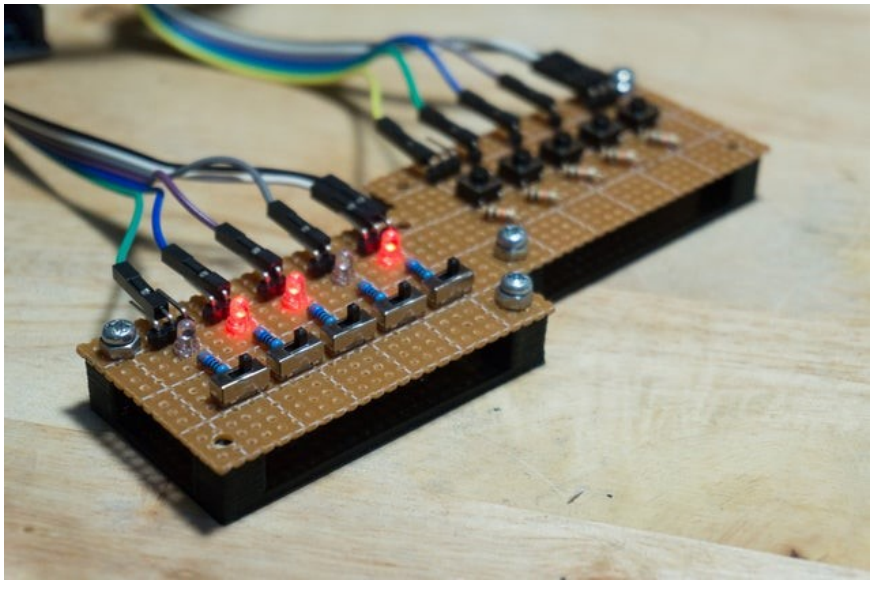


The above picture shows a typical project with jumper wires connecting Arduino board to the breadboard. This arrangement is good, and rapid when the external circuit is not very complex and requires few components. However it becomes really tedious and complicated when the required number of electronic components grow and their interconnections be-

come complex. This entails many jumper connections on the breadboard as well as from breadboard to Arduino board.

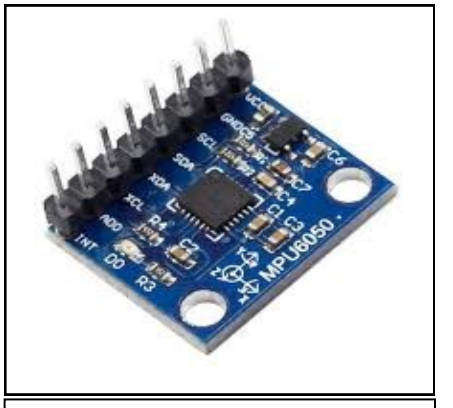


Sometimes this complexity is natural as the required project is complex, but many a times its unnecessary as well as repetitive. By repetitive I mean that there are some components that are very frequently required in most projects, like displays, LEDs or buttons etc. when building these circuits of breadboard they tend to add to the complexity of breadboard wiring as well as they need to be made every time in new projects. A Simple solution for such a



problem is to readymade modules that one can build for his own personal lab. You can make many other similar modules like for LCD display, seven segment displays and so on. These modules are usually soldered on perfboard and are quite stable and useful. Soldering and making permanent electrical connections, reduces the clutter of wires.

A major problem that I have observed with breadboard projects, is the loose connections of wires with the breadboard contacts. Moreover the disorganized connections makes it really difficult to trace any fault if the project is not behaving as per expectation. When using such modules the number of connections are dramatically reduced.



MPU6050 3-Axis Gyroscope Module

Many commercial companies have started producing modules for this purpose. Sometimes the external circuitry required is based upon special purpose chips, like Bluetooth communication, wireless communication or interfacing with TFT displays etc. The circuits of these devices are so complex and delicate that they are beyond the reach of a student or hobbyist to make on breadboard. These modules offer the same level of convenience and flexibility as of homemade modules.

Jumper Wires

When connecting Arduino to the breadboard or other modules jumper wires are used. The ends of these wires contain terminals that should fit the connector and establish a secure connection. By secure connection I mean the connection should be fitting snuggly, without any loose connections, and they should be secure that accidental movements of hand or otherwise should not dislodge the connection.


Male-Male Jumper Wires

Since Arduino boards specially Arduino UNO and Mega have ‘female’ type headers for connections and the breadboard also has ‘female’ type sockets for connections, male -male jumper wires are frequently used. In my personal experience there are various ‘Qualities’ of wires available in market. Some have very short and thin connectors, that don’t fit well in the Arduino sockets as well as breadboard sockets.

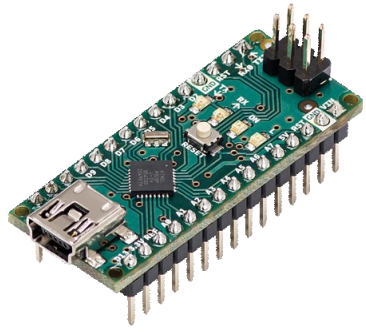


Consequently most commercial modules provide male type headers on their module, so that you need a female-type jumper connector to connect to them. This connection in my personal experience is more secure both in terms of secure connection and accidental dislodgment. Since Arduino board has female header so we use these jumper wires with male connector on one side and female connector on other side. Use of male connections on Ar-

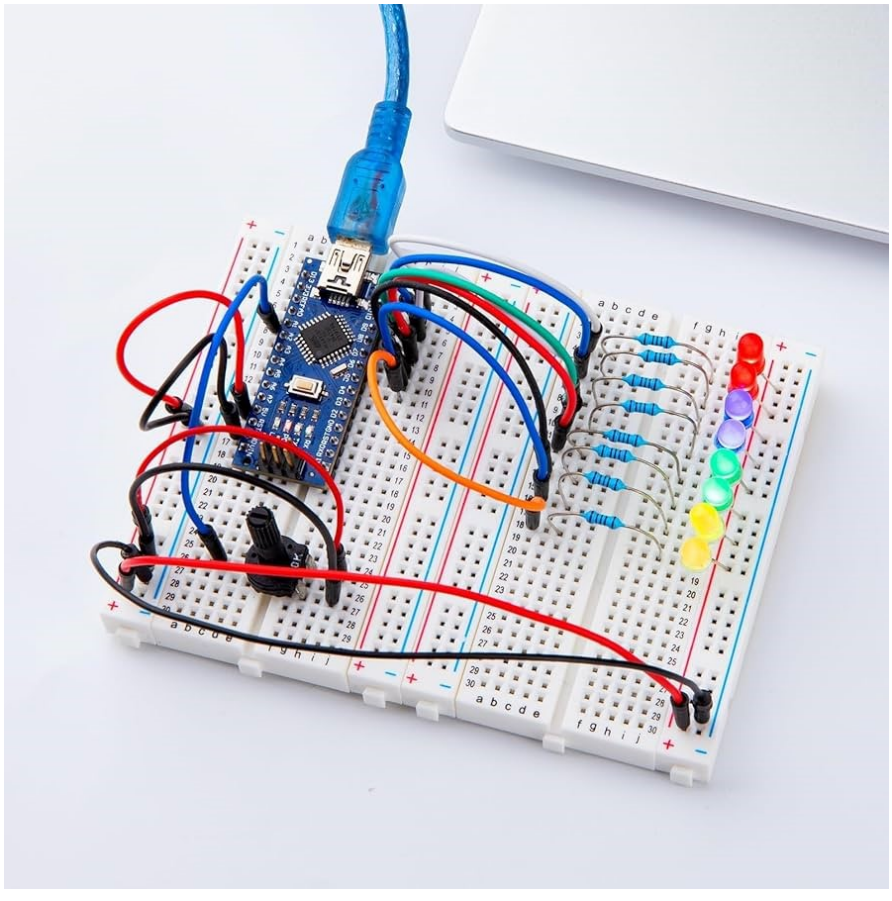
duino still makes these connections a less attractive option. But still at least connections with module are secure.

Arduino Nano

Although Arduino UNO is the flagship board of Arduino family, another variant and compact sized board from Arduino family is 'Arduino Nano'. It has the same microcontroller as on Arduino UNO, and has similar resources like power supply and regulator etc. but on a small scale factor. Also the connections are usually as male header so that it can be treated just lie a 40 pin integrated circuit.



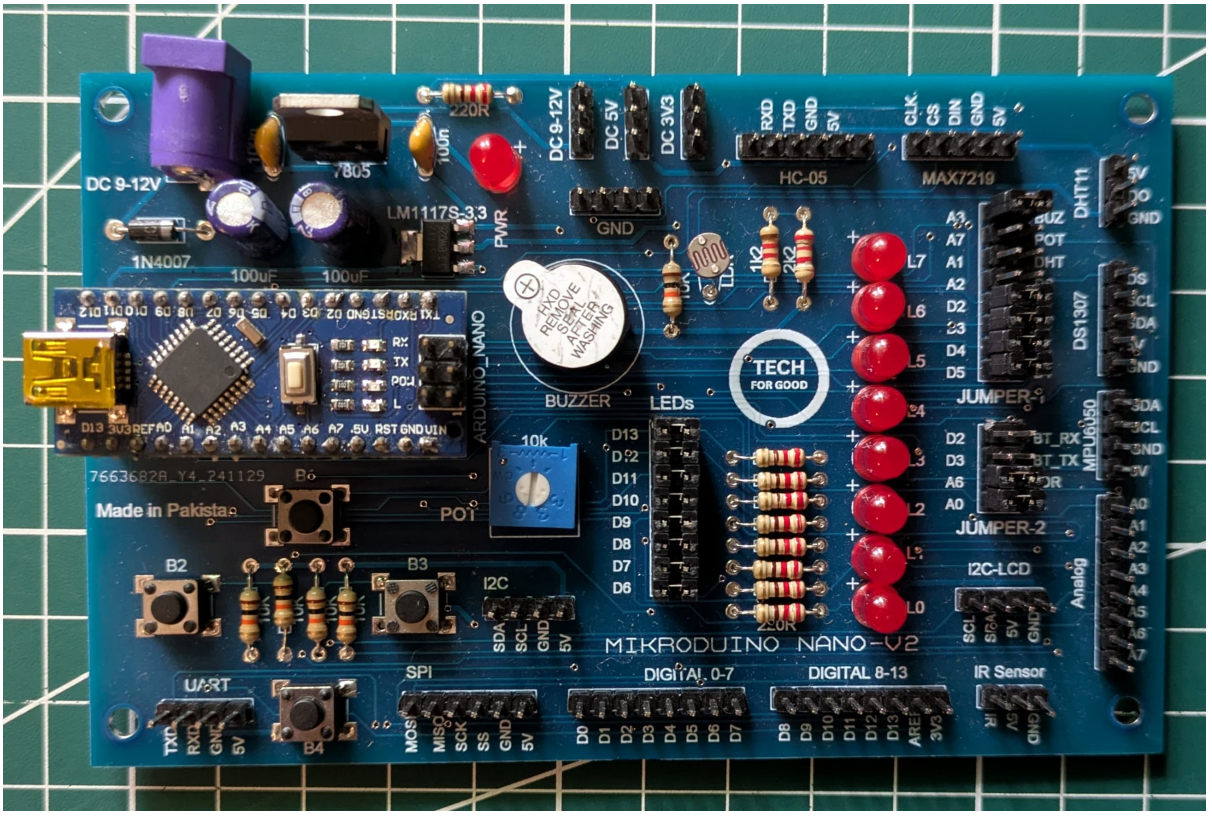
This board can be connected to modules using female -female jumper wires, or can even be plugged in directly on breadboard. In that case further connections will still need male-male jumper wires. The header pins of Arduino nano are usually quite thick and strong and usually make good contact with the breadboard.



MikroDuino Nano Platform

MikroDuino Nano is an adapter board developed by Mikrotronics Pakistan, to make it further easy for the developer to rapidly prototype the projects, and reduce the need of external modules, or breadboard jumper wires. Although it is not possible to totally eliminate the breadboard and jumper wires from the learning process, yet we can reduce it while at the

same time offering same level of flexibility and liberty. The MikroDuino nano board has been designed to keep the things simple, neat at the same time 100% flexible. The board is based upon Arduino Nano, which easily plugs in the board. And a host of commonly used electronic components are already on board that can be connected or disconnected to the Arduino pins using jumper tabs (to predefined pins) or using female-female jumper wires to pins of your choice if required by the project.

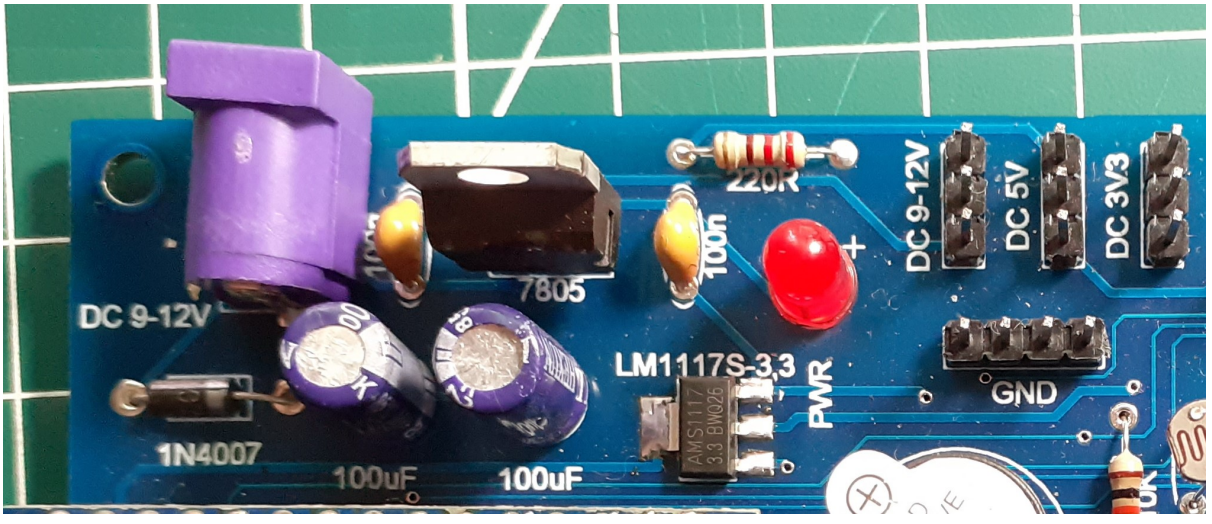


Additionally the board has a number of headers to connect various commercially available modules directly to the board. These headers have pin arrangements that are similar to the pin arrangements on module, making it simple to connect. Moreover some modules that use I2C communication or SPI communication need to be connected to the specified pins of Arduino only, so there is not question of choice of IO line for them, the headers connect these pins directly to the module pins. There are only one set of I2C pins on Arduino board, while the literature says you can connect a number of devices together on the same lines. In case you want to connect more than one I2C compliant devices you need expansion boards, or connectors like breadboard. This board offers the I2C and SPI lines not only on the IO header pins, but also dedicated headers for external devices. Indeed the board also has headers for I2C LCD, I2C OLED and I2C EEPROM, I2CDS1307 RTC etc. Lets dive in a little bit to explore the various parts of this board.

Power Supply

All projects when running need a reliable power supply. Most projects do not need very heavy power supply so they can get the supply from Arduino boards. This board can get the power supply directly from the Arduino Nano, which gets is directly from the USB port and

in most cases from your computer. The supply current depends upon your computer's USB port. Usually these ports can supply up to a maximum of 500mA current. The Arduino Nano board provides the same 5V to the microcontroller as well as to its distribution pins for use by modules.



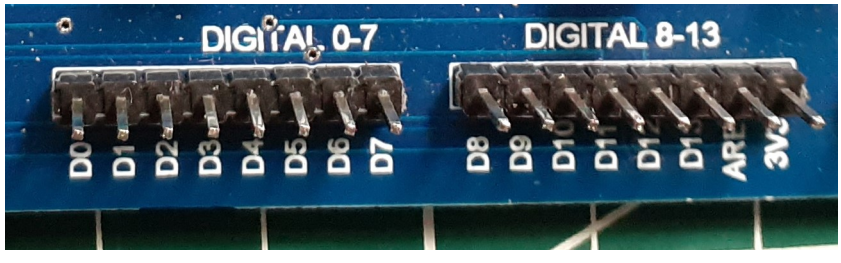
There is also a 3.3V regulator on Arduino nano (LP2985-33 or similar) that can provide only 50mA of current. Though 3.3V is not used by Arduino Nano board, yet there are some modules out there that run on 3.3V so it becomes handy to have this power supply to use those modules readily.

MikroDuino Nano board gives an external source of power supply as well. So you can get the supply from USB connector of Arduino Nano, or when not connected to your PC from this external source. The external source has a barrel connection for DC power adapter. Although you can give up to 30V DC, but we recommend giving 9-12V DC supply. There is a reverse polarity protection diode that prevents powering the board if the polarity of power supply is reversed. The board has a robust 5V linear voltage regulator (LM7805) that can power up to 1.5A of current. The board has its own 3.3V regulator (LM1117) that can supply up to 1A of 3.3V current.

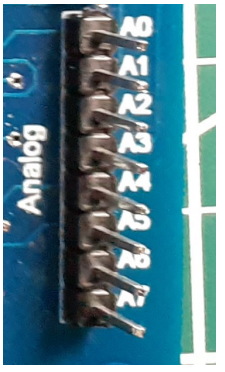
The 5V power supply is routed to the 5V rail of Arduino nano, as well as to headers for various modules. The board has 3 headers for supplying the board power to external modules. Header labelled DC9-12V gets direct power from the barrel jack. So whatever you provide appears here. DC 5V and DC3.3V headers provide 5V and 3.3V regulated power for external modules if required.

Arduino IO Lines

Arduino nano has 22 general purpose IO lines, also called GPIO lines. These lines are grouped as "digital" and "analog". Digital lines are numbered from D0 to D11 and analog lines as A0 to A7. Some digital lines have PWM modules attached to them, so these can be used to produce PWM signals. While some lines have internal connec-



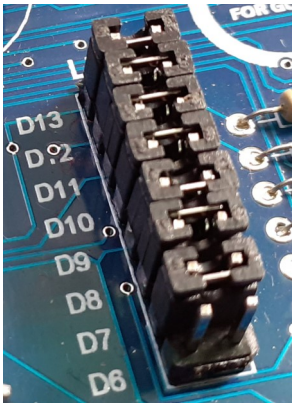
tions to I2C module and SPI module. MikroDuino board gives access to these lines as two headers, one exposing digital pins D0 to D7 and other D8 to D11. PWM pins are D3, D5, D6, D9, D10, D11.



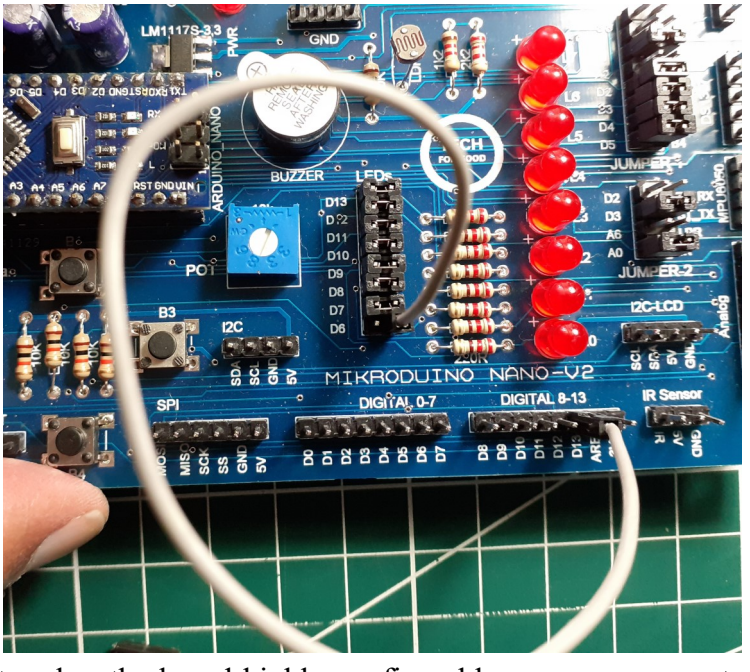
Analog pins are provided on a separate header and labelled as A0 to A7. The analog pins can also be used as digital, in that case their numbers continue from D11 onwards, like A0 can be used as D12 and so on. A6 and A7 are however purely analog input and these cannot be used for digital data.

The analog pins are connected to internal Analog to digital converter module. These pins therefore can read analog voltage as input. These pins can not produce analog output.

Configuration Jumpers



As previously said the board has number of commonly used components, like LEDs, Buttons, potentiometer etc. Instead of hard wiring these devices to the GPIO pins, we have provided jumper headers. On one side of jumper header are the device pins (Like LEDs) and on the other are digital GPIO lines as selected by us. Placing a jumper tab on the header will connect the corresponding device pin to the labelled GPIO pin. And disconnecting the tab will free the pin from this device. Additionally is your project requires the device to be connected to a different IO line other than the one, we have selected, just remove the tab, and using a jumper wire connect the device pin to GPIO pin of your choice.



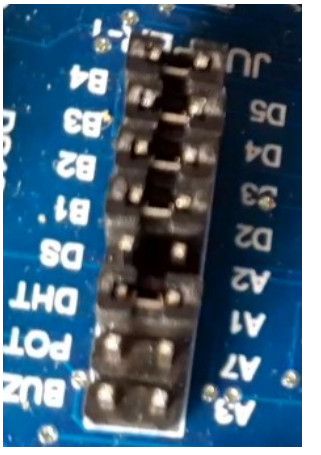
This arrangement makes the board highly configurable, you can connect a device pin to a preselected GPIO, or to a GPIO of your choice or even disconnect it, if you want that pin to be used in some other module or simply don't want this device in your project.

Status LEDs (D6:D13)

The MikroDuino board has 8 status LEDs connected as common cathode. The anodes of

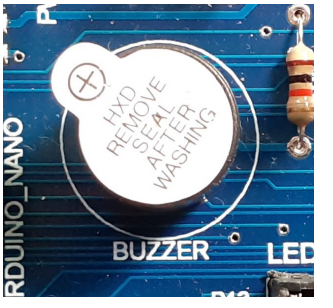
these LEDs are connected via 220 Ohms resistors to the LED header. The other side of LED header is connected to digital pins, D6 to D13. remember D12 and D13 are also A0 and A1. So when using A0 and A1, disconnect the Led 7 and Led 8.

Push Buttons (D2,D3,D4,D5)



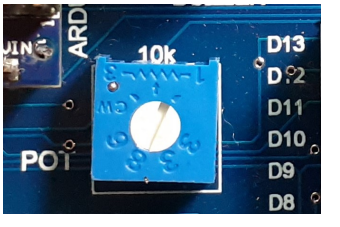
There are 4 momentary push buttons on board. Arranged in a cube style. The buttons have external 10K pullup resistors attached to them. Although Arduino pins have internal configurable week pullup resistors, yet a 10K external pullup gives better noise tolerance. The header for buttons on jumper 1. Button B1 connects to D2, B2 to D3 and so on. These connections are clearly marked on the PCB. In case you want the button to be connected to some other GPIO line, ject disconnect the header tab and use a jumper wire to connect the button pin to any GPIO line of your choice. No additional debouncing circuitry has been added, so in its native form you will learn to implement debouncing in coding.

Active Buzzer (A3, D17)



Producing sounds like alerts, or giving feedback is also commonly used in application development. The active buzzer provided on MikroDuino board also has its jumper on jumper-1 and by default connects to A3 pin. Although A3 pin is analog input in most cases, but it can be configured to be used as digital. Just use it as D15. The buzzer has internal 1K Hz Oscillator, so when the corresponding pin is turned high it starts producing the tone.

Potentiometer (A7)

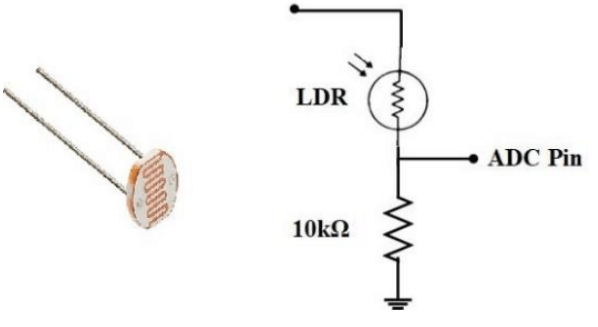


Potentiometer is just a variable resistor, its two ends are connected to 5V and GND supply the center tap is available for connecting to Arduino. Its jumper is also located on Jumper-1 and by default connects to A7 pin of Arduino nano. Potentiometers are useful in testing the analog input to simulate a sensor, or just as an analog source.

Light Dependent resistor, LDR (A6)

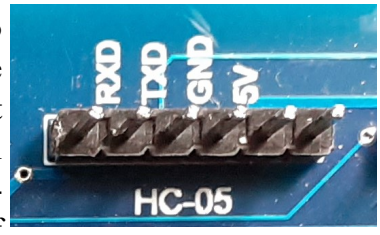
The board has an LDR which is configured as a voltage divider with a 10K resistor.

The output is analog voltage that corresponds to the ambient light. The jumper tab for LDR is located on Jumper-2 header. By default it connects to A6 analog pin of Arduino nano.



HC-05 Bluetooth Module Header (RX:D2, TX:D3)

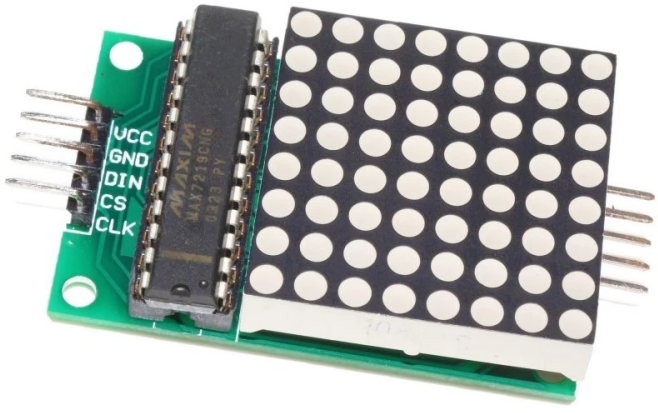
HC-05 module is commonly used in Arduino applications for giving Bluetooth connectivity. The Bluetooth module communicates through serial communication. We can connect its transmit and receiver pins to the USART (D1 and D1) pins of Arduino. Since D0 and D1 are also used for programming the Arduino, most projects connect HC-05 to software controlled serial port, for which any digital GPIO line can be used. We have therefore provided a tab jumper for its these two pins, The default connections are RX of Bluetooth module is connected to D2 and TX is connected to D3. The jumpers are located in Jumper-2 header. The RX pin of this module requires 3.3V signal, therefore a voltage divider has been built on the board to allow it connecting to 5V output of Arduino pins.



Max7219 LED Header

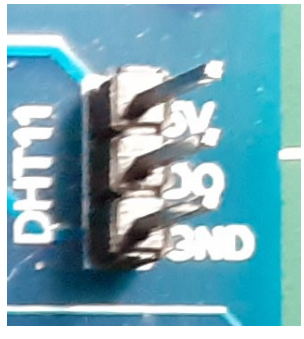
MAX7219 is a popular led driving chip, it has built in circuits to repeatedly scan the eight rows of LEDs. It is commonly used to drive an LED Matrix module that contains eight rows and eight columns. It uses SPI communication protocol, therefore we have hardwired the header to the SPI module of Arduino nano. All you need is just to plugin the MAX7219 module. The required 5V power supply ids also provided on the module. Max7219 based seven segment 8 digit LED modules are also available and can be connected to this header.





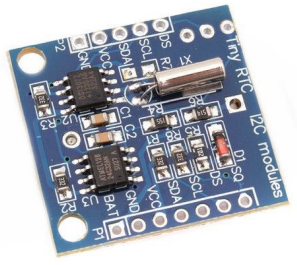
DHT-11 Humidity and Temperature Sensor Module Header (A1:D13)

DHT11 is a commonly used sensor, available as a module that can sense the environmental humidity and temperature. It is a digital sensor and emits its data as digital signal on querying. MikroDuino board has dedicated header to connect this module directly. The jumper Tab is located on header 1 and by default connects it to pin A1. Pin A1 can be treated as digital pin D13.



DS1307 Real Time Clock and EEPROM Module

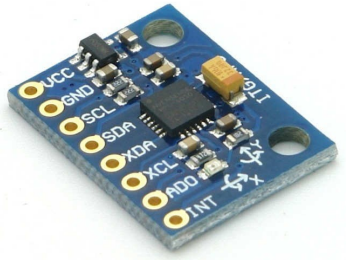
DS1307 is commonly used for keeping a digital clock and calendar. The DS137 module actually is 3 in one. It has a DS1307 chip that keeps the clock and calendar running. Along with that it has 24C32 EEPROM chip that runs on the same I2C bus but with a different address. Optionally the module has pads for DS18B20 which is a temperature sensor, but works on a different protocol, called Dallas one wire protocol. You will need to solder this sensor on board in order to use it. Since both DS1307 and EEPROM work on I2C bus, the module is hard wired to the dedicated I2C pins (SDA and SCL) of the Arduino nano. So there is no need to configure the jumpers. Jumper for DS18B20 is however provided on Jumper 1 header and it connects to pin A2 which is digital pin D14. This is however optional if you are using DS18B20 sensor on module. I would recommend to do so, because for learning purpose you got to learn a new technology.



MPU6050 3-Axis Accelerometer and Gyro meter Header

MPU6050 is a very good sensor that can sense the position of your device. It has two different functions built-in the same chip.

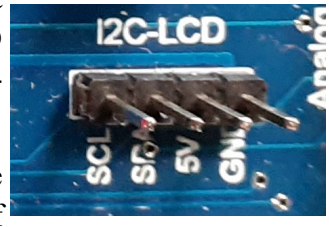
The chip communicates through I2C interface, therefore its SDA and SCL connections are hard-wired to the Arduino I2C module pins. Rest of the pins XDA , XCL etc. are not commonly used therefore not provided on header.



I2C LCD Header

Displaying data is usually required in Arduino projects, there are many types of displays that can be used with Arduino. Character LCD is one of the most popular one. The display however requires at least 6 digital IO lines for communication. There are I2C modules that can be connected to these displays and then use I2C bus to display data.

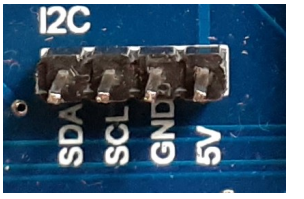
MikroDuino Board has a dedicated header to connect this device directly to the Arduino I2C bus. The pin layout is as per pinout of the module.





There are also I2C seven segment displays that you can plug into this header. The pin arrangements might be different, but they can be connected. I2C OLED Display can also be connected to this bus.

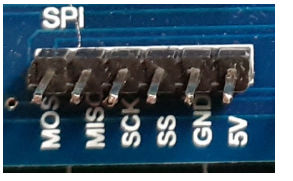
I2C Expansion Bus



Since many devices are available that can communicate via I2C a dedicated I2C header along with 5V power supply is provided to connect any I2C compliant device.

SPI Expansion Bus

The SPI pins are also dedicate GPIO pins on Arduino. In addition to MPU6050, the MikroDuino nano board has a dedicated SPI expansion header to connect any other SPI device.



IR Remote Control Sensor Module Header

IR remote control is a very commonly used device to get user input on just a single GPIO line. This allows a wireless system of input, and gives you a large number of buttons as input for the user. You can use any remote control with it.

The jumper header for IR remote control is located on Jumper02 and default connection is to pin A0, which is used as D12 when used as digital.

So this was a basic introduction to the MikroDuino nano board, that we will be using in our various demonstrations throughout this book, or manual whatever you want to call. The applications developed will be generic, and in no way dependent upon MikroDuino nano, it simply makes it more convenint to make and learn when using it.

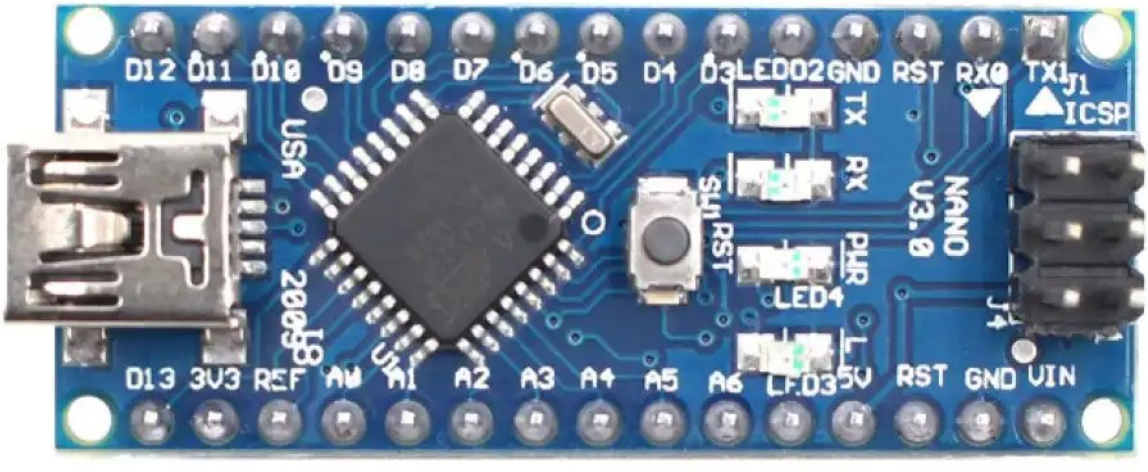
CHAPTER 3

Arduino Nano | AVR MCU

In this chapter we will dive a little deep to understand the Arduino nano board, and the AVR microcontroller that it has. Though using Arduino IDE all this knowledge is not required, because the beauty of Arduino IDE is in abstracting all this detail. I however believe that one should always know the fundamentals and atomic level structure of the things. This helps better understanding, and ability to modify the code or even libraries if you want.

ARDUINO Nano Board

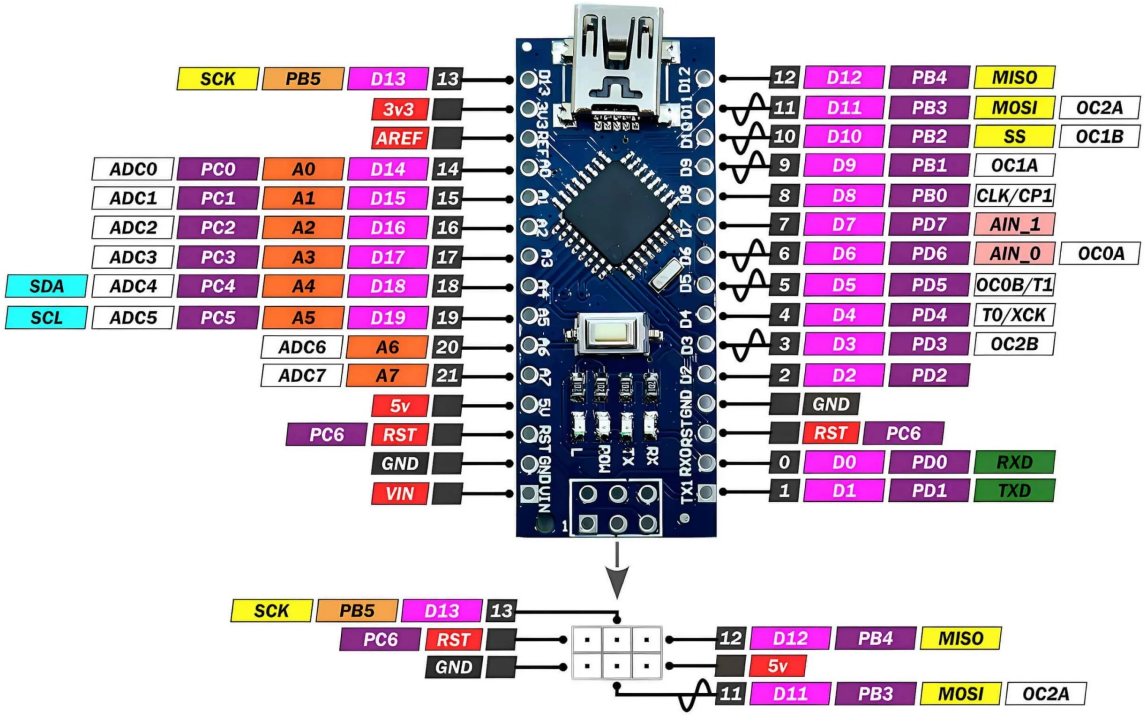
The Arduino nano board Is a small form factor duplicate of the flagship Arduino UNO board. Arduino nano has the same microcontroller as on Arduino UNO. ATMega328P microcontroller is used in Arduino nano.



The board has all the necessary circuitry to support the functionality of ATMega328P microcontroller. The board has a USB to serial conversion chip that allows communication with PC using USB port. Most of the Arduino nano boards contain CH340G chip, its driver is usually automatically detected by windows. The board has one user programmable LED which is connected to D13 pin of the MCU. In addition there are LEDs for serial communication, labelled as Tx and Rx, and there is one LED for power. There is a reset button on the board. The board gets power supply from USB port, which is always 5V. However it can also be powered through Vin pin which goes to 5V regulator chip and then the 5V rail is powered up. The 5V regulator is small scale, and can give only upto 500mA of current. The

board itself needs very little power, however the 5V rail is available through jumper header for expansion. The limitation of this 5V regulator current should be kept in mind. The 5V supply also passes through a 3.3V regulator, which is also available in header for expansion modules.

All GPIO Lines marked as digital and Analog are available as header on both sides of the board. The Arduino nano has 14 digital lines, numbered from D0 to D13 and 8 Analog lines numbered from A0 to A7. Pins A0 to A5 can also be used as digital using digital labels, D14 to D19. Pin A6 and A7 are however purely analog inputs.



The picture above shows the pinout map of Arduino nano. Pins labelled in pink are digital pins, and are used in Arduino programming language to access the digital pins. Pins shown in orange are for Analog to digital conversion, and are numbered from A0 to A7. Notice the analog pins A0 to A5 are also digital and can be addressed as D14 to D19. However A6 and A7 are only analog.

There are three communication modules in ATMega328 microcontroller. These are USART (Green Pins), SPI (yellow pins) and I2C (sky Blue pins). Each of these modules have their own specific pins that are hard wired to certain pins of Arduino nano board. When using these modules the external circuitry must be connected to “these” pins only.

Digital pins numbered D3,D5,D6, D9,D10 and D11 can be provided with six independently controlled 8 bits Pulse Width Modulation (PWM) Signals. The pins are indicated by a wave like symbol on them.

Pina labelled in violet are the same as digital or analog, but these names like PD0, PD1, PD2 etc. are used by the native compiler that is provided by the manufacturer. The datasheet of microcontroller also name these pins as these labels, indeed they actually group them as 8 bit Ports, like PORTD, PORTB etc. each having its bits named as PD0, PD1 and so on. When programming the Arduino board using Microchip Studio, and native C++ program-

ming you access these pins by these names. Arduino has simplified the process to access these individually named pins, by replacing or mapping an internal array thus allowing us to access these pins just by using a linear scale of integer numbers, from 0 to all the way to 19.

> Although We label digital pins as D0, D1 etc in this text, they re represented by the integer number only in program. So D13 will be accessed as 13 only in program.

This Arduino scheme of mapping the individual Ports and their bits, to a linear scale has dramatically simplified the application development. This ease of application development however comes at some 'Cost'. Arduino generates an extra code that is run every time you access the pin by Arduino numbering system. This translation takes some CPU time, and thereby affects the speed of performance. Most of our applications are not that much time critical, so this small reduction in speed is acceptable in view of the convenience that it provides.

The underlying compiler running in Arduino is same as provided by Microchip in its Microchip Studio. Therefore you are free to use the native C++ naming convention and bit accessing while in Arduino IDE. I will show you these methods in appropriate sections.

Pins labelled in White are special function bits of certain modules, specially timer modules, that are used to control the PWM output. The analog pins are also shown in white, these pin names are used by the compiler when accessing these pins as analog inputs.

Programmer Pins

There are a group of six pins, on a separate edge of the board. All these pins are also available from the Arduino standard side header, but they have been grouped together to allow a hardware programmer. This device is a separate hardware, not provided as part of Arduino, and there are actually a number of devices available from different vendors. These devices put the microcontroller into programming mode and allow transfer of compiled program from PC into the program memory of microcontroller. In addition to programming these devices also provide access to inside registers directly at run time allowing low level debugging of the application. The same devices also gives you access to the "Fuse Bits" which are special protected area of the microcontroller and configures various aspects of the microcontroller, like protecting your code from being copied, or configuring a different speed of crystal etc.



Arduino Programming

Another great thing Arduino developers have done, is that they have written a piece of code called "Bootloader", this code is already present in every Arduino board. This code occupies a small part of program memory, and whenever the microcontroller is reset, either by a signal on reset pin, or power up, this code starts execution. It will monitor the Serial port (USART) for a new compiled user program, Arduino IDE sends its compiled program, over the USB using USART commands. If the bootloader gets a new program, it erases the previous user program in microcontroller, and writes the new program in its place. When new user program is in place, it hand overs the control to the user program. In case of reset or

power up, if there is no new user program coming, the bootloader simply starts execution of the existing user program. All this process is automated by Arduino IDE, and when you press the program button, the IDE compiles the user program, sends a reset signal to the Arduino board and transfers the new program to be executed.

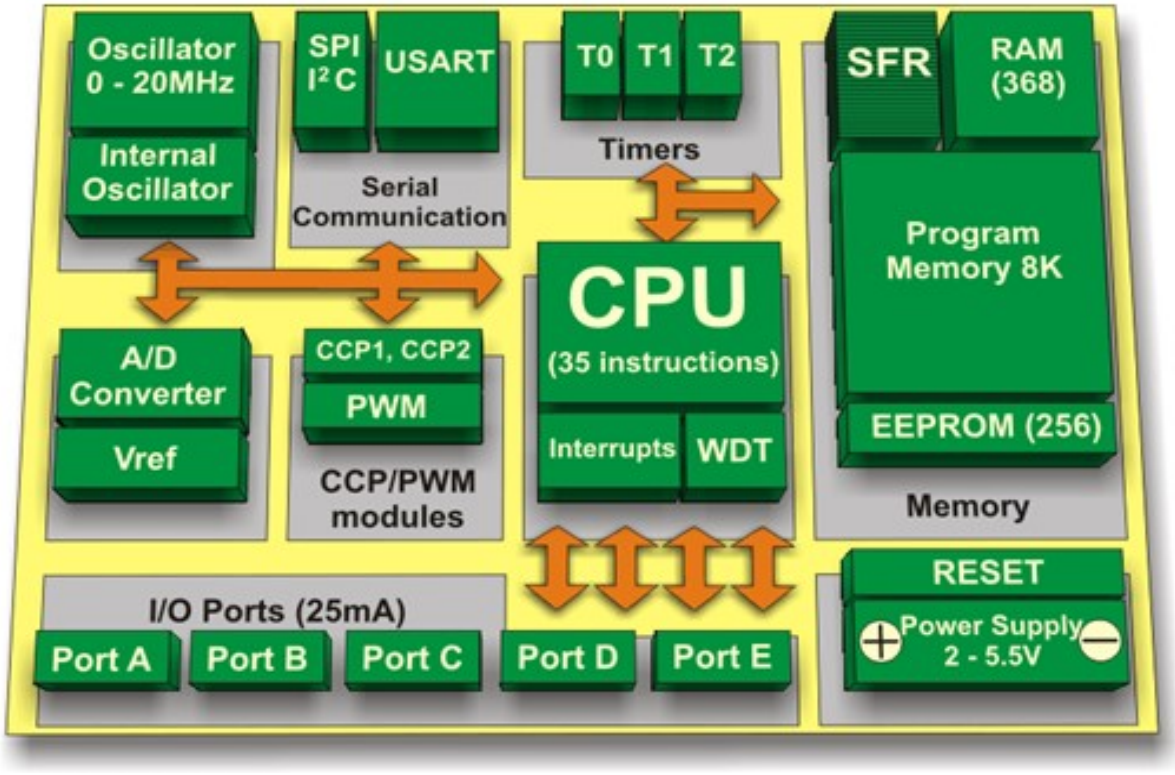
This is fairly good for most of the times, but this bootloader does not allow you to access the fuse bits, and special function registers at run time, thus limiting the debugging and device settings.

So if you get a new chip from market (ATMega328) it cannot be programmed via Arduino IDE, unless the bootloader is first transferred into it using standard programmer like USB-ASP or you may program it using USB-ASP only as you like.

What is a Microcontroller

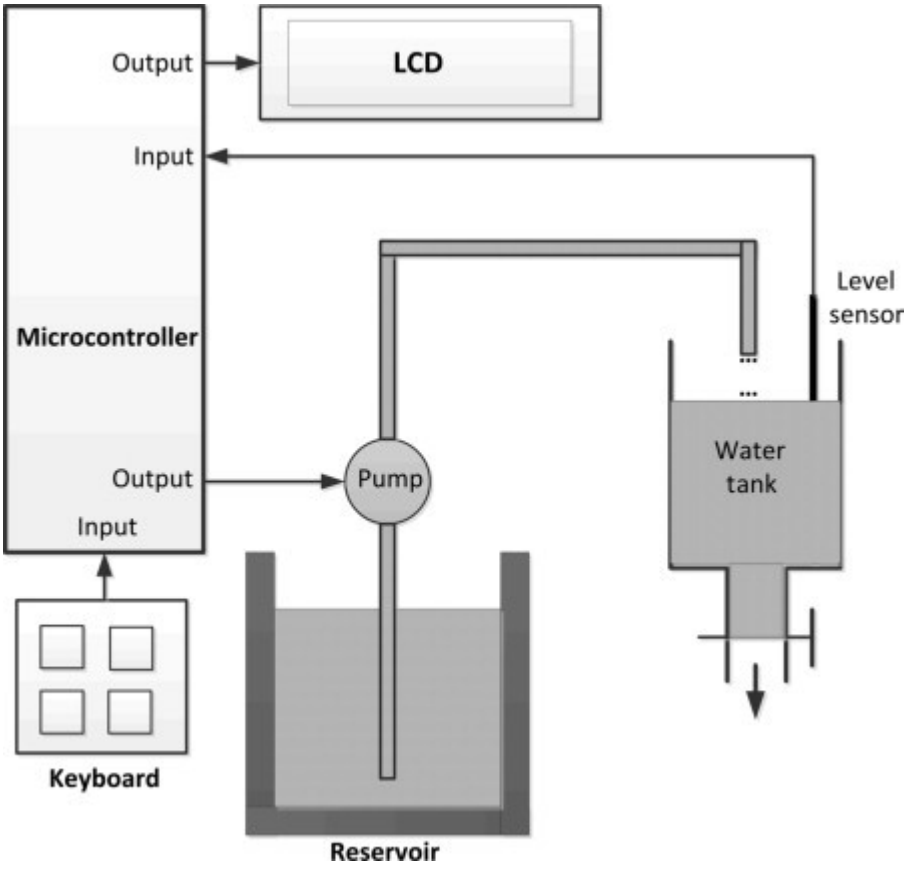
While programming the Arduino using Arduino IDE does not require you to understand the underlying architecture and hardware of the microcontroller itself, yet I believe knowledge is power and the more you know about your hardware the more control and command you have over it. This is just like driving a car, most of us only require to know our dashboard, and important paddles that car manufacturer has provided us to communicate with the underlying hardware of the car. You don't need to know how the fuel jets work, when you press the accelerator, and how the gears shift when you change the gear lever and so on and so forth. The knowledge of underlying hardware becomes important when troubleshooting. Same thing goes about microcontrollers. If you know what you are driving and playing with you understand better what's happening and what limitations it has.

A microcontroller is a mini computer with a handful of modules useful in communication with other electronic components. All packed together in one integrated circuit with pins for



connecting the microcontroller with other electronic devices on board. The microcontroller is able to read data from external environment, or other specialized chips. Process that data according to a set of requirements as determined by a program, and then produce an output to affect the controlling device.

A microcontroller control system usually consists of sensing the environment, getting user input, displaying output for user information, and then producing an appropriate control signal to control the application.



ATMEGA328P Introduction

This is the right time to know the features, powers and limitations of your microcontroller. Always make it a habit to go through the manufacturer documentation to get the true picture of your hardware. When we deal with third party resources like Arduino, or other community acquired libraries, they may or may not harvest the full potential of your hardware, and force you to play within their own boundaries.

ATMega328 is an 8 bit microcontroller manufactured by Microchip corporation. It belongs to AVR series of family. This family was originally designed and produced by Atmel microcontrollers company, later it has been acquired by microchip. The AVR series of microcontrollers are based upon advanced RISC architecture. RISC stands for “Reduced Instruction Set Computing”. The instruction set has been reduced to essential instructions, resulting in reduced CPU complexity and circuitry. This enables reduced power consumption and improved performance. The ATMega328 microcontroller can run at a maximum clock frequency of 16MHz, though we can make it run using slower speed crystals, or even using an

internal **8MHz oscillator**. Most of the instructions of AVR series microcontrollers can complete their execution within 1 clock cycle. This means if the CPU is running at 16MHz, it is executing 16 million instructions per second.

High clock frequency delivers better computational performance, but also makes the application power hungry. Higher is the frequency higher is the power consumption. So when designing applications that will actually run on battery this factor becomes really important. So you choose the frequency according to system requirements. High speed is not always the best choice. The Arduino system however is designed to run only at 16MHz, limiting your choice of power consumption.

There are three different types of memories inside the microcontroller. It has **32KB of flash memory** dedicated to keep the application software. Usually it is called 'Program memory'. This is nonvolatile memory, which means the contents of memory remain preserved even when power is turned off. Second type of memory is **2KB of SRAM**. SRAM is volatile memory, and keeps the contents alive till the power is on. On reset and power off this memory gets cleared. The speed of reading and writing data to this memory is however quite fast and is mainly used to keep the runtime variables, and objects. Third type of memory is **1KB EEPROM**. Technically its similar to flash memory, but separate from it. It is also nonvolatile and is used to record the program settings and some configuration variables, that are generated during program execution. It has **23 general purpose I/O pins**, supports six PWM channels, eight 10 bit ADC channels and multiple communication interfaces such as **USART, SPI and I2C**. The microcontroller has a wide operating voltages from 1.8V to 5.5V. With nano power consumption modes and deep sleep mode it is suitable for battery powered applications. It has 3 programmable timers 2 8-bit and 1 16-bit.

Feature	Specification
Architecture	8-bit AVR RISC
Package	SMD (TQFP-32, QFN-32)
Operating Voltage	1.8V – 5.5V
Clock Speed	Up to 20 MHz (16 MHz typical in Arduino)
Flash Memory	32 KB
SRAM	2 KB
EEPROM	1 KB
GPIO Pins	23 (Digital + Analog)
Digital I/O	14 (including 6 PWM channels)
PWM Channels	6 (D3, D5, D6, D9, D10, D11)
Analog Inputs	8 (10-bit ADC, A0-A7)
Communication	USART, SPI, I2C (TWI)
Timers	3 (Two 8-bit, One 16-bit)
External Interrupts	2 (INT0, INT1)
Power Consumption	Low-power modes available
Bootloader Support	Yes (Used in Arduino boards)

ATmega328P USART Module and Its Features

The **ATmega328P** microcontroller includes a **Universal Synchronous and Asynchronous Receiver-Transmitter (USART) module**, which enables efficient serial communication. This module is a crucial interface for exchanging data between the microcontroller and other devices such as computers, sensors, or other microcontrollers. The USART module in ATmega328P supports **full-duplex communication**, meaning it can send and receive data simultaneously. It operates with a **baud rate generator**, allowing flexible speed selection for communication. The module is integrated with **hardware buffer registers** to ensure smooth and efficient data transfer without software intervention.

Key features of the ATmega328P USART module include:

- Asynchronous and Synchronous communication support
- Full-duplex operation (separate TX and RX lines)
- Configurable baud rate generator
- Frame format selection (5, 6, 7, 8, or 9 data bits with 1 or 2 stop bits)
- Parity bit support (None, Even, or Odd)
- Double-speed mode (U2X bit) for faster communication
- Multi-processor communication mode
- Hardware-based data transmission and reception buffer
- Interrupt-based communication for efficiency
- Modes of Operation

The ATmega328P USART module operates in the following modes:

1. Asynchronous Normal Mode
    - The most commonly used mode, where data transmission and reception occur without an external clock signal.
    - A dedicated **baud rate generator** determines the speed of communication.
    - It supports data frames of **5 to 9 bits** with selectable parity and stop bits.
2. Asynchronous Double-Speed Mode (U2X = 1)
    - Reduces the number of clock cycles required per bit, effectively **doubling the baud rate**.
    - Provides faster communication while maintaining the same clock frequency.
3. Synchronous Mode
    - Uses an external **clock signal** for communication, making it more reliable in timing-critical applications.
    - Requires a **synchronized transmitter and receiver** to operate correctly.

- Less commonly used in general applications but useful in certain embedded systems.

4. Master SPI Mode
    - The USART can be configured as an **SPI Master** to communicate with SPI-compatible devices.
    - This mode is enabled by setting the **UMSEL bits** to SPI mode and configuring the USART clock.

USART Features Used in Arduino

In the **Arduino environment**, the USART module is primarily used in **asynchronous mode** for serial communication between the microcontroller and a computer via USB. The **Arduino Serial library (`Serial.begin()`, `Serial.print()`, `Serial.read()`, etc.)** provides an abstraction over the USART module, making communication simple. The baud rate is configurable using `Serial.begin(baud_rate)`, commonly set to **9600 bps** by default. The Arduino **Serial Monitor** is used to send and receive data from the microcontroller, useful for debugging and real-time interaction.

By default, **8-bit data frames with no parity and 1 stop bit (8N1 format)** are used in Arduino. The **Double Speed Mode (U2X = 1)** is also enabled in the Arduino core to allow higher baud rates with improved accuracy. However, synchronous communication and multi-processor modes are not typically used in the Arduino framework.

ATmega328P SPI Module and Its Features

The **ATmega328P** microcontroller includes a **Serial Peripheral Interface (SPI) module**, which is a high-speed, full-duplex communication protocol used for connecting peripheral devices such as **SD cards, sensors, displays, and other microcontrollers**. SPI operates using a **master-slave architecture**, where the master device controls the communication and can communicate with multiple slaves by selecting them individually using chip select (**SS**) lines. The SPI module in ATmega328P supports **clock speeds up to one-fourth of the system clock (Fosc/4 by default, configurable to Fosc/2, Fosc/8, Fosc/16, Fosc/32, Fosc/64, or Fosc/128)**, making it much faster than asynchronous serial communication (UART).

Key features of the ATmega328P SPI module include:

- **Full-duplex communication** (simultaneous data transmission and reception).
- **Master and slave mode support**.
- **Four clock speed options** (Fosc/4, Fosc/16, Fosc/64, Fosc/128 in normal mode, and Fosc/2, Fosc/8, Fosc/32, Fosc/64 in double speed mode).
- **Configurable clock polarity (CPOL) and phase (CPHA)** for different SPI modes.

    - **Uses dedicated hardware shift registers for efficient data transfer**.
    - **Supports multiple slaves with chip select (SS) pin management**.
- Interrupt-driven operation for efficient data handling.

The SPI communication relies on four primary signals:

- MOSI (Master Out, Slave In - Pin 11 on Arduino Nano/Uno)
- MISO (Master In, Slave Out - Pin 12 on Arduino Nano/Uno)
- SCK (Serial Clock - Pin 13 on Arduino Nano/Uno)
- SS (Slave Select - Pin 10 on Arduino Nano/Uno, used to enable the slave device)

How SPI is Used in Arduino

In the **Arduino framework**, the SPI module is accessed using the **SPI library (`<SPI.h>`)**, which simplifies the configuration and usage of the SPI bus. The **Arduino Nano/Uno uses digital pins 10-13 for SPI communication**, where pin **10 is typically used as SS (Slave Select) for master mode**.

In most applications, **Arduino acts as the SPI master** and communicates with peripherals like **SD cards, displays (e.g., OLED, TFT, MAX7219 LED matrix), shift registers, and external EEPROMs**. The SPI library provides functions like `SPI.begin()`, `SPI.transfer()`, and `SPI.setClockDivider()` to configure and send data over SPI.

Since SPI is much **faster** than I2C and UART, it is preferred for high-speed data transfer applications like **real-time sensor data acquisition and display interfaces**. However, **unlike I2C, SPI does not support multiple masters**, making it best suited for **single-master, multi-slave** configurations.

ATmega328P I2C Module and Its Implementation in Arduino

The **ATmega328P** microcontroller includes a **Two-Wire Interface (TWI)**, commonly referred to as **I2C (Inter-Integrated Circuit)**, which enables multi-device communication using only two lines: **SDA (Serial Data) and SCL (Serial Clock)**. This module allows communication between the microcontroller and various peripherals such as **sensors (e.g., MPU6050, BMP280), EEPROMs, real-time clocks (RTC), and display modules (e.g., OLED, LCD with I2C backpack)**. I2C supports both **master and slave modes**, allowing a single microcontroller to control multiple devices on the same bus. Each device on the I2C bus has a **unique 7-bit or 10-bit address**, enabling efficient communication without requiring multiple select lines like SPI. The **ATmega328P supports standard (100 kHz), fast (400 kHz), and high-speed (up to 3.4 MHz) I2C communication**.

I2C Module Implementation in Arduino

In **Arduino**, the **I2C module** of the **ATmega328P** is implemented using the **Wire library**, which simplifies communication between the microcontroller and various peripherals such as sensors, memory modules, and displays. The library provides functions to initialize the bus, set device addresses, transmit and receive data, and handle interrupts for slave devices. Arduino uses **7-bit addressing** by default, while the ATmega328P hardware also supports **10-bit addressing**, though this is not commonly used in the Wire library. The standard **I2C clock speed is 100 kHz**, but the library allows operation at **400 kHz (fast mode)** using the `Wire.setClock()` function. However, **high-speed mode (3.4 MHz)**, multi-master arbitration, clock stretching control, and fine-tuned low-power optimizations available in the ATmega328P's TWI module are **not fully utilized in the standard Wire library**. Additionally, while the hardware supports advanced bus error handling, the library does not implement extensive error detection or recovery mechanisms, which can sometimes lead to bus hangs or unresponsive devices. Despite these limitations, the Wire library remains the most commonly used interface for I2C communication on Arduino due to its simplicity and ease of integration.

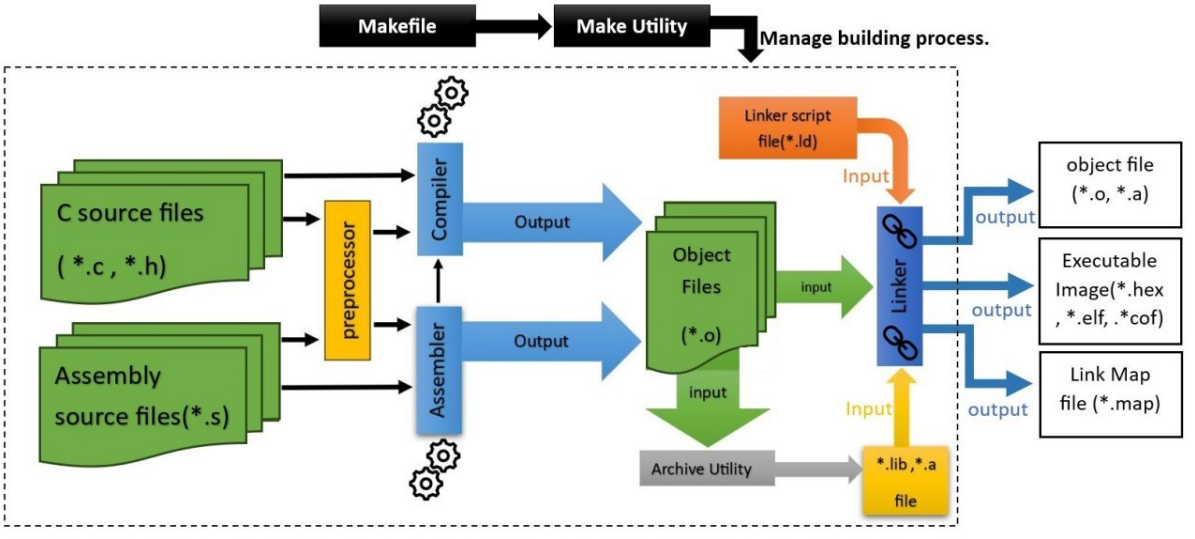
Summary

In general ATMega328 microcontroller is quite powerful and highly configurable to fit requirements of a number of projects. Taping its best powers however requires programming at chip level and understanding its architecture. Arduino IDE has done a great job at simplifying most tasks, but also does not allow user to harness all the powers of the microcontroller. This has been highlighted only to emphasize the importance of knowledge of the underlying architecture, and not relying on easy to use libraries and tools. Always be prepared to accept the challenges and dig deeper.

CHAPTER 4

Arduino IDE and Toolchains

In order to write programs for Arduino board, or more specifically ATMega328 you need a chain of tools, or software. These tools include an editor, where you can write abd edit the program, a compiler that will compile your program into microcontroller understandable code, a Linker that would incorporate already developed libraries into the executable code and produce final application. Then you need a programmer software that would take the compiled executable file and transfer it to the microcontroller over some communication system, like USB or serial port.

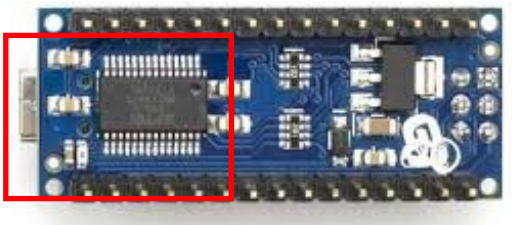


All these tools are practically separate software. Arduino has combined them together in one, and called it a 'Development Environment'. That is why the Arduino software that you install on your PC is called Arduino IDE (Integrated Development Environment). You can download Arduino IDE free from their official website. The installation is simple and straightforward process so I will not go into the details of downloading and installing the IDE. When you install the Arduino IDE it automatically installs the GNU C++ compiler and AVRDude which is the programmer that communicates with Arduino board and transfers the program.

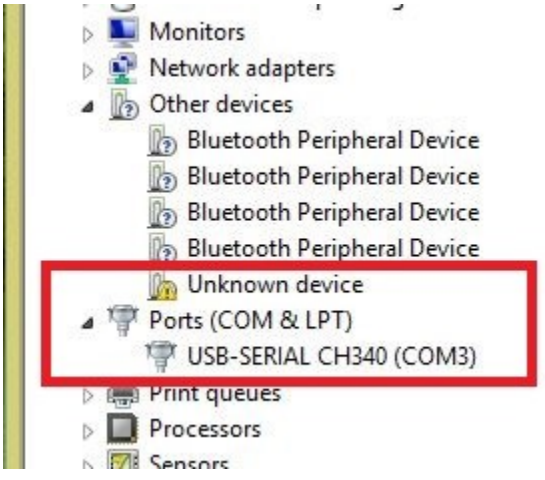
USB To Serial Converter

Truly speaking Arduino board, particularly Arduino nano and UNO both communicate using USART or serial communication. Since present day laptops and most desktop computers now do not have serial ports in them, we need a way to communicate over USB ports.

All Arduino boards have a dedicated chip that performs this function. When connecting to the PC, this chip must be recognized by your PC as a bridge from USB to Serial converter. A number of chips are available that perform this function. Since Arduino boards are open-source many companies are manufacturing them and they are using different USB to Serial converter chips, as per their availability and cost.



The Arduino nano boards usually have CH340 chip. All these chips need a device driver to be installed on your PC in order to be recognized. CH340 is fortunately automatically detected by windows and it has its driver already built into it. I am not sure about Linux or IOS operating systems, but I am sure they also have its driver. Otherwise you got to download and install the device driver.

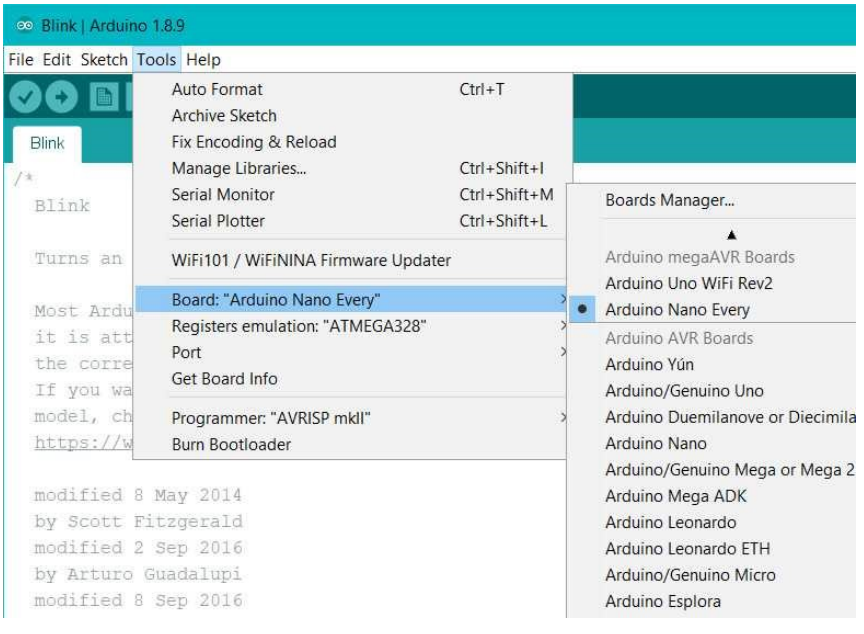


Virtual Serial Port

When you connect the Arduino board to your PC using the USB cable, and driver is successfully installed, the USB to Serial convert chip should appear to your computer as a virtual Serial port like COM4 or something similar. You can check for this virtual port in device manager. This serial port will appear only when the Arduino board is connected to your PC.

Selecting Arduino Board and Serial Port in Arduino IDE

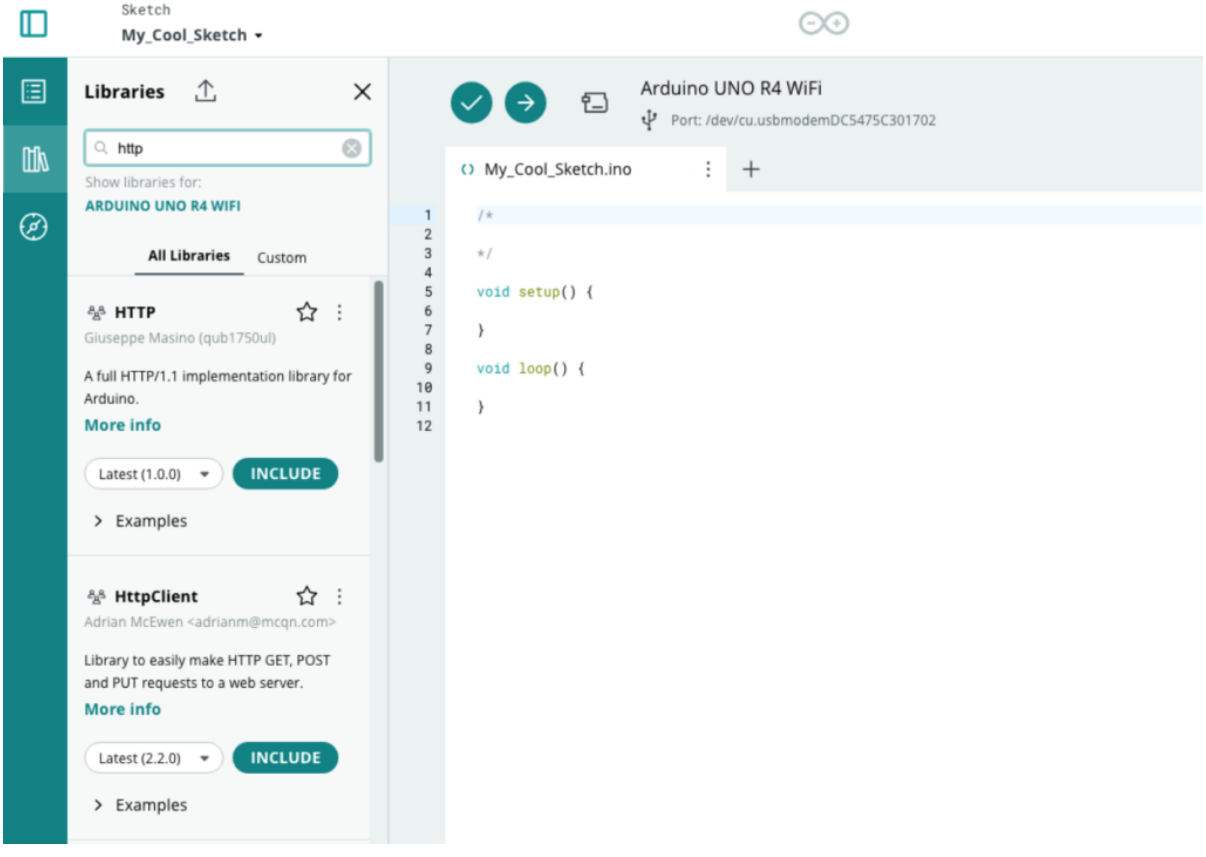
There are a number of boards that are manufactured by Arduino. These boards differ in hardware design and microcontroller. Therefore it is important to tell the Arduino about your Arduino hardware.



Below the Board manager, is the choice of Port. You will have to select the appropriate Arduino port.

Arduino Libraries: Philosophy, Benefits, and Limitations

Arduino libraries are a core part of the **Arduino ecosystem**, designed to simplify programming by providing **pre-written functions** for handling hardware interfaces, communication protocols, and complex algorithms. These libraries follow Arduino's philosophy of **ease of use, abstraction, and portability**, allowing users—especially beginners—to develop applications without deep knowledge of low-level hardware operations. Libraries like **Wire (I2C), SPI, EEPROM, LiquidCrystal (LCD), and Servo** abstract complex register-level operations into simple, intuitive function calls, making embedded programming more accessible. The benefits include **faster development, reusable code, and compatibility across different Arduino boards**, enabling quick prototyping and experimentation. However, these libraries come with limitations, such as **increased memory usage, reduced flexibil-**

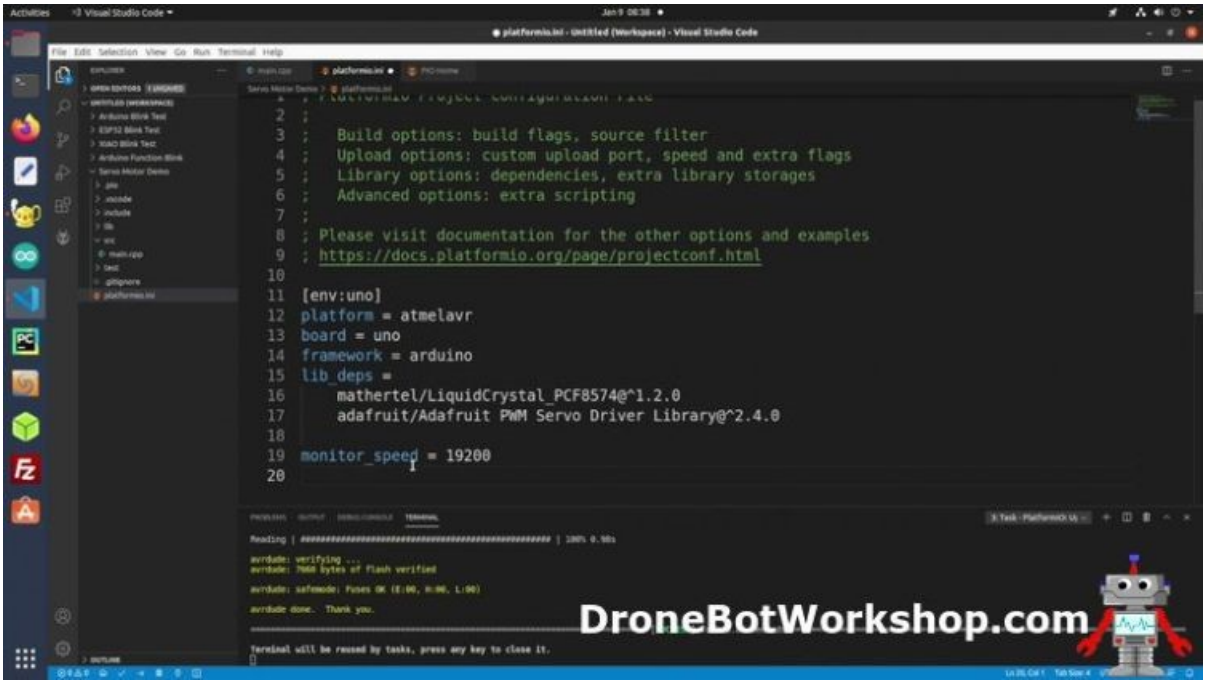


ity, and sometimes inefficient implementations compared to direct register manipulation. Additionally, some libraries may **lack support for advanced hardware features**, limiting their utility in high-performance applications. Despite these shortcomings, Arduino libraries remain **essential tools** for rapid prototyping and educational purposes, making hardware development more approachable for beginners and professionals alike.

Visual Studio Code and PlatformIO

Many professional programmers who are more used to working in VSCode, popular development environment by Microsoft. Visual Studio code is very commonly used in profes-

sional software development specially by web developers. It has a number of editing facilites like intellisense, autocompletion etc. In order to make it more effective number of add-on plugins are available to make it more user friendly and effective. In order to program Arduino, a popular plugin called 'PlatformIO' is available.



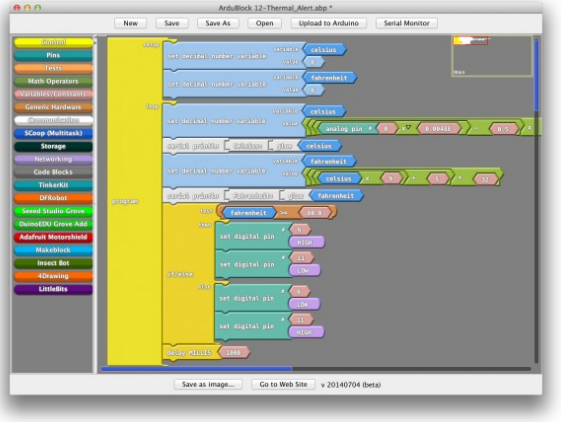
I will not go into the details of this application here, but mention here that many professional developers consider it a better alternative to the Arduino IDE. The interface is however little more technical and you need to configure the things. I would recommend it only for experienced users, for beginners standard Arduino IDE is fine.

Other Development Options

Arduino also provides an online version of its IDE, using it you can make and compile the programs online, without need to install anything on your computer. This option is good where you are working in a shared environment.

ArduBlock -- A Visual Programming Arduino Extension

ArduBlock is a **graphical programming** add-on to the default Arduino IDE. Instead of memorizing cryptic functions, forgetting semicolons, and debugging code, ArduBlock allows you to build your Arduino program by dragging and dropping interlocking blocks.

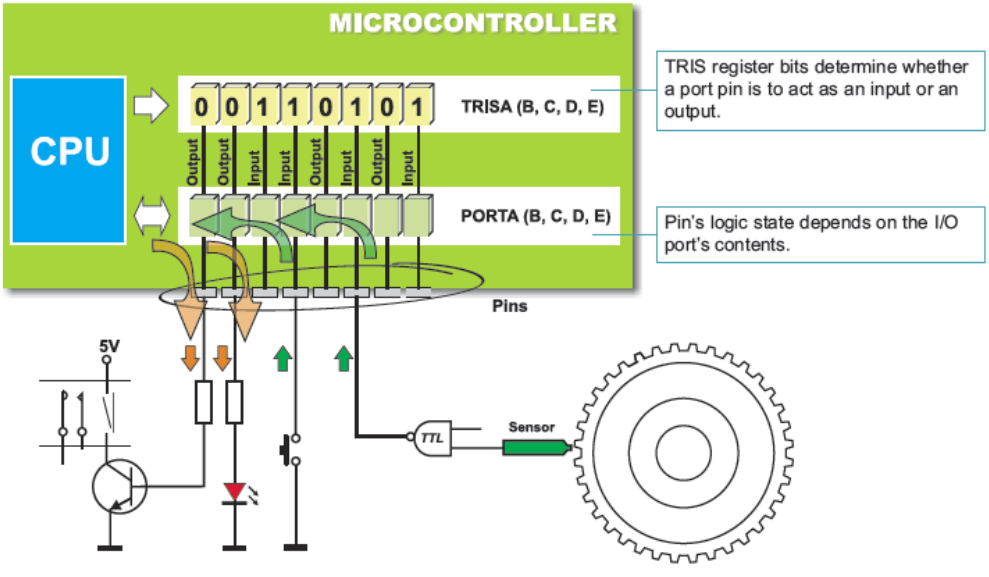


CHAPTER 5

Arduino GPIO

Beginning with GPIO, in microcontroller world is like the 'hello world' in any other programming language. GPIOs are the most basic form of microcontroller devices. As the name suggests these are simple digital pins that can take logical data (High or Low) from external environment or send logical data to external environment. In its most simple form we can set a GPIO pin as HIGH or LOW. When the pin is set to HIGH (binary 1) the pin gets 5V (or VCC) supply on it and is ready to source a small current to the output as well. The Arduino pins can source a maximum of 25mA current, which is enough to light an LED, or to turn a transistor ON. Similarly when a GPIO pin is set to LOW (binary 0) the pin gets connected internally to ground or 0V. The pin then effectively can sink a current of up to 25mA. The Arduino nano has digital pins labelled as D0 to D13. the 'D' is used only to indicate pure digital pins. These pins can perform other functions like serial communication etc., but essentially they can assume only a HIGH or LOW state and all serial communication will take place through same two states. The pins A0 to A5 are commonly used for acquiring analog inputs, but they can also be used as digital pins, labelled as D14 to D19. In your Arduino code you can access the digital pins by referring them as a numeric number. The number can be a constant, or a variable.

Each of these 19 GPIO pins can be individually programmed, and used without affecting each other's state. Before using a pin, we have to set its 'Mode'. The mode of pin can be 'OUTPUT' or 'INPUT'. When set to OUTPUT the program can set the state of pin as



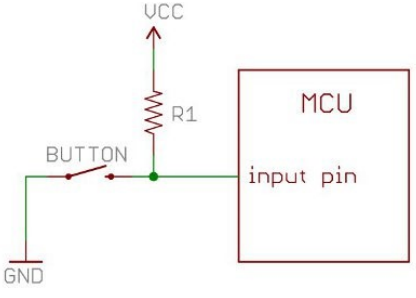
HIGH or LOW, while when set to INPUT the program can read the externally applied signal on the pin. Usually in any given project we know before programming which of the pins would be OUTPUT and which ones will be INPUT. So these settings are set only once during the program startup. Nevertheless if required the mode of pin can be changed from OUT to INPUT and vice versa during program execution, as in case when the same pin is used to send and receive data.

When a pin is set for OUTPUT keep it in mind that it can source or sink only up to 25mA of current. Therefore you cannot drive a heavy load like a motor, or a relay directly from this pin. Similarly when the pin is acting as INPUT the externally applied voltage must not exceed the pin's rated voltage. For Arduino Nano, all its pins are 5V tolerant. So any external signal should not be more than that. Application of higher voltage, like 12V from some external device will damage the microcontroller pin.

> Some boards like ESP32, or STM32 etc. are powered by 3.3V and most of their pins are not 5V tolerant. However some of the pins can accept 5V signals. Always check datasheet of your microcontroller while designing the hardware.

PULL-UP Resistor

When a pin is configured as INPUT, it should always be reading a HIGH or LOW signal. In no circumstances it should be left open. If left open it will read noise from external environment and program may malfunction. If the pin is connected to an external signaling device that ensures a HIGH or LOW state weather its communicating or not, there is no need of a PULL-UP resistor. In other situations, like in push buttons, the pin is left open when button is not pressed. In this situation a pull-up resistor is must.



Here in this picture R1 is a pull-up resistor. The value of R1 is not critical, but a general rule is to use 10K of external resistor. In this configuration when button is 'Not' being pressed the microcontroller will read the pin as 'HIGH', and when the button is pressed, it will read the pin as 'LOW'

Just like Pull-up you can also use a pull-down resistor. Arduino has no objection, but a general usage is to use a pull-up resistor.

Internal PULL-UP Resistor

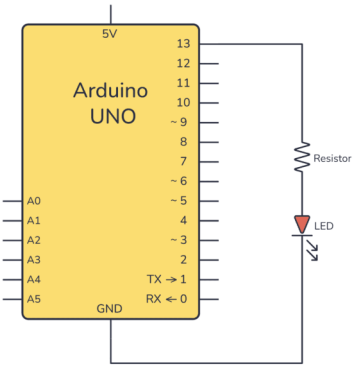
Each Arduino pin has an internal pull-up resistor (about 20K to 50K). So it is unnecessary to have external resistor with Arduino, as you can always enable its internal pull-up resistor.

The internal pull-up resistor is enabled when setting the 'Mode' of pin as INPUT. When setting the pin as INPUT without pull-up the command is, `pinMode(pin, INPUT);` When you want to enable internal pull-up resistor use command, `pinMode(pin, INPUT_PULLUP);`

Driving An LED with Arduino Pin

The simplest project that you can start with is with LEDs. The Arduino pin will source the

current to the LED, so when the pin is HIGH, it will turn ON, and when the pin is LOW it will turn OFF.



LEDs are current driven devices, since they are diode, the PN junction needs at least 1.5V to turn ON. When the LED turns ON its resistance dramatically drops and therefore draw a huge current. This huge rush of current can in fact damage the LED itself and can damage the source as well. It is therefore necessary to limit the current flow through the LED. The current limit has to be according to the requirements of an LED. Different LEDs need different current levels to glow at optimal brightness. The standard 5mm Red LED needs about 20mA of current. In order to limit the current from Arduino pin, we have to insert a resistance in series. The value of resistance depends upon the current limit and supply volts. As per Ohm's law:

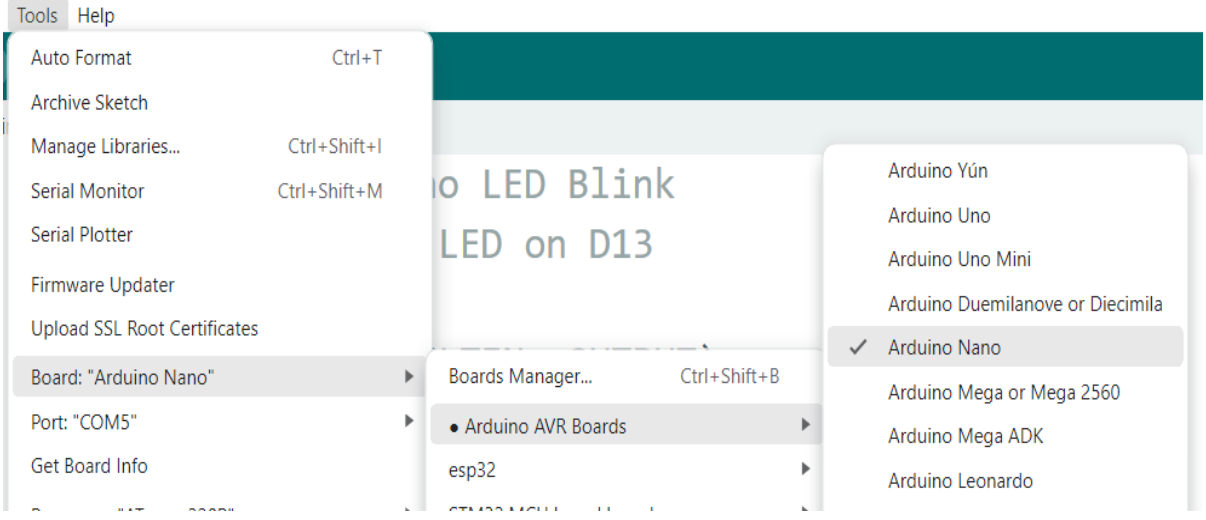
R=V/I, Where V=5V and I=20/1000=0.02A.

```
// Blink LED on D13
void setup() {
    pinMode(LED_BUILTIN, OUTPUT);
  }

  void loop() {
    digitalWrite(LED_BUILTIN, HIGH);
    delay(500);
    digitalWrite(LED_BUILTIN, LOW);
    delay(500);
  }
```

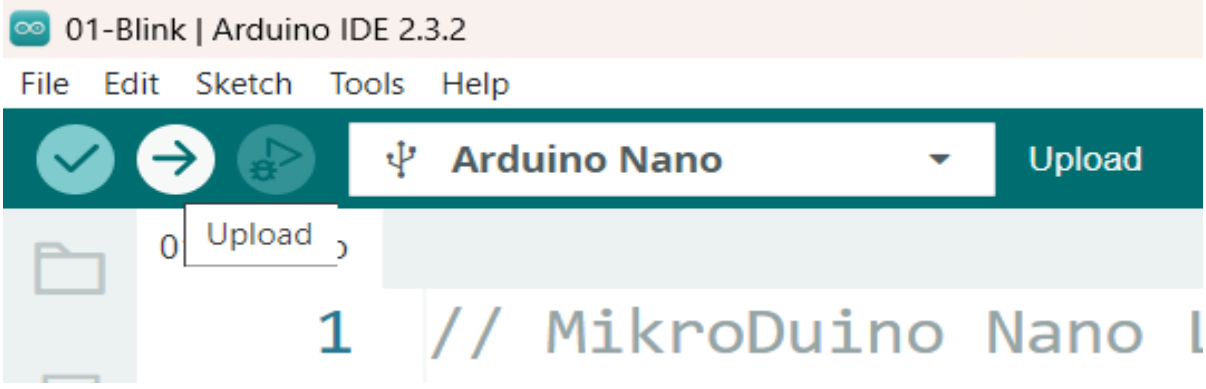
R=5/0.02 = 250 Ohms.

The nearest value that is commercially available is 220 Ohms. So we will use 220 Ohms



resistance in series with the LED to source it from Arduino.

Now Select the board as Arduino Nano from your PC. If using standard Arduino or any other board select appropriate one. Also select the Port, like COM5 or whatever has been created on your system, and you are good to go.



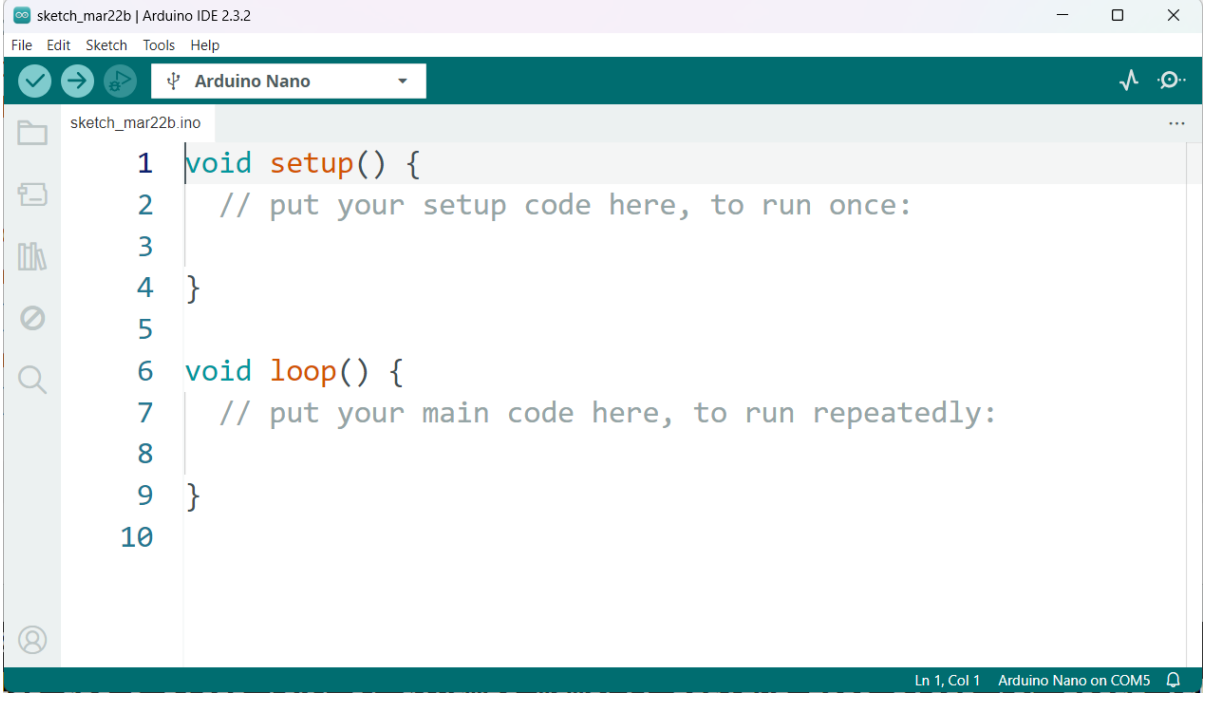
Once you have selected the board and port, you can now compile and program the board.

Notice the two buttons with a check mark and arrow on the upper left corner of screen. The check mark only compiles the program, and reports if there are any errors. The Arrow button, is compile and program or upload the code.

The results of compilation and uploading are shown in the Output window below. If everything is OK, the program starts executing on the board and an Built-in LED on Arduino board starts flashing.

Congratulations, you have just completed the complete cycle of writing a program and uploading to your board.

Now lets have a look at the code. Since this book is mainly about coding the Arduino, we shall concentrate more on that. Unlike the standard C/C++ program (if you already have



experience on it), the Arduino code (Also called sketch in Arduino community) does not contain the main() function. Instead the Arduino IDE by default makes two functions, the setup() and loop().

The void setup() function is where the program first starts execution. This part of code is runs every time the microcontroller starts execution. Once the code in this part is finished executing, the control is handed over to the loop() function. As the name suggests the loop function, keeps on executing the code line by line, endlessly.

Usually we write the various configuration commands like setting up the modes of pins, or initializing some variables etc. in the setup() function.

```
pinMode(LED_BUILTIN, OUTPUT);
```

The pinMode() function is used to set the mode of a particular pin to OUTPUT, INPUT or INPUT_PULLUP. The First argulent of the function is the pin number and second is the mode. The constant LED_BUILTIN is already defined in the Arduino as 13 which is the digital pin (D13) of the Arduino to which an onboard LED is connected. If you want some other pin to set its mode, just use its number here. Remember write only number not the 'D' part. So if you want the led connected on D6 to blink with this code, just replace LED_BUILTIN with 6.

```
digitalWrite(LED_BUILTIN, HIGH);
```

digitalWrite() is the function that actually turns the port pin HIGH or LOW. Again it has two arguments, the first one is the pin number, that you want to affect and second is the state you want. HIGH or LOW. Notice the words high and low are written in upper case. This is the C++ style of writing the constants. HIGH and LOW are constants defined by Arduino compiler as logic words 1 and 0. you can write 1 instead of HIGH and 0 instead of LOW. However writing HIGH and LOW makes the code more easy to read and understandable to you and others if you share the code.

```
delay(500);
```

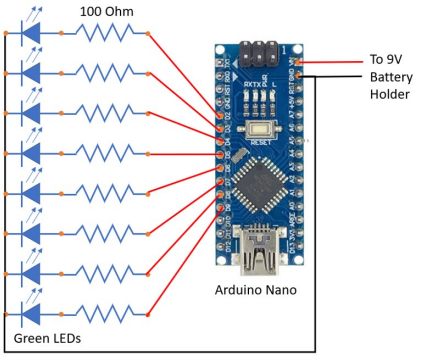
Delay() function takes only one argument, and it's a number. This number indicates time in milliseconds. Thus 500 means 500mS of delay. The program actually halts here for half a second. During this time the previous state of pins remains unchanged. After a delay, or waiting for half a second the program proceeds to next statement, which is again a digitalWrite() and that turns the LED off. Again a wait of 500mS and the loop function, starts over again. This cycle keeps on repeating endlessly so the LED turn ON for half a second and then turns OFF for half a second.

Assignments:

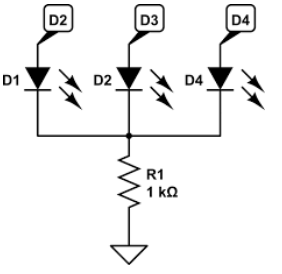
1. Change the Blink cycle to 1 second On and 1 Second OFF
2. Change the Blink Cycle to 1/10 second ON and 1/2 second OFF (=> 100mS and 500mS)
3. Connect an LED to Digital Pin 6 (D6) and repeat the above examples
4. Make two LED D6 and D13 to alternate On and OFF

Controlling More LEDs

When it comes to controlling more LEDs with Arduino, there are many options, that will be discussed in appropriate sections of this book. Here we will contrate on controlling individual LEDs connected to individual GPIO lines.



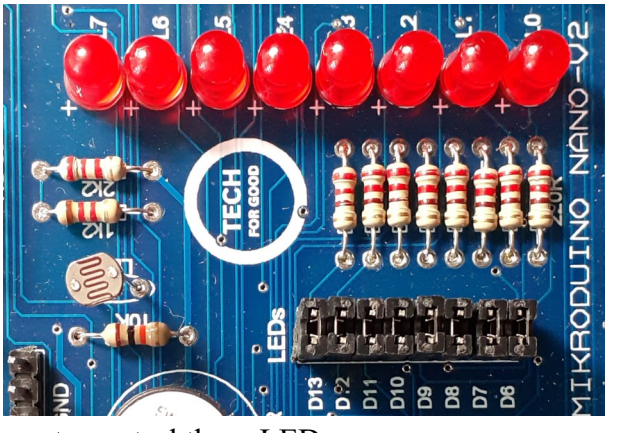
Ideally speaking each LED should have its own current limiting resistor. Secondly when talking absolutely maximum current, the ATMega328 datasheet says each IO pin can source up to 25mA current, while the entire microcontroller current at any given time should not exceed 200mA. Thus when driving individual LEDs its OK, but when multiple LEDs turn ON simultaneously they tend to add the current.



Another approach is to have a single current limiting resistor for all the LEDs. This approach may work, but is not ideal. Since total amount of current passing is limited by 1 resistor, when 1 LED is turned ON all current passes through it, but when multiple LEDs are ON at the same time, the current is distributed among LEDs while total current remains same. This results in dimmed illumination. Thus brightness of LEDs does not remain constant.

Fun With LEDs

Lets have 8 LEDs connected to some digital pins of Microcontroller and experiment as to how to control these LEDs. The objective of these exercises is not to produce LED effects, but to show you various ways of controlling the GPIO lines. The LEDs actually represent an ultimate device that we want to control, like a fan, a heater, a compressor, or a relay etc.



We are going to use the MikroDuino nano board, that has 8 LEDs with individual current limiting resistors. The tabs are connected So LED0 gets connected to digital pin D6, LED1 to D7 and so on. So we have 8 LEDs connected from D6 to D13. Lets explore various programming techniques as to how to control these LEDs.

First task is to set individual pins as OUTPUT in the setup function. One simple method is to write a line of code to individually set the pinMode of individual pin.

```
void setup() {
  pinMode(6, OUTPUT);
  pinMode(7, OUTPUT);
  pinMode(8, OUTPUT);
```

```
  pinMode(9, OUTPUT);
  pinMode(10, OUTPUT);
  pinMode(11, OUTPUT);
  pinMode(12, OUTPUT);
  pinMode(13, OUTPUT);
}
```

The above technique is simplest and most flexible, however it needs more typing and will generate more code as similar lines of code are being executed repeatedly. Since our output pins are from 6 to 13 in a simple ascending order sequence, we can make use of a variable as counter and implement it in a loop.

```
void setup() {
  for(int i=6;i<14;i++){
    pinMode(i, OUTPUT);
  }
 }
```

For loop is a common method in C to implement a counter type loop. Here a local variable i has been declared as type int (integer) and its initial value has been set to 6. the pinMode functions accepts this variable as first argument, thus first time when loop begins the value is 6, and pin 6 gets its mode. The counter is then incremented by 1 (i++) and the pinMode() function is called again, this time I has a value of 7 now, so pin 7 gets its mode set. The loop continues up to 13, when the value of i is incremented to 14 the loop terminates, thus setting pinMode of all the pins from 6 to 13.

Lets write a simple program that should turn all these LEDs from D6 to D13 one by one ON, and then OFF in the same sequence.

```
void loop() {
  // Turning LEds ON one by One
  for(int a=6;a<14;a++){
    digitalWrite(a, HIGH);
    delay(500);
  }
// Turning LEds OFF one by One
  for(int a=6;a<14;a++){
    digitalWrite(a, LOW);
    delay(500);
  }
}
```

In the above example we have omitted the setup() function, I hope you understand that in setup() function all pins from 6 to 13 should be set for OUTPUT. In the loop() section, there are two for loops, the first loop runs from 6 to 13 and during each cycle of loop sets the corresponding pin HIGH, after that another loop runs that turns the corresponding LEDs OFF

and the cycle repeats endlessly.

In this second program, we will write code to make a chasing LED, in which one LED turns ON, stays ON for a while and then turns OFF. Same steps are repeated for all LEDs in sequence.

```
void loop() {
// One LED Chase
  for (int i = 6; i <= 13; i++) {
    digitalWrite(i, HIGH);
    delay(200);
    digitalWrite(i, LOW);
  }
}
```

In our next example we will modify the above code to make a ping-pong effect that is LED chases forward and backwards continuously.

```
// LED Ping Pong Back and Forth
void loop() {
    for (int i = 6; i <= 13; i++) {
    digitalWrite(i, HIGH);
    delay(100);
    digitalWrite(i, LOW);
  }
  for (int i = 12; i > 6; i--) {
    digitalWrite(i, HIGH);
    delay(100);
    digitalWrite(i, LOW);
  }
}
```

Notice in second loop, we are counting back, from 12 to 6, and decreasing the counter variable using i-- statement.

Binary Data in a Byte

When we store a number in a variable, the number is stored as a binary number. Which is sequence of 1s and 0s. Same mathematical number can be represented in a program in a number of formats. Simplest is binary, that uses only two digits or two possible states, 0 and 1. Others are decimal, that we commonly use, that has 10 digits 0 to 9 and hexadecimal that has 16 digits 0 to 9 and A to F. As an example lets represent a number 210 (decimal) in all three formats.

Binary	0b11001000
Decimal	200
Hexadecimal	0xC8

Internally every number is stored in binary format, however in program, we can use any format we like. The prefix 0b is written before the binary number (in program) to indicate that the digits that follow are binary representation. Prefix 0x is used before the hexadecimal numbers. Decimal numbers do not need any prefix.

Reading individual bit from a byte or variable

Some times we need to extract the individual bit setting in a variable. Say for example in a byte sized variable, that has 8 bits, we want to know the value of 5th bit weather it is set to 1 or 0.

Unfortunately C language does not have any direct command to extract this information. What we do is do a bitwise manipulation, or bitwise mathematics to extract this information.

Bitwise And Operation

Bitwise operations are used to manipulate individual bits in a stored variable (or register). A detailed discussion is beyond this text, and you may refer to various sources on internet to read more about them and their uses. Here we will outline only the basic functionality as required in this context.

Bitwise Shift Operations.

There are two bitwise shift operations, called shift right (>>) and shift left (<<). In shift right operation all bits in the variable are shifted 1 position to the right, the right most bit (bit 0) is dropped out. In Shift Left operation opposite happens and all bits are shifted to left by 1 bit and most significant bit is dropped off.

As an example if we have a Binary number: n=0b10011001 a shift right operation would be (n >> 1) the result will be a new number 0b01001100 we can shift any number of bits, like (n>>4) will shift 4 bits not just 1. When we shift the bits, the right most bit is dropped an a new bit is entered in most significant bit, and its value is 0.

Testing a Bit if its 1 or 0

To test a bit in a variable if its set to 1 or 0, we use a bitwise AND operation. First we shift the required bit to 0 bit position and then do an AND operation with a number with 0th bit set to 1.

Letes have a byte containing 0b10011001 number and we want to test if bit no 3 is 1 or 0.

n=0b10011001;

(n>>3) & 1

The second statement will shift the bits to right by 3 positions, making sure that bit no 3 is at position 0. Now we AND it with 1 (0b00000001) so is the n>>3 has 0th bit set to 1 this operation will return 1 otherwise 0.

Using this understanding now lets write a program that would show the bit pattern in a byte on the 8 LEDs we have. Essentially we will have a byte sized variable and put some value in it. Then test each of its bits from 0 to 7th if its 1 or 0 and then set the corresponding LED ON or OFF.

Since our LEDs are beginning from pin 6 we will show the 0th bit on pin 6 and 1st bit on pin 7 and so on.

```
// LED Binary
void loop() {
  int count = 210;
  for (int i = 0; i < 8; i++) {
    digitalWrite(i + 6, (count >> i) & 1);
  }
}
```

The test variable is count that has a value 210. Although the variable int is of two bytes (int in C is 2 byte) the value 210 is within scope of single byte, so it will be stored in lower byte of the count variable. Alternately you could take unsigned char as variable type, that will ensure an 8 bits variable.

```
digitalWrite(i + 6, (count >> i) & 1);
```

This line is the key. (count >> i) shifts the ith bit to 0 position, then its ANDed with 1 and the result is either 1 or 0 that is placed as second argument of digitalWrite function (1 is HIGH and 0 is LOW).

Assignment:

1. Convert the above program to make a Binary counter that should display the entire range from 0 to 255 on LEDs with a delay of 500mS for each number.

*** This Program is Available in Accompanied Examples Folder as : "\Examples\GPIO\06-LED_Binary2"

Random Number

This example will introduce you to a random number generator function in Arduino. The random() function lakes two arguments, minimum number and maximum number and it will generate a random number in between these two limits, We will make an example to generate a random number between 6 and 13 (as our Led pins are from 6 to 13) and then light the LED corresponding to the number generated. The random number generator gener-

```
void setup() {
  for (int i = 6; i < 14; i++) {
    pinMode(i, OUTPUT);
    digitalWrite(i, LOW);
  }
}
// LED Random
void loop() {
  int led = random(6, 14);
  digitalWrite(led, HIGH);
  delay(1000);
  digitalWrite(led, LOW);
}
```

ates a number 1 less than maximum number. That's why in program we have used random (6,14).

Assignment:

1. Write a Program to simulate a rolling Dice. When program is turned on it just sets one led from LED0 to LED5 indicating the dice as its rolled.

Scope of Variables

Strictly speaking this is a topic of C/C++ language, but I feel its important to discuss various programming language features and tips in between the microcontroller related topics. C programming language uses { … } these curly brackets to group various statements. Like the statements that will be part of a function, or part of loop etc. the variables are declared when the variable name is preceded by its data type like int, char, float etc. whenever a variable is declared within the control structure, that is within the body of a loop, or within the {…} brackets, it remains active and accessible only within this part. As soon as program is done with this part of code, the variable is cleared from memory and the space occupied in SRAM is freed to be used by other variables. This has two major effects, first you can use the same variable name, in multiple parts of the program, without one affecting other. Secondly the SRAM is used in an optimistic way.

A variable that is declared outside any of these structures, specially outside the setup() or loop functions, is declared as global or public. This variable remains active throughout the life of program, and is accessible from any part of the program. Usually we use global variables to assign various setup data that we want to be accessible and modifiable across various parts of the program. Beware of using too many global variables, because they keep the SRAM reserved and thus limit the resources available for rest of the program.

Knight Rider Effect (Chasing LEDs)

Earlier in this chapter we have made an example, where a single LED scrolls forward and backward in a loop to demonstrate the effect of swinging pendulum. In 1980's there was a popular TV serial 'Knight Rider' this was a fiction series where an autonomous hi-tech self driving intelligent car was the central point of show. This car had a series of lights in its front that used to scroll left to right and back. Two Lights were On at one time and when the next light turns on then previous one turns off. Here is an example for this.

```
int delayTime = 100; // Global variable
void loop() {
  for (int i = 6; i <= 12; i++) {
    digitalWrite(i, HIGH);
    digitalWrite(i + 1, HIGH);
    delay(delayTime);
    digitalWrite(i, LOW);
  }
  for (int i = 13; i >= 7; i--) {
    digitalWrite(i, HIGH);
```

```
        digitalWrite(i - 1, HIGH);
        delay(delayTime);
        digitalWrite(i, LOW);
      }
    }
```

What if the LEDs are not connected to pins in a linear order !

So far we have connected LEDs to Arduino pins in a linear sequence, that is from D6 to D13. The pin numbers are easily accessible in linear order by adding 1 to the counter or indexing variable. This is usually not possible in practical projects. The LEDs may be displayed physically in an order, but they may be connected to pins in different order. Like you can say LED1 and 2 are connected to pins D6 and D9, while LED3 and 4 are connected to D13 and D14 etc. Then accessing them individually in code makes the code little more complex. Sometimes you have to, as there is no alternative. But sometimes you can organize them in an indexed variable called Array.

Array is a data structure of C language, where a number of variables with same name are created, however they are accessed through their index number, that varies from 0 to the number of elements you have defined.

```
int leds[8]={2,8,5,4,7};
```

This statement creates an array variable named led of type int. It has 8 elements or 8 variables, indexed from 0 to 7. So first variable will be led[0], second will be led[1] and so on. The values in {…} are the initial values of these variables. So led[0] contains 2, led[1] contains 8 and so on.

So in a program we can assign the pin numbers of LEDs to the array elements, and then in program array can be accessed using its index number. As an example I will show you the Setup() function with an array. Since pin definitions will be required to be accessible across the program we will declare the array as global.

```
//global array to hold pin numbers
int led[8]={6,7,8,9,10,11,12,13};
 void setup() {
  for(int i=0;i<8;i++){
    pinMode(led[i], OUTPUT); // accessing pin number throug array
  }
}
```

Now if you have LEDs connected to different pins in different order, just change the numbers in array, and the code remains same.

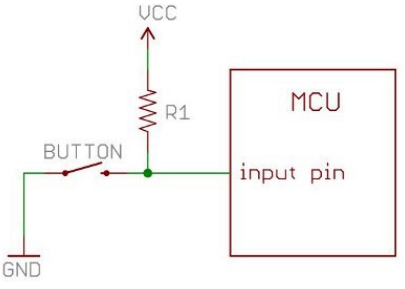
Reading Push Buttons | Digital Read

So far we have explored various ways to use the GPIO system as output. Now lets explore the other part of GPIO, that is reading data from the IO lines. The digital data can come from various sources, these can be user devices like buttons, or some sensors, or it can be

another system sending its data. Digital data may arrive at a single pin, or multiple pins as a combination. Arduino gives a very simple and easy to use function digitaRead() that reads the sate of pin, at a given moment in time. Simple way to get input is to use a push button. Push buttons are mechanical switches that make a contact when pressed. Normally the contact is open, and when pressed contact closes. Commercially you will find buttons in a number of designs, but the function is same, make a contact when pressed. The figure shows a lever switch that is frequently used to detect the limit of some moving object, like door etc. for user input a simple push button is used.



You can simply attach a button to the Arduino pins, and connect other side of the button to GND (Active low configuration) or VCC (Active High configuration). Its up to you which configuration you like.



The Arduino pin should not remain floating. The button when pressed gives GND signal to the pin but when not connected the pin remains open. A pull-up resistor is therefore required. That will provide the 5V when button is not pressed. The R1 resistor can be external as in this case, or we can enable an internal pull-up resistor through code. Therefore in most hobby codes you don't need this external pull-up resistor, however in final projects its better to use the external pullup instead of depending upon the internal one.

```
// MikroDuino Button Read and toggle LED
#define BUTTON_PIN 2  // Button Pin
#define LED_PIN 6     // LED PIN
bool ledState = false;

void setup() {
  pinMode(BUTTON_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
}


void loop() {
  if (digitalRead(BUTTON_PIN) == LOW) {
    ledState = !ledState;
    digitalWrite(LED_PIN, ledState);
    delay(500);  // Allow time to release button
  }
}
```

This program will read the button attached to pin 2 of Arduino and toggle the state of LED attached to digital pin 6. The MikroDuino board has this default setting so just enable the jumpers of LED0 and SW1.

Macros in C

Macros are special compile time identifiers that are commonly used to represent constant types of data. Like in this case BUTTON is a macro and it maps to number 2. When compiling the program the compiler first scans the entire program to see if you have used BUTTON word anywhere, if used it will be replaced (in temporary file) with number 2. then the program will compile. This way it is convenient to change the constant value at one place and the entire code will adapt itself. Secondly since change takes place at compile time, it does not consume SRAM. It is traditional to name the macro names in upper case to differentiate them from variables.

So the program defines two macros, one for BUTTO_PIN and other for LED_PIN.

```
bool ledState = false;
```

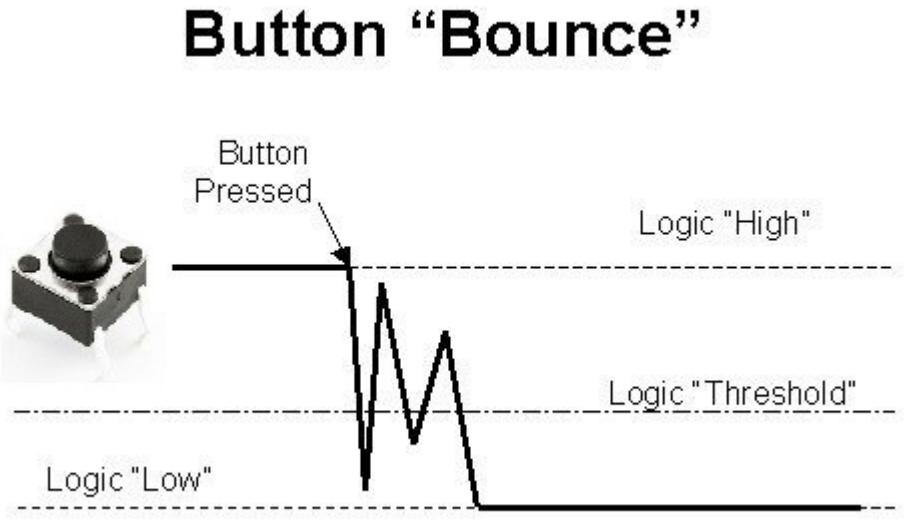
This statement defines a global variable, of type Boolean. It can take only two states, like 1 or 0. Arduino defines these as true and false. This makes program more readable.

```
digitalRead(BUTTON_PIN)
```

This function reads the current state of digitalpin 2 (as defined by macro BUTTON_PIN) if button is pressed this pin will read LOW (Gnd). The result is compared in an if() structure and if button press is detected, the state of Boolean variable is reversed (!ledState) the ! (Not) operation sets true to false and false to true. The ledState is the used to set the status of LED_PIN.

Button Debouncing

Buttons are mechanical devices, when a mechanical contact is made, its not instantaneous, rather there are a series of very short contacts, loss of contacts, like thing, before finally setting for a good contact. You may call it a noise, or we can think of it as a micro 'spark' type thing that happens when a contact is made and current flows through it.

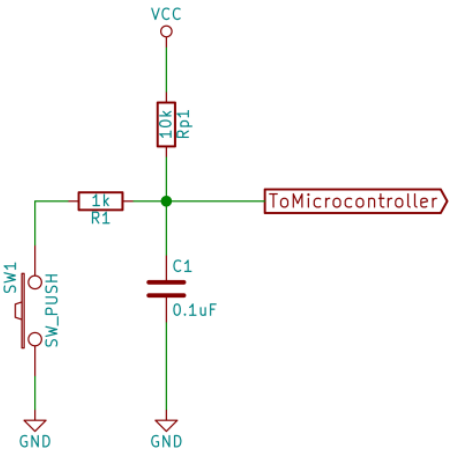


Since microcontroller is reading the pin for a very short period of time, and if its in a rapid loop like thing, it may read the button press and release many times, whereas in fact the in-

tention of user is to press the button only once. This may produce unwanted effects in the application. To counteract this, rapid bouncing of button various techniques are applied. Some of these techniques involve additional hardware, while others use a software approach to counteract this. In this text we discuss both, and then its up to you which one to choose. Most people however choose software techniques, to reduce additional hardware.

Fix it in Hardware

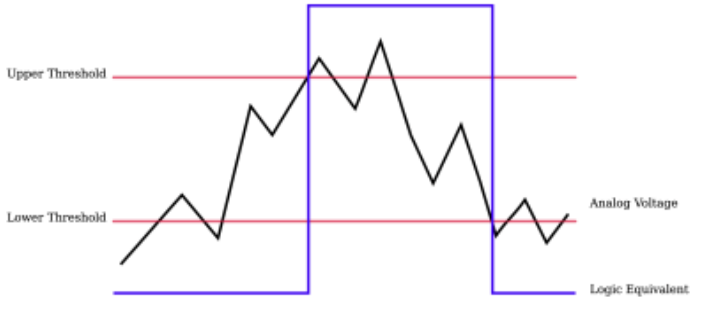
The basic ide of hardware fix, it to implement a low pass filter. That should absorb the redundant noise and let the signal pass through in a clean state. A simple solution is to use an RC Filter. I will not go into the details of design, and calculations, but only mention that using a pull-up resistor and a capacitor can solve the problem to a large extent. Since most microcontrollers have internal pull-up resistor, you can use it and simply use one capacitor.



We need to know the time duration during which the oscillations take place, and we want to smooth out that transition time. Usually this noise period lasts only 1mS in most buttons. Here in this circuit C1 and RP1 are forming the network filter. You choose these values, so that the product R*C (R in Ohms and C in Farads) gives you the time constant for which this filter would be active. For a 10K resistor and 0.1uF capacitor this turns out to be 0.001S of 1mS. The resitor R1 essentially forms a voltage divider when the button is pressed. You may eliminate it, or if want to use it, its value must be significantly lower than pull up resistor, to ensure the voltage produced on button press is close to 0V threshold of the microcontroller. If you eliminate the 1K resistor, button press will discharge the capacitor rapidly and may produce unwanted high frequency noise. While the filter smooths things out fairly well, it doesn't guarantee that you don't get unlucky bounces that wiggle around just as the voltage on the capacitor is reaching 2.5 V. If you pick the product of RC large enough, you'll get away with it most of the time.

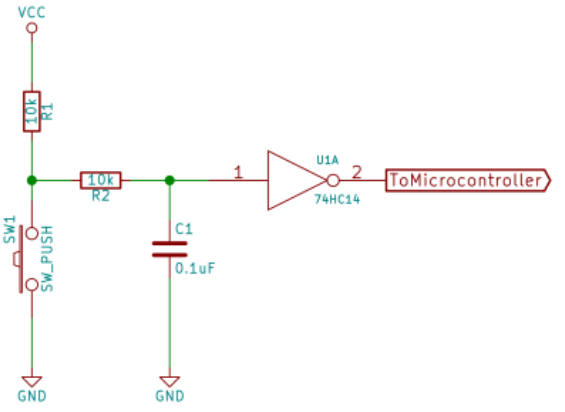
Hysteresis: The Full Fix

Logic chips like the 74HC14 or 40106 will do just that for you. Hysteresis means that instead of one voltage threshold, right in the middle, there are two: one-third and two-thirds of the maximum logic voltage are common. A switch with hysteresis uses the upper threshold when it's last seen the low state, and the lower threshold when it's in the high state. This leaves the range between the two thresholds as a "dead band" where no switching ever takes place, and this solves our problem perfectly.



(Note that thermostats work exactly this way — they trip the heater on at a lower set point

than they turn it off again. If a thermostat didn't work this way, your heater would be oscillating back and forth with every change of fractions of a degree.)



The ideal hardware solution is straightforward: a resistor and a capacitor to smooth the ripples out and a 74HC14 to provide hysteresis. A cheap chip and a handful of passives can take care of up to six buttons with ease, and you don't have to think about it at all on the software side.

In conclusion, fixing contact bounce in hardware is straightforward, at least on this timescale. If you're feeling lucky, you can filter the button's signal with a resistor and capacitor. If you've got really dirty buttons, or you just want to do the job right, an inverter with hysteresis buys a lot of peace of mind and covers six buttons for just few rupees.

Fix it in Software

I'm usually a fan of fixing hardware problems in hardware, rather than software. On the other hand, besides requiring no additional parts, the beauty of the software solution is that it's a once-and-for-all fix: get yourself comfortable with a particular debouncing routine and you can import that one library for the rest of your life. Or maybe you'll want a couple. Anyway, writing the code is an up-front cost that you pay just once. Besides, debouncing routines can be fun. There are a number of algorithms used to fix the debouncing problem. You don't need to learn and master all, just be comfortable, with any one routine, make yourself a library and viola, just use that library for the rest of your life.

Keep the picture of noise spikes in your mind, after all this is what we are trying to cope with. Broadly speaking there are three main types of techniques:

1. The first, and simplest, attempts to wait until after the bouncing has stopped before declaring a button press or release. If the switch is still bouncing after a delay time, it delays again until the switch is stable. I'll call these routines **delay debouncers** for obvious reasons, and that's what I'll work through today.
2. There's also **counter- or integrator-based debouncers**, with or without hysteresis in the switching threshold. These are the equivalent of the `RC` filter above, or the `RC` filter with inverter respectively. We'll not cover these in detail.
3. Finally, what I'd call a **pattern-based debouncer**, that takes the overall pattern of the switch's voltage output over a relatively long time period into account.

Time-Out Debounce

1. A simple approach, is to detect if the button is pressed, and then wait for a definite amount of time during which you are sure that button would have been released. I usually take is as 200mS (my usual button press time). This technique however does not take into consideration if someone is slow enough and keeps the button pressed for a longer period of

```
#define BUTTON_PIN 2
#define LED_PIN 13

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  if (digitalRead(BUTTON_PIN) == LOW) {  // Button Pressed
      delay(50);  // Simple debounce delay
      digitalWrite(LED_PIN, !digitalRead(LED_PIN));  // Toggle LED
      }
}
```

time.

2. The above code is most simple form, it just reads the button, and on very first encounter that it has ben pressed it weights for 50ms (for the bouncing noise to finish) and then assumes that button has settled in its final position. There are two problems with this approach, one it is blocking in nature. The delay function does not let other parts of program like sensors, or displays to be updated, second it does not detect when button is lifted. A continuous press will keep the LED toggle multiple times.

3. One simple addition to it can be a code, that should ensure the program to proceed when

```
void loop() {
  if (digitalRead(BUTTON_PIN) == LOW) {  // Button Pressed
    delay(50);  // Simple debounce delay
    if (digitalRead(BUTTON_PIN) == LOW) {  // Confirm press
      digitalWrite(LED_PIN, !digitalRead(LED_PIN));  // Toggle LED
    }
    while (digitalRead(BUTTON_PIN) == LOW);  // Wait for button re-
lease
  }
}
```

the button is released.

4. The code above is still blocking in nature, but has feature to wait endlessly for the button to be released.

```
5. while (digitalRead(BUTTON_PIN) == LOW);
```

1 The ';' at the end of while loop, has no executable statements and keeps on looping within, till the button is pressed.

2Non Blocking Debounce Using State Variables

3 One efficient method is **state tracking**, where we store the button's previous state and

detect changes between HIGH and LOW. Instead of responding immediately to a press, we introduce a short delay (usually **10–50ms**) to allow the button to stabilize before confirming the press.

4 How It Works

1. **Read the current button state** from the input pin.
2. **Compare it with the previous state** to detect a change.
3. **If the state changes from HIGH to LOW**, introduce a short delay (debounce time).
4. **After the delay, check the button state again**—if it remains LOW, register a valid press.
5. **Store the new state** as the last known state to prevent multiple detections.

```
#define BUTTON_PIN 2
#define LED_PIN 13

bool lastButtonState = HIGH;  // Previous button state

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  bool buttonState = digitalRead(BUTTON_PIN);

  if (buttonState == LOW && lastButtonState == HIGH) {  // Detect press
    delay(50);  // Debounce delay
    if (digitalRead(BUTTON_PIN) == LOW) {  // Confirm press after delay
      digitalWrite(LED_PIN, !digitalRead(LED_PIN));  // Toggle LED
    }
  }
  lastButtonState = buttonState;  // Update button state
}
```

This method does not have a blocking loop to wait for button release and toggle the LED many times. However it still has a small 50ms delay to overcome the debounce period.

Using millis() for non-blocking Debounce

In embedded systems, **blocking code** (such as using delay()) **halts execution** for a set period, making the system **unresponsive** to other inputs or tasks. If we use delay(50), the Arduino will **ignore everything else** during that time, which is inefficient in a real-time system. To solve this, we use **non-blocking** code with millis(), which allows the microcontroller to **continue running other tasks** while keeping track of time for debounce processing.

How the millis() Approach Works

Instead of stopping everything with delay(), we:

Read the button state continuously in the loop.
Detect when the state changes (from HIGH to LOW or vice versa).
Store the time (millis) of this change and wait until a stable period (e.g., 50ms) has passed.
If the button remains in the same state after this time, **we confirm the press** and take action

(e.g., toggle an LED).
The loop keeps running, allowing other tasks to execute simultaneously.

```
#define BUTTON_PIN 2
#define LED_PIN 13

bool buttonState = HIGH;         // Current button state
bool lastButtonState = HIGH;     // Previous button state
bool ledState = LOW;             // LED state
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50; // Debounce time in milliseconds

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  bool reading = digitalRead(BUTTON_PIN); // Read button state

  // If button state changes, reset debounce timer
  if (reading != lastButtonState) {
    lastDebounceTime = millis();
  }

  // Check if debounce time has passed and state is stable
  if ((millis() - lastDebounceTime) > debounceDelay) {
    if (reading == LOW && buttonState == HIGH) { // Detect a press event
      ledState = !ledState;  // Toggle LED state
      digitalWrite(LED_PIN, ledState);
    }
    buttonState = reading;  // Update stable button state
  }

  lastButtonState = reading;  // Update last raw reading
}
```

The non blocking code runs without blocking other commands, and it can update the displays, and read other sensors while monitoring if the button has been pressed. The technique is slightly more complicated but it is scalable and can be used in many situations.

Interrupt-Based Button Handling (Best for Real-Time)

Although we have not discussed interrupts in this text, till now, but just for the completion of topic I have included this technique here. We shall discuss interrupts later in appropriate section. Here I would say only that interrupts are the core of real world programming. An interrupt does not waste time in loops to continuously check the state of a system. Instead the microcontroller's hardware keeps checking the state of system, while other tasks are running. As soon as it detects an event, it triggers a special call, this special call is called interrupt handler. The interrupt handler will address the needful done, and then resume the task it was already doing. A simple example of interrupt is a man working in his office, when suddenly phone bell rings. The phone bell is a trigger that you need to attend. The

man, pauses what he was doing and picks up the phone, answers the call and then hangs back the phone, and resume what he was doing.



Similarly in Arduino the microcontroller does not keep checking for a button to be pressed, specially when we don't know if and when the button will be pressed. So we instruct the microcontroller to keep watching the button pin, and notify us when the pin status is changed.

```
#define BUTTON_PIN 2
#define LED_PIN 13

volatile bool buttonPressed = false;  // Flag for button press
unsigned long lastDebounceTime = 0;   // Stores last valid press time
const unsigned long debounceDelay = 50;  // Debounce time in milliseconds

void handleButtonPress() {
  unsigned long currentTime = millis();

  // Debounce: Accept only if enough time has passed
  if (currentTime - lastDebounceTime > debounceDelay) {
    buttonPressed = true;  // Set flag for processing in loop()
    lastDebounceTime = currentTime;
  }
}

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);

  attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), handleButtonPress, FALL-
ING);
}

void loop() {
  if (buttonPressed) {
    buttonPressed = false;  // Reset flag
    digitalWrite(LED_PIN, !digitalRead(LED_PIN));  // Toggle LED
  }
}
```

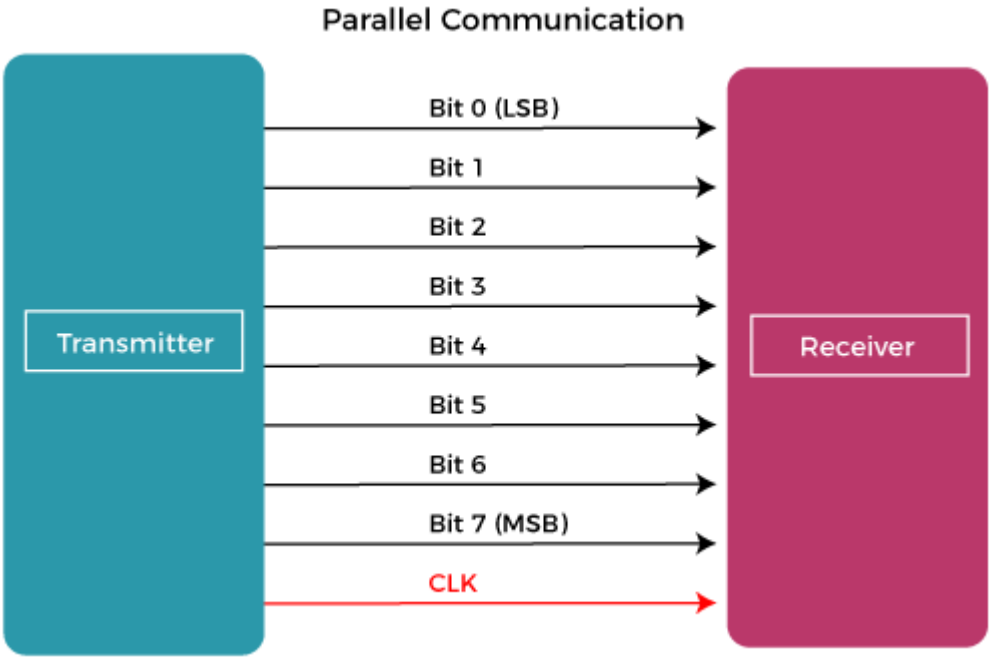
Assignment:

1. Write a program to simulate a dice of 1 to 6 which should select an LED from1 to 6 at random on press of a button.

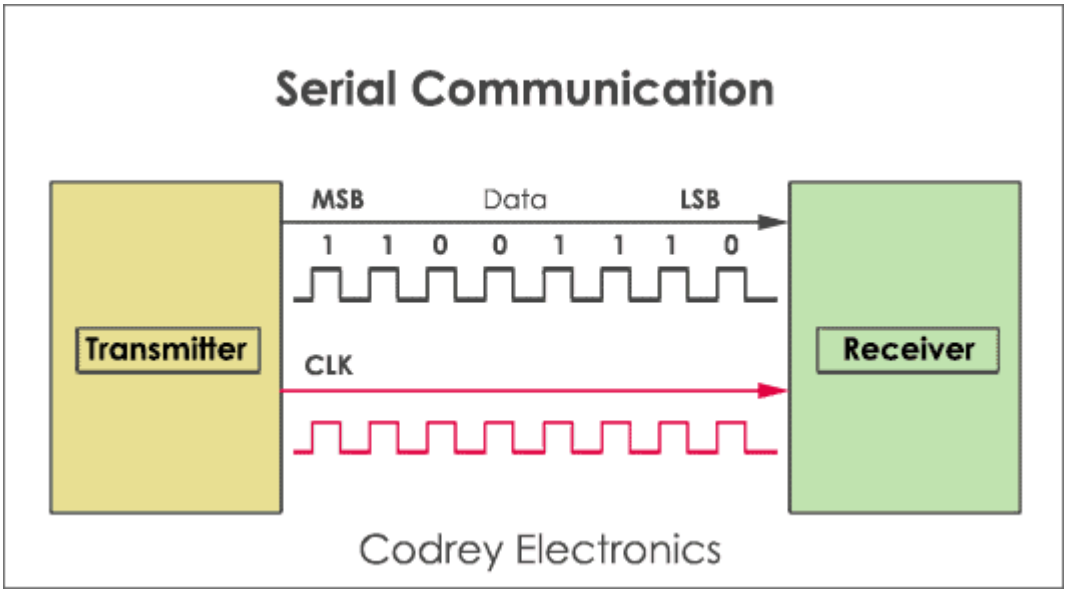
CHAPTER 6

Serial Communication | USART

Communication is one of the most talked topic in embedded systems. Almost every microcontroller application has to have a way to communicate with other devices, that are either part of the same hardware or are separate devices all together. The smallest unit of data is a bit, but microcontrollers and computer systems, group them as a byte and treat byte as a unit. Thus all internal working of a microcontroller is byte based. A byte consists of 8 bits, and therefore when communicating with another device we need to exchange 8 bits of data. In simple terms this would need 8 digital IO lines between sender and receiver. An additional clock line is required to synchronize data communication. The transmission of data will be almost instantaneous as entire byte is transferred in a single clock pulse. The problem with this method however is that it would require a large number of PCB tracks from one chip to another, and if more than one chips are involved the routing will become even more complex. The complexity of routing makes PCB size larger, as well as chances of introduction of external noise in any one track may affect the reliability of data exchange. This becomes more important when it comes to communication beyond the same hardware, to an external device, like PC, or another control system. The cable will contain a large number of wires, each likely to catch the environmental noise and distort the signal. A single misread-



ing of 1 or 0 will distort the message. The alternate is to keep the 8 bit version inside the microcontroller and transfer the data using a single sire, in a series of bits, one by one.

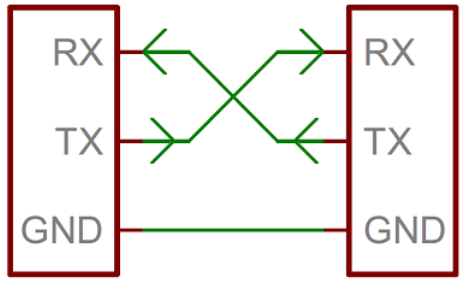


This will also require a clock line to synchronize data between transmitter and receiver. Now we have to take care of only two lines, its easier to route and even shield when transmitting over longer distance.

There are many protocols as to how the data will be encoded, over the lines. One of the most commonly used is USART or its simplified version UART. The USART protocol requires a clock line for synchronization, whereas the UART does not require a synchronizing clock signal. Instead it depends upon the commonly agreed speed of data transmission between sender and receiver system.

Wiring and Hardware

A serial bus consists of just two wires - one for sending data and another for receiving. As such, serial devices should have two serial pins: the receiver, **RX**, and the transmitter, **TX**. And a common GND of course.



It is important to note that Rx of one device should be connected to the Tx of other and vice versa. When communicating there can be one way traffic, or two way traffic. The data transmission is define as Half-Duplex or Full-Duplex. In half-duplex the communication is either sending or receiving at any given moment. While in Full-duplex simultaneous sending and receiving of data is practiced.

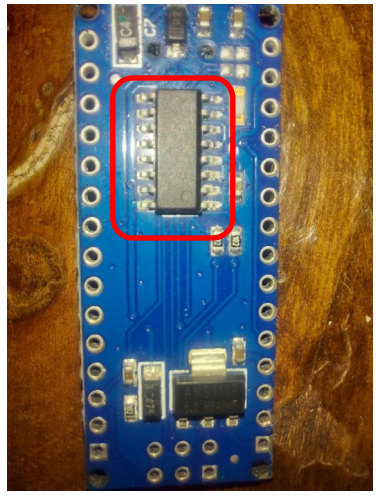
The Arduino Serial Communication System

Arduino nano, has a hardware module inside it called UART module. The Rx and Tx lines of this module are shared with the GPIO pins D0 and D1. We will discuss about this module and its features later in this chapter. To start with you just need to know you can connect any other UART compliant serial device to these two lines.

The UART module expects data in a particular format. This defined by the internationally accepted UART frame protocol. Each frame consists of one byte of data. The start of frame is indicated by a special 'Start bit' followed by data bits and then a parity bit followed by 'stop bit'. The entire sequence is repeated for next byte.

USB to UART Conversion

Most PCs now have only USB ports for communication. USB operates on a different protocol, which is quite complex and contains number of codes for the types of communication devices, like keyboard, mouse, mass storage devices etc. All these are connected to same USB port but have different communication protocols. The USB cannot communicate with the Arduino UARD system directly. In order to connect these two different systems, we need an intermediate translator or adapter that should convert USB type data to UART type and UART type to USB type. The Arduino board has a dedicated chip for that. When connected to the USB of your laptop, it creates a virtual UART port in your PC. This port is named as COM1, COM2 or any other similar. Every computer can assign this number differently. You have already done so, when setting up the Arduino environment. The output of this chips, Tx and Rx pins are connected to the Arduino's Rx and Tx pins. When transferring a new program from Arduino IDE the same chip is used and it transmits data to the Rx and Tx lines of Arduino.



If you connect any other external device to these pins, may be an LED, or even another UART compliant device, it may interfere with the data transmitted by the USB to Serial conversion chip while programming, and programming might fail. In that case just unplug the additional circuitry from these lines while programming. Once programmed you may connect the external circuit.

UART Settings

Reliable UART communication depends upon the appropriate settings on both sides. The transmitter and receiver must agree on the same settings in order to ensure message is clearly reached.

Baud Rate

Baud Rate defines the speed of communication (bits per second). Common baud rates are: 9600, 19200, 38400, 57600, 115200. Note the generation of baud rate depends upon the performance of your microcontroller, its timers, and the clock speed. Not all baud rates are generated exactly as intended. This produces some error. Both sender and receiver must use the same baud rate for communication.

Data Bits

This defines the number of bits that will form one byte. In early days when only text was transmitted 7 bits of data were used. Now however data may consist of binary information, so an entire 8 bits are used.

Stop Bits

Stop bits indicate the end of transmission. Common standards are 1, 1.5 and 2 stop bits (1 stop bit is common).

Parity Bit

Parity bit is used to detect error during communication. Option are none, Even parity or Odd parity. The parity bit represents the number of 1s in data.

Flow Control

This prevents data loss by managing flow of data between sender and receiver. It nan be None (most commonly used), Software (XON/XOFF) or Hardware (RTS/CTS or DTR/DSR)

Synchronization

This setting can be asynchronous or synchronous. In synchronous setting, an additional clock signal is required to synchronize. In asynchronous the communication reliability depends upon baud rate and data frame. We use asynchronous most of the times.

Serial Communication in Arduino

Arduino offers two types of serial communication. One uses the dedicated hardware module of the Arduino, and other use software techniques to implement it. Hardware module UART is better and used most of the times. The communication takes place through dedicated Tx and Rx pins. Arduino IDE offers a built-in object called 'Serial'. You don't need to define this object.

Setting baud rate in Arduino is all that is required to start communication.

Set using `Serial.begin(baudRate);`

Example: `Serial.begin(9600);` (sets baud rate to 9600 bps).

Data Bits, Parity and Stop Bits

These are set optionally as a second parameter of the Serial.begin() function.

Configured using `Serial.begin(baudRate, configuration);`

Default setting: 8 data bits, no parity, 1 stop bit (8N1).

After you initialize the Serial object, make sure the receiver is also configured to use the same settings.

```
Serial.begin(115200, SERIAL_8N1); // 8 data bits, no parity, 1 stop bit
Serial.begin(9600, SERIAL_8E1);  // 8 data bits, even parity, 1 stop bit
Serial.begin(19200, SERIAL_7O2); // 7 data bits, odd parity, 2 stop bits
```

Sending and Receiving Data

After you have initialized the Serial object data can be sent and received using Aerial ob-

jects, methods to do that.

Sending Data

Receiving Data

While receiving you first check if there is a data present in the receive buffer or not. Since it

```
Serial.print("Hello");  // Sends "Hello" as ASCII characters
Serial.println(123);    // Sends "123" followed by a newline
Serial.write(65);       // Sends binary value 65 (ASCII 'A')
```

is not known when the sender will send data, in its absence the receive buffer is empty. The

```
if (Serial.available()) { // Check if data is received
    char data = Serial.read(); // Read a single byte
}
```

UART buffer in ATMega328 is 64 bytes. There are separate buffers for Transmitter and Receiver parts. *You can change this setting in 'hardwareSerial.h' file in Arduino installation if required.*

How to check the Available Data in Buffer

```
int availableBytes = Serial.available();
```

This function returns the number of bytes present in the buffer. When you read one byte, that byte is removed from buffer.

If we want to ensure that all data in Tx buffer is sent and its clear, before sending further data:

```
Serial.flush(); // Waits for outgoing data to be transmitted
```

Arduino does not support hardware flow control, and software flow control must be implemented if required.

What if we need two Serial Ports.

The ATmega328 has only one hardware serial module, and therefore only one set of Tx and Rx pins. In case we need another port in Arduino, we can use software library to implement an additional serial communication port. The software library can use any digital GPIO lines to act as TX or Rx pins. The software serial communication however generates more code, and consumes more program memory.

The above code shows how to implement the software Serial library. We have used pin 10 and 11 for Rx and Tx, you can use any two digital pins.

```
#include <SoftwareSerial.h>
SoftwareSerial mySerial(10, 11); // RX, TX on pins 10 and 11
mySerial.begin(9600);
mySerial.println("Hello from SoftwareSerial");
```

Example Programs to Demonstrate Serial Communication

Basic Serial Communication (Hello World)

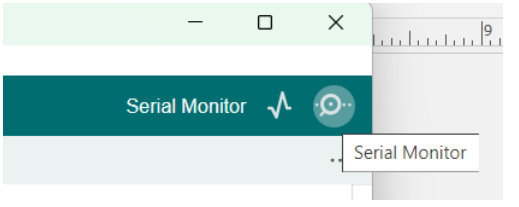
This program sends the message “Hello Arduino!” to the serial monitor every second. Each message is on a new line.

To view this data you must turn on your serial monitor. This is built-in available within Ar-

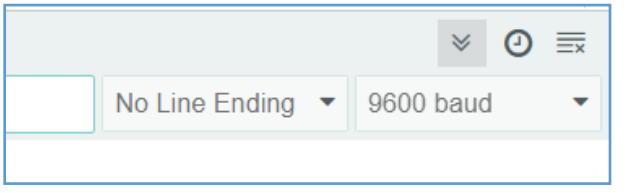
```
void setup() {
    Serial.begin(9600); // Start serial communication at 9600 baud
}

void loop() {
    Serial.println("Hello, Arduino!"); // Send data to Serial Monitor
    delay(1000); // Wait for 1 second
}
```

duino IDE. Its available as a search glass icon on the upper right corner of the Arduino IDE. Once monitor is turned on, you must select the appropriate baud rate and other settings if required.



Expected output in Serial Monitor



Sending and Receiving Data from

```
Hello, Arduino!

Hello, Arduino!

...
```

Serial Monitor

Now lets write a program, that reads data from serial device and also writes data back to the serial device.

```
void loop() {
    if (Serial.available()) { // Check if data is available
        char receivedChar = Serial.read(); // Read the received character
        Serial.print("You sent: ");
        Serial.println(receivedChar); // Echo back the received character
    }
}
```

We have omitted the setup() where you will initialize the Serial port using Serial.begin() command. Now open the Serial monitor and type any character and press enter. The Ar-

duino will respond by sending the same character back.

Assignment:

1. Change the above program to return back the upper case letter. Like if you type ‘h’ it should return ‘H’

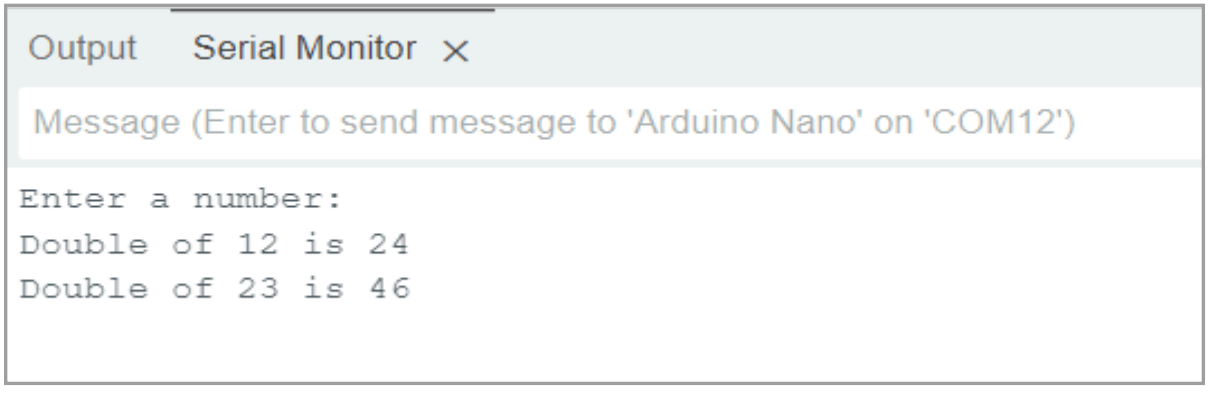
Sending a Number from Serial Monitor and Performing an Operation

This next program, reads the numeric data from the serial monitor, and converts the , text based data into integer values. Serial monitor sends data one character at a time, like if you send 106, it will actually be sent as ‘1’, ‘0’, ‘6’. The numbers 1,0 and 6 will be transmitted as ASCII characters, in order to make it an integer value, you have to do some math. Fortu-

```
// Read a number from serial and convert it to integer
void setup() {
    Serial.begin(9600);
    Serial.println("Enter a number:");
}

void loop() {
    if (Serial.available()) {
        int num = Serial.parseInt(); // Read an integer
        Serial.print("Double of ");
        Serial.print(num);
        Serial.print(" is ");
        Serial.println(num * 2);
    }
}
```

nately Serial object, gives us a function parseInt() that reads the numeric value from serial device and converts it into an integer value.



The Result is:

Controlling an LED via Serial Commands

This program turns an LED on or off using Serial Monitor commands (`ON` or `OFF`). This

program demonstrates another useful command with serial object, readString(). This function reads the entire line of data and converts it into a string type data.

Assignment:

1. Read a value from 2 to 16 from Serial monitor and return a times table (1 to 10) for the given Number.

Number Guessing Game

In this program, we make a simple game, in which Arduino thinks about a number between 1 to 100 and you are supposed to guess it. The game uses UART communication and the terminal. This game demonstrates the parsing of integer data user types in terminal, and then using the integer data in program. The program also demonstrates the use of functions

```
// Number Guessing Game
int targetNumber;  // The number to guess
bool gameActive = false; // Track game state

void setup() {
    Serial.begin(9600);  // Initialize Serial Communication
    Serial.println("Welcome to the Number Guessing Game!");
    Serial.println("Type 'start' to begin.");
}

void loop() {
    if (Serial.available()) {
        String input = Serial.readString();  // Read user input
        input.trim();  // Remove any newline characters

        if (input == "start") {
            startGame();
        }
        else if (gameActive) {
            int guess = input.toInt();  // Convert input to integer

            if (guess >= 1 && guess <= 100) {
                checkGuess(guess);
            } else {
                Serial.println("Invalid input! Enter a number between 1 and
100.");
            }
        }
        else {
            Serial.println("Type 'start' to begin a new game.");
        }
    }
}

// Function to start a new game
void startGame() {
    targetNumber = random(1, 101);  // Pick a random number between 1 and 100
    gameActive = true;
    Serial.println("Game started! Guess a number between 1 and 100.");
```

```
}

// Function to check the user's guess
void checkGuess(int guess) {
    if (guess > targetNumber) {
        Serial.println("Too High! Try again.");
    }
    else if (guess < targetNumber) {
        Serial.println("Too Low! Try again.");
    }
    else {
        Serial.println("?? Congratulations! You guessed the correct num-
ber! ??");
        Serial.println("Type 'start' to play again.");
        gameActive = false;  // End the game
    }
}
```

to keep the program organized.

```
// Number Guessing Game 2
int low = 1, high = 100, guess;
bool gameActive = false;

void setup() {
    Serial.begin(9600);  // Initialize Serial Communication
    Serial.println("Think of a number between 1 and 100.");
    Serial.println("Type 'start' when you're ready.");
}

void loop() {
    if (Serial.available()) {
        String input = Serial.readString();  // Read user input
        input.trim();  // Remove extra spaces and newline characters

        if (input == "start") {
            startGame();
        }
        else if (gameActive) {
            if (input == "H") {
                high = guess - 1;
                makeGuess();
            }
            else if (input == "L") {
                low = guess + 1;
                makeGuess();
            }
            else if (input == "C") {
                Serial.println("?? Yay! I guessed your number! ??");
                Serial.println("Type 'start' to play again.");
                gameActive = false;
            }
            else {
```

```
                Serial.println("Invalid input! Reply with 'H' for too high, 'L'
for too low, or 'C' for correct.");
            }
        }
        else {
            Serial.println("Type 'start' to begin the game.");
        }
    }
}

// Start a new game
void startGame() {
    low = 1;
    high = 100;
    gameActive = true;
    makeGuess();
}

// Make a guess using binary search
void makeGuess() {
    if (low <= high) {
        guess = (low + high) / 2;
        Serial.print("Is your number ");
        Serial.print(guess);
        Serial.println("? (Reply with 'H' for too high, 'L' for too low, 'C' for
correct)");
    } else {
        Serial.println("Something went wrong! Let's start over. Type 'start' to
try again.");
        gameActive = false;
    }
}
```

Think of a Number and Arduino Guesses it.

This is reverse of the above game, in which you think of a number and Arduino guesses it.

Simple Math Calculator

Here is another program that accepts an expression from user like 5+3 and evaluates it to calculate the sum. The example demonstrates how to parse an expression and separate the

```
void setup() {
    Serial.begin(9600); // Initialize Serial Communication
    Serial.println("Simple Calculator");
    Serial.println("Enter an expression (e.g., 5 + 3) and press Enter:");
}

void loop() {
    if (Serial.available()) {
        String input = Serial.readString(); // Read user input
        input.trim(); // Remove spaces and newlines

        int num1, num2;
        char operation;
```

```
        // Scan input for a number, an operator, and another number
        if (sscanf(input.c_str(), "%d %c %d", &num1, &operation, &num2) == 3) {
            float result;
            bool validOperation = true;

            // Perform the calculation
            switch (operation) {
                case '+': result = num1 + num2; break;
                case '-': result = num1 - num2; break;
                case '*': result = num1 * num2; break;
                case '/':
                    if (num2 != 0) result = (float)num1 / num2;
                    else {
                        Serial.println("Error: Division by zero!");
                        validOperation = false;
                    }
                    break;
                default:
                    Serial.println("Error: Invalid operator! Use +, -, *, /");
                    validOperation = false;
            }

            // Print the result
            if (validOperation) {
                Serial.print("Result: ");
                Serial.println(result);
            }
        }
        else {
            Serial.println("Invalid input! Use format: number operator number
(e.g., 5 + 3)");
        }

        Serial.println("Enter another expression:");
    }
}
```

two integers, and the operand from expression. Then undergo mathematical evaluation and give the result.

The key function here is sscanf() that takes the input string as an argument, and number of parameters to be separated as arguments. The & before arguments indicates the address of variable, so that function can write values directly into the variable.

Industrial Communication protocols for UART

So far we have covered the basics of Serial communication and then we have seen how to implement it in Arduino. Now lets dive a little deeper to see how all this can be applied in industrial or even in a practical environment.

TTL Communication

So far we are communicating with the standard digital logic levels that is 5V and 0V as log-

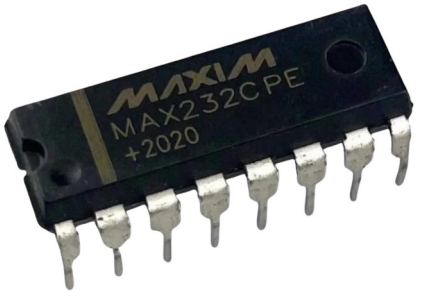
ic 1 and logic 0 respectively. Some other microcontrollers will use 3.3V as logic 1. this is alright for communication within the same hardware or for short distances. When the distance increases, the ling wires tend to increase the resistance resulting in loss of signal strength as well as tend to act like ‘antennas’ and pick up the noise from surroundings. The result is poor communication and unreliable transmission.

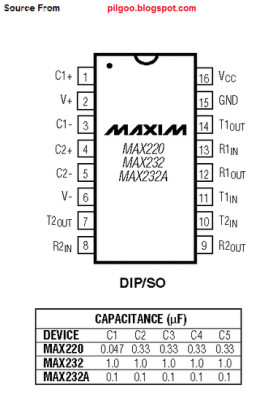
RS232 standard for data transmission

To counteract this limitation a new standard of data transmission wa introduced, named as RS232. This is strictly speaking a transmission protocol, and not a data protocol. The message is encoded the same way as for TTL level of transmission, but definitions for 1 and 0 is changed. In RS232 logic 1 is -15V and logic 0 is +15V. This is in sharp contrast to the TTL logic. The broad gap in voltage levels ensures noise immunity as well as increased distance. The maximum distance RS232 can cover depends on the baud rate (speed of communication). As the baud rate increases, the distance decreases due to signal degradation and electromagnetic interference (EMI). The maximum distance is 150m at 9600 baud rate.

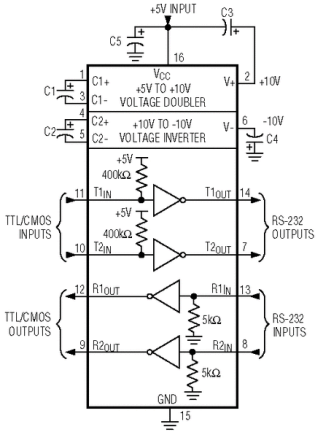
TTL to RS232 Converters

Since microcontrollers and processors need the standard TTL signals, you need a two way translator. MAX232 is a hardware chip that is frequently used to translate the signals be-





CAPACITANCE (µF)					
DEVICE	C1	C2	C3	C4	C5
MAX220	0.047	0.33	0.33	0.33	0.33
MAX232	1.0	1.0	1.0	1.0	1.0
MAX232A	0.1	0.1	0.1	0.1	0.1



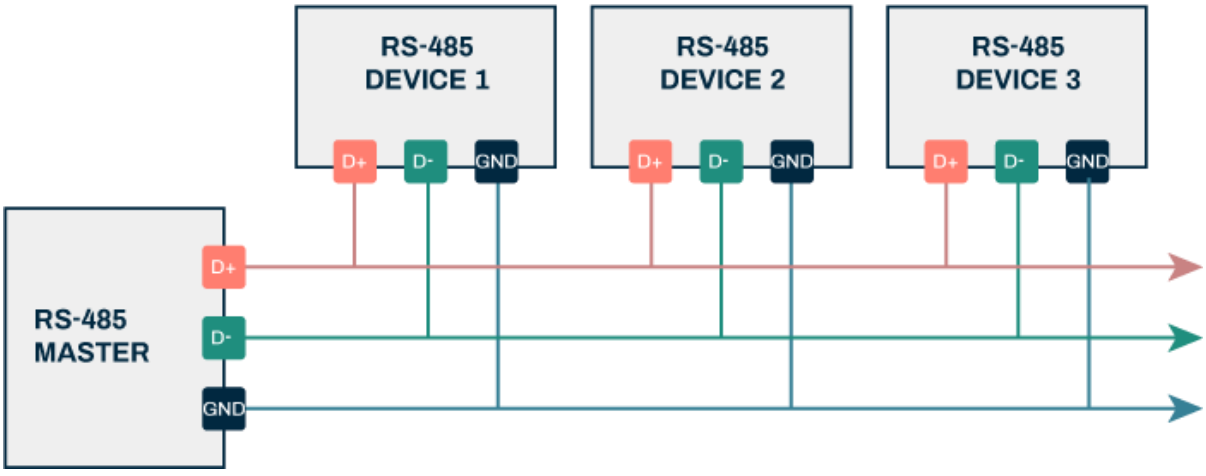
tween TTL and RS232 levels.

RS232 is an old protocol, and has been used in industry. It has been largely replaced by other transmission protocols. Some industries still use this standard. Moreover RS232 is point to point communication system, where only two devices can communicate with each other.

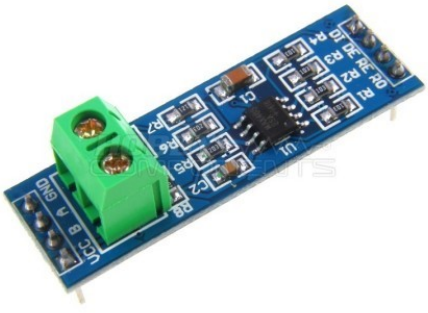
RS485 Protocol

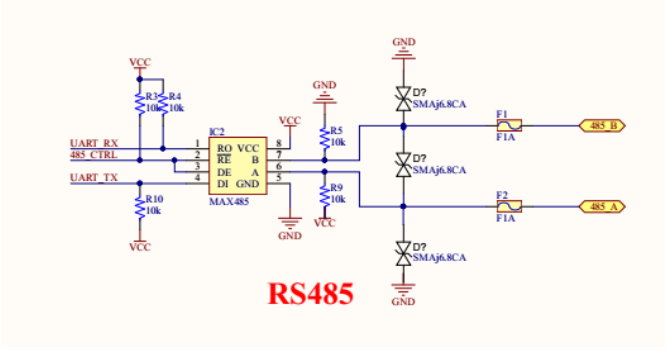
RS485 (Recommended Standard 485) is an enhanced version of RS232, designed for long-distance, high-speed, and multi-device communication. It is widely used in industrial automation, embedded systems, and building control systems. RS485 is a multipoint communication system, where multiple devices can be connected on the same line. It supports a speed up to 10MBits pers second and distance up to 1.2Km. The voltage levels are +1.5V and -1.5V. It uses two twisted wires for each line. For two way communication you need a cable with 4 wires.

The two wires are called A and B and it uses differential voltage levels during transmission. This differential gives a very high noise immunity. An addition of GND shield to the cable



adds further noise immunity. 120 ohms terminal resistors are needed to reduce signal reflections.





MAX485 is an easily available chip that does the job. You can give it TTL logic and it will convert the data to RS485 standards. You use one module for half duplex, and two modules for full-duplex communication.

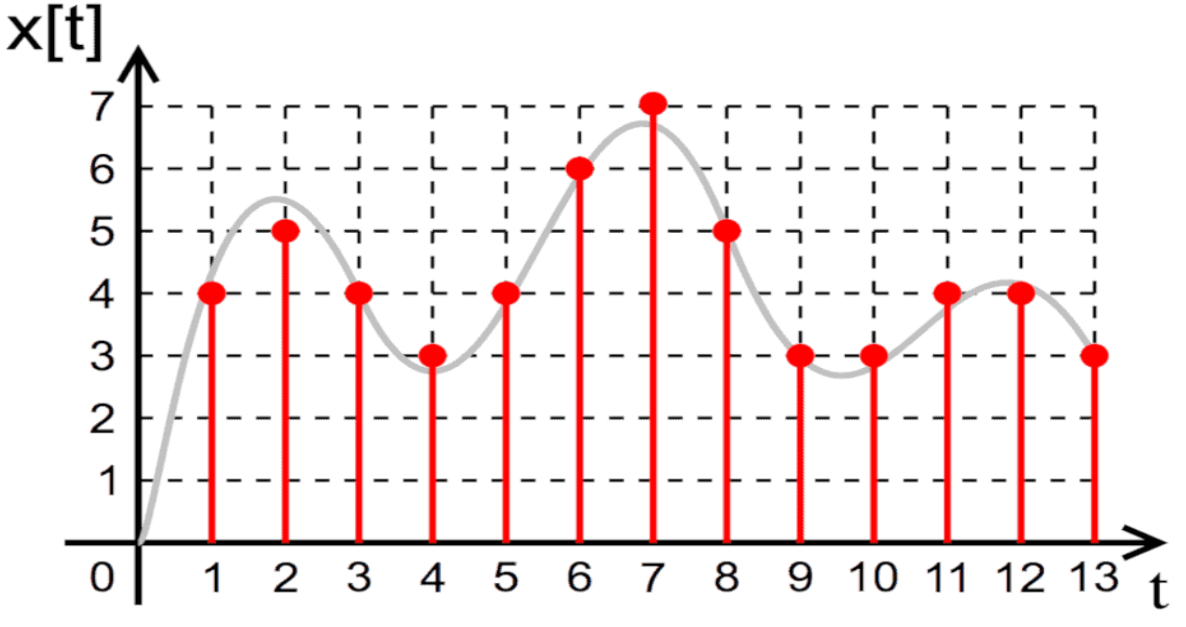
CHAPTER 7

Analog to Digital Conversion

We live in an analog world, but process this information in digital domain. The analog data is abundant in nature. Almost all physical properties of matter and event quantum mechanics are actually analog values. Though there are certain matters that can not be quantified as analog, but we know they do exist. One of these is fragrance, we can appreciate fragrance but cannot quantify it, similarly taste, beauty, softness etc these are all physical qualities we come across and use in our everyday life but we cannot quantify them, or its not easy to quantify them reliably.

There are so many ways the analog data exists in nature, like if we talk about light only, it has intensity, color, frequency, wavelength, save shape or speed and may be many other parameters that describe a given sample of light. Our electrical systems, unfortunately have a handful ways of detecting or quantifying these properties.

Simplest is voltage, if we can somehow convert an analog property into equivalent or representative voltage, we can read it in our electrical systems. Second is current, provided voltage remains constant, current flow can be used as a parameter to assess some physical prop-



erty. Others are change in resistance or capacitance of a substance that can be influenced by the physical parameter. The change in resistance or capacitance can be translated into two measurable parameters, voltage or oscillating frequency.

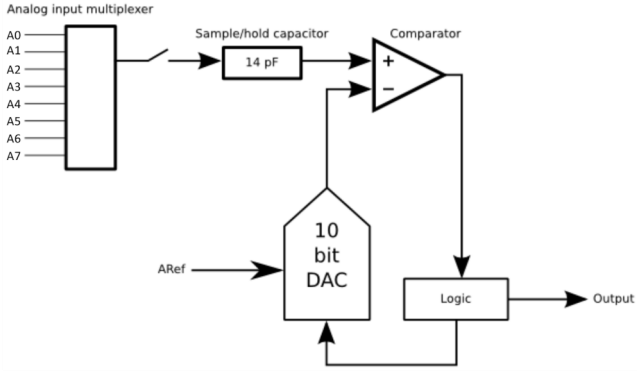
Analog Voltage

Analog voltage is a continuous variable, that contains unlimited number of values. Even if we have an absolute upper and lower limits, the physical property may translate into voltage, that can have unlimited values. Unfortunately our digital measuring systems have a limit, depending upon the width of variables we choose. By width I mean number of bits. This is expressed as the resolution of digital system to acquire analog data. Second parameter is how fast the system can acquire the analog data, that is the time slice, before the analog volts change. This is termed as sampling rate.

The analog data or voltage can have rapid fluctuations, and a slower acquisition speed is likely to miss the signal value in between the samples. So there are practically two issues with digital conversion of analog data. First limited quantification, and secondly chances of missing a data in between the samples. Moreover the actual data or voltage can also have negative voltage, that is difficult to measure by the regular microcontroller electronics.

Arduino ADC

The Arduino microcontroller ATMega328P has an analog to digital conversion module that uses a simpler but effective technique of acquiring analog data. The Arduino nano has 8 channels to acquire data, but they all are internally connected to one ADC module. So you can select the channel to read, but only one channel can be read at any given time.



Internally a small capacitor is charged with the incoming voltage, Which is fed to the input of a comparator. The other side of comparator is connected to a digital to analog converter (DAC), like an R2R resistor network. Now the microcontroller tries to approximate the voltage of capacitor by setting different values in the DAC, until the two voltages become equal and comparator output becomes 0.

The Arduino has 10 bits of DAC so the input sample can have a 2^{10} number of discreet values. This turns out to be 1024. The input sample is therefore broken down into 1024 possible levels, that this module can measure. So if we are measuring a maximum of 5V the possibility of input voltages can be 0-5V this 5V spectrum will be divided in 1024 steps (0-1023). So that when the voltage is 0V the output is digital number 0. when it is 5V the output is 1023 (10 bit digital number). This gives a resolution of 5/1024 = 0.00488V or 4.88mV per step. So if voltage is 4.88mV the digital output will be number 1. with this resolution we cannot measure the voltage of 2mV for example. So all measurements will be in steps of 4.88mV.

To improve upon this ADC uses another technique, in which the range is reduced, but number of bits remain same. So if we know that our input signal will never be more than 3.3V we use this as a standard and instruct the ADC to consider 3.3V as highest (instead of 5V) and distribute the 1024 steps from 0-3.3V. This will effectively decrease the voltage step

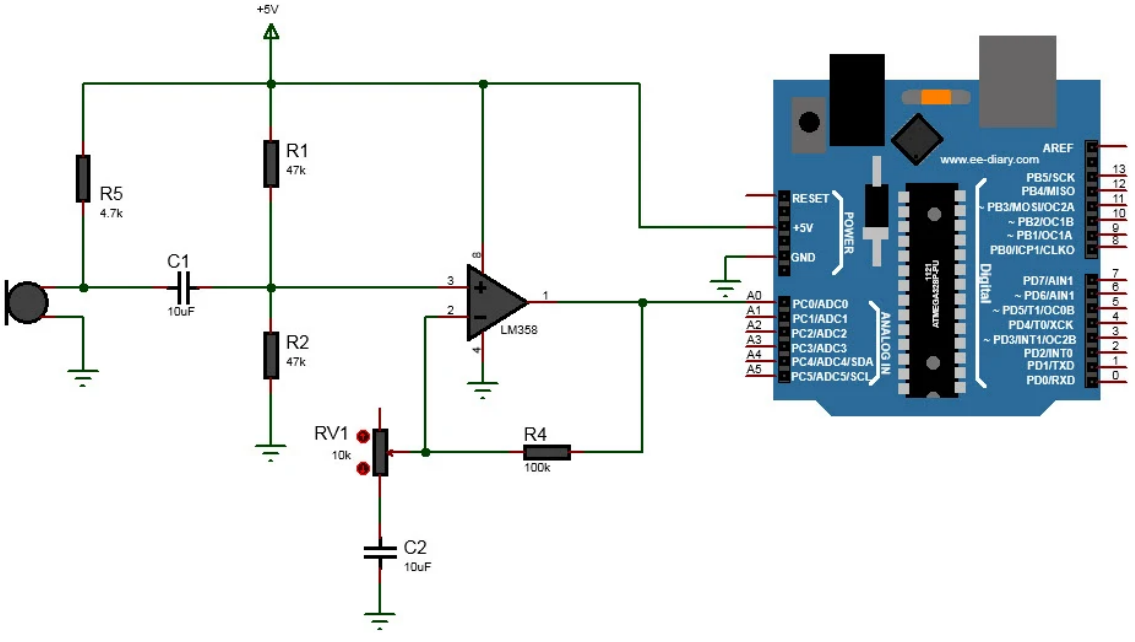
size. Thus 3.3/1024 = 0.0032 or 3.2mV. This will however unable to differentiate in voltages above 3.3V. This instruction of ceiling voltages is called 'Reference Voltage'. Usually Arduino uses the supply voltage (5V) as a reference by default and therefore we tend to have a measuring steps of 4.88mV. Other possible references are an internal calibrated 1.1V reverence. Using this as reference will give a maximum input volts of 1.1V but give steps of

1.1/1024 = 0.00107 or 1.07mV steps. There is a special pin labelled as Aref on Arduino. We can configure the ATMega328 to use the voltage applied on this pin as reference for the upper limit. Usually we apply an external 2.5V to this pin. This gives steps of 2.5/1024=0.00244 or 2.44mV steps.

In any case the Arduino pins cannot have a negative voltage, and no more than 5V on signal. More than 5V will damage the microcontroller pin.

Acquiring small Signals

Although when using an internal 1.1V reference we can sample signals as small as 1.07mV but truly speaking this is usually not practical. Noise of digital circuits inside the microcontroller can slightly affect the lower range of signals, similarly external noise on pins can also deteriorate the lower end signals. This practically reduces the usable range of 10 bits resolu-

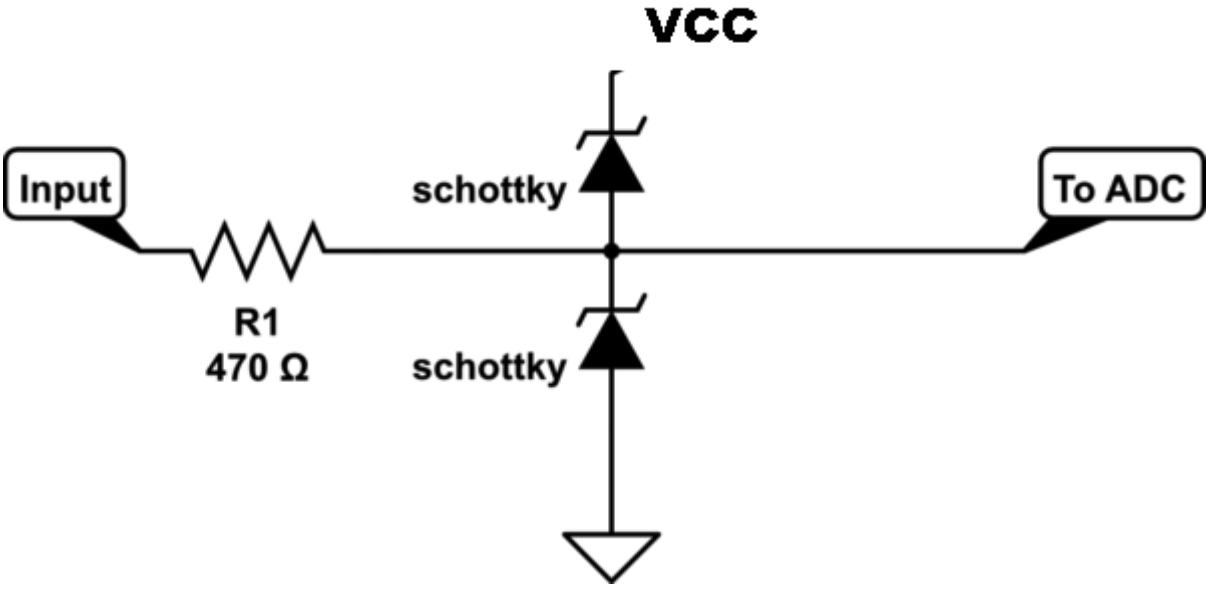


tion. A practical method of acquiring weak signals in reliable way is to use a preconditioning circuit or amplifier. An operational amplifier can be used to amplify the weak signals. You can find many different amplifier designs to acquire small analog signals into Arduino.

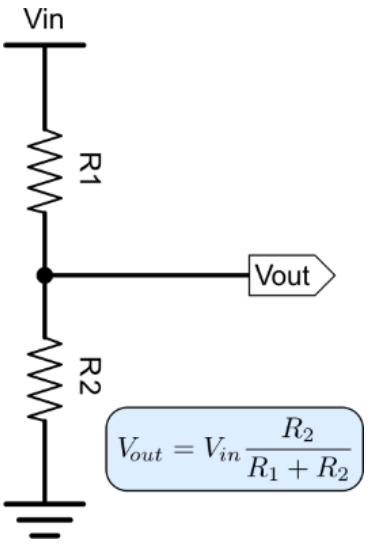
Pre-Conditioning Large Signals

Arduino pins can take only positive voltages as input, and the applied voltage must not be greater than 5V. Most industrial sensors and equipment use 12V signals, these must be scaled down to safe levels before applying to the Arduino pins. Moreover the industrial

noise may include negative voltage spikes that can damage the pins. Although the manufacturers of ATMega328 and other microcontrollers have clipping diodes present on the pins, that tend to protect the pins from negative spikes, but these diodes are of limited capacity.



Use schottky diodes like BAT54 or 1N5819 in the above configuration. When a negative voltage spike comes it shunts the spike to GND. The schottkey diodes have low forward voltage (~0.3V) so they start conducting early. Also a 1K to 10K resistor in series protects against rush of current through these diodes.



If your signal is above 5V it is essential to scale it down to bring it into safe 5V range. Use a simple voltage divider to scale down the input voltage.

Choose resistor values in K Ohms, so that the signal current is low, and voltages of signal do not drop erroneously due to current sinking.

I prefer to reduce the input voltage by 3 times or more depending upon the input voltage. Then calculate the scale by dividing the input with output, so if the scale 2.4 for example I will multiply the sensed voltage with 2.4 to get the original voltage in program.

I choose R2=4.7K and R1=10K this gives a scale down of 3.1 times. Always combine this with Schottky diode protection to protect against ESD spikes. Though voltage divider gives a good cushion, as 12V will be scaled down to 3.8V, still an accidental 48V signal will give higher voltages. In case such a possibility exist, add a 5V Zener diode in parallel to the pin.

Sampling Rate

Analog to digital conversion takes some time, this conversion time depends upon the underlying technology used for this conversion. Arduino uses ‘Successive Approximation’ tech-

nique. In this technique the sample voltage or signal is applied to the input of a comparator (sometimes using a charging capacitor) and then using a DAC, various voltages are applied to the other input of comparator, till both inputs become equal. The digital value that finalizes the output is reported to the MCU.

The ADC conversion depends upon the ADC clock, which is derived from system clock. The maximum ADC clock is 200KHz for accurate reading, usual ADC clock is 125KHz. A single ADC conversion takes 13 ADC Clock cycles. This translates to approximately 104 uS per conversion at 125KHz ADC clock resulting in a sample rate of approximately 9.6KHz.

Pre-scaler

Pre-scaler is a technique used to reduce the clock frequency. The Arduino ADC needs a clock frequency between 50KHz to 200KHz for accurate conversion. Since clock is derived from system clock of 16MHz, it needs to be divided many times to achieve this level of scale down.

The default prescaler of Arduino is 128 that results in ADC clock frequency of 125KHz. Lower the frequency better is adc accuracy. Increasing frequency increases speed but reduces accuracy.

You can change the ADC pre-scaler or other settings by direct register manipulation. Setting appropriate bits in ADCSRA register will set the different pre-scaler.

How to implement ADC in Arduino

As already explained, Arduino IDE by default sets the most commonly used and acceptable settings. So all you need is to read the appropriate channel. Arduino UNO has 6 ADC channels, named from A0 to A5 while Arduino Nano has 8 channels, A0 to A7. Since ADC returns a 10 bit value, the variable to hold the digital value must be able to hold 10 bit number. Since ‘int’ datatype is two bytes long, and it has 16 bits, it can easily store the 10 bits value.

```
int sensorValue = analogRead(A0); // Reads the analog voltage at pin A0
```

The analogRead() function does the magic. It accepts an argument which is channel number. A0 is internally defined as number 14. You can use number 14, but preferred way for Arduino is to use the internally defined pin numbers as A0, A1 etc. This function reads the analog pin A0 for analog signal and converts it to equivalent integer value. 0V=0 and 5V = 1023 similarly 2.5V = 512 and so on. The conversion takes 104 uS and the function waits for the conversion to finish.

Setting Analog Reference Volts

By default the analog system of Arduino uses the supply voltage (5V or 3.3V whatever it is) as the reference volts. This voltage is used to map the digital output of all 10 bits of ADC output set to 1. This means 5V will return 1023 value.

Reference can also be set to internal calibrated 1.1V or external whatever volts we apply on AREF pin/

```
analogReference(INTERNAL);  // Use internal 1.1V reference


analogReference(EXTERNAL);  // Use external volts on AREF pin as reference
```

Reading Potentiometer from Analog Pin

This is the first and basic example of Arduino where we connect the potentiometer, as a source of variable volts to the analog input pin and then read its value. The MikroDuino Nano board has a potentiometer on it and by default it connects to A7 pin of Arduino Nano. The potentiometer gives voltage between 0V and 5V output. Connect the jumper of POT and it will be connected to A7.

Run the program and turn on the serial monitor, it should display a value between 0 and 1023. now turn the potentiometer and the values should change.

```
// Read Analog data from potentiometer
// display raw ADC value of serial monitor
#define POT_PIN A7
void setup() {
    Serial.begin(9600); // Start serial communication
}

void loop() {
    int sensorValue = analogRead(POT_PIN); // Read the analog input (0-1023)
    Serial.println(sensorValue);      // Print the value to Serial Monitor
    delay(500); // Delay for readability
}
```

Converting the ADC value to Volts

Since ADC value given corresponds to the measured volts, we can convert the digital value into volts by applying simple math.

$$V = \frac{\text{ADC Reading} \times 5.0}{1023}$$

```
// Read analog pin and convert ADC value to volts
#define POT_PIN A7 // POT pin on MikroDuino Board
void setup() {
    Serial.begin(9600);
}

void loop() {
    int sensorValue = analogRead(POT_PIN); // Read analog input
    float voltage = sensorValue * (5.0 / 1023.0); // Convert to voltage
    Serial.print("ADC Value:" );
    Serial.print(sensorValue);
```

```
    Serial.print(" Voltage: ");
    Serial.print(voltage);
    Serial.println(" V");
    delay(500);
}
```

float voltage = sensorValue * (5.0 / 1023.0);

Note 5/1023 is written as 5.0/1023.0 to ensure that its calculated as floating point value. Otherwise 5/1023 as integer would give 0.

Reading an LDR (Light Sensor) and Controlling an LED

LDR is an analog sensor that changes its resistance when visible light falls on it. The resistance depends upon intensity of light. The resistance decreases as the light intensity increases. They typically have very high resistance, about 1M Ohms in full dark and low resistance about 10K in full light. In order to use it with Arduino, usually it is used as part of the resistor divider network, giving analog voltage corresponding to the intensity of light. Its up to you if you want LDR on VCC side or on GND side. In MikroDuino nano board we have an LDR with 10K resistance as voltage divider.

```
// LDR Demo
#define LDR_PIN A6
#define LED_PIN 6

void setup() {
    pinMode(LED_PIN, OUTPUT);
    Serial.begin(9600);
}

void loop() {
    int lightValue = analogRead(LDR_PIN);
    Serial.println(lightValue);

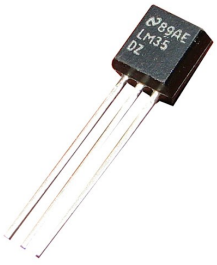
    if (lightValue < 500) { // Adjust threshold as needed
        digitalWrite(LED_PIN, HIGH); // Turn ON LED
    } else {
        digitalWrite(LED_PIN, LOW);  // Turn OFF LED
    }

    delay(500);
}
```

Reading Temperature from LM35 Temperature Sensor

The LM35 is a precision temperature sensor that provides an analog voltage output proportional to the ambient temperature. It is widely used in electronics and embedded systems due to its accuracy, ease of use, and low power consumption. Unlike thermistors, the LM35 does not require external calibration and provides a linear output in degrees Celsius. The output voltage is 10mV per degree centigrade. It has a working range from 0 to 100 degrees

centigrade. It can operate from 4 to 30V. It looks like a transistor but in fact is an integrated circuit. It has internal circuitry that is automatically calibrated to give a linear output response.




```
// LM35 Temperature demo
#define TEMP_SENSOR A0

void setup() {
    Serial.begin(9600);
}

void loop() {
    int sensorValue = analogRead(TEMP_SENSOR);
    float temperature = sensorValue * (5.0 / 1023.0) * 100; // Convert to °C
    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" °C");
    delay(1000);
}
```

Assignments:

1. Using 8 LEDs, make a bar-graph to correspond the ambient light. When its total dark all 8 LEDs should be ON and when full bright All LEDs Off.
2. Make a bar-graph using 8 LEDs to indicate the ambient temperature. All lights on when upper limit of set temperature is reached, and one LED when lower limit is reached.
3. Make a temperature alarm system, get the upper limit of set temperature from USART (Range between 30-50) and monitor temperature using LM35. On reaching set limit, turn an LED On (indicating Alarm) when temp lowers, it should turn Off

```
// LED Bar Graph display on reading potentiometer

#define POT_PIN A7  // Potentiometer connected to A7
#define NUM_LEDS 8  // Number of LEDs

// LED Pins from D6 to D13
int ledPins[NUM_LEDS] = {6, 7, 8, 9, 10, 11, 12, 13};

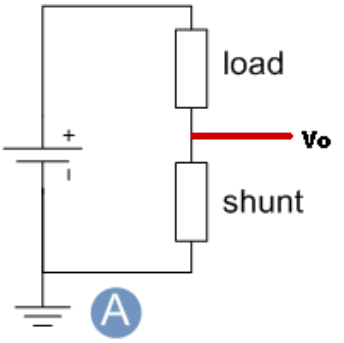
void setup() {
    for (int i = 0; i < NUM_LEDS; i++) {
        pinMode(ledPins[i], OUTPUT);  // Set LED pins as output
    }
}
```

```
void loop() {
    int potValue = analogRead(POT_PIN);  // Read potentiometer value (0-1023)
    int ledLevel = map(potValue, 0, 1023, 0, NUM_LEDS); // Map to LED range (0-
8)

    // Turn ON LEDs up to ledLevel
    for (int i = 0; i < NUM_LEDS; i++) {
        if (i < ledLevel) {
            digitalWrite(ledPins[i], HIGH);
        } else {
            digitalWrite(ledPins[i], LOW);
        }
    }

    delay(100); // Small delay for smooth updating
}
```

Measuring Current with Shunt Resistor

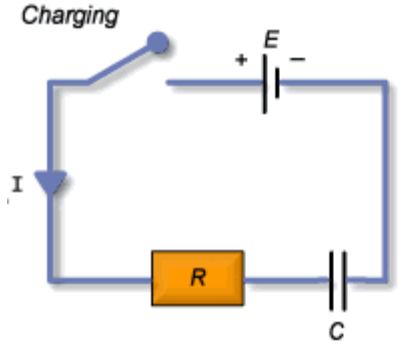


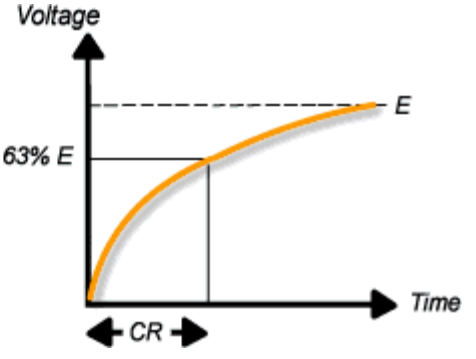
Arduino is good enough to measure the voltage using its ADC module, but when it comes to measuring current there is no direct method available. One simple and often used method is to use a 'Shunt' resistor, and let the current pass through it. Then measuring the voltage across the resistor is proportional to the flowing current.

The shunt resistance is essentially a very small value resistor, usually 0.47 Ohms, but in fact you can use anything from 0.01 to 10 Ohms. The shunt resistor is placed in series to the ground. Thus all current passes through it. The voltage generated across it can be measured using Arduino ADC and using Ohms's law converted into current passing through.

Measuring Capacitance of a Capacitor

Capacitance of a capacitor is not easy to measure directly. Even most multimeters do not have the capacitance measuring into them. To measure the capacitance of a capacitor it is important to convert the capacitance into some parameter that Arduino can measure. One method is to put the capacitor into an oscillator circuit and then measure the frequency of oscillation. Another method is to charge the capacitor with a known resistor and voltage, and then measure the time taken by the capacitor to get charged.





In the following discussion we will use this later method to determine the capacitance of an unknown capacitor.

Theory behind it

When a capacitor is charged through a resistor, the charging rate and therefore capacitor voltage rise is not constant. Rather it follows a curve, that is determined by the following equation:

$$V(t) = V_{supply} \times \left(1 - e^{-t/(R \times C)}\right)$$

Where:

- $V(t)$ is the voltage across the capacitor at time t.
- V_{supply} is the applied voltage.
- R is the resistance value.
- C is the unknown capacitance.
- e is Euler's number (~2.71828).

$$C = \frac{t}{R}$$

At **63.2% of supply voltage**, the time **t** equals exactly $R \times C$ (i.e., one time constant, τ).

So, by measuring the time it takes for the capacitor voltage to reach ~63% of the supply voltage, we can calculate the capacitance as:

When connecting to Arduino, we can charge the capacitor through a known resistor 10K, first discharge the capacitor by setting the pin LOW, and then making it HIGH. This will start charging the capacitor. Start monitoring and counting time. Charge the capacitor volts till it reaches 63% of the supply volts. For Arduino 5V supply its 3.15V or analogread value 645. When this value is reached timer count is stopped and t is calculated. If time is measured in Seconds capacitance is given in Farads, if measured in microSeconds capacitance is given as uF.

```
// ADC Measuring capacitence of an unknown capacitor.

const int chargePin = 7;      // Output to charge the capacitor
const int analogPin = A0;     // Analog input to monitor voltage
const int threshold = 645;  // 63.2% of 1023 (5V) ~ 3.15V
const float R = 10000.0;      // 10k ohms

void setup() {
  Serial.begin(9600);
}

void loop() {
  // Discharge capacitor
  pinMode(chargePin, OUTPUT);
  digitalWrite(chargePin, LOW);
```

```
  delay(100);

  // Prepare to measure
  pinMode(chargePin, INPUT);  // Hi-Z state
  delay(10);

  // Start charging
  pinMode(chargePin, OUTPUT);
  digitalWrite(chargePin, HIGH);

  unsigned long startTime = micros();

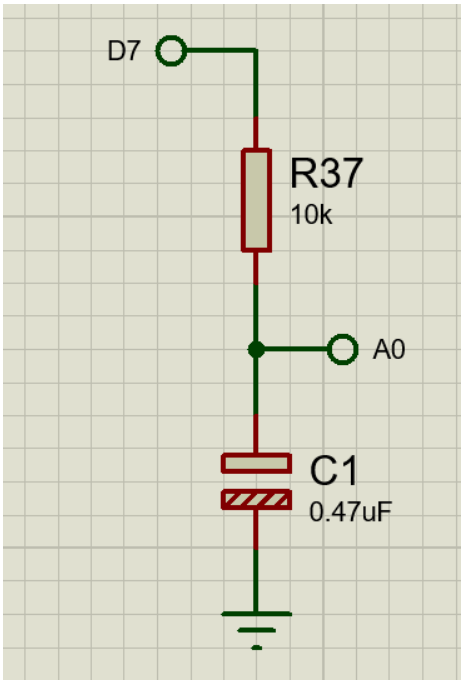
  while (analogRead(analogPin) < threshold);  // Wait until 63%

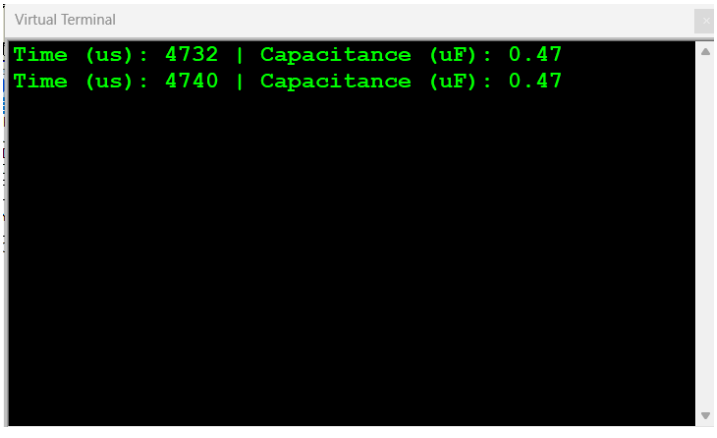
  unsigned long elapsedTime = micros() - startTime;

  float capacitance = (float)elapsedTime / R;  // C in microfarads
(µF)

  Serial.print("Time (us): ");
  Serial.print(elapsedTime);
  Serial.print(" | Capacitance (uF): ");
  Serial.println(capacitance);

  delay(2000);
}
```

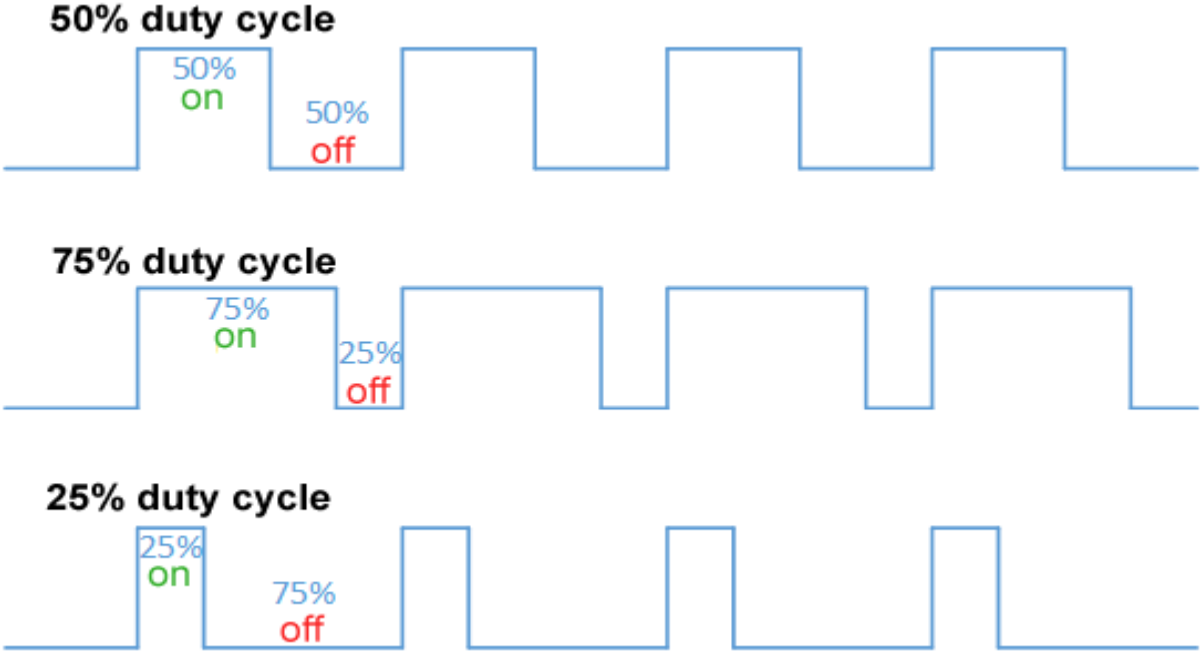




CHAPTER 8

Pulse Width Modulation

Modulation as a text is used in a number of situations, like Frequency Modulation, Amplitude Modulation or pulse width Modulation. The term modulation actually means a way to encode some information into some electrical signals. Pulse width modulation is very commonly used in digital electronics, where only two levels of binary states exist. We add another dimension, 'time' in this two state level system and get a way to encode some information. We can encode many different types of information in this format. Some of these are proprietary and have been standardized, while others are devised individually for specific jobs.



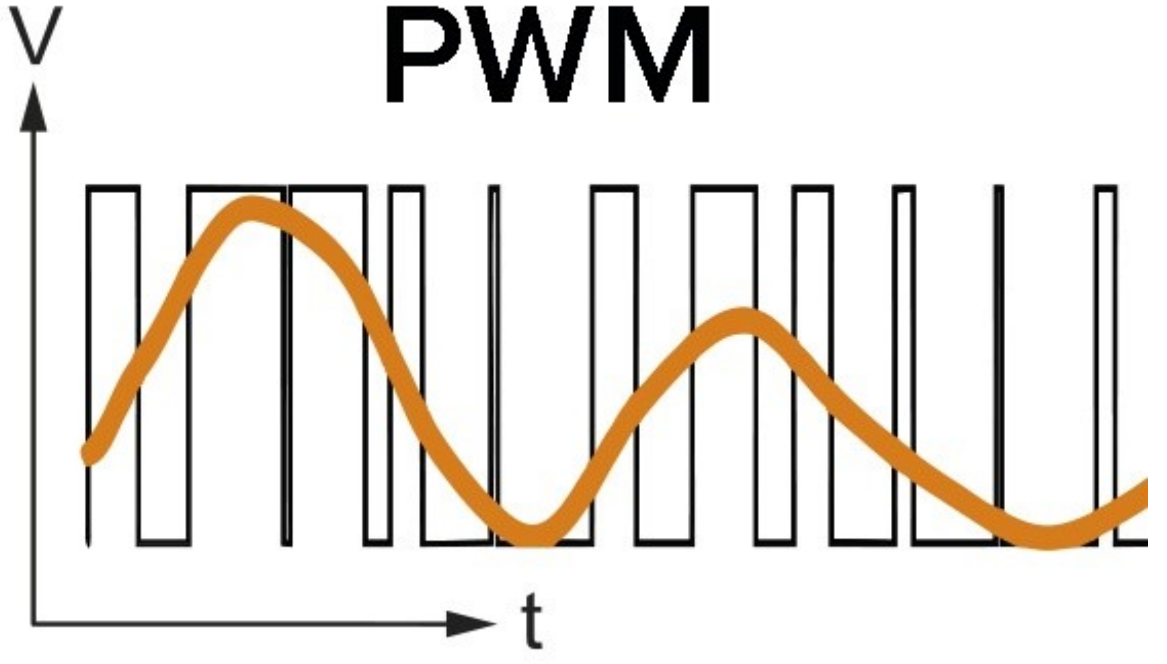
Pulse width modulation or PWM, is a waveform with two levels of pulses, On and Off. One complete wave cycle starts from 0 to 1, stays 1 for some time, and then goes low and remains low for some time. And then the cycle repeats. The total time it take for one cycle is the wavelength. PWM relates the ratio between On time and OFF time during a cycle. Irrespective of the total time of cycle. Thus if the On and Off times are equal it is termed as 50% duty cycle. It is customary to define a PWM signal as the percentage of On time. The Off time is automatically assumed to be 100-duty cycle.

Although as I have already said we can encode information on this, using a defined protocol

to describe what our data is. But in general PWM technique is used to produce Analog Signals or to transfer a varying power to a device using digital signals.

Just imagine if the duty cycle is 100%, it means the pin, or output is continuously ON, this means that the target device is getting full power. However if we turn the pin ON for half the time, and keep it OFF half the time, the target device is getting intermittent power, and averaging it over time, we can say it is getting 50%of the total power.

This technique is used to control the speed of motors, brightness of LEDs or even produce various analog waveforms. The analog waveforms cannot be described as truly analog, but you can say they resemble analog signals.



Due to its widespread use in electronics and projects, almost every microcontroller is equipped with hardware modules to continuously produce PWM signals. Although these signals can be produced in a simple loop, yet the streaming of PWM signal will be affected when microcontroller is busy in some other loop or task. The internal timers in microcontrollers can run autonomously in background and can be used to produce stream of PWM signals.

PWM in Arduino

Theoretically PWM can be produced on any digital GPIO line, but since internal timers are used to produce PWM, specific GPIO pins are dedicated to produce PWM signals. On Arduino boards a ~ sign is printed along with pin number to indicate the PWM capability of the pin. The Arduino nano has 6 independently controllable pins that can be used to produce PWM signals. These are D3, D5, D6, D9, D10 and D11. The `analogWrite(pin,`

`value)` function is used to generate PWM signals, where the `value` can range from 0 (0% duty cycle) to 255 (100% duty cycle). PWM on Arduino typically runs at a frequency of around 490 Hz or 980 Hz, depending on the pin.

Pin	Default PWM Frequency	Timer Used
3	~490 Hz (or ~980 Hz)	Timer 2
5	~980 Hz	Timer 0
6	~980 Hz	Timer 0
9	~490 Hz	Timer 1
10	~490 Hz	Timer 1
11	~490 Hz (or ~980 Hz)	Timer 2

The table above shows various frequencies at which PWM waves are generated on Arduino pins. Since PWM generation depends upon timers, any other functions that influence timer settings, are likely to influence the PWM behavior.

How PWM Frequency Influences the Projects

Although when we talk about PWM, we talk only about the duty cycle. To a beginner a 50%duty cycle on pin 10 and pin 6 means same thing. The PWM frequency can have dramatic effects on your project. Different devices tend to behave differently on low and high frequency. When controlling the intensity of an LED light a low frequency PWM (like 490 Hz) will cause the flicker to be recognized by a video camera, or if seen from periphery of the eye (peripheral vision in eye has fast recovery).

Motors generally behave well when a PWM frequency between 1KHz and 20KHz is used. Frequencies below 1KHz may result in jerky movements or an audible noise. Higher frequencies reduce noise but may produce heating if the driver circuit cannot handle fast switching.

PWM can produce audio sounds if frequency is modulated between audible frequency range. For producing musical notes the PWM frequency must match the desired pitch.

So when designing an electronics project, you also need to adjust the PWM frequency. The Arduino PWM frequency is just too low for any useful project. Its good to experiment but not good enough to be used in a true project. Using timer registers directly to produce PWM signals a PWM frequency up to 62KHz can be produced in ATMega328 microcontroller. Using registers give you full control over PWM frequency and resolution which is important for precise control.

LED Dimming in Arduino

Using standard PWM frequency generators, producing an LED dimming effect is quite simple. Though not ideal, but it works in most projects. LED dimming with Arduino is

achieved using **Pulse Width Modulation (PWM)**. By changing the **duty cycle** of a PWM signal on a PWM-enabled pin, the brightness of the LED can be smoothly increased or decreased. The Arduino function `analogWrite(pin, value)` is used, where `value` ranges from `0` (LED off) to `255` (LED fully on).

```
// PWM DEMO
// Pin connected to LED
const int ledPin = 9; // Use any PWM pin (e.g., 3, 5, 6, 9, 10, 11 on UNO)

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // Increase brightness
  for (int brightness = 0; brightness <= 255; brightness++) {
    analogWrite(ledPin, brightness);
    delay(10);
  }

  // Decrease brightness
  for (int brightness = 255; brightness >= 0; brightness--) {
    analogWrite(ledPin, brightness);
    delay(10);
  }
}
```

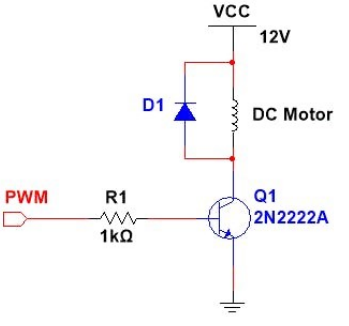
Since PWM is being controlled by timers, once you set a duty cycle, it tends to remain in effect, no matter what else the program is doing.

Assignment:

1. Using a potentiometer control the brightness of an LED

Controlling Fan Speed with PWM

While LED can be driven directly with microcontroller pin, a DC motor needs way too high current, and sometimes different voltage levels too. We therefore need a driver circuitry that should accept TTL level signals as input and provide sufficient power output to drive a motor, or even any other heavy DC load.



The choice of transistor depends upon the amount of current your DC motor would need. 2N2222 can source up to 500mA of current. For heavier loads I prefer Darlington pair, TIP120 transistor that can easily take up to 5A of current. Another important thing to consider is the use of D1 diode. When the DC load is inductor in nature, like a coil in a relay or a motor, the electromotive induction tends to produce a heavy spike of reverse voltage when the load is disconnected. This heavy spike can damage the driver cir-

cuitry. This diode protects against this spike. Rest of the discussion remains same. When the source of power is different, like 12V in this case, make sure the Ground connection of both supplies are tied together.

```
// PWM Fan Speed Control
const int fanPin = 10; // PWM pin connected to transistor base via resistor

void setup() {
  pinMode(fanPin, OUTPUT);
}

void loop() {
  // Ramp up
  for (int speed = 0; speed <= 255; speed++) {
    analogWrite(fanPin, speed);
    delay(10);
  }

  // Ramp down
  for (int speed = 255; speed >= 0; speed--) {
    analogWrite(fanPin, speed);
    delay(10);
  }

  delay(1000);
}
```

Fading an RGB LED with PWM

Now that we can increase or decrease the brightness of an LED, what about controlling three LEDs with Red, Blue and Green colors to produce varying brightness and mixing these colors to produce new colors.

Though you can use three independent LEDs, but the easier way is to use the RGB LED that contains three LEDs built in the same casing.



RGB LEDs are available as common Anode and common cathode. Use a current limiting resistor with each LED pin to limit the current flow.

```
const int redPin = 9;
const int greenPin = 10;
const int bluePin = 11;

void setup() {
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);
}

void loop() {
```

```
  analogWrite(redPin, 255);   // Red full
  analogWrite(greenPin, 0);
  analogWrite(bluePin, 0);
  delay(1000);

  analogWrite(redPin, 0);
  analogWrite(greenPin, 255); // Green full
  analogWrite(bluePin, 0);
  delay(1000);

  analogWrite(redPin, 0);
  analogWrite(greenPin, 0);
  analogWrite(bluePin, 255);  // Blue full
  delay(1000);
}
```

Now lets have the fun part, of controlling multiple LEDs in RGB LED, to mix colors.

```
//PWM RGB LEDs
const int redPin = 9;
const int greenPin = 10;
const int bluePin = 11;

void setup() {
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);
}

void loop() {
  // Fade red up, others off
  for (int i = 0; i <= 255; i++) {
    analogWrite(redPin, i);
    analogWrite(greenPin, 0);
    analogWrite(bluePin, 0);
    delay(10);
  }

  // Fade green up, red down
  for (int i = 0; i <= 255; i++) {
    analogWrite(redPin, 255 - i);
    analogWrite(greenPin, i);
    analogWrite(bluePin, 0);
    delay(10);
  }

  // Fade blue up, green down
  for (int i = 0; i <= 255; i++) {
    analogWrite(redPin, 0);
    analogWrite(greenPin, 255 - i);
    analogWrite(bluePin, i);
    delay(10);
  }

  // White light mix (R + G + B full)
```

```
  analogWrite(redPin, 255);
  analogWrite(greenPin, 255);
  analogWrite(bluePin, 255);
  delay(1000);

  // All off
  analogWrite(redPin, 0);
  analogWrite(greenPin, 0);
  analogWrite(bluePin, 0);
  delay(1000);
}
```

You will see various color mixes in the RGB LED.

Assignment:

1. Using three potentiometers make program to mix various colors.

Here is a version to show different colors using a push button. The program also demonstrates use of a function.

```
//RGB LED Color selection using Button
const int redPin = 9;
const int greenPin = 10;
const int bluePin = 11;
const int buttonPin = 2;

int mode = 0;
bool lastButtonState = HIGH;

void setup() {
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
}

void loop() {
  bool buttonState = digitalRead(buttonPin);

  if (buttonState == LOW && lastButtonState == HIGH) {
    mode++;
    if (mode > 6) mode = 0;
    delay(200); // debounce
  }
  lastButtonState = buttonState;

  switch (mode) {
    case 0: setColor(255, 0, 0); break;   // Red
    case 1: setColor(0, 255, 0); break;   // Green
    case 2: setColor(0, 0, 255); break;   // Blue
    case 3: setColor(255, 255, 0); break; // Yellow
    case 4: setColor(0, 255, 255); break; // Cyan
    case 5: setColor(255, 0, 255); break; // Magenta
```

```
    case 6: setColor(255, 255, 255); break; // White
  }
}

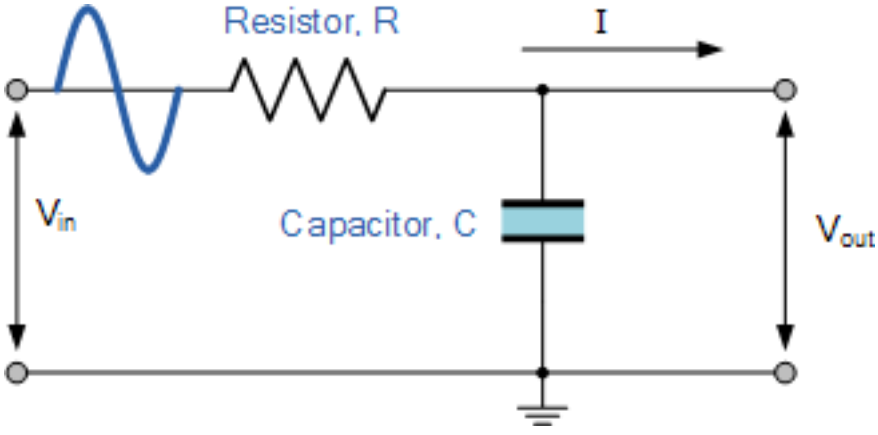
void setColor(int r, int g, int b) {
  analogWrite(redPin, r);
  analogWrite(greenPin, g);
  analogWrite(bluePin, b);
}
```

Analog Wave Generation using PWM

Since PWM can produce varying levels of power output by changing its duty cycle, with a suitable filtering circuit we can obtain analog volts as output. The PWM signal in fact is a wave of digital signals, and has a frequency of about 980Hz on pins 5 and 6 and frequency of 490Hz on pins 3,9,10 and 11. We need to separate the digital frequency from the PWM induced power to produce analog signal.

LOW Pass Filter

A Low pass filter is used to filter out the high frequency and output the low frequency signal modulated on it.



A low pass filter essentially consists of a resistor and a capacitor. The values of resistor and capacitor are important in selecting the frequency that needs to be passed and filter out other frequencies. The cutoff frequency is calculated using this formula.

$$f_c = \frac{1}{2\pi RC}$$

Where:

- f_c is the **cutoff frequency** (Hz)
- R is resistance in ohms (Ω)
- C is capacitance in farads (F)

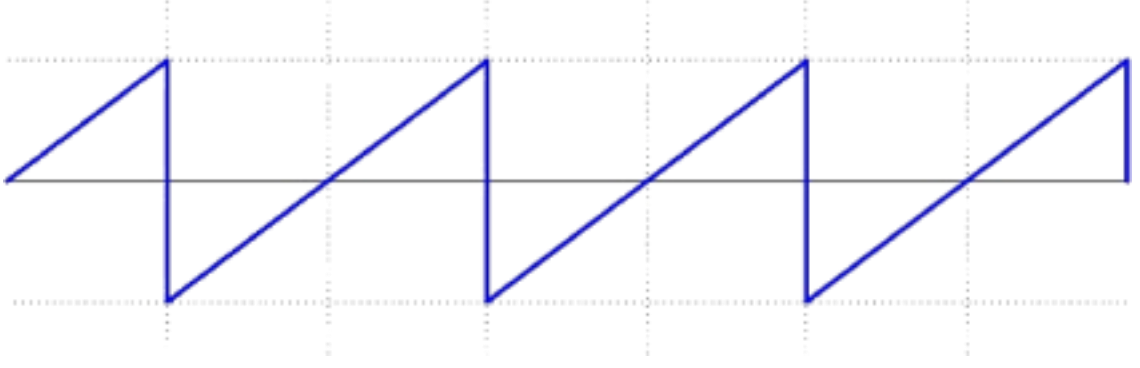
The carrier frequency of PWM signal is 980Hz so the signal that we are going to produce on it must be below this frequency. Lets suppose we want a signal to be 100-200Hz to be modulated on this PWM frequency.

- $R = 1\text{k}\Omega$
- $C = 1\mu\text{F}$

$$f_c = \frac{1}{2\pi \cdot 1000 \cdot 1 \times 10^{-6}} \approx 159 \text{ Hz}$$

Using a resistor of 1K and a capacitor of 1uF we can easily produce a wave on PWM. You will need an oscilloscope to see the signal.

Let's create an Arduino program that generates a sawtooth wave using PWM output and your RC low-pass filter (designed for ~980 Hz PWM). The RC filter will smooth the PWM signal into an analog-style sawtooth wave. A sawtooth wave gradually increases in amplitude, reaching maximum and then abruptly drops to zero and start over.

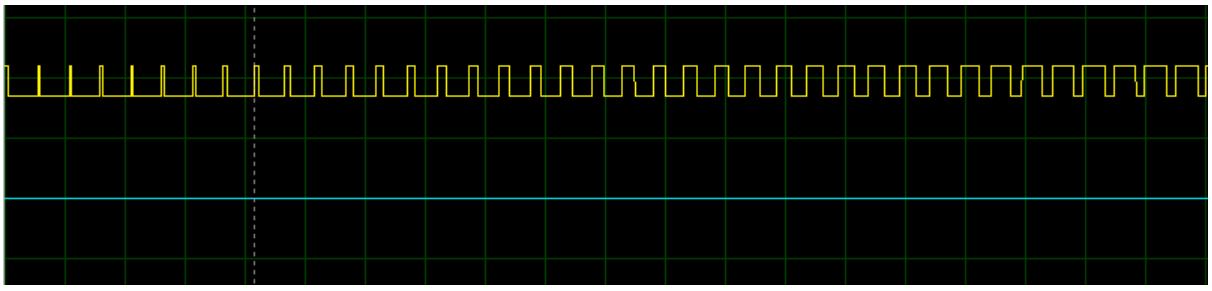


Using PWM we will modulate this as gradually increasing the PWM duty cycle from 0 to 255 and then dropping back to 0 and repeat. The slope of wave will be determined by delay in between the changing duty cycles.

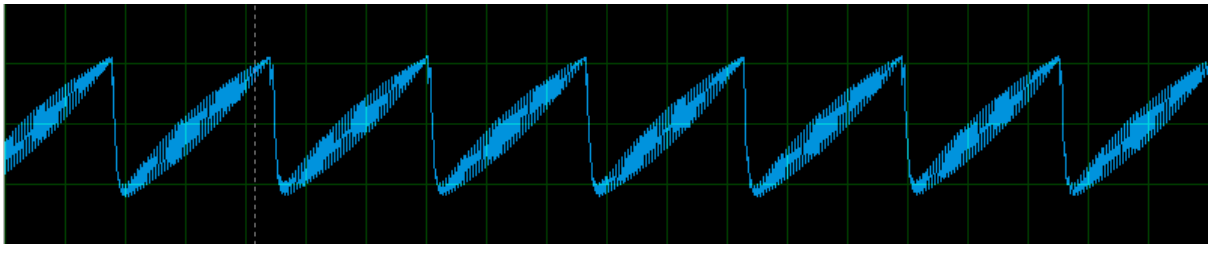
```
// Sawtooth Wave useing PWM
const int pwmPin = 5;       // PWM-capable pin (pins 5 or 6 have ~980Hz PWM)
const int maxValue = 255;   // 8-bit PWM range

void setup() {
  pinMode(pwmPin, OUTPUT);
}

void loop() {
  for (int value = 0; value <= maxValue; value++) {
    analogWrite(pwmPin, value);  // Set PWM duty cycle
    delayMicroseconds(200);      // Controls the frequency of the wave
  }
}
```



This trace of oscilloscope has been takes from D5 pin output this shows the PWM signal. Notice how the duty cycle is changing from left to right. With narrow pulses to start with and gradually increasing the pulse width.



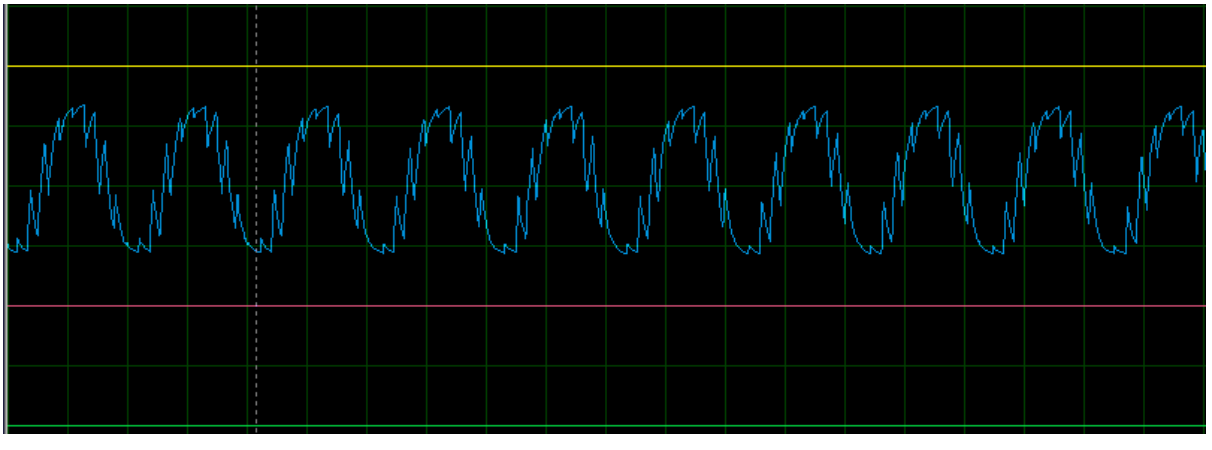
This trace shows the output of the filter. Notice How the voltage is gradually increasing and then dropping. Though PWM signal is digital, with filter we can convert it to something like analog. This is not truly analog as it contains the switching noise. With better filters and improving frequency a better wave can be produced.

Waveform shape: Linear rising voltage from 0V to ~5V

Frequency control: Delay × 256 steps = one full cycle Example: `delayMicroseconds(200)` → ~50 Hz wave (`200 µs × 256 ≈ 51.2 ms per cycle → ~19.5 Hz`)

Try reducing the delay for a faster wave (e.g., `100 µs` for ~40 Hz).

Similarly you can produce a variety of signals, like Ramp Wave, Sine Wave, Triangular wave etc.



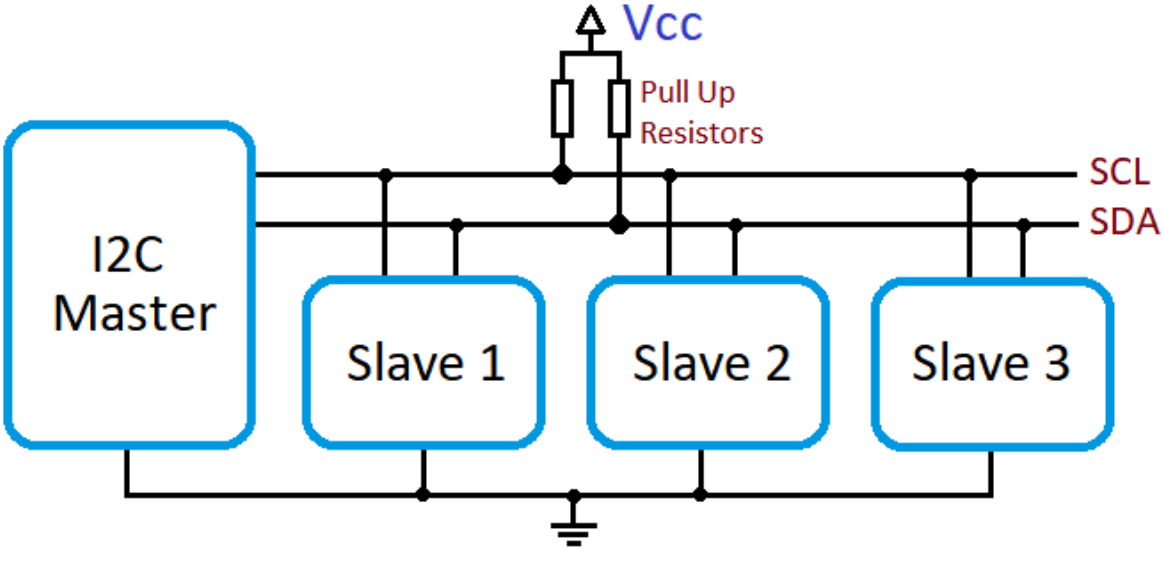
Here is output of a sine wave generator.

CHAPTER 9

I²C Communication

Inter Integrated Circuit Communication or I2C is a very popular protocol for multiple device communication within a system. It was developed by Philips Semiconductor (now NXP) in the 1980s. It is designed for short distance, low speed communication between multiple devices on the same circuit board. This communication protocol is particularly useful in embedded systems and microcontroller projects. This protocol uses two digital lines, named as SDA and SCL, SDA line is bidirectional and used to send and receive data while SCL is clock line and used to synchronize communication. The SCL line is controlled by the master device, all other devices are called slave devices that follow the master commands. The standard practice is to have one master device and one or more slave devices, but it is possible to have more than one masters, called multi-master system. In any case only one master is communicating at one time.

Each slave device has a unique 7-bit address that master can use to communicate selectively. Some systems now allow 10-bit addressing too, but this 7-Bit addressing is fairly standard. The commercially produced integrated circuits and devices have their unique addresses

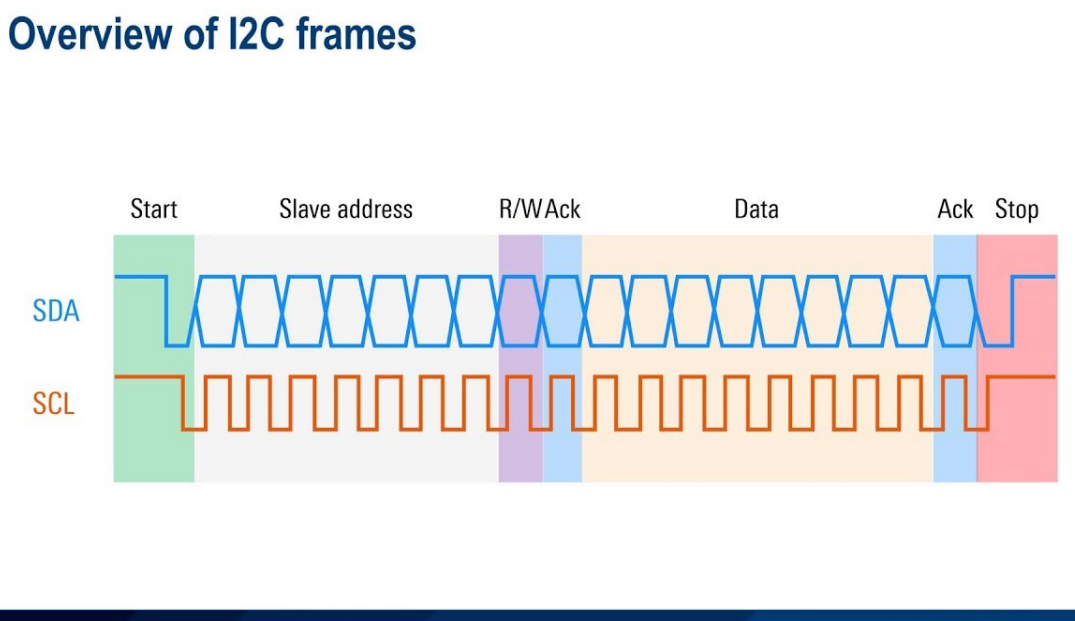


assigned by the I2C Bus Committee run by NXP semiconductors. In large scale for commercial applications conflict of address by some devices can cause serious issues.

The address of a particular I2C device is hard coded in the chip, however some manufacturers hard code a few bits of the address and allow few other bits to be set by the user to allow

flexibility and add more devices to the circuit. The same bus is shared by all devices, the entire bus needs to have a single set of Pull-Up resistors.

Various communication speed modes are available, the standard is 100 Kbps others are 400 Kbps, 1 Mbps and 3.4 Mbps. (bits per second). The master device initiates communication by sending a start condition, followed by the address of the target slave and a read/write bit. Data is then exchanged in 8-bit frames, each followed by an acknowledgment (ACK) bit. Communication ends with a stop condition sent by the master.



I²C is commonly used to interface, EEPROMs, RTCs (Real-Time Clocks), Temperature sensors, Displays (like OLED or LCD modules), ADC/DACs and many other devices. The advantage of this bus, is minimal pin usage, support for multiple devices on the same bus, availability of hardware modules in many microcontrollers and easy routing of printed circuit boards. The disadvantage is however limited speed, as compared to SPI and relatively short distance of communication, preferably on the same PCB.

ATMega328 I²C Module

The AtMega328 microcontroller has a built-in module to manage I2C communication. This is also referred to as two wire interface. It manages all I2C protocol requirements including Start/Stop conditions, ACK/NACK responses and data shifting etc. It supports both master and slave modes, and can generate interrupts on events like data received, transmission complete or start / stop conditions detected.

The module has several registers, that modify the behavior of this module. One of these registers is TWBR, that controls the speed of I2C bus. By default Arduino sets this speed to 100 Kbps.

The Arduino 'wire.h' Library

The Arduino IDE contains a pre-installed library that simplifies the I2C communication.

This is called ‘wire’ library. All you need is to include this library in your project. We will explore various uses of this library in up coming chapters when we use various I2C compliant devices. Just to give you a summary. The wire library can configure your Arduino device as ‘Master’ or as ‘Slave’. The Master mode is by far most common and default.

Master Mode Commands (default)

```
#include <Wire.h>
Wire.begin();                // Join I²C bus as master
Wire.beginTransmission(addr); // Start talking to a device
Wire.write(data);            // Send data (single byte or buffer)
Wire.endTransmission();      // Send stop condition

Wire.requestFrom(addr, n);  // Request 'n' bytes from device
Wire.read();                 // Read a byte from the buffer

```

First you need to include the <Wire.h> library. Notice the ‘W’ is capital. You can also include it by selecting wire library from library manager in Arduino IDE. Rest of the commands are self explanatory. The Wire object is automatically created. Many other libraries use this library in background, so they may require you to include this library in the project.

Slave Mode Commands (Rarely used)

```
#include <Wire.h>
Wire.begin(address);        // Join I²C bus as slave with given address
Wire.onReceive(func);       // Set function to run when data is received
Wire.onRequest(func);       // Set function to run when data is requested

```

The wire library uses interrupts to manage I2C events in the background. It uses the internal 32 Bytes storage registers for I2C data.

Things to Keep in Mind:

- Connect **pull-up resistors (typically 4.7kΩ or 10kΩ)** to SDA and SCL lines.
- All devices on the bus must share a **common ground**.
- Be careful to avoid **address conflicts**.

When all I2C devices are connected you can access them using their address. The address is provided by the manufacturer. However since address is a 7 bit number, we can write a scanning program to try to ping all the numbers, and see which ones respond.

This I2C scanning program is very common among the Arduino community. Sometimes it is essential because the manufacturers, that are not registered with NXP semiconductors, tend to assign their own addresses which may not be the ones mentioned in datasheets.

```ino
//I2C Address Scanner

#include <Wire.h>

void setup() {
  Wire.begin();

  Serial.begin(9600);
  while (!Serial); // Leonardo: wait for Serial Monitor
  Serial.println("\nI2C Scanner");
}

void loop() {
  int nDevices = 0;

  Serial.println("Scanning...");

  for (byte address = 1; address < 127; ++address) {
    // The i2c_scanner uses the return value of
    // the Wire.endTransmission to see if
    // a device did acknowledge to the address.
    Wire.beginTransmission(address);
    byte error = Wire.endTransmission();

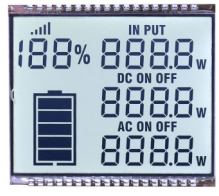
    if (error == 0) {
      Serial.print("I2C device found at address 0x");
      if (address < 16) {
        Serial.print("0");
      }
      Serial.print(address, HEX);
      Serial.println("  !");

      ++nDevices;
    } else if (error == 4) {
      Serial.print("Unknown error at address 0x");
      if (address < 16) {
        Serial.print("0");
      }
      Serial.println(address, HEX);
    }
  }
  if (nDevices == 0) {
    Serial.println("No I2C devices found\n");
  } else {
    Serial.println("done\n");
  }
  delay(5000); // Wait 5 seconds for next scan
}
```

CHAPTER 10

Liquid Crystal Display | I²C

Displays are an important part of embedded systems. They tend to provide a convenient interface for the embedded system to communicate with the user. Just like buttons and various other input devices that are used to get user input, displays are used to give outputs in text or graphical form to the user. Apart from LEDs which are the simplest way of giving system status to the user, seven segment, LED display, LED matrix displays and many other devices that are based upon LEDs are used as output devices.



Liquid crystal displays come in various forms, some are general purpose while others are customized for a particular device. Liquid crystal displays are in market since long, as they are not energy hungry like LEDs, and they are easy to manufacture. They have been frequently used in wrist watches, calculators or fuel stations. One of the important features of this type of display is that it has a very good contrast ratio, and readability is very good in daylight. In dim light or room applications you need an illumination system, either a side LED light or a backlight. In the absence of this LED the native liquid crystals require a very few current and therefore are battery friendly.

If you look at the physics of these displays, the liquid crystals spin under the influence of applied voltage and therefore conduct or block the light to pass through, in other words they change the polarization of light resulting in black display segments. These crystals cannot be held in this position for a long time by constantly applying the volts, as in case of LEDs. Rather they need an alternating current signal at a frequency of about 100 Hz to keep them spinning and display the segment. A special display driver is therefore needed to keep these segments oscillating with an alternating current. Some manufacturers make these drivers along with the display, while others just sell the bare display with connections, and let the electronics engineer to work around the display.

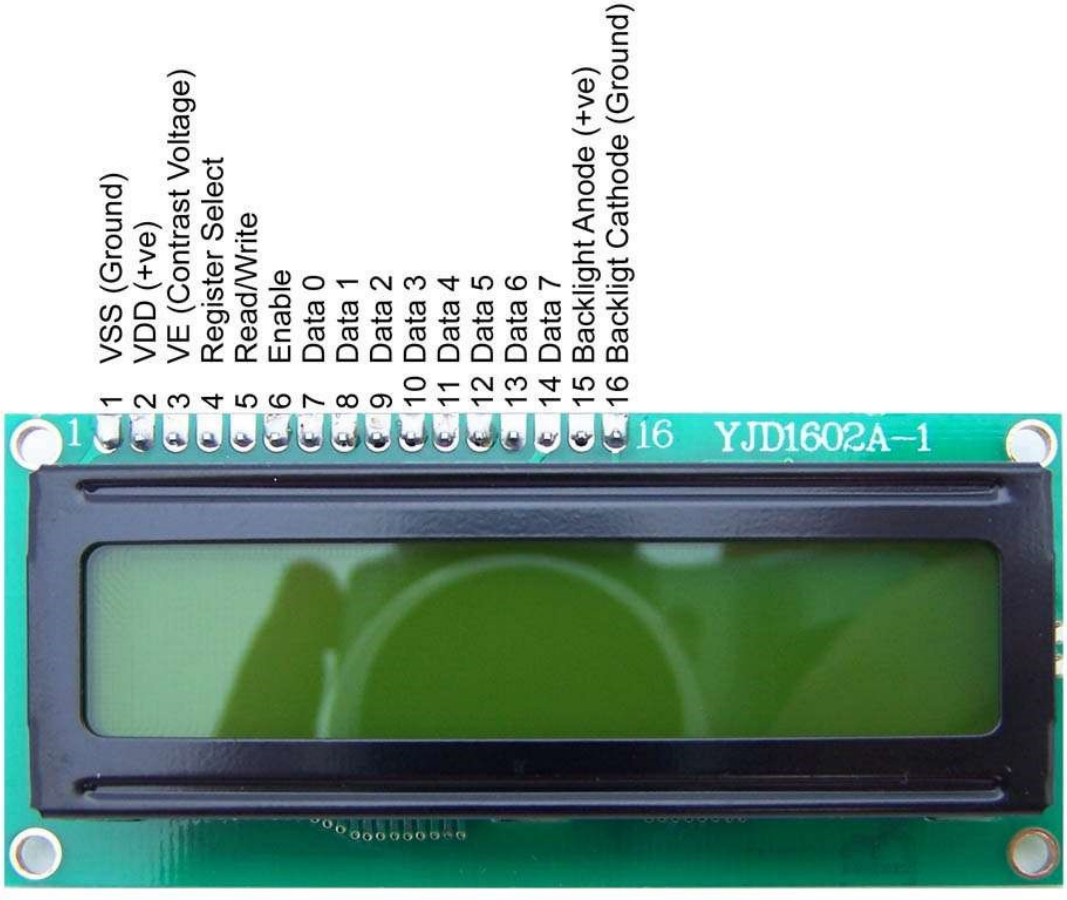


Character LCD

In the display above the characters or segments are arranged as digits, or fixed symbols. However in order to make a general purpose display a character LCD matrix was developed. This matrix contains LCD dots about 5x8 in each character. The entire display contains several such characters. Various charac-

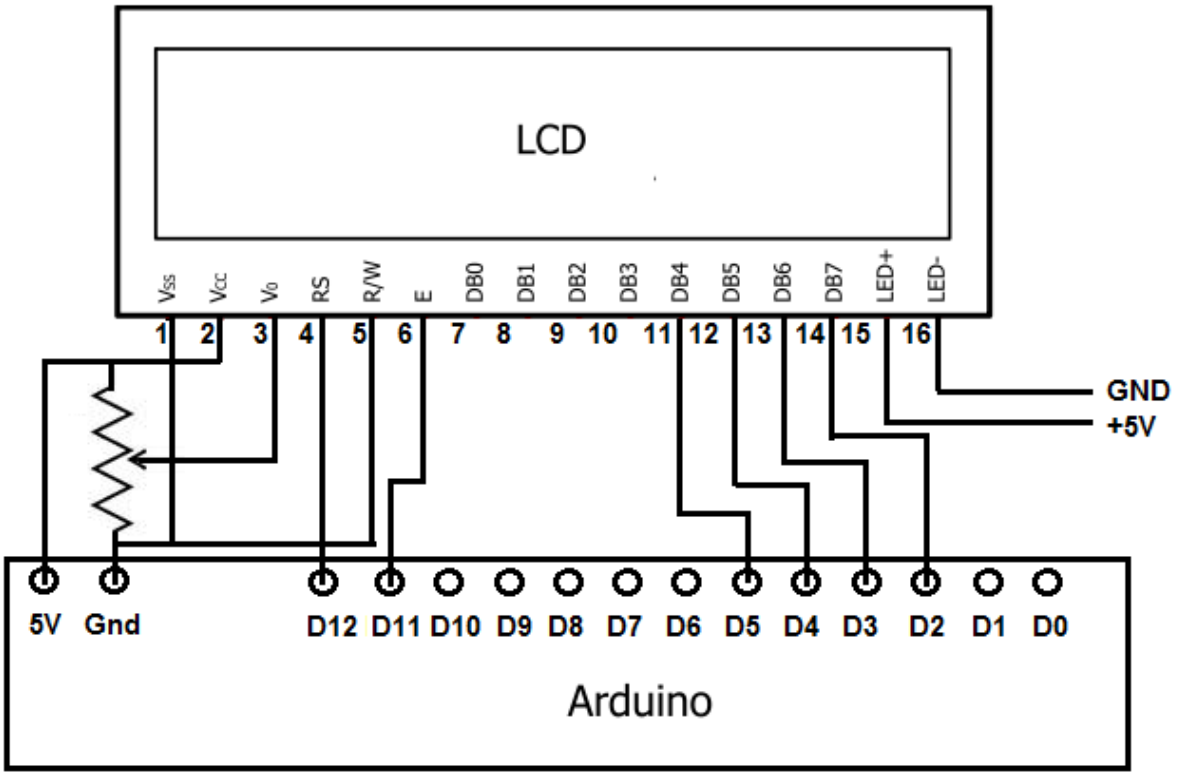
ters and graphics can therefore be displayed on these matrix characters. Many manufacturers used to make these displays, and they tend to have their own driver circuitry. Since drawing characters, and managing the alternating current signals to each dot (or pixel) in display is a huge task.

Hitachi company introduced an LCD controller HD44780, it was easy to program, and use. Displays using this controller became widely used in industry and among electronic equipment manufacturers. Today HD44780 controller based character LCDs are available in general market as standalone products that can be used in any project. These LCDs are being manufactured by many different manufacturers, but they all comply to the same standard. They may differ in the quality of display but interfacing is same.



These displays typically have 16 pins for power, contrast, control , data and LED. The display can be driven from 3.3V to 5V. VE is the contrast pin, and a potentiometer is usually attached to this pin to adjust the contrast, Register select pin is used to select the two registers, either a command register or data register. Read/Write pin is used to select the operation, if we want to write to the LCD or read data from it. Most of the times we do not need to read from LCD, so this pin is permanently configured as ‘write’ by connecting it to GND. Enable pin is used to push the data into the LCD. D0 to D7 are 8 bits of data. The LCD can be configured to use either all of these 8 bits, or only 4 bits. In 4 bit mode only pins D4-D7 are used to send a byte of data in two chunks. The LED backlight can be hard wired to power or through a transistor and controlled via microcontroller pin. When using this display in 4 bit mode, which is the most commonly used mode to save the microcontroller digital IO

lines, You need 4 lines for data, and two for control register like RS and E. So at least 6 digital lines are required to use the character LCD with a microcontroller.



When using the character LCD in this way, there is a built-in Arduino library to control it.

```
// Liquid Crystal Display
// include the library code:
#include <LiquidCrystal.h>

// initialize the library by associating any needed LCD interface pin
// with the arduino pin number it is connected to
const int rs = 6, en = 7, d4 = 8, d5 = 9, d6 = 10, d7 = 11;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

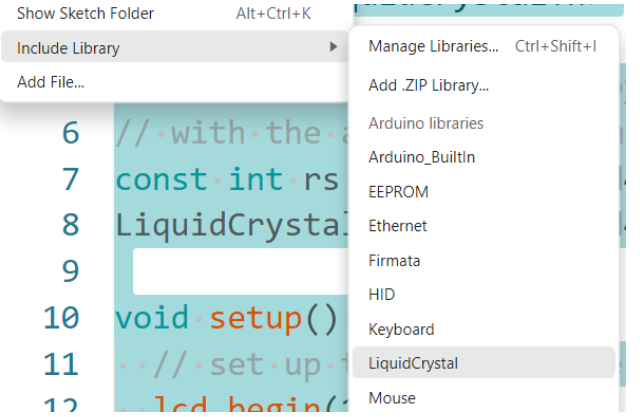
void setup() {
  // set up the LCD's number of columns and rows:
  lcd.begin(16, 2);
  // Print a message to the LCD.
  lcd.print("hello, world!");
}

void loop() {
  // set the cursor to column 0, line 1
  // (note: line 1 is the second row, since counting begins with 0):
  lcd.setCursor(0, 1);
  // print the number of seconds since reset:
  lcd.print(millis() / 1000);
}
```

Using Arduino Libraries

Although libraries are a common concept in C, they just need to be included by the header file, and call the public functions. Arduino libraries are little different. These are actually classes of objects, and therefore to use the library you have to include its class, by including the .h header file. After that you have to instantiate an object of type, library, this object will contain all the code of the library. The object oriented nature of Arduino libraries, exposes only the public functions and variables to the user (programmer) and hides all other private functions and internal variables. Moreover the library allows you to create more than one objects of the type, and access them completely independently.

```
#include <LiquidCrystal.h>
```



Includes the preinstalled library in your project. Liquid crystal library is provided the Arduino as part of IDE and you just need to include it in you program. Either by click on the library name from the library manager, or typing in directly. Remember the C++ is case sensitive language, and you have to write the library name exactly as it appears in the library manager.

Liquid Crystal Library Usage Commands Summary

Here are the commonly used commands and settings of the Liquid Crystal library.

`LiquidCrystal(rs, en, d4, d5, d6, d7)`

This command initializes the LiquidCrystal object and sets which Arduino pins are connected to the LCD. These pins are the RS (Register Select), EN (Enable), and the four data pins D4 through D7. This must be declared before using other functions of the LCD.

Usage: `LiquidCrystal lcd(12, 11, 5, 4, 3, 2);`

`lcd.begin(cols, rows)`

This sets the dimensions of the LCD. Most commonly used sizes are 16 columns by 2 rows (16x2) or 20x4. It must be called inside the `setup()` function.

Usage: `lcd.begin(16, 2);`

`lcd.print(data)`

Prints characters, strings, or numbers to the LCD screen starting from the current cursor location. It works like `Serial.print()`.

Usage: `lcd.print("Hello, World!");`

`lcd.write(byte)`

Writes a raw byte to the display. It is mainly used to write custom characters that were previously defined.
Usage: `lcd.write(0);`

`lcd.setCursor(col, row)`
Sets the position of the cursor on the screen. Both column and row numbers start from 0.
Usage: `lcd.setCursor(0, 1);`

`lcd.clear()`
Clears all content on the LCD and moves the cursor to the top-left corner (0,0).
Usage: `lcd.clear();`

`lcd.home()`
Moves the cursor to the top-left corner without clearing the display content.
Usage: `lcd.home();`

`lcd.display()`
Turns the display back on if it was previously turned off with `noDisplay()`. Content remains unaffected.
Usage: `lcd.display();`

`lcd.noDisplay()`
Turns the display off without erasing its contents. Useful for power saving or blinking effects.
Usage: `lcd.noDisplay();`

`lcd.cursor()`
Displays an underscore-style cursor at the current position.
Usage: `lcd.cursor();`

`lcd.noCursor()`
Hides the underscore-style cursor from the screen.
Usage: `lcd.noCursor();`

`lcd.blink()`
Enables a blinking block cursor on the screen at the current location.
Usage: `lcd.blink();`

`lcd.noBlink()`
Disables the blinking cursor.
Usage: `lcd.noBlink();`

`lcd.scrollDisplayLeft()`
Scrolls all display text one position to the left.
Usage: `lcd.scrollDisplayLeft();`

`lcd.scrollDisplayRight()`
Scrolls all display text one position to the right.
Usage: `lcd.scrollDisplayRight();`

`lcd.leftToRight()`
Sets text direction from left to right, which is the default behavior.
Usage: `lcd.leftToRight();`

`lcd.rightToLeft()`
Reverses text direction to right to left, useful for some languages or effects.
Usage: `lcd.rightToLeft();`

`lcd.autoscroll()`
Enables automatic scrolling of the display when new characters are printed.
Usage: `lcd.autoscroll();`

`lcd.noAutoscroll()`
Disables automatic scrolling, returning to the default behavior.
Usage: `lcd.noAutoscroll();`

`lcd.createChar(num, data[])`
Defines a custom character to be stored in one of the 8 available CGRAM slots (0–7). Each character is defined by 8 bytes.
Usage: `lcd.createChar(0, heart);`

Arduino Demo: All LiquidCrystal Commands .

```
// Liquid Crystal Library Demonstration
#include <LiquidCrystal.h>

// Initialize the LCD with RS, EN, D4, D5, D6, D7
LiquidCrystal lcd(6, 7, 8, 9, 10, 11);

// Define a custom character (heart symbol)
byte heart[8] = {
  0b00000,
  0b01010,
  0b11111,
  0b11111,
  0b11111,
  0b01110,
  0b00100,
  0b00000
};

void setup() {
  lcd.begin(16, 2);            // Set up the LCD's number of columns and rows
  lcd.createChar(0, heart);    // Create custom character at location 0
}

void loop() {
  demoPrint();
  demoCursor();
  demoBlink();
  demoScrollLeft();
  demoScrollRight();
  demoTextDirection();
  demoAutoscroll();
  demoDisplayToggle();
  demoCustomChar();
  demoFinish();
}

// Print "Hello, World!" and delay
void demoPrint() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Hello, World!");
  delay(2000);
}

// Show and hide cursor
void demoCursor() {
  lcd.clear();
  lcd.print("Cursor Test");
  lcd.setCursor(13, 1);
  lcd.cursor();        // Show cursor
  delay(2000);
  lcd.noCursor();      // Hide cursor
}

// Show and stop blinking cursor
```

```
void demoBlink() {
  lcd.clear();
  lcd.print("Blink Cursor");
  lcd.setCursor(13, 0);
  lcd.blink();         // Enable blinking
  delay(2000);
  lcd.noBlink();       // Disable blinking
}

// Scroll text to the left
void demoScrollLeft() {
  lcd.clear();
  lcd.print("Scroll Left >>>");
  delay(1000);
  for (int i = 0; i < 5; i++) {
    lcd.scrollDisplayLeft();
    delay(300);
  }
}

// Scroll text to the right
void demoScrollRight() {
  lcd.clear();
  lcd.print("<<< Scroll Right");
  delay(1000);
  for (int i = 0; i < 5; i++) {
    lcd.scrollDisplayRight();
    delay(300);
  }
}

// Demonstrate changing text direction
void demoTextDirection() {
  lcd.clear();
  lcd.leftToRight();    // Default text direction
  lcd.setCursor(0, 0);
  lcd.print("LTR Text");
  delay(1000);

  lcd.rightToLeft();    // Reverse text direction
  lcd.setCursor(15, 1);
  lcd.print("RTL Text");
  delay(2000);

  lcd.leftToRight();    // Restore direction
}

// Demonstrate autoscroll effect
void demoAutoscroll() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.autoscroll();   // Enable autoscroll
  const char* word = "AutoScroll";
  for (int i = 0; word[i] != '\0'; i++) {
    lcd.print(word[i]);
```

```
    delay(300);
  }
  lcd.noAutoscroll(); // Disable autoscroll
  delay(1000);
}

// Turn display off and on again
void demoDisplayToggle() {
  lcd.clear();
  lcd.print("Display OFF");
  delay(1000);
  lcd.noDisplay();   // Turn off display
  delay(1500);
  lcd.display();     // Turn display back on
  delay(1000);
}

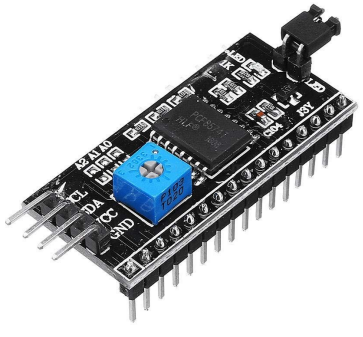
// Show custom character
void demoCustomChar() {
  lcd.clear();
  lcd.print("Custom Char: ");
  lcd.write(byte(0));  // Display the custom heart character
  delay(3000);
}

// End of demo message
void demoFinish() {
  lcd.clear();
  lcd.print("Demo Over!");
  delay(2000);
}
```

HD44780 liquid crystal displays come in various sizes. Like 16x2 in which there are two rows of display characters and 16 characters per row. Similarly there are 16x4 or even 20x4 LCD displays. All have the same interface, same commands and same connections.

I2C Liquid Crystal Displays

Unlike the HD44780 liquid crystal displays, there are other controllers, that have an I2C connectivity. This gives flexibility on circuit design, as it will consume only two GPIO lines. The I2C protocol allows connecting multiple devices on the same bus. Thus an LCD can be connected on the same two lines along with other devices like real time clock, memories etc. There are some proprietary I2C LCD displays, and you will need to read their datasheets in order to know the various commands their controller understands.



We are going to use an adapter board for HD44780 displays, that uses PCF8574 an I2C to 8 lines data expander IC. The PCF8574 is a general purpose I2C compliant IC that provides I2C connectivity and allows the sent data to be displayed on 8 data lines. An adapter board based upon this IC is available for attaching the standard HD44780 LCD display on it. Then using appropriate software librar-

ies we can send the LCD display commands over the I2C bus. The adapter board has 4 pin header for i2C Bus. Two data wires (SCL and SDA) and two power pins. The board has 16 pin header to accept the HD44780 character LCD. You can connect any HD44780 LCD to it, may it be 16x2 or 20x4. The board has a header pin marked LED. If you connect a jumper across it (which usually is the case) the LCD backlight gets power from I2C Bus supplied power. If you want to power it externally, remove the jumper and apply external power.

I2C Address of LCD

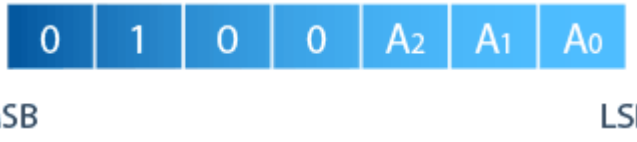
If you have multiple devices on the same I2C bus, you may need to set a different I2C address for the LCD adapter to avoid conflicts with other I2C devices.



For this purpose, the adapter comes with three solder jumpers/ pads (labeled A0, A1, and A2). For a single LCD, you can leave these untouched; however, if you connect multiple I2C devices or LCDs, you'll need to make sure each has a unique address by modifying these jumpers. You can change the address by shorting a jumper with a small blob of solder. There is a base address of the PCF8574 and these three bits add to the this base address.

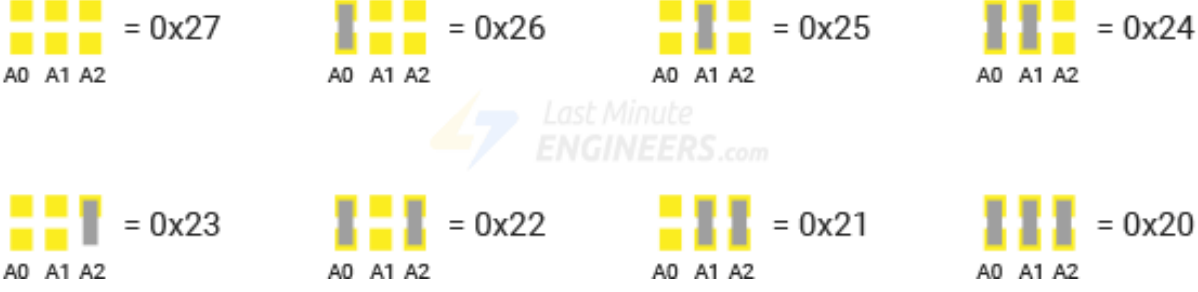
If your module has Texas Instruments PCF8574 chip it has A0 to A3 addresses mapped to least significant three bits of the address. By default these bits are set to 1s with internal pullup resistors.

> PCF8574 is manufactured by many different manufacturers and they tend to have different base address.



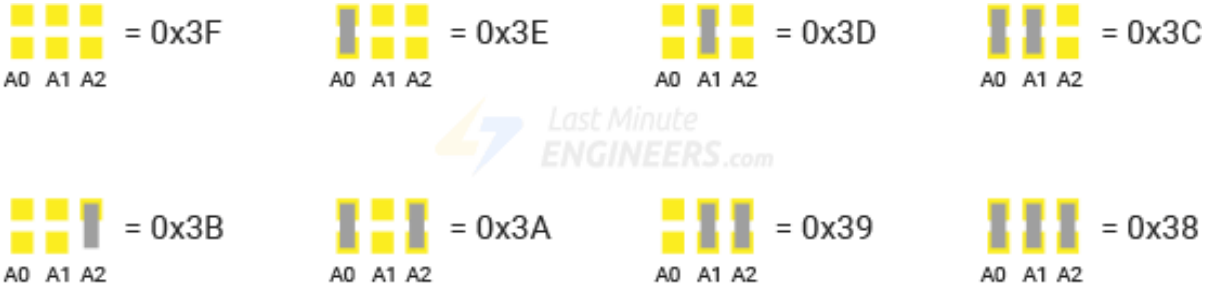
Since there are three address inputs that can be either HIGH or LOW, there are eight possible combinations (2^3 = 8) of addresses.

The default address of this board therefore is 0x27. When you short the A0 to A3 jumpers, you bring that particular bit LOW. You can set the I2C addresses like this:



If your Module has NXP manufactured PCF8574 chip the base address is 0x3F. The three A0 to A2 bits are located at the lower three bit positions of the address byte.

The display can have following addresses:



Connecting I2C LCD to Arduino

The power lines obviously will go 5V or 3.3V and GND. The SDA line will be connected to SDA and SCL to SCL of Arduino. These lines are fixed and you cannot connect to just any digital IO lines.

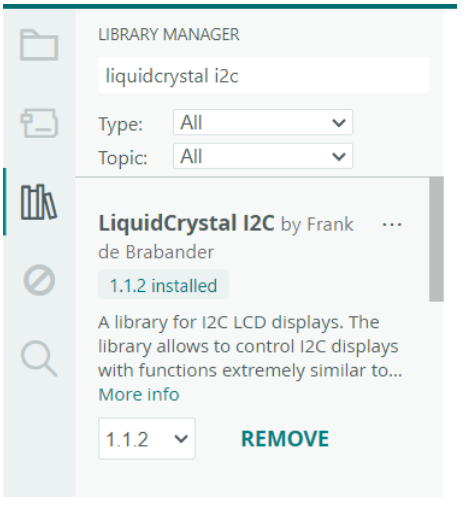


On MikroDuino board there is a specific header for I2C where you can connect it directly. It will connect the I2C lines to respective I2C bus lines.

There is a potentiometer on the module to adjust the display contrast.

The I2C LCD Library

Since I2C adapter is an external hardware, Arduino IDE does not have its supporting library to send commands to the display. You need to install appropriate library to do this job, or you have to write your own custom code to do so. Libraries make it simple for us, as someone has worked hard to write a low-level code and expose us only relevant commands that we can use to drive this display.



In the Arduino IDE open the Library manager. Search for 'LiquidCrystal I2C' a number of libraries will pop up. All tese are good, each has its own pros and conns. We recommend the one written by 'Frank de Barbender', as it has many good reviews, just click install and it will be added to your projects folder and available for projects.

Basic Arduino Program to Display "Hello world" on I2c LCD

With the hardware connected and the library installed, we can write a simple Arduino sketch to display text on the LCD. In this example, we'll print 'Hello World!' on the first line of the LCD and 'LCD Tutorial' on the second line.

```
I2C Scanner
Scanning...
I2C device found at address 0x27  !
done
```

You must know the I2C Address of your LCD. Mine is 0x27. There is an example pro-

gram in Arduino IDE under the section of wire library, that can scan the system and give you addresses of all I2C devices connected.

```
// I2C LCD Test
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27,16,2);  // set the LCD address to 0x3F for a 16 chars
and 2 line display

void setup() {
  lcd.init();
  lcd.clear();
  lcd.backlight();      // Make sure backlight is on

  // Print a message on both lines of the LCD.
  lcd.setCursor(2,0);   //Set cursor to character 2 on line 0
  lcd.print("Hello world!");

  lcd.setCursor(2,1);   //Move cursor to character 2 on line 1
  lcd.print("LCD Tutorial");
}

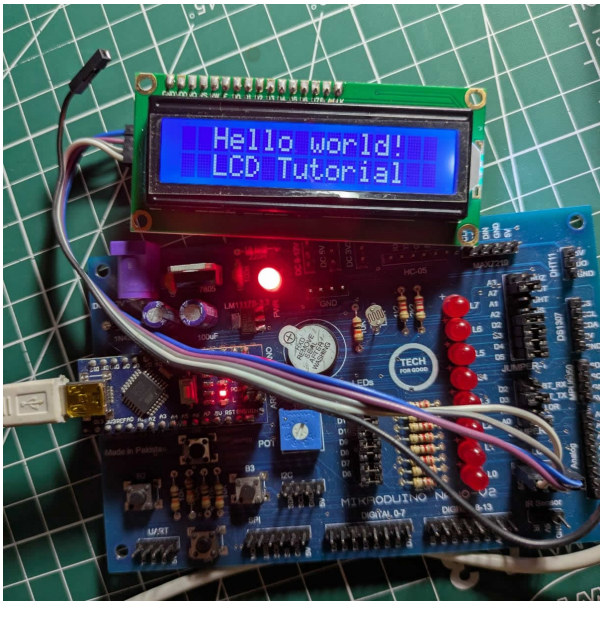
void loop() {
}
```

First you include the installed library, and initialize the LCD object.

```
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27,16,2);  // set the LCD address to 0x3F for a 16 chars
and 2 line display
```

The LiquidCrystal_I2C is the name of class that is defined in the library. Usually it bears the same name as include file. The lcd(0x27,16,2) actually creates an object by the name of lcd and calls its constructor (a function that initializes the object). You can give any name to your object. 0x27 is the i2C Address and 16,2 are the LCD type used that is 16x2. If you have 20x4 you will write 20.4 instead.



In the setup part you initialize the lcd, by using various commands that library offers.

`lcd.init()`
Description: Initializes the LCD display.
Usage:

lcd.init();

`lcd.backlight()`
Description: Turns on the LCD backlight.

`lcd.noBacklight()`
Description: Turns off the LCD backlight.

`lcd.setCursor(col, row)`
Description: Moves the cursor to the specified column and row.

`lcd.print(data)`
Description: Prints text or numbers to the current cursor position.

```
lcd.print("Hello");
```

`lcd.write(character)`
Description: Writes a single character or custom character.

```
lcd.write('A');
```

`lcd.clear()`
Description: Clears the display and resets the cursor to (0, 0).

`lcd.home()`
Description: Moves the cursor to (0, 0) without clearing the display.

`lcd.cursor()`
Description: Shows an underline cursor on the screen.

`lcd.noCursor()`
Description: Hides the underline cursor.

`lcd.blink()`
Description: Enables blinking cursor at current position.

`lcd.noBlink()`
Description: Disables the blinking cursor.

`lcd.scrollDisplayLeft()`
Description: Scrolls the entire display one character to the left.

`lcd.scrollDisplayRight()`
Description: Scrolls the entire display one character to the right.

`lcd.autoscroll()`
Description: Enables automatic scrolling when text reaches the end of the display.

`lcd.noAutoscroll()`
Description: Disables automatic scrolling.

`lcd.createChar(index, byteArray)`
Description: Defines a custom character in one of the 8 available memory slots.

```
byte smiley[8] = {
  0b00000,
  0b01010,
  0b01010,
  0b00000,
  0b10001,
  0b01110,
```

```
  0b00000,
  0b00000
};
lcd.createChar(0, smiley);
lcd.setCursor(0, 0);
lcd.write(byte(0)); // Display the custom character
```

These are the various commands that LiquidCrystal_I2C Library exposes. You can use them, just by calling them in your code.

Assignments:

1. Make a Counter using two buttons to increment or decrement its value and display on I2C LCD
2. Add a reset Functionality on long press of a button
3. Make a Password entry system. Using two buttons get user input of 4 digits, when 4th digit is entered compare it to previously stored number and turn an LED On.
4. Reaction Time Game: Turn an LED ON, and as the LED turns ON the user is supposed to press the button. Display the time in milliseconds between LED ON and Button press.
5. Make a two user reaction time game, where the user who presses the button first wins

CHAPTER 11

OLED Display I2C

In general an LED is made up of silicon with PN junction. An organic LED in contrast are certain organic compounds that emit light on electrical stimulation (just like the light insect). Such organic material are widely found in nature, like many marine animals have light emitting properties.



OLED (Organic Light Emitting Diode) displays have become increasingly popular in hobby electronics due to their vibrant colors, excellent contrast, and low power consumption. Unlike traditional LCDs that require a backlight, OLED displays emit light directly from each pixel. This results in deep blacks, sharp images, and high visibility even in low-light environments. These displays consume very little power and are therefore useful in battery operated projects. A number of interface controllers are available, but commonly an SPI and I2C are used. In this chapter we will explore the I2C based OLED displays that are easily available from hobby electronics stores. The I2C OLED displays are commonly available as graphic displays, where we can control individual pixel to turn On or Off. The come in resolution 128x64 pixel resolution. The physical size however can be 0.96” or 1.3” in diameter. The number of pixels are same however. These displays have only one color for the pixels, either blue or white. Some have different zones, like upper zone has amber color and lower zone has white color. These displays are driven by an I2C compliant controller SSD1306 that is integrated on the module. An even smaller version is also available, that has a resolution of 128x32 pixels having the same driving interface.



Connecting OLED Display

The connection header has 4 pins, two for power and two for I2C bus. You can power it from 3.3V or 5V. The SDA and SCL lines are connected to the I2C specific pins of Arduino. The MikroDuino Nano board has a header for I2C LCD, you can connect the OLED display to the same connector, or there is also a separate I2C header, it can be connected there.

Interfacing I2C OLED Displays with Arduino

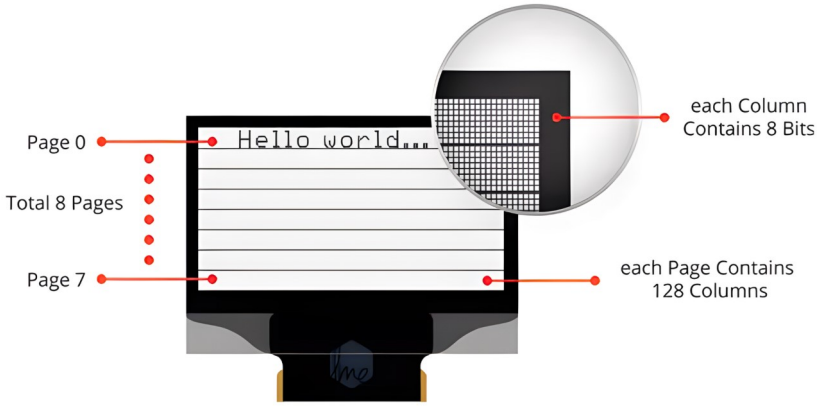
Tired of using character LCD displays in your Arduino projects over and over? Well! They are, in fact, a thing of the past. Enter the fantastic OLED (Organic Light-Emitting Diode)

displays! They're extremely light, almost paper-thin, theoretically flexible, and produce a brighter, crisper image. These displays come in two types of interfaces, an I2C and SPI. The SPI is in general faster than I2C, but requires little more IO lines. The I2C on the other hand takes only two lines, and these two lines can be shared across other i2C devices.

The SSD1306 controller operates at 1.65V to 3.3V, while the OLED panel itself requires a 7V to 15V supply voltage. All of these various power requirements are fulfilled by internal charge pump circuitry on the module. This makes it possible to connect the display to an Arduino or any other 5V logic microcontroller without requiring a logic level converter.

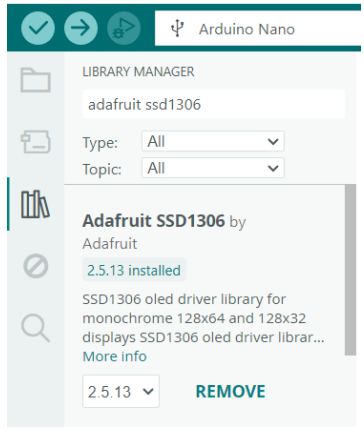
OLED Memory Map

Although knowledge of internal structure of the controller is not necessary for interfacing it, yet I believe the more you know, better you are equipped. Each pixel on display is controlled by a bit in the memory of controller. The SSD1306 has 1KB of graphics display memory or GDDRAM. The memory is arranged in 8 pages (0-7) and each page has 128 col-



umns. Each column has one byte consisting of 8 bits.

OLED Display Library



The SSD1306 controller on this display is quite flexible, but at the same time has a complex interface. It needs extensive knowledge of the memory map and other control registers to get it working. Fortunately the Adafruit SSD1306 library abstracts all the inner details, and exposes a set of easy to use functions, that make it quite simple to use this display.

Navigate to library manager in Arduino IDE and search for 'Adafruit SSD1306' click install library. This library needs an accompanying library 'Adafruit GFX' library that has supporting functions. Install this too. The Adafruit SSD1306 library provides hardware control of the display, while GFX library provides commands for drawing various graphics, like lines, rectangles or circles etc.

Display Text on OLED

Now lets begin the exciting part, to write some text on the display. Just like any other startup programs, it will be a hello world program.

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// OLED display size (width, height)
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

// I2C address for SSD1306 (common one is 0x3C)
#define OLED_ADDRESS 0x3C

// Create display object
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);  // -1 means
no reset pin

void setup() {
  display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDRESS);
    // Clear buffer
   display.clearDisplay();

  // Set text size and color
  display.setTextSize(1);      // Size: 1 = small, 2 = medium, 3 = large
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 10);    // (x, y)

  // Print message
  display.println("Hello, World!");

  // Show on screen
  display.display();
}

void loop() {
  // Nothing to repeat
}
```



Library Inclusions

So the first task is to include the appropriate libraries. Not only the Adafruit libraries that are meant for deriving the display, but also graphics library that they have made to draw text, fonts, and other graphics. Since the display has to use I2C communication, the Adafruit libraries depend upon the Arduino supplied ‘wire.h’ library.

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

Creating the Display Object

```
// Create display object
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);  // -1 means
no reset pin
```

Next we have to create an object, based upon the class defined in Adafruit library. Here we have created an object named 'display' you can give it any name you want. We need to pass this object 4 parameters, when initializing it. First two are the display dimensions, so these are 128 and 64. We have defined them as macros in program, so that it becomes more readable and adaptable. Third parameter is the address (the & indicates address) of the structure to hold the I2C communication data. It is defined in the wire.h library. The last parameter is the reset pin. Most modules do not have this reset pin, it is managed internally by the controller, so we provide -1 for it.

Initializing The Display

Next we have to initialize the display. This is done once in the setup part, and needs to call a .begin function in the display object. It takes two parameters, the power mode, which tells the library how the module will generate 15V supply for the display, and the I2C Address. SSD1306_SWITCHCAPVCC is a constant defined in the Adafruit library to tell the onboard voltage booster. The I2C address of our display is 0x3C. This is by far used by most OLED display, Some other displays like 128x32 use 0x3D. Use the I2C scanner program to see your address.

```
display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDRESS);
```

This function returns 0 or 1, depending upon if it could initialize the display or not. It can be used to give warning about the status of display device.

Clearing the Display

display.clearDisplay();

This function clears the display buffer. It does not have immediate effect. The display screen actually clears when display. Display() function is called.

Setting Text Properties

```
// Set text size and color
  display.setTextSize(1);      // Size: 1 = small, 2 = medium, 3 = large
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 10);    // (x, y)
```

These functions define how your text appears. setTextSize() sets the font size (1 = normal, 2 = double size, etc.). setTextColor() sets the text color—usually SSD1306_WHITE. The OLED is monochrome, so there are no colors other than white and black. setCursor(x, y) sets the position where text will begin (top-left corner is 0,0).

Printing and Displaying Text

```
  display.println("Hello, World!"); / /Print message
  display.display(); // Show on screen
```

println() works just like in the Serial Monitor—it prints text and moves the cursor to the next line. The display.display() command is essential—it sends all buffered graphics to the

screen. Without this, nothing will show up.

Advanced OLED Text Features

Now lets take a step forward to the **next level** with the OLED SSD1306 display by exploring advanced text features such as **larger fonts**, **scrolling text**, and **simple text animation**. These techniques not only make your project look cooler but also help in displaying more information in creative ways.

Larger Fonts

You can always increase the font size, using .setTextSize() function. Size 1 is normal, 2 and 3 are larger. The size and font are defined in the Adafruit GFX library.

Scrolling Text

The SSD1306 controller has built-in commands to scroll the display left, right, or diagonally. The Adafruit library includes simple functions for that, such as:

```
startscrollright(0x00, 0x0F)

startscrollleft(0x00, 0x0F)

stopscroll()
```

This is great for marquee-style displays or when your message is longer than the screen width.

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_ADDRESS 0x3C

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

void setup() {
  display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDRESS);
  display.clearDisplay();

  // Display large text
  display.setTextSize(2);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(10, 10);
  display.println("BIG!");
  display.display();
  delay(2000);

  // Scroll demo
  display.clearDisplay();
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("Scrolling right...");
  display.display();
  display.startscrollright(0x00, 0x0F);
```

```
  delay(3000);
  display.stopscroll();

  // Animated text bounce
  String msg = "Bouncing Text!";
  int x = 0;
  int dx = 2;

  for (int i = 0; i < 100; i++) {
    display.clearDisplay();
    display.setCursor(x, 30);
    display.setTextSize(1);
    display.println(msg);
    display.display();
    delay(100);
    x += dx;
    if (x < 0 || x > (SCREEN_WIDTH - msg.length() * 6)) dx = -dx;
  }
}

void loop() {
  // Nothing in loop
}
```

Text Animation

You can create custom animations by manually updating the text position in a loop and using `clearDisplay()` and `setCursor()` to simulate movement. It's simple and flexible for any text motion, like bouncing text.

Assignments:

1. Make a button counter on OLED display to show the number of times a button is pressed
2. Read Temperature from an LM35 temperature sensor and display it on OLED
3. Make a Button counter, one button increases the count and other decreases it. Long press resets the counter. Limit the count between 0 and 20
4. Make a stopwatch,

StopWatch

Lets write a program to implement a simple stopwatch, it uses millis() function to get the system clock in milliseconds, and uses two buttons S3 and S4)

```
//Simple stopwatch
//S3 to start and stop
//S4 to reset
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
```

```
#define SCREEN_HEIGHT 64
#define OLED_ADDRESS 0x3C

#define BUTTON_START 3
#define BUTTON_RESET 4

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

bool running = false;
unsigned long startTime = 0;
unsigned long elapsedTime = 0;

bool lastStartState = HIGH;
bool lastResetState = HIGH;

char timeString[16];  // Buffer for formatted time

void setup() {
  pinMode(BUTTON_START, INPUT_PULLUP);
  pinMode(BUTTON_RESET, INPUT_PULLUP);

  display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDRESS);
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(2);
  updateDisplay();
}

void loop() {
  // Handle start/stop button
  bool currentStartState = digitalRead(BUTTON_START);
  if (currentStartState == LOW && lastStartState == HIGH) {
    running = !running;
    if (running) {
      startTime = millis() - elapsedTime;
    } else {
      elapsedTime = millis() - startTime;
    }
    delay(200);  // Debounce delay
  }
  lastStartState = currentStartState;

  // Handle reset button
  bool currentResetState = digitalRead(BUTTON_RESET);
  if (currentResetState == LOW && lastResetState == HIGH) {
    running = false;
    elapsedTime = 0;
    updateDisplay();
    delay(200);  // Debounce delay
  }
  lastResetState = currentResetState;

  // Update elapsed time
  if (running) {
```

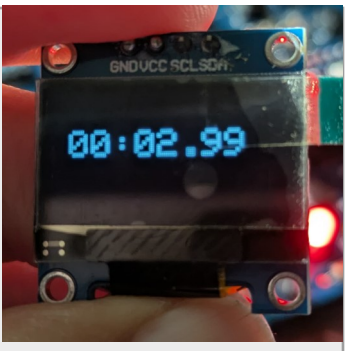
```
    elapsedTime = millis() - startTime;
  }

  updateDisplay();
  delay(50);  // Smooth refresh rate
}

void updateDisplay() {
  unsigned long centi = (elapsedTime / 10) % 100;
  unsigned long seconds = (elapsedTime / 1000) % 60;
  unsigned long minutes = (elapsedTime / 60000);

  sprintf(timeString, "%02lu:%02lu.%02lu", minutes, seconds, centi);

  display.clearDisplay();
  display.setCursor(10, 20);
  display.print(timeString);
```



In this program we have use a command called sprintf(), this is equivalent of C command, printf that formats the print data. Sprint(), formats the print data into a string and then we print that string on display.

Displaying a Menu System

Menu systems are commonly required as user input or user interface. Here is an example code to implement a simple menu of text. Use S3 button to navigate among menu items and S4 to select an item.

```
//Making a Menu System using S3 and S4 Buttons
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_ADDRESS 0x3C

#define BUTTON_DOWN 3
#define BUTTON_SELECT 4

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

const char* menuItems[] = { "Start", "Settings", "About", "Exit" };
const int menuLength = sizeof(menuItems) / sizeof(menuItems[0]);
int currentSelection = 0;

bool lastDownState = HIGH;
bool lastSelectState = HIGH;

void setup() {
  pinMode(BUTTON_DOWN, INPUT_PULLUP);
  pinMode(BUTTON_SELECT, INPUT_PULLUP);

  display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDRESS);
```

```
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.clearDisplay();
  display.display();
  showMenu();
}

void loop() {
  bool downState = digitalRead(BUTTON_DOWN);
  bool selectState = digitalRead(BUTTON_SELECT);

  if (downState == LOW && lastDownState == HIGH) {
    currentSelection = (currentSelection + 1) % menuLength;
    showMenu();
    delay(200);  // debounce
  }

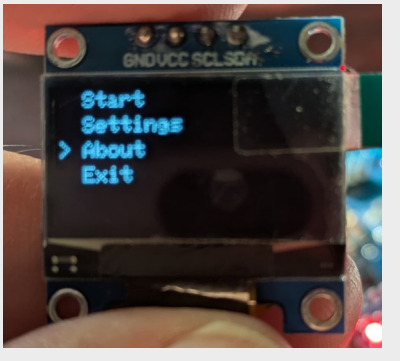
  if (selectState == LOW && lastSelectState == HIGH) {
    handleSelection(currentSelection);
    delay(300);  // debounce
  }

  lastDownState = downState;
  lastSelectState = selectState;
}

void showMenu() {
  display.clearDisplay();
  display.setTextSize(1);
   for (int i = 0; i < menuLength; i++) {
    if (i == currentSelection) {
      display.setCursor(0, i * 12);
      display.print("> ");
    } else {
      display.setCursor(0, i * 12);
      display.print("  ");
    }
    display.print(menuItems[i]);
  }
  display.display();
}

void handleSelection(int selection) {
  display.clearDisplay();
  display.setCursor(0, 20);
  display.setTextSize(1);
  display.print("You selected:");
  display.setCursor(0, 35);
  display.setTextSize(2);
  display.print(menuItems[selection]);
  display.display();

  delay(1500);  // Show the selection before returning
  showMenu();   // Go back to menu
}
```



Assignments:

1. Make menu system to toggle 4 LEDs

Graphics on OLED with Adafruit Library

The **Adafruit_SSD1306** library, combined with the **Adafruit_GFX** library, turns the OLED into a mini graphical canvas. While the SSD1306 handles the communication with the display hardware, it's the **GFX library** that provides a rich set of graphic primitives like lines, rectangles, circles, and even bitmaps. This allows you to create dynamic visual interface, display icons, shapes and animations. You can even draw graphics and make games.

Common Graphic Functions

Here are a few graphical functions provided by Adafruit GFX:

Function	Description
`drawPixel(x, y, color)`	Sets a single pixel
`drawLine(x0,y0,x1,y1)`	Draws a straight line
`drawRect(x,y,w,h)`	Draws rectangle outline
`fillRect(x,y,w,h)`	Draws filled rectangle
`drawCircle(x,y,r)`	Draws circle outline
`fillCircle(x,y,r)`	Draws filled circle
`drawTriangle(x1,y1,x2,y2,x3,y3)`	Draws triangle
`drawBitmap(x, y, bitmap, w, h, color)`	Draws custom image from array

```
// Draw Shapes on OLED
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_ADDR 0x3C

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

void setup() {
  display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR);
  display.clearDisplay();

  // Rectangle
  display.drawRect(10, 10, 50, 30, SSD1306_WHITE);
```

```
  // Circle
  display.drawCircle(90, 20, 15, SSD1306_WHITE);

  // Line
  display.drawLine(0, 0, 127, 63, SSD1306_WHITE);

  display.display();
}

void loop() {
  // No animation here
}
```

Bouncing Ball

Here is a program that demonstrates an animation of a ball bouncing when it comes in contact with display edge.

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// Display dimensions
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1  // Reset pin not used
#define OLED_ADDR 0x3C // I2C address

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// Ball properties
int ballX = 10;
int ballY = 10;
int ballRadius = 4;
int dx = 2;
int dy = 1;

void setup() {
  // Initialize the display
  if (!display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
    Serial.println(F("SSD1306 allocation failed"));
    for (;;); // Don't proceed, loop forever
  }

  display.clearDisplay();
  display.display();
}

void loop() {
  // Clear previous frame
  display.clearDisplay();

  // Draw the ball
  display.fillCircle(ballX, ballY, ballRadius, SSD1306_WHITE);
```

```
  // Display the ball
  display.display();

  // Update position
  ballX += dx;
  ballY += dy;

  // Check boundaries and reverse direction
  if (ballX - ballRadius <= 0 || ballX + ballRadius >= SCREEN_WIDTH) {
    dx = -dx;
  }
  if (ballY - ballRadius <= 0 || ballY + ballRadius >= SCREEN_HEIGHT) {
    dy = -dy;
  }

  delay(20); // Adjust speed
}
```

This program creates a small ball that moves around on the OLED screen and bounces off the edges.

Ball Position and Movement

We start with the ball's current position (`ballX`, `ballY`) and how much it moves each time (`dx`, `dy`). These are like the ball's speed and direction in the x and y directions.

Drawing the Ball

In each round of the loop:

The screen is **cleared** so we can draw the ball in a new place. A **filled circle** is drawn at the new position of the ball using `fillCircle()`.We then show this new drawing using `display.display()`.

Moving the Ball

After drawing, we **update the position** of the ball:

`ballX += dx;` moves the ball horizontally.

`ballY += dy;` moves the ball vertically.

Bouncing from the Edges

The ball checks if it's touching any side of the screen:

If it's at the **left or right edge**, it **reverses its x-direction** (`dx = -dx`).

If it's at the **top or bottom edge**, it **reverses its y-direction** (`dy = -dy`).

This makes the ball appear to bounce back whenever it hits a wall.

Delay

We add a small `delay(20);` to **slow down the animation**, so it's smooth and visible to our eyes.

Introduction to the Snake Game on OLED

The **Snake Game** is a classic video game where the player controls a growing line (the snake), trying to eat "food" items while avoiding collisions with the walls or its own body. This game is simple yet addictive and is a fantastic way to learn **game programming logic** using **Arduino**, an **OLED display**, and **push buttons**.

By building this game, you learn:

- Handling user input via buttons
- Drawing graphics on OLED
- Implementing real-time movement
- Detecting collisions
- Using arrays to store game objects (the snake's body)

How the Snake Game is Implemented

1. Display & Grid System

The OLED screen (128x64 pixels) is divided into small square blocks (e.g., 4x4 pixels). Each block represents one unit of the game space. The snake and food are drawn using these blocks.

2. Snake Representation

The snake's body is stored in two arrays:

```
int snakeX[MAX_LENGTH];

int snakeY[MAX_LENGTH];
```

Each index of these arrays represents a segment of the snake (head, body, tail).

3. Movement

- A direction (dx, dy) is used to move the snake.
- On each loop, the snake moves by shifting its body positions: each segment takes the position of the one in front of it.
- The head moves forward based on the current direction.

4. Food & Growth

- A food block appears randomly on the screen.
- When the snake's head touches the food, the snake grows by increasing the length and a new food is placed.

5. Collision Detection

- If the snake's head touches the wall (`x < 0` or `x >= screen width`) or touches its own body, the game ends.
- This is checked by comparing the head's position with the rest of the body segments.

6. User Input

- Four buttons are used to control the direction.
- We check that the snake does not move directly back into itself (i.e., no reverse direction).

7. Drawing to OLED

Each frame clears the display, draws the updated snake and food positions using `fillRect()`, and then refreshes the screen using `display.display()`.

```
//Snake Game on OLED
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Wire.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define OLED_ADDR 0x3C

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// Button pins
#define UP_BUTTON 2
#define LEFT_BUTTON 3
#define RIGHT_BUTTON 4
#define DOWN_BUTTON 5

// Snake variables
#define BLOCK_SIZE 4
#define MAX_LENGTH 64

int snakeX[MAX_LENGTH];
int snakeY[MAX_LENGTH];
int length = 3;
int dx = BLOCK_SIZE;
int dy = 0;

int foodX, foodY;
bool gameOver = false;

void spawnFood() {
  foodX = (random(SCREEN_WIDTH / BLOCK_SIZE)) * BLOCK_SIZE;
  foodY = (random(SCREEN_HEIGHT / BLOCK_SIZE)) * BLOCK_SIZE;
}
```

```
void setup() {
  pinMode(UP_BUTTON, INPUT_PULLUP);
  pinMode(LEFT_BUTTON, INPUT_PULLUP);
  pinMode(RIGHT_BUTTON, INPUT_PULLUP);
  pinMode(DOWN_BUTTON, INPUT_PULLUP);

  display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR);
  display.clearDisplay();
  display.display();

  randomSeed(analogRead(0));

  // Initialize snake position
  for (int i = 0; i < length; i++) {
    snakeX[i] = 40 - i * BLOCK_SIZE;
    snakeY[i] = 20;
  }

  spawnFood();
}

void loop() {
  if (gameOver) {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(30, 30);
    display.print("Game Over!");
    display.display();
    delay(2000);
    return;
  }

  // Handle input
  if (digitalRead(UP_BUTTON) == LOW && dy == 0) { dx = 0; dy = -BLOCK_SIZE; }
  if (digitalRead(DOWN_BUTTON) == LOW && dy == 0) { dx = 0; dy = BLOCK_SIZE; }
  if (digitalRead(LEFT_BUTTON) == LOW && dx == 0) { dx = -BLOCK_SIZE; dy = 0; }
  if (digitalRead(RIGHT_BUTTON) == LOW && dx == 0) { dx = BLOCK_SIZE; dy = 0; }

  // Move snake
  for (int i = length - 1; i > 0; i--) {
    snakeX[i] = snakeX[i - 1];
    snakeY[i] = snakeY[i - 1];
  }
  snakeX[0] += dx;
  snakeY[0] += dy;

  // Check for collision with self or wall
  if (snakeX[0] < 0 || snakeX[0] >= SCREEN_WIDTH || snakeY[0] < 0 || snakeY[0]
>= SCREEN_HEIGHT) {
    gameOver = true;
  }
```

```
  for (int i = 1; i < length; i++) {
    if (snakeX[0] == snakeX[i] && snakeY[0] == snakeY[i]) {
      gameOver = true;
    }
  }

  // Check for eating food
  if (snakeX[0] == foodX && snakeY[0] == foodY) {
    if (length < MAX_LENGTH) length++;
    spawnFood();
  }

  // Draw everything
  display.clearDisplay();

  // Draw snake
  for (int i = 0; i < length; i++) {
    display.fillRect(snakeX[i], snakeY[i], BLOCK_SIZE, BLOCK_SIZE,
SSD1306_WHITE);
  }

  // Draw food
  display.fillRect(foodX, foodY, BLOCK_SIZE, BLOCK_SIZE, SSD1306_WHITE);

  display.display();
  delay(150); // Control speed
}
```

Dodge the Blocks – A Simple OLED Game

Dodge the Blocks is a beginner-friendly arcade-style game built using an **OLED display** and two **push buttons**. The player controls a small block (representing a character) positioned at the **bottom of the screen** and must move left or right to **avoid falling obstacles**. If the player is hit by a block, the game ends and displays a **"Game Over"** message before restarting.

Programming Techniques Used

1. Graphics with OLED

We use the `Adafruit_SSD1306` and `Adafruit_GFX` libraries to draw shapes (like rectangles) on the OLED screen. The player and the falling blocks are drawn using `fillRect(x, y, width, height)`.

2. Player Movement

Two push buttons (connected to digital pins 3 and 4) are used to move the player block **left** and **right**. Button states are checked using `digitalRead()` inside the `loop()` function, and the player's position is updated accordingly.

3. Obstacle Logic

An obstacle (block) starts from a random `x` position at the **top of the screen** and moves downward at a constant speed. When it reaches the bottom or collides with the player, the game either **resets the obstacle** or **ends the game**.

4. Collision Detection

Collision is checked by comparing the coordinates of the **falling block** and the **player block**. If their rectangular areas overlap, a collision is detected and the game ends.

5. Game Loop and Frame Refresh

The game logic runs inside the `loop()` function, which:

Handles input

Updates object positions

Checks for collisions

```
//Dodging Game
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Wire.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define OLED_ADDR 0x3C

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// Button pins
#define LEFT_BUTTON 3
#define RIGHT_BUTTON 4

// Player variables
#define PLAYER_WIDTH 10
#define PLAYER_HEIGHT 4
int playerX = SCREEN_WIDTH / 2 - PLAYER_WIDTH / 2;
const int playerY = SCREEN_HEIGHT - PLAYER_HEIGHT;

// Obstacle variables
int obstacleX;
int obstacleY = 0;
const int obstacleSize = 8;
int obstacleSpeed = 2;

bool gameOver = false;

void spawnObstacle() {
  obstacleX = random(0, SCREEN_WIDTH - obstacleSize);
  obstacleY = 0;
}

void setup() {
```

Clears and redraws the display

```
  pinMode(LEFT_BUTTON, INPUT_PULLUP);
  pinMode(RIGHT_BUTTON, INPUT_PULLUP);

  display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR);
  display.clearDisplay();
  display.display();

  randomSeed(analogRead(0));
  spawnObstacle();
}

void loop() {
  if (gameOver) {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(30, 30);
    display.print("Game Over!");
    display.display();
    delay(3000);
    // Restart game
    playerX = SCREEN_WIDTH / 2 - PLAYER_WIDTH / 2;
    gameOver = false;
    spawnObstacle();
  }

  // Handle input
  if (digitalRead(LEFT_BUTTON) == LOW && playerX > 0) {
    playerX -= 3;
  }
  if (digitalRead(RIGHT_BUTTON) == LOW && playerX < SCREEN_WIDTH - PLAYER_WIDTH)
{
    playerX += 3;
  }

  // Move obstacle
  obstacleY += obstacleSpeed;

  // Check collision
  if (obstacleY + obstacleSize >= playerY &&
      obstacleX + obstacleSize > playerX &&
      obstacleX < playerX + PLAYER_WIDTH) {
    gameOver = true;
  }

  // Respawn obstacle if it goes off screen
  if (obstacleY > SCREEN_HEIGHT) {
    spawnObstacle();
  }

  // Draw everything
  display.clearDisplay();

  // Draw player
```

```
  display.fillRect(playerX, playerY, PLAYER_WIDTH, PLAYER_HEIGHT,
SSD1306_WHITE);

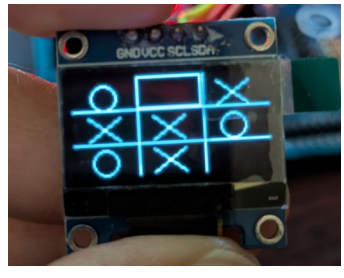
  // Draw obstacle
  display.fillRect(obstacleX, obstacleY, obstacleSize, obstacleSize,
SSD1306_WHITE);

  display.display();
  delay(50); // controls game speed
}
```

This is repeated at regular intervals (using `delay(50)`) to control the game speed.

Tic Tac Toe on OLED using 4 Buttons

Tic Tac Toe is a simple two-player game played on a 3×3 grid. The players take turns placing either an "X" or an "O" in empty squares, and the first to get three of their marks in a row—horizontally, vertically, or diagonally—wins the game. If all 9 squares are filled without a winner, the game ends in a draw. This game is ideal for learning programming concepts like:



Arrays
Decision-making (if/else)
Loops
Button input handling
Simple graphics
Game logic and state management

How the Code Works: Step-by-Step Notes

1. Display Setup

The game uses a 128×64 OLED display with the Adafruit SSD1306 library to draw:

- The 3×3 grid
- The X and O marks
- A blinking or outlined cursor to show the current selection
- Winning or draw messages

2. Button Configuration

We use 4 buttons connected to digital pins:

- **D2** for Up
- **D3** for Left
- **D4** for Right
- **D5** for Down

Players move the cursor with these buttons and perform a "long press" on any button to **place their mark** in the current cell.

3. Game Board and Cursor Logic

The board is stored in a 2D array:

```
char board[3][3];
```

The cursor is controlled with:

```
int cursorX = 0;

int cursorY = 0;
```

Movement is limited within the 3×3 grid using boundary checks.

4. Drawing the Game

The `drawGrid()` function:

- Draws vertical and horizontal lines to form the grid
- Displays current X or O symbols
- Draws a rectangular cursor box over the selected cell

5. Placing a Move

If a long press is detected and the cell is empty, the current player's symbol (X or O) is placed using `markCell()`.

6. Switching Players

After a valid move, the code switches between players:

7. Checking for a Winner

After every move, the program checks:

- Rows
- Columns
- Both diagonals Using the `checkWinner()` function.

8. Detecting a Draw

If all 9 cells are filled without a winner, the `isDraw()` function returns true and displays a draw message.

9. Resetting the Game

After a win or draw, the screen shows the result for 3 seconds, then the game is automatically reset.

```
//Tic Tac Toe game
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Wire.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define OLED_ADDR 0x3C

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// Button pins
#define UP_BUTTON 2
#define LEFT_BUTTON 3
#define RIGHT_BUTTON 4
#define DOWN_BUTTON 5

// Game state
char board[3][3];
int cursorX = 0;
int cursorY = 0;
char currentPlayer = 'X';
bool gameOver = false;

// Button timing
unsigned long lastPressTime = 0;

void drawGrid() {
  display.clearDisplay();

  // Draw vertical lines
  display.drawLine(42, 0, 42, 64, SSD1306_WHITE);
  display.drawLine(85, 0, 85, 64, SSD1306_WHITE);

  // Draw horizontal lines
  display.drawLine(0, 21, 128, 21, SSD1306_WHITE);
  display.drawLine(0, 43, 128, 43, SSD1306_WHITE);

  // Draw Xs and Os
  for (int y = 0; y < 3; y++) {
    for (int x = 0; x < 3; x++) {
      int cellX = x * 42 + 10;
      int cellY = y * 21 + 5;

      if (board[y][x] == 'X') {
        display.drawLine(cellX, cellY, cellX + 20, cellY + 15, SSD1306_WHITE);
        display.drawLine(cellX + 20, cellY, cellX, cellY + 15, SSD1306_WHITE);
      } else if (board[y][x] == 'O') {
        display.drawCircle(cellX + 10, cellY + 8, 8, SSD1306_WHITE);
```

```
      }
    }
  }

  // Draw cursor
  int cursorPixelX = cursorX * 42;
  int cursorPixelY = cursorY * 21;
  display.drawRect(cursorPixelX, cursorPixelY, 42, 21, SSD1306_WHITE);

  display.display();
}

bool isButtonPressed(int pin) {
  return digitalRead(pin) == LOW;
}

bool isCellEmpty(int x, int y) {
  return board[y][x] == ' ';
}

void markCell(int x, int y, char player) {
  board[y][x] = player;
}

char checkWinner() {
  for (int i = 0; i < 3; i++) {
    // Check rows and columns
    if (board[i][0] != ' ' &&
        board[i][0] == board[i][1] &&
        board[i][1] == board[i][2])
      return board[i][0];

    if (board[0][i] != ' ' &&
        board[0][i] == board[1][i] &&
        board[1][i] == board[2][i])
      return board[0][i];
  }

  // Diagonals
  if (board[0][0] != ' ' &&
      board[0][0] == board[1][1] &&
      board[1][1] == board[2][2])
    return board[0][0];

  if (board[0][2] != ' ' &&
      board[0][2] == board[1][1] &&
      board[1][1] == board[2][0])
    return board[0][2];

  return ' ';
}

bool isDraw() {
```

```
  for (int y = 0; y < 3; y++)
    for (int x = 0; x < 3; x++)
      if (board[y][x] == ' ') return false;
  return true;
}

void displayEnd(char winner) {
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(20, 20);
  if (winner != ' ')
    display.print(String("Winner: ") + winner);
  else
    display.print("It's a draw!");
  display.display();
  delay(3000);
}

void resetGame() {
  for (int y = 0; y < 3; y++)
    for (int x = 0; x < 3; x++)
      board[y][x] = ' ';
  cursorX = 0;
  cursorY = 0;
  currentPlayer = 'X';
  gameOver = false;
  drawGrid();
}

void setup() {
  pinMode(UP_BUTTON, INPUT_PULLUP);
  pinMode(LEFT_BUTTON, INPUT_PULLUP);
  pinMode(RIGHT_BUTTON, INPUT_PULLUP);
  pinMode(DOWN_BUTTON, INPUT_PULLUP);

  display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR);
  display.clearDisplay();
  display.display();

  resetGame();
}

void loop() {
  if (gameOver) {
    delay(1000);
    resetGame();
    return;
  }

  if (isButtonPressed(LEFT_BUTTON)) {
    if (cursorX > 0) cursorX--;
    delay(200);
  }
```

```
  if (isButtonPressed(RIGHT_BUTTON)) {
    if (cursorX < 2) cursorX++;
    delay(200);
  }

  if (isButtonPressed(UP_BUTTON)) {
    if (cursorY > 0) cursorY--;
    delay(200);
  }

  if (isButtonPressed(DOWN_BUTTON)) {
    if (cursorY < 2) cursorY++;
    delay(200);
  }

  // Long press any button to mark
  if (isCellEmpty(cursorX, cursorY)) {
    if (millis() - lastPressTime > 800 && (
        isButtonPressed(LEFT_BUTTON) ||
        isButtonPressed(RIGHT_BUTTON) ||
        isButtonPressed(UP_BUTTON) ||
        isButtonPressed(DOWN_BUTTON))) {

      markCell(cursorX, cursorY, currentPlayer);
      char winner = checkWinner();
      if (winner != ' ' || isDraw()) {
        displayEnd(winner);
        gameOver = true;
        return;
      }
      currentPlayer = (currentPlayer == 'X') ? 'O' : 'X';
    } else if (
        isButtonPressed(LEFT_BUTTON) ||
        isButtonPressed(RIGHT_BUTTON) ||
        isButtonPressed(UP_BUTTON) ||
        isButtonPressed(DOWN_BUTTON)) {
      lastPressTime = millis();
    }
  }

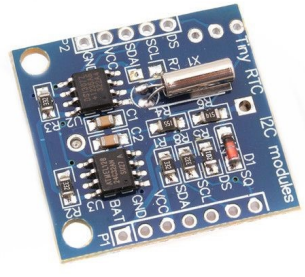
  drawGrid();
}
```

CHAPTER 12

Real Time Clock | I2C

Real-time clocks (RTCs) are essential components in many embedded systems and microcontroller-based projects that require accurate timekeeping. Unlike the internal timers of microcontrollers, which lose track of time when the device is powered off, RTC modules maintain precise time and date information using a dedicated oscillator and a small backup battery. This makes them ideal for applications such as data logging, alarms, event scheduling, access control systems, and time-stamped displays.

In the hobbyist market, several RTC modules are available, each offering different features and communication protocols. Popular RTC chips include the DS1307, DS3231, and PCF85263, among others. The DS1307 is one of the most commonly used RTC modules due to its affordability, I2C communication interface, and compatibility with a wide range of development platforms including the Arduino. The DS1307 maintains time in seconds, minutes, hours, day, date, month, and year format and can operate in either 12-hour or 24-hour modes. It features an onboard 32.768 kHz crystal oscillator and a backup battery socket, allowing the module to keep track of time even when the main power is removed. In this chapter, we will explore how to interface the DS1307 with the Arduino Nano, display the current time and date on an I2C OLED screen, and use push buttons connected to digital pins D2, D3, D4, and D5 to interact with the system. As usual we will use our MikroDuino Nano as a platform to connect DS1307 module, as well as OLED display for these projects. You can use your own setup for this.



The DS1307 Module also contains, an I2C EEPROM chip, which is separate from DS1307 and can be used separately for EEPROM functionality. Additionally this module allows you to solder a digital temperature sensor DS18B20 that can also be individually accessed through its header pins.

DS1307 RTC Chip

At the center of this module is DS1307, a chip by maxim electronics. It's a low cost, highly accurate real time clock and calendar chip, having an I2C interface. The DS1307 can keep track of seconds, minutes, hours, days, dates, months, and years. It's smart enough to know how many days are in each month and even adjusts automatically for months with fewer than 31 days. It also handles leap years correctly—though only up to the year 2100.

You can choose to set the time in either 12-hour format (with AM and PM) or 24-hour for-

mat, depending on your preference.

There's also a special pin on the chip called SQW (short for square wave). You can set this pin to produce a steady pulse signal at one of four frequencies: 1Hz, 4kHz, 8kHz, or 32kHz. This can be useful in projects that need a regular timing signal.

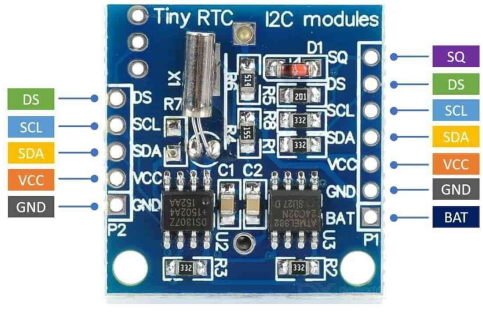
> The DS1307 uses an external 32kHz crystal to keep track of time. This crystal is what helps the chip measure each passing second. However, there's a small issue: the crystal's frequency can change slightly when the temperature around it changes.
> Even though the change is very tiny, over time, it can cause the clock to slowly become less accurate. In fact, the time can drift by about five minutes each month.
>
> This might sound like a big problem, but for most basic projects, it's not a big deal. The DS1307 is still a reliable and popular real-time clock for many everyday uses where super precise timing isn't critical.

The DS1307 chip includes a battery backup feature that helps keep the clock running even if the main power gets disconnected. On the back of the module, there's a battery holder designed to fit a 20mm 3V lithium coin cell battery. Inside the chip, there's a built-in smart power-sensing circuit that constantly checks whether the main power is on. If this circuit detects that the main power has been lost, it automatically switches to the backup battery.

In addition to timekeeping, the DS1307 includes 56 bytes of battery-backed SRAM, which can be used to store small amounts of user data that must be preserved across power cycles. Thanks to its low power consumption, easy interfacing through I2C, and wide availability as ready-to-use modules, the DS1307 remains a favorite choice for hobbyists and educators when building clocks, alarms, timers, and time-stamped data logging systems.

Connecting to Arduino

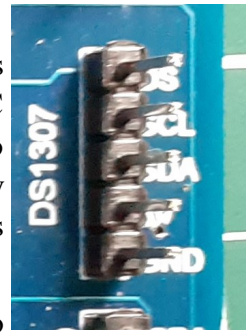
DS1307 uses I2C interface, and can share the same I2C lines (SDA and SCL) with other I2C devices. The DS1307 has an I2C address of 0x68.



The module has two sets of pins, a 5 pin header, that has power SDA and SCL pins, and pin for DS. The DS pin has nothing to do with DS1307, it's the output of DS18B20 temperature sensor. This digital temperature sensor, has footprints on the PCB, but not soldered. It will be a good exercise to get a DS18B20 and solder it on this module, this way we have another device to play and learn with.

The second set of connections is 7 pin header, that has an additional output of SQ and BAT. The SQ is an optional output of DS1307 to give out square wave outputs at regular intervals, that can be used in projects for various timing related events. BAT is the output of bat-

tery and can be used in applications to monitor the battery voltage etc.

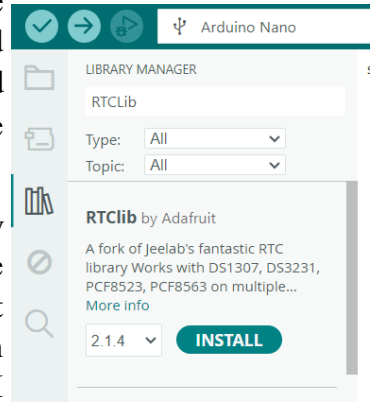


MikroDuino Nano board has a dedicated header with 5 pins to plug in this module. Since the module connects directly to SDA and SCL lines of I2C there is no need to set jumper tabs, or to reposition these connections to any other pins. The DS pin (DS18B20) however can be connected to any digital line, and therefore has a tab jumper option. We will consider this in appropriate chapter.

So to experiment with this module, you need to connect this module to the dedicated header and optionally (rather better) to have a display, like I2C OLED or I2C LCD to display the DS1307 output.

Arduino Library to work with DS1307 RTC

To simplify communication with the DS1307 RTC module, several Arduino libraries are available that provide ready-made functions to read and write time and date information easily. Some of the popular libraries include the **RTClib** by Adafruit, the **DS1307RTC** library by Paul Stoffregen, and the **RtcDS1307** library. These libraries abstract the low-level I2C communication details and offer user-friendly functions such as `now()`, `adjust()`, and `dayOfTheWeek()`, allowing developers to interact with the clock effortlessly.



Among these, **RTClib** by Adafruit is one of the most widely used and well-maintained libraries. It supports not only the DS1307 but also other RTC chips like the DS3231, making it versatile for future projects. RTClib is simple to install through the Arduino Library Manager and is known for its clean API and reliable performance. In this chapter, we will use the **RTClib** library because of its ease of use, rich set of functions, and excellent documentation, which will make our learning process smooth and effective. Open the library manager, and search for RTCLib by Adafruit. Install the library and you are good to go. When installing the library it will ask if you want to install Adafruit BusIO library, select yes, or install all, as this library will be used by the RTCLib for background communication.

Here is the first demonstration on connecting your Arduino to the RTC module, and getting

```
// DS1307 RTC test program
#include <Wire.h>      // Include the Wire library for I2C communication
#include <RTClib.h>    // Include the RTClib library for RTC functions

RTC_DS1307 rtc;        // Create an RTC object

void setup() {
  Serial.begin(9600);  // Start serial communication for debugging
  if (!rtc.begin()) {  // Initialize the RTC
    Serial.println("Couldn't find RTC");
    while (1);         // Stop the program if RTC is not found
  }
```

data from the chip. The program initializes the chip and stats the timing function. Display

```
  if (!rtc.isrunning()) { // Check if the RTC is running
    Serial.println("RTC is NOT running!");
    //Uncomment the next line to set the RTC to the time when the sketch is com-
piled
    rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
  }
}

void loop() {
  DateTime now = rtc.now();  // Read current time and date from RTC

  Serial.print(now.year());
  Serial.print('/');
  Serial.print(now.month());
  Serial.print('/');
  Serial.print(now.day());
  Serial.print(" ");
  Serial.print(now.hour());
  Serial.print(':');
  Serial.print(now.minute());
  Serial.print(':');
  Serial.print(now.second());
  Serial.println();

  delay(1000);  // Wait for 1 second before updating the time again
}
```

the chip registers or time data on the serial monitor.

When the chip is first powered up, the clock is not running, and it needs to be set to an initial value to start working. If you have inserted the coin battery, this needs to be done only once, as after that when mains power is turned off, the battery keeps the timer running. Here is a breakdown of the various RTCLib commands and how they are used in this context.

Explanation of the Important Functions:

`#include <Wire.h>`
: This library is needed for I2C communication between Arduino and the DS1307.

`#include <RTClib.h>`
: This is the main library that provides easy functions to interact with the DS1307.

`RTC_DS1307 rtc;`
: This line creates an object `rtc` that represents our DS1307 device.

`rtc.begin();`
: Initializes communication with the RTC module. It must be called in the `setup()` function. If it fails (for example, if the module is not connected), the program stops.

`rtc.isrunning();`

Checks if the RTC is actively keeping time. If the backup battery is missing or the clock is not yet set, it will report that the RTC is not running.

`rtc.adjust(DateTime(F(__DATE__), F(__TIME__)))` (Optional)

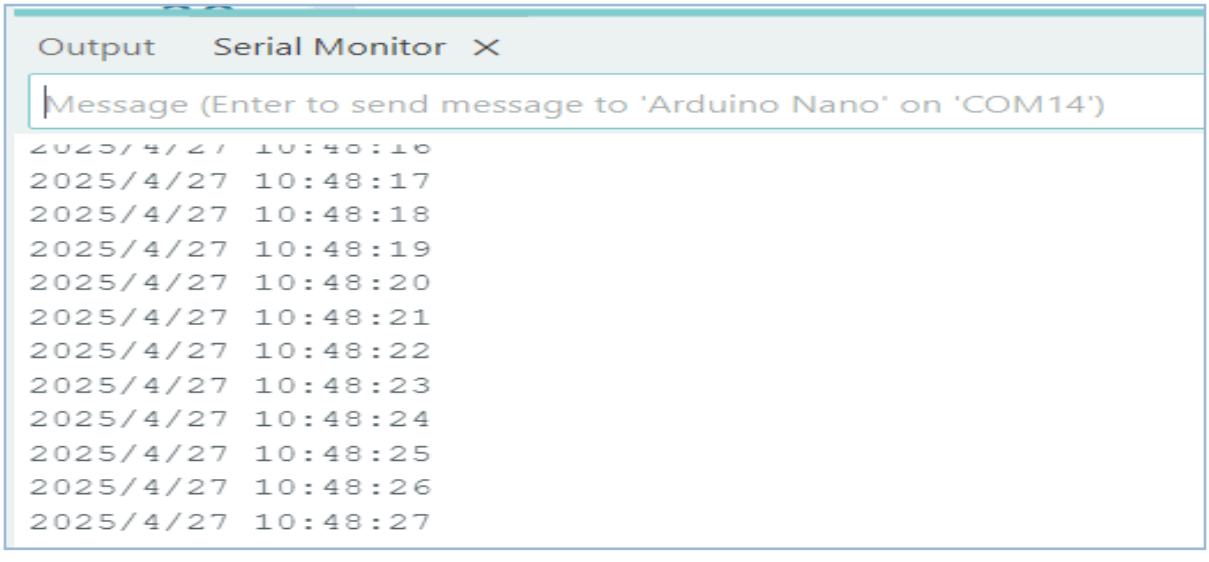
This function sets the RTC time and date to the moment the sketch was compiled. It's used if you need to set the clock initially.

`rtc.now();`

Reads the current time and date from the RTC and returns it as a `DateTime` object.

`DateTime` object

This object (`now` in the example) provides methods like `.year()`, `.month()`, `.day()`, `.hour()`, `.minute()`, and `.second()` to access the individual components of time.



The above code sets the date and time of the RTC as system generated date and time when the program is compiled. The (__DATE__) and (__TIME__) are the compiler generated global constants that contain the date and time of you PC when the program was compiled. You can use the global constants not only to set the date and time of the RTC, but also in many other places where compiled date and time are needed. This can include version control, or even conditional compiling. The letter F before (__DATE__) and time macros indicate that this information is stored in the program memory and RAM is not used for these. This is done because we have more program memory available than the RAM that is used for runtime variables.

```
if (!rtc.isrunning()) {
    Serial.println("RTC is NOT running!");

    // Manually set the date and time: (Year, Month, Day, Hour, Minute, Second)
    rtc.adjust(DateTime(2025, 4, 26, 15, 30, 0));
    // This sets the RTC to 26th April 2025, 3:30:00 PM
  }
```

In case you want to set the date and time of RTC to some custom values, you do so using this code.

> **Tip for Students:**
> *"Whenever you want your Arduino to know when its code was last updated, you can use __DATE__ and __TIME__. They save you from manually typing the date every time you update your project!"*

Realtime clock on OLED

Now lets make a complete application, to display the clock and date on the I2C OLED. As already discussed, both these devices, the RTC and OLED will share the same I2C bus lines, and due to their different addresses the system will be able to communicate with them individually.

```
//RTC clock on OLED
#include <Wire.h>               // For I2C communication
#include <RTClib.h>             // For DS1307 RTC
#include <Adafruit_GFX.h>       // Core graphics library
#include <Adafruit_SSD1306.h>   // OLED driver library

#define SCREEN_WIDTH 128        // OLED display width, in pixels
#define SCREEN_HEIGHT 64        // OLED display height, in pixels
#define OLED_RESET     -1       // Reset pin (not used with I2C)

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

RTC_DS1307 rtc;                 // Create an RTC object

void setup() {
  Wire.begin();
  rtc.begin();

  // Initialize the OLED
  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { // 0x3C is a common I2C address
    for(;;); // Don't proceed, loop forever
  }

  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);

  // Optional: Adjust RTC once if needed
  // rtc.adjust(DateTime(2025, 4, 26, 15, 45, 0));  // Year, Month, Day, Hour, Minute, Second

  if (!rtc.isrunning()) {
    display.clearDisplay();
    display.setCursor(0, 0);
    display.println("RTC not running!");
    display.display();
```

Notice how we have included both libraries and their dependencies in the code, also defined the constants for each library as we have used them before.

```
    while (1); // Halt if RTC not running
  }
}

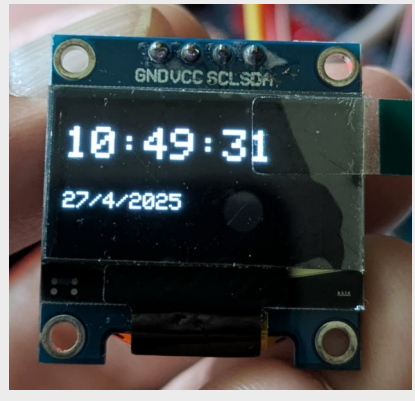
void loop() {
  DateTime now = rtc.now(); // Read current time

  display.clearDisplay();

  display.setCursor(0, 10);
  display.setTextSize(2);   // Make text bigger for time display
  display.print(now.hour());
  display.print(':');
  if (now.minute() < 10) display.print('0'); // Leading zero for minute
  display.print(now.minute());
  display.print(':');
  if (now.second() < 10) display.print('0'); // Leading zero for second
  display.print(now.second());

  display.setTextSize(1);   // Smaller text for date
  display.setCursor(0, 40);
  display.print(now.day());
  display.print('/');
  display.print(now.month());
  display.print('/');
  display.print(now.year());

  display.display(); // Update the OLED screen
  delay(1000);       // Update every second
}
```



How this code works:

Wire.begin() starts the I2C bus.

rtc.begin() initializes communication with the DS1307 RTC.

display.begin() initializes the OLED display.

rtc.now() reads the current time and date from the DS1307.

display.setCursor(x, y) sets where text will appear.

display.print() writes time and date onto the OLED.

display.display() updates the OLED screen after drawing.

Including Buttons to Set Time and Date

Now lets dive little bit more into a practical clock project, we have four push buttons on MikroDuino nano board. Lets use them to set the custom date and time.

Lets plan first, to define the functionality of buttons.

- **Button 1 (D2)** = **Select Field** (Hours → Minutes → Day → Month → Year)
- **Button 2 (D3)** = **Increment Field**
- **Button 3 (D4)** = **Decrement Field**
- **Button 4 (D5)** = **Save/Exit Setting Mode**

Full Program with Button-based Date and Time Adjustment

```
//RTC with buttons to set time and date
#include <Wire.h>
#include <RTClib.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET    -1

#define BTN_SELECT    2   // Button to move selection
#define BTN_INC       3   // Button to increment
#define BTN_DEC       4   // Button to decrement
#define BTN_SAVE      5   // Button to save/exit

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
RTC_DS1307 rtc;

bool settingMode = false;
uint8_t selectedField = 0; // 0: Hour, 1: Minute, 2: Day, 3: Month, 4: Year

int setHour, setMinute, setDay, setMonth, setYear;
bool showColon = true;

void setup() {
  Wire.begin();
  rtc.begin();

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    while (1);
  }

  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);

  pinMode(BTN_SELECT, INPUT_PULLUP);
  pinMode(BTN_INC, INPUT_PULLUP);
  pinMode(BTN_DEC, INPUT_PULLUP);
  pinMode(BTN_SAVE, INPUT_PULLUP);

  if (!rtc.isrunning()) {
    rtc.adjust(DateTime(__DATE__, __TIME__)); // Auto set time if RTC stopped
  }
```

```
}

void loop() {
  if (digitalRead(BTN_SELECT) == LOW) {
    delay(200); // Debounce
    if (!settingMode) {
      DateTime now = rtc.now();
      setHour = now.hour();
      setMinute = now.minute();
      setDay = now.day();
      setMonth = now.month();
      setYear = now.year();
      settingMode = true;
      selectedField = 0;
    } else {
      selectedField = (selectedField + 1) % 5; // Move to next field
    }
  }

  if (settingMode) {
    adjustDateTime();
    showSetTimeScreen();
  } else {
    showNormalClock();
  }

  if (settingMode && digitalRead(BTN_SAVE) == LOW) {
    delay(200); // Debounce
    rtc.adjust(DateTime(setYear, setMonth, setDay, setHour, setMinute, 0));
    settingMode = false;
  }
}

void adjustDateTime() {
  if (digitalRead(BTN_INC) == LOW) {
    delay(200);
    switch (selectedField) {
      case 0: setHour = (setHour + 1) % 24; break;
      case 1: setMinute = (setMinute + 1) % 60; break;
      case 2: setDay = (setDay % 31) + 1; break;
      case 3: setMonth = (setMonth % 12) + 1; break;
      case 4: setYear++; break;
    }
  }

  if (digitalRead(BTN_DEC) == LOW) {
    delay(200);
    switch (selectedField) {
      case 0: setHour = (setHour + 23) % 24; break;
      case 1: setMinute = (setMinute + 59) % 60; break;
      case 2: setDay = (setDay == 1) ? 31 : (setDay - 1); break;
      case 3: setMonth = (setMonth == 1) ? 12 : (setMonth - 1); break;
      case 4: setYear--; break;
    }
  }
```

```
}

void showNormalClock() {
  DateTime now = rtc.now();

  display.clearDisplay();

  display.setTextSize(3);
  display.setCursor(10, 10);

  if (now.hour() < 10) display.print('0');
  display.print(now.hour());

  if (showColon) display.print(':');
  else display.print(' ');

  if (now.minute() < 10) display.print('0');
  display.print(now.minute());

  display.setTextSize(1);
  display.setCursor(20, 50);
  if (now.day() < 10) display.print('0');
  display.print(now.day());
  display.print('/');
  if (now.month() < 10) display.print('0');
  display.print(now.month());
  display.print('/');
  display.print(now.year());

  display.display();

  showColon = !showColon;
  delay(1000);
}

void showSetTimeScreen() {
  display.clearDisplay();

  display.setTextSize(2);
  display.setCursor(0, 0);
  display.print((selectedField == 0) ? ">" : " ");
  if (setHour < 10) display.print('0');
  display.print(setHour);
  display.print(":");

  display.print((selectedField == 1) ? ">" : " ");
  if (setMinute < 10) display.print('0');
  display.print(setMinute);

  display.setTextSize(1);
  display.setCursor(0, 30);
  display.print((selectedField == 2) ? ">" : " ");
  if (setDay < 10) display.print('0');
  display.print(setDay);
  display.print("/");
```

```
  display.print((selectedField == 3) ? ">" : " ");
  if (setMonth < 10) display.print('0');
  display.print(setMonth);
  display.print("/");

  display.print((selectedField == 4) ? ">" : " ");
  display.print(setYear);

  display.setCursor(0, 55);
  display.setTextSize(1);
  display.println("Press Save to confirm");

  display.display();
}
```

To set the time press button S1, the clock will enter the set mode, and an arrow will appear indicating the currently selected field. Use S2 and S3 to increment or decrement value, press S1 to move to next field. Press S4 to save and exit setup mode.

CHAPTER 13

SPI Communication and MAX7219

In the world of microcontrollers, communication between devices is essential. Whether it's talking to sensors, displays, or memory chips, efficient communication protocols allow our projects to become more powerful and interactive. One such widely used protocol is **SPI** — the **Serial Peripheral Interface**.

SPI is a high-speed, full-duplex communication protocol commonly used in embedded systems. It allows a microcontroller, like an Arduino, to exchange data quickly with one or more peripheral devices, such as sensors, displays, or memory chips. Compared to simpler methods like using individual digital pins, SPI offers a structured and faster way to send and receive large amounts of data — all while using just a few wires.

In this chapter, we will dive into the basics of SPI communication:

- How it works,
- What the common signal lines mean (MOSI, MISO, SCK, SS),
- And how Arduino makes it easy to use SPI through its built-in libraries.

After understanding the fundamentals, we'll apply this knowledge by interfacing the **MAX7219** — a popular display driver chip — to control an **8x8 LED Matrix Display**. The MAX7219 simplifies the task of managing multiple LEDs by handling the scanning and multiplexing operations internally, freeing up the Arduino to focus on other tasks.

By the end of this chapter, you will:

- Understand how SPI communication works and when to use it,
- Learn how to wire a MAX7219-based LED matrix to your Arduino,
- Write programs to display characters, patterns, and animations on the LED matrix.

This hands-on project will give you practical experience with both SPI and display management — two important skills for building more advanced Arduino projects.

Let's get started by first understanding what SPI is and why it's so useful!

What is SPI

SPI stands for **Serial Peripheral Interface**. It's a method for two or more electronic devices to quickly and reliably exchange data using a few dedicated wires.

Imagine you're at a fast-food counter:

- You (the customer) place an order.
- The cashier (the server) listens to you and passes the message to the kitchen.
- The kitchen prepares your meal and hands it back to the cashier, who gives it to you.

In SPI communication:

- The **Arduino** acts like the **Master** (the customer).
- The **connected device** (like a sensor or display) acts like the **Slave** (the cashier and kitchen).
- The Master always starts the communication and controls when and how data is exchanged.

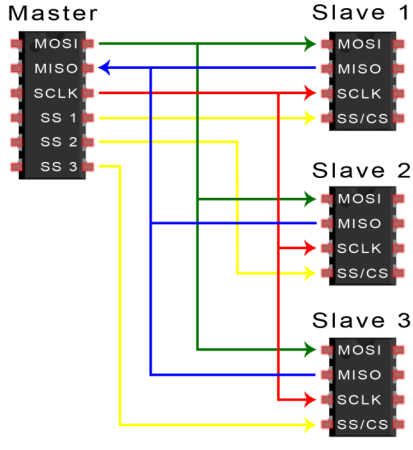
The Four SPI Wires

SPI typically uses **four lines** to connect devices:

Wire Name	Meaning	Arduino Pin (Uno)
MOSI	Master Out, Slave In (data from Arduino to device)	Pin 11
MISO	Master In, Slave Out (data from device to Arduino)	Pin 12
SCK	Serial Clock (clock signal from Arduino)	Pin 13
SS or CS	Slave Select or Chip Select (chooses which device to talk to)	Usually Pin 10

MOSI carries data **out** from the Arduino to the device.

MISO carries data **back** from the device to the Arduino (not always used; MAX7219 only receives data).



SCK provides a clock signal so that both the Arduino and the device know exactly when to send or receive bits.

SS/CS tells the device when it should listen. It's like tapping someone on the shoulder to get their attention.

How SPI Communication Works

SPI communication is **synchronous**, meaning it sends data based on the clock signal. Here's a simple step-by-step of how SPI works:

- The Master (Arduino) pulls the **CS** line LOW to select a device.
- The Master sends a clock pulse through **SCK**.

- For every clock pulse, one bit of data is sent through **MOSI** and/or received through **MISO**.
- After the communication is complete, the Master sets **CS** HIGH again to deselect the device.

Because data can move on every clock pulse, SPI can be **very fast** — often faster than other protocols like I²C or UART.

Understanding the MAX7219 Chip

The **MAX7219** is a serially interfaced LED driver designed by Maxim Integrated (now part of Analog Devices). It is a popular chip used to control large numbers of LEDs with minimal effort. Specifically, it can manage up to 64 individual LEDs arranged in an 8×8 matrix, eight 7-segment displays, or bar graph displays. What makes the MAX7219 particularly powerful is its ability to communicate with a microcontroller like Arduino through just a few wires, while internally handling the refresh and brightness control of the LEDs automatically.

Without the MAX7219, controlling dozens of LEDs would require connecting each LED separately and writing complex programs to turn them on and off at the right speed. The MAX7219 solves this problem by acting as a smart manager between the Arduino and the LEDs. It receives commands over a simple communication link (SPI), stores the information internally, and refreshes the LEDs continuously to make them appear steadily lit to human eyes.

One of the standout features of the MAX7219 is its support for **Serial Peripheral Interface (SPI)** communication. This allows it to receive data very quickly using only three control lines: Data In (DIN), Clock (CLK), and Chip Select (CS). Additionally, the chip provides 16 levels of brightness control through internal pulse-width modulation (PWM), which means you can adjust the display's brightness without additional circuitry or complex programming. The MAX7219 also has the capability to cascade multiple chips, enabling users to create much larger displays simply by connecting the output of one chip to the input of the next.

Internally, the MAX7219 is organized with an 8×8 matrix driver, a 16-bit shift register, and a controller that manages display refreshing and decoding. When data is sent to the MAX7219, the Arduino provides 16 bits of information: the first 8 bits indicate which digit (or row) to update, and the second 8 bits specify the pattern to display. The chip then stores this data and takes care of continuously lighting the LEDs based on the stored information, freeing the Arduino from the task of constantly refreshing the display.

The communication between Arduino and MAX7219 always involves sending two bytes (16 bits) together. The first byte represents the address — which digit, column, or row you are targeting — and the second byte contains the data, determining which LEDs should be on or off in that position. For instance, if you want to turn on every other LED in row 1, you would send the address for row 1 along with a data byte like `10101010` in binary.

Several important internal registers allow you to configure the MAX7219's behavior. These include the Decode Mode register (for working with 7-segment displays), the Intensity reg-

ister (for adjusting brightness), the Scan Limit register (to set how many digits/rows are active), the Shutdown register (to turn the display on or off), and the Display Test register (which lights up all LEDs to check if the hardware is working properly). By setting these registers appropriately during initialization, you prepare the MAX7219 to operate exactly as needed for your project.

A typical setup procedure for the MAX7219 includes taking it out of shutdown mode, setting the number of digits or rows to display, choosing the desired brightness level, and clearing the display. After that, sending new data to update the display becomes very easy.

The MAX7219 offers many practical advantages. It simplifies wiring significantly — instead of dozens of connections, only five or six are needed even for a full 8×8 matrix. It makes larger, scalable projects possible since multiple modules can be chained together. It also reduces coding complexity, because you do not need to manually refresh LEDs at a fast rate to avoid flickering; the MAX7219 handles that internally. Finally, it offers stable, flicker-free performance even when operating displays at various brightness levels.

Interestingly, many LED matrix modules sold online for Arduino projects are based on the MAX7219 chip, though it may be hidden underneath the module or beneath the LED matrix itself. Thanks to its ease of use, the MAX7219 is extremely popular in applications like digital clocks, scrolling text displays, simple games, counters, and animated signs.

History and Applications of the MAX7219

The MAX7219 was introduced by Maxim Integrated in the late 1980s, during a period when digital displays were becoming increasingly important for both industrial and consumer electronics. Before chips like the MAX7219 were available, creating a digital display meant manually wiring dozens of LEDs or seven-segment displays to a microcontroller or a set of discrete drivers, a process that was not only tedious but also unreliable for larger designs.

The MAX7219 was a breakthrough because it packaged all the necessary functions — decoding, refreshing, brightness control, and easy serial communication — into a single, compact 24-pin IC. This dramatically simplified the hardware design and reduced the workload on microcontrollers, many of which at that time were much less powerful than even today's basic Arduino boards. The chip quickly gained popularity among engineers developing control panels, instrumentation displays, and early digital signage systems.

As the hobby electronics movement grew in the 1990s and early 2000s, fueled by easier access to microcontrollers like PIC, AVR, and later Arduino, the MAX7219 found a new and enthusiastic audience. Hobbyists and students realized that with just a few wires and very little code, they could create impressive LED projects — from clocks to counters to mini message boards. Its ability to cascade easily also made it ideal for modular, expandable systems.

Today, the MAX7219 remains a favorite in the maker community. It is commonly used in a wide range of projects, including:

Digital clocks that display time across 4 or more seven-segment displays.

Scrolling text displays, often seen on DIY electronic signboards.

Simple LED animations and games, like snake or pong on an 8×8 matrix.

Counters and scoreboards for sports or event tracking.

Electronic dashboards and information panels.

Educational projects for learning SPI communication and display control.

Wearable displays and artistic installations using flexible matrices.

Even with the arrival of more modern LED drivers and matrix controllers, the MAX7219 continues to be widely used because of its simplicity, reliability, and the huge number of libraries and ready-made modules available for it. It remains an excellent starting point for beginners who are learning about serial communication and LED matrix programming.

In short, the MAX7219 is not just a chip — it is a bridge between simple electronics and dazzling digital displays, helping generations of engineers, students, and hobbyists bring their ideas to life.

Now that we understand the basics, lets start diving in, and start exporing the MAX7219 LED matrix display using Arduino and SPI protocol.

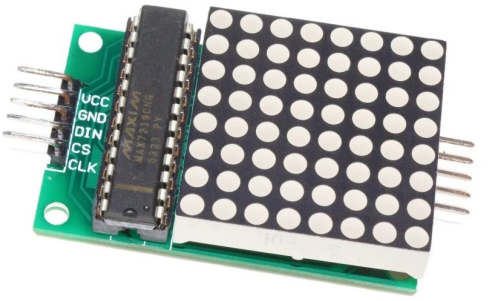
The MAX7219 LED Matrix Module



The MAX7219 LED Matrix Module

The **MAX7219 LED Matrix Module** is a convenient, ready-to-use module that leverages the MAX7219 chip to control a matrix of LEDs. It typically comes with an **8×8 LED matrix** and requires only a few pins from the microcontroller for operation. The beauty of this module is that it simplifies the process of controlling multiple LEDs, handling tasks such as multiplexing and refreshing internally, so the microcontroller only needs to focus on send-

ing data.



Pinout of the MAX7219 LED Matrix Module

The MAX7219 module has five main pins, typically arranged as follows:

VCC – Connects to the 5V supply from the Arduino.

GND – Connects to ground (0V).

DIN – Data In, this pin is used to send data from the microcontroller (e.g., Arduino) to the MAX7219. It is connected to the **MOSI** (Master Out Slave In) pin of Arduino.

CLK – Clock, this pin is used to synchronize the data being sent. It is connected to the **SCK** (Serial Clock) pin of Arduino.

CS (Chip Select) – This pin is used to activate the MAX7219 chip. It should be connected to any available **digital I/O pin** on the Arduino.

Cascading Multiple MAX7219 Modules

One of the most powerful features of the MAX7219 is its ability to cascade multiple modules together to form larger displays. You can link as many modules as you want in series, and the data will automatically be sent to each module in sequence. This is possible because the MAX7219 has a **DOUT** (Data Out) pin, which transmits the data from one module to the next.

How to Cascade MAX7219 Modules

To cascade multiple MAX7219 LED matrix modules, follow these simple steps:

Connect the First Module: The first module's **DIN** pin should be connected to the **MOSI** pin of the Arduino, as usual. The **CLK** pin (Clock) should also connect to the Arduino's **SCK** pin, and the **CS** (Chip Select) pin should be connected to any available **digital I/O** pin on the Arduino.

Link Additional Modules: For the next module in the chain, connect the **DOUT** (Data Out) pin of the first module to the **DIN** (Data In) pin of the second module. Continue this for each additional module you want to add to the chain.

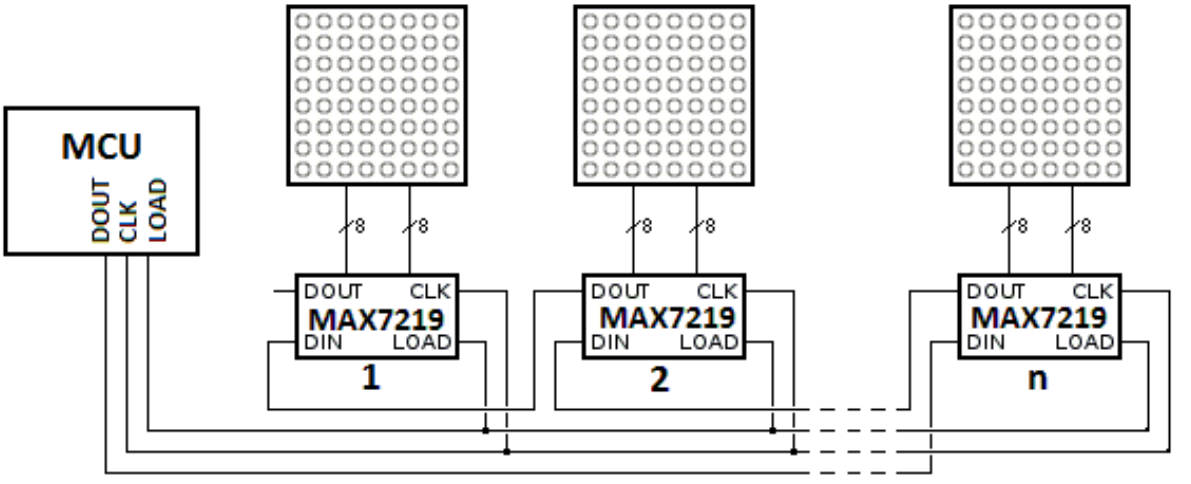
Powering the Modules: Ensure that all modules share a common **ground** (GND), and each module is powered by the same **VCC** (5V) line from the Arduino or an external power source.

Controlling Multiple Modules: When you send data from the Arduino, the first module receives the data for the first LED row, the second module gets the data for the second row, and so on. As data flows through each module, the LEDs on each module light up according to the patterns sent.

Addressing Multiple Modules: When cascading modules, the MAX7219 automatically addresses each module in sequence. However, if you're controlling many modules, you may

need to modify the **Scan Limit Register** to ensure that all modules are being addressed.

This cascading feature allows you to create long, continuous displays with ease. You can even control displays of **32×8**, **64×8**, or more LEDs by simply connecting more MAX7219 modules together.



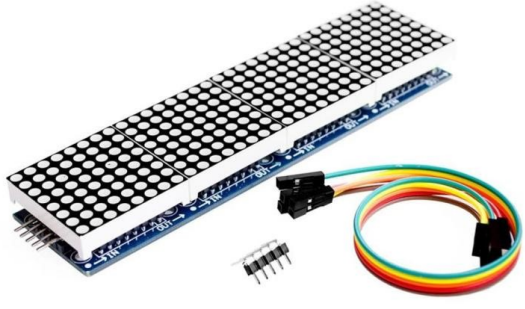
MAX7219-Based LED Matrix Module with Four Cascaded Modules

A popular type of **MAX7219 LED Matrix Module** on the market is one where **four 8×8 matrix modules are already cascaded together in a single unit**. These modules come pre-connected in a chain, and the data from the microcontroller is automatically split across the four displays. This arrangement saves you the time and effort of physically connecting the modules in a chain yourself.

Features of These Pre-Cascaded Modules

Convenience: Four 8×8 LED matrices are wired in series, so you only need to interface with a single module, rather than worrying about cascading each module individually.

Larger Displays: These pre-cascaded modules create a **32×8 matrix** (4 modules × 8 rows), allowing you to display more complex patterns, text, or graphics than a single 8×8 matrix.



Compact and Organized: Since the modules are already chained together, the wiring is cleaner and easier to manage. You only need to connect the **DIN**, **CLK**, and **CS** pins to the Arduino, while the data flows seamlessly through the chained modules.

Ease of Programming: The Arduino or any compatible microcontroller can send data to the first module, and the library you are using will automatically handle the data distribution across the

four modules.

Applications of Cascaded MAX7219 Modules

Modules with pre-cascaded MAX7219 chips are popular for large, multi-digit displays and scrolling text signs. They are commonly used in:

Public display boards for messages and announcements.

Scoreboards for sports or event tracking.

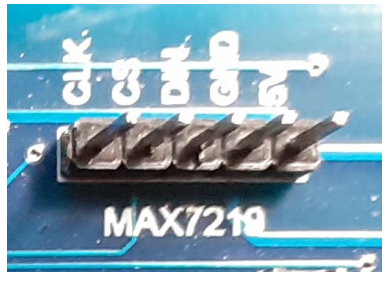
LED clocks or time displays.

Dynamic art installations that require larger LED screens.

Text scrolling banners for advertisements or notifications.

Now lets begin practical, with MAX7219 and single module 8x8 display to explore SPI communication and make use of the display. Later we will use the cascaded version of the display to make more elegant displays.

MAX7219 module can be connected to hardware SPI module in Arduino microcontroller or it can be controlled using any digital GPIO pins and software techniques. The later method gets complex, as it has to rely on code to manage data. We shall be using the hardware module, that does most of the job internally. You have to connect the MAX7219 inputs to the dedicated GPIO pins for the SPI.



DIN → Pin **11** (MOSI)

CLK → Pin **13** (SCK)

CS (LOAD) → Pin **10 (or any other GPIO)**

VCC → 5V

GND → GND

MikroDuino Nano board has a dedicated header for MAX7219 module, The header makes all the connections to the SPI module dedicated pins, and you do not need to set any jumper tabs.

Choice of Libraries for MAX7219 Displays

The MAX7219 is a powerful driver chip that controls 64 LEDs (in an 8×8 matrix) using just **three signal lines** through SPI-like communication. While it's possible to control the chip manually by writing SPI code and managing all 16 registers, this is **tedious and error-prone**—especially as the complexity of your project increases.

To make development faster, more readable, and scalable, we use libraries that abstract away low-level communication and provide easy-to-use functions.

Popular Libraries for MAX7219 with Arduino

Several open-source libraries exist for driving MAX7219 displays. The most commonly used include:

1. LedControl Library

- Simple and lightweight
- Good for small projects (1–4 modules)
- Uses **software SPI**, not hardware optimized
- Limited features (no font/text/scrolling support)

2. MD_MAX72XX (by Majic Designs)

- Highly configurable and optimized
- Supports **hardware SPI** for speed and efficiency
- Designed for **multiple cascaded modules**
- Logical coordinate addressing (row, column)
- Supports multiple hardware layouts
- Works seamlessly with the **Parola text animation library**

3. Parola for Arduino (also by Majic Designs)

- Built on top of MD_MAX72XX
- Adds text scrolling, animations, effects
- Ideal for message displays, clocks, and signage

In this book we will use MD_MAX72XX library along with Parola library to get animations and effects. This has been chosen because of advanced functionalities it offers and because the student will be able to make professional level devices using this library.

Installation of the libraries is simple, just search for MD_MAX72XX in the library manager and select the library by "Majic Designs" also search for and install MD_Parola library by the same group "Majic Designs".

Types of MAX7219 LED Matrix Modules Available in the Market

MAX7219-based LED matrix display modules are widely used for displaying scrolling text, symbols, and simple graphics using 8x8 LED matrices. While the MAX7219 chip remains the core controller, the **form factor, PCB layout, orientation, and wiring** of these modules can differ depending on the manufacturer and design. These differences can affect software compatibility, especially when using libraries like `MD_MAX72XX`.

Common Types of Modules

FC-16 Module (most common)

- This is perhaps the most widely available type.
- It has four 8x8 matrices soldered together in a straight line.

- The MAX7219 IC is usually placed **behind each matrix**, with components facing upward.
- The LED matrices are often arranged in **column-major order**, and the origin (0,0) of each matrix may start in the **bottom-left** or **top-left** depending on how the module is mounted.
- **Connectors**: Usually labeled `IN` and `OUT`, using 5 or 6 pins (VCC, GND, DIN, CS, CLK, and sometimes LOAD).

Generic Chinese Modules

- These may look similar to FC-16 but might have differences in wiring or matrix orientation.
- Can be used with most libraries if the correct hardware type is specified.
- Some may have **reversed data directions** or rotated matrices.

Parola-Compatible Modules

- Designed to be fully compatible with the **Parola display library** from Majic Designs.
- May follow a different internal layout but offer better alignment with the `MD_MAX72XX` and `MD_Parola` libraries.
- Easier for chaining multiple displays seamlessly.

DIY or Custom Modules

- Often found in hobby kits or educational products.
- Require manual configuration of orientation and connections in code.

How to Identify Which Module Type You Have

Before using a library like `MD_MAX72XX`, it's essential to know your hardware type to correctly display characters or graphics. Here's how to identify it:

Inspect the PCB Label: Check for markings like **"FC-16"** printed on the back of the PCB. Some may be labeled as "Generic," "Parola," or have no markings at all.

Check the Orientation: Upload a basic example using a known configuration (like FC-16). If characters appear **flipped**, **mirrored**, or **scrambled**, the layout in code needs to be adjusted.

Use the MD_MAX72XX Hardware Test Sketch: The `MD_MAX72XX` library includes an **example sketch** named `MD_MAX72XX_HW_Mapper`. This sketch helps you test various hardware types visually to find the correct setting.

Try different `#define HARDWARE_TYPE` options like `FC16_HW`, `PAROLA_HW`, or `GENERIC_HW`.

Observe IC Position: If the MAX7219 chips are located **behind** each matrix or in line on the side, that gives clues about the design and cascading direction.

Tip for Your Projects

When buying MAX7219 modules, **stick to a single hardware type** for a project to avoid configuration headaches. Also, **label your modules** with the correct type if you're maintaining a parts inventory for students or multiple kits.

Turn On a Single LED at (3,4)

Lets write the first code to Blink an LED in row 3 and column 4 of the 8x8 LED matrix with MAX7219 controller IC.

```
#include <MD_MAX72xx.h>
#include <SPI.h>

// MAX7219 8x8 LED Matrix
// Define hardware type and SPI pins
#define HARDWARE_TYPE MD_MAX72XX::FC16_HW
#define MAX_DEVICES 1
#define CS_PIN 10

MD_MAX72XX mx = MD_MAX72XX(HARDWARE_TYPE, CS_PIN, MAX_DEVICES);

void setup() {
  mx.begin();
  mx.clear();
}

void loop() {
  mx.setPoint(3, 4, true);  // row 3, col 4
  delay(1000);
  mx.setPoint(3, 4, false);
  delay(1000);
}
```

Here is a summary of the commands used to control the MAX7219 in the above code:

Explanation of Commands

`#include <MD_MAX72xx.h>`

This includes the main library that allows communication and control of MAX7219-based LED matrix displays. It abstracts the SPI interface and provides easy-to-use functions for graphics, text, and animation.

`#include <SPI.h>`

This includes the Arduino SPI library. Even though `MD_MAX72xx` internally uses SPI, this include ensures proper linkage and support for SPI communication.`#define HARDWARE_TYPE MD_MAX72XX::FC16_HW`

This defines the type of LED matrix hardware being used. `FC16_HW` is the most common con-

figuration for MAX7219 8x8 modules. Other types include `GENERIC_HW` and `PAROLA_HW`. Choosing the correct hardware type is essential for correctly orienting the characters and ensuring proper addressing of the LEDs.

`#define MAX_DEVICES 1`
Defines how many 8x8 LED matrix modules are connected in series (daisy-chained). Here, only one module is used.

`#define CS_PIN 10`
Defines the **Chip Select** pin (also called LOAD or SS) used to select the MAX7219 device on the SPI bus. Digital pin 10 is often the default SPI CS pin on most Arduino boards like the Uno.

`MD_MAX72XX matrix = MD_MAX72XX(HARDWARE_TYPE, CS_PIN, MAX_DEVICES);`
This line **creates an instance** of the `MD_MAX72XX` class called `matrix`. It is initialized with:

- The hardware type (`FC16_HW`)
- The CS pin (`10`)
- The number of devices (`1`)

This object handles all interaction with the matrix display.

`matrix.begin();`
Initializes the display. This sets up SPI communication and prepares the MAX7219 to receive commands and data.

`matrix.clear();`
Clears the entire display, turning off all LEDs. This ensures the screen starts in a blank state before anything is shown.

`matrix.setIntensity(5);`
Sets the **brightness level** of the LEDs. The value ranges from `0` (dim) to `15` (bright). A mid-level brightness like `5` is comfortable for most indoor uses.

Draw a Diagonal Line

```
//Draw a diagonal Line
#include <MD_MAX72xx.h>
#include <SPI.h>

// MAX7219 8x8 LED Matrix
// Define hardware type and SPI pins
#define HARDWARE_TYPE MD_MAX72XX::FC16_HW
#define MAX_DEVICES 1
#define CS_PIN 10

MD_MAX72XX mx = MD_MAX72XX(HARDWARE_TYPE, CS_PIN, MAX_DEVICES);

void setup() {
  mx.begin();
  mx.clear();
}

void loop() {
```

```
  mx.clear();
  for (uint8_t i = 0; i < 8; i++) {
    mx.setPoint(i, i, true);  // draw diagonal
    delay(100);
  }
  delay(1000);
}
```

The above code simply plots a line diagonally on the display. It increments the row and column, sequentially to light up the LEDs.

Display A Smily Face

In the next code example we will display a smiley face. First we need to make a mask of bits that should represent the bit pattern that we want to display. Since one row in our case is 8 LEDs, and one Byte is 8 bits, so we can easily represent one row as a byte. We define the map as a set of eight bytes, and using mx.setRow(0, row, smiley[row]); function display the bitmap pattern.

```
//Draw a diagonal Line
#include <MD_MAX72xx.h>
#include <SPI.h>

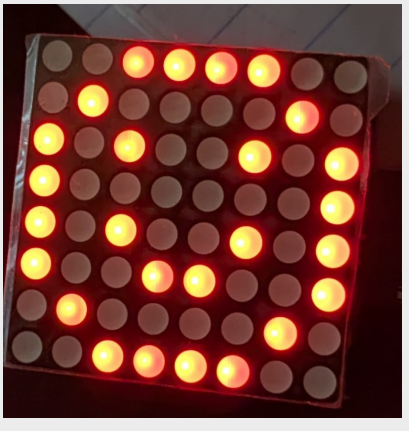
// MAX7219 8x8 LED Matrix
// Define hardware type and SPI pins
#define HARDWARE_TYPE MD_MAX72XX::FC16_HW
#define MAX_DEVICES 1
#define CS_PIN 10

MD_MAX72XX mx = MD_MAX72XX(HARDWARE_TYPE, CS_PIN, MAX_DEVICES);

void setup() {
  mx.begin();
  mx.clear();
  mx.control(MD_MAX72XX::INTENSITY, 5);
}

uint8_t smiley[8] = {
  B00111100,
  B01000010,
  B10100101,
  B10000001,
  B10100101,
  B10011001,
  B01000010,
  B00111100
};

void loop() {
  mx.clear();
  for (uint8_t row = 0; row < 8; row++) {
    mx.setRow(0, row, smiley[row]);
  }
  delay(2000);
}
```



mx.control(MD_MAX72XX::INTENSITY, 5);

Use this statement to control the intensity of display. The value 5 is the intensity level and it can be from 0 to 15.

Multiple Frames Animation

As we have displayed a bitmap image of smiley face, you can draw a sequence of bitmaps to make an animation.

```
//Draw Multiple Frames
#include <MD_MAX72xx.h>
#include <SPI.h>
// Define hardware type and SPI pins
#define HARDWARE_TYPE MD_MAX72XX::FC16_HW
#define MAX_DEVICES 1
#define CS_PIN 10

MD_MAX72XX mx = MD_MAX72XX(HARDWARE_TYPE, CS_PIN, MAX_DEVICES);

void setup() {
  mx.begin();
  mx.clear();
  mx.control(MD_MAX72XX::INTENSITY, 5);
}

uint8_t frames[2][8] = {
  {
    B00011000,
    B00111100,
    B01111110,
    B11111111,
    B11111111,
    B01111110,
    B00111100,
    B00011000
  },
  {
    B00000000,
    B00011000,
    B00111100,
    B01111110,
    B00111100,
    B00011000,
    B00000000,
    B00000000
  }
};

void loop() {
  for (uint8_t f = 0; f < 2; f++) {
    for (uint8_t row = 0; row < 8; row++) {
      mx.setRow(0, row, frames[f][row]);
    }
    delay(500);
  }
}
```

Displaying Text with MD_MAX72XX Library

The `MD_MAX72XX` library provides powerful functions to render characters and text on 8x8 LED matrix modules driven by the MAX7219 chip. It supports ASCII characters from a built-in 5x7 font and offers precise control over character placement, alignment, and animation.

? Basics of Text Display

Each 8x8 LED module is treated as a "device" by the library. Characters are displayed using a 5x7 dot matrix font, with a column of space between characters. Text can be written character by character, or scrolled across multiple modules. You can position characters manually or use helper functions for scrolling.

Display a Single Character on One Module

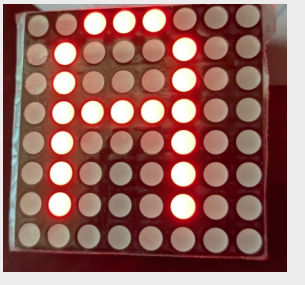
```
//Displaying Single Character
#include <MD_MAX72xx.h>
#include <SPI.h>

#define HARDWARE_TYPE MD_MAX72XX::FC16_HW
#define MAX_DEVICES 1
#define CS_PIN 10

MD_MAX72XX matrix = MD_MAX72XX(HARDWARE_TYPE, CS_PIN, MAX_DEVICES);

void setup() {
  matrix.begin();
  matrix.control(MD_MAX72XX::INTENSITY, 5);
  matrix.clear();
  matrix.setChar(6  , 'A');
}

void loop() {
  // Do nothing, static display
}
```



matrix.setChar(6 , 'A');

This function displays a character on the display. The number 6 is the column number from where to start drawing the character.

Scrolling Text

The space on display is always limited, while the length of message display may be long. Even when you have multiple modules connected, you may find the display area has a limited window. In such situations we can scroll the text on display. Scrolling can be in any direction, but for English it is usually from Right to Left. If using a display for an elevator, it is more useful to animate the floor number appearing as vertical scroll.

In this code we have made use of MD_Parola library instead of the standard MD_MAX72XX library. The MD_Parola library depends upon the MD_MAX72XX library to provide the basic commands to control and display text on the display. The MD_Parola extends this library to give features like animations, display strings of text and scrolling text

etc.

display.displayText("WELCOME", PA_CENTER, 100, 0, PA_SCROLL_LEFT, PA_SCROLL_LEFT);

Here display object is of type MD_Parola and its displayText function does all the magic.

```
//Scrolling text
// Use cascaded 4 modules
#include <MD_Parola.h>
#include <MD_MAX72xx.h>
#include <SPI.h>

#define MAX_DEVICES 1  // change the number of modules you are using
#define CS_PIN 10
#define HARDWARE_TYPE MD_MAX72XX::FC16_HW

MD_Parola display = MD_Parola(HARDWARE_TYPE, CS_PIN, MAX_DEVICES);

void setup() {
  display.begin();
  display.setIntensity(5);
  display.displayClear();
  display.displayText("WELCOME", PA_CENTER, 100, 0, PA_SCROLL_LEFT,
PA_SCROLL_LEFT);
  display.displayAnimate();
}

void loop() {
  if (display.displayAnimate()) {
    display.displayReset();
  }
}
```

In order to display and animate the text, we need two part statements. First we prepare the text that is to be displayed, and then we issue the animate function to actually affect the display.

```
display.displayText("WELCOME", PA_CENTER, 100, 0, PA_SCROLL_LEFT,
PA_SCROLL_LEFT);
```

This line **sets up the text to be scrolled** on the matrix.

`"WELCOME"`: The actual message to be displayed.

`PA_CENTER`: The alignment of the text (could also be `PA_LEFT` or `PA_RIGHT`).

`100`: Scroll speed in milliseconds between animation frames. Lower means faster.

`0`: Pause time (in milliseconds) after one scroll cycle completes.

`PA_SCROLL_LEFT`: Entry animation — how the text appears.

`PA_SCROLL_LEFT`: Exit animation — how the text leaves.

This setup prepares the message, speed, alignment, and effect, but **doesn't actually scroll it yet** — that's the job of the next function.

`display.displayAnimate();`
This function **runs the animation** defined in `displayText(...)`. It returns `true` when the animation has completed.You must call this function **repeatedly in** `loop()` to advance the animation frame by frame.

`if (display.displayAnimate()) { display.displayReset(); }`
This construct checks **whether the current scroll cycle has completed**. If it has:

`display.displayReset();` starts the same animation again from the beginning.

This loop gives the effect of continuous scrolling — it keeps resetting after each pass.

Elevator Counter

You must have encountered counters in elevators, that scroll up or down when the elevator reaches a floor. Here is an animation for this.

```
//Elevator Counter
#include <MD_Parola.h>
#include <MD_MAX72xx.h>
#include <SPI.h>

#define MAX_DEVICES 1 // Use the number of devices you have
#define CS_PIN 10
#define HARDWARE_TYPE MD_MAX72XX::FC16_HW

MD_Parola display = MD_Parola(HARDWARE_TYPE, CS_PIN, MAX_DEVICES);

int currentFloor = 0;
int direction = 1;  // 1 = up, -1 = down
char floorText[4];  // Enough to hold "5" or even "G", "10" etc.

void setup() {
  display.begin();
  display.setIntensity(5);
  display.displayClear();

  sprintf(floorText, "%d", currentFloor);
  display.displayText(floorText, PA_CENTER, 100, 1000, PA_SCROLL_UP,
PA_SCROLL_UP);
}

void loop() {
  if (display.displayAnimate()) {
    currentFloor += direction;

    if (currentFloor > 5) {
      currentFloor = 5;
      direction = -1;
    } else if (currentFloor < 0) {
      currentFloor = 0;
      direction = 1;
    }
```

```
    // Select direction of scroll
    textEffect_t scrollEffect = (direction == 1) ? PA_SCROLL_UP :
PA_SCROLL_DOWN;

    // Prepare new text in persistent buffer
    sprintf(floorText, "%d", currentFloor);
    display.displayText(floorText, PA_CENTER, 100, 1000, scrollEffect,
scrollEffect);
  }
}
```

The heart of this code is again displaytext() function in the MD_Parola library. The text to display, is obtained by converting the floor number into a text by using sprintf() the %d is the formatting for conversion.

Assignments:

1. Modify the elevator code to respond to press of a button
2. Modify the elevator code to respond to two buttons, one for up and one for down (range 0 to 5)
3. Using LM35 temperature sensor display room temperature on display, update every 5 seconds
4. Using LDR sense the ambient light, and adjust the display brightness

Digital Dice

Lets make a program to randomly select a number between 1 to 6 and display the number as dots (real dice like) on the display.

```
// Digital dice
// Use D2 Button to scroll
#include <MD_MAX72xx.h>
#include <SPI.h>

#define HARDWARE_TYPE MD_MAX72XX::FC16_HW
#define MAX_DEVICES 1
#define CS_PIN 10
#define BUTTON_PIN 2

MD_MAX72XX mx = MD_MAX72XX(HARDWARE_TYPE, CS_PIN, MAX_DEVICES);

int currentDice = 1;
bool lastButtonState = HIGH;

byte dicePatterns[6][8] = {
  // Dice 1
  {
    B00000000,
    B00000000,
    B00000000,
    B00010000,
    B00010000,
    B00000000,
    B00000000,
    B00000000
```

```
  },
  // Dice 2
  {
    B00000000,
    B00000000,
    B00000010,
    B00000000,
    B00000000,
    B01000000,
    B00000000,
    B00000000
  },
  // Dice 3
  {
    B00000000,
    B00000010,
    B00000000,
    B00010000,
    B00000000,
    B01000000,
    B00000000,
    B00000000
  },
  // Dice 4
  {
    B00000000,
    B01000010,
    B00000000,
    B00000000,
    B00000000,
    B01000010,
    B00000000,
    B00000000
  },
  // Dice 5
  {
    B00000000,
    B01000010,
    B00000000,
    B00010000,
    B00000000,
    B01000010,
    B00000000,
    B00000000
  },
  // Dice 6
  {
    B00000000,
    B01000010,
    B00000000,
    B01000010,
    B00000000,
    B01000010,
    B00000000,
    B00000000
```

```
  }
};

void setup() {
  mx.begin();
  pinMode(BUTTON_PIN, INPUT_PULLUP);  // use internal pull-up
  displayDice(currentDice);
}

void loop() {
  bool buttonState = digitalRead(BUTTON_PIN);

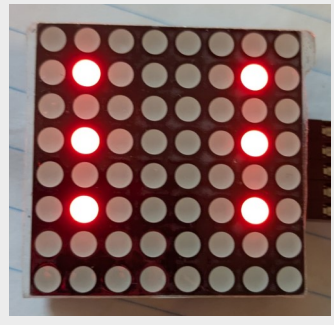
  // Detect button press (falling edge)
  if (lastButtonState == HIGH && buttonState == LOW) {
    rollDice();
  }

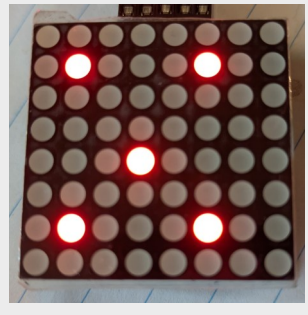
  lastButtonState = buttonState;
}

void rollDice() {
  // Simulate roll animation
  for (int i = 0; i < 10; i++) {
    int r = random(1, 7);
    displayDice(r);
    delay(100);
  }

  // Set final number
  currentDice = random(1, 7);
  displayDice(currentDice);
}

void displayDice(int number) {
  mx.clear();
  for (int row = 0; row < 8; row++) {
    mx.setRow(0, row, dicePatterns[number - 1][row]);
  }
}
```





Display Serial Data on Scrolling Display

One of the most demanding application of these scrolling displays is to send serial data, either from PC, or through some other mechanism, like mobile phone and Bluetooth device and display the text as scrolling message on the display. Here is a complete code for this project.

After you compile the program, the message to type a text in terminal appears. You open the terminal window and send a text, This message immediately starts appearing as scrolling text on display.

```
//Display text, transmitted by serial terminal
#include <MD_Parola.h>
#include <MD_MAX72xx.h>
#include <SPI.h>

#define HARDWARE_TYPE MD_MAX72XX::FC16_HW
#define MAX_DEVICES 1         // Number of 8x8 modules
#define CS_PIN 10             // Chip Select for SPI

MD_Parola display = MD_Parola(HARDWARE_TYPE, CS_PIN, MAX_DEVICES);

String inputText = "";

void setup() {
  Serial.begin(9600);
  display.begin();
  display.setIntensity(5);      // Brightness: 0–15
  display.displayClear();
  display.displayScroll("Send Text via Serial", PA_LEFT, PA_SCROLL_LEFT, 50);
  display.displayAnimate();
}

void loop() {
  if (Serial.available()) {
    inputText = Serial.readStringUntil('\n');
    inputText.trim();   // Remove trailing newline or spaces

    if (inputText.length() > 0) {
      display.displayClear();
      display.displayScroll(inputText.c_str(), PA_LEFT, PA_SCROLL_LEFT, 50);
    }
  }

  if (display.displayAnimate()) {
    // Automatically restart animation
    display.displayReset();
  }
}
```

Displaying Graphics on MAX7219 LED Matrix

The 8x8 LED matrix is not limited to text—it's also a great platform for basic graphics. With the `MD_MAX72XX` library, you can control individual **pixels**, **rows**, and **columns**, and even draw custom **bitmaps**.

Required Library

`MD_MAX72XX` by MajicDesigns (No need for MD_Parola if you're only working with graphics only)

Key Graphics Functions

Here are some commonly used functions from `MD_MAX72XX` for drawing graphics:

Function	Description
`setPoint(row, col, true/false)`	Set or clear a single pixel

`setRow(dev, row, value)`	Set all 8 bits of a row
`setColumn(dev, col, value)`	Set all 8 bits of a column
`transform(dev, t)`	Apply transformation (rotate, flip, etc.)
`clear()`	Clears the display
`update()`	Force a refresh (rarely needed)

PONG Game

```
//Pong game
// use buttons D3 and D4
#include <MD_MAX72xx.h>
#include <SPI.h>

#define HARDWARE_TYPE MD_MAX72XX::FC16_HW
#define MAX_DEVICES 1
#define CS_PIN 10

MD_MAX72XX mx = MD_MAX72XX(HARDWARE_TYPE, CS_PIN, MAX_DEVICES);

// Button pins
const int buttonLeft = 4;
const int buttonRight = 3;

// Paddle position
int paddleX = 3;

// Ball position and direction
int ballX = 3, ballY = 6;
int ballDirX = 1; // +1 = right, -1 = left
int ballDirY = -1; // -1 = up, +1 = down

void setup() {
  mx.begin();
  pinMode(buttonLeft, INPUT_PULLUP);
  pinMode(buttonRight, INPUT_PULLUP);
  mx.clear();
  mx.update();
}

void loop() {
  mx.clear();

  // Read buttons
  if (digitalRead(buttonLeft) == LOW && paddleX > 0) {
    paddleX--;
  }
  if (digitalRead(buttonRight) == LOW && paddleX < 6) {
    paddleX++;
  }

  // Draw paddle (2 LEDs wide)
  mx.setPoint(7, paddleX, true);
```

```
  mx.setPoint(7, paddleX + 1, true);

  // Move ball
  ballX += ballDirX;
  ballY += ballDirY;

  // Bounce from left and right walls
  if (ballX < 0 || ballX > 7) {
    ballDirX = -ballDirX;
    ballX += ballDirX;
  }

  // Bounce from top wall
  if (ballY < 0) {
    ballDirY = -ballDirY;
    ballY += ballDirY;
  }

  // Check for collision with paddle
  if (ballY == 7) {
    if (ballX == paddleX || ballX == paddleX + 1) {
      ballDirY = -ballDirY;
      ballY += ballDirY;
    } else {
      // Missed! Reset game
      ballX = 3; ballY = 6;
      ballDirX = 1; ballDirY = -1;
      paddleX = 3;
    }
  }

  // Draw ball
  mx.setPoint(ballY, ballX, true);

  mx.update();
  delay(150); // adjust for speed
}
```

CHAPTER 14

Accelerometer | Gyroscope | MPU6050

As you advance in the world of embedded systems and Arduino development, you'll inevitably encounter the need to understand your project's movement, orientation, or balance. Whether you're building a robotic arm, a self-balancing vehicle, or a wearable fitness tracker, motion sensing becomes a crucial part of the design. In this chapter, we will explore how to interface and use the **MPU6050**, a popular 6-axis motion tracking device, with an Arduino.

We'll utilize **I2C communication**, which allows efficient data exchange using only two pins, making it ideal for projects where pin availability is limited. In addition to reading sensor data, we'll also explore ways to visualize the output using peripherals like the **MAX7219-based LED matrix display**, the **SSD1306 OLED display**, and **user buttons** for interaction and mode selection.

The MPU6050 Sensor: Overview

The **MPU6050** is a compact and powerful **6-axis motion tracking sensor** manufactured by InvenSense (now part of TDK). It combines:

A **3-axis gyroscope** (measuring angular velocity),

A **3-axis accelerometer** (measuring linear acceleration),

A **Digital Motion Processor (DMP)** that can compute complex motion-related calculations internally.

This single-chip device allows precise detection of orientation, motion, and tilt, and can output raw or processed data through an **I2C interface**, which makes it Arduino-friendly.

Key Features:

Gyroscope Range: ±250, ±500, ±1000, ±2000 °/s

Accelerometer Range: ±2g, ±4g, ±8g, ±16g

16-bit ADCs for each channel

Built-in DMP for computing motion processing algorithms

Auxiliary I2C Bus to connect additional sensors like magnetometers

Power Supply: 3.3V to 5V (depending on the breakout module)

Common Applications of MPU6050

The MPU6050 is found in a wide range of real-world applications, such as:

Self-balancing robots

Quadcopters and drones

Motion-controlled gaming devices

Gesture-based interfaces

Fitness tracking wearables

Fall detection systems

Tilt-compensated compasses (when used with a magnetometer)

The MPU6050 Module (Breakout Board)

While the MPU6050 chip is surface-mount and difficult to handle directly, many manufacturers provide **breakout boards** that are easy to use with Arduino. These modules typically include:

- On-board voltage regulator (to allow operation at 5V supply)
- Pull-up resistors for the I2C lines (SCL and SDA
- Pins for I2C connection: **VCC, GND, SCL, SDA**
- Optional interrupt (INT) pin for advanced usage

Pin	Description
VCC	Power supply (3.3V or 5V depending on module)
GND	Ground
SCL	I2C Clock Line
SDA	I2C Data Line
INT	Interrupt Output (optional)



These modules are compact and easily fit into a breadboard
or can be soldered into a permanent project. Many are also compatible with Arduino libraries such as **Wire**, **I2Cdev**, and **MPU6050** by Jeff Rowberg.

How the MPU6050 Works

The MPU6050 operates as a **6-degree-of-freedom (6-DoF)** motion sensing device by combining a **3-axis gyroscope** and a **3-axis accelerometer** on a single silicon chip. The **accelerometer** measures **linear acceleration** along the X, Y, and Z axes. This helps determine the sensor's orientation with respect to gravity—useful for detecting tilt and motion. The **gyroscope** measures **angular velocity** (how fast the device is rotating) around the same three axes. This is essential for tracking changes in orientation over time.

Internally, each axis is connected to a **16-bit Analog-to-Digital Converter (ADC)**, enabling high-resolution digital output. The chip also contains an onboard **Digital Motion Processor (DMP)** that can process raw motion data and output values like pitch, roll, and yaw, reducing the burden on the microcontroller.

Communication with MPU6050: I2C Protocol

The MPU6050 uses the **I2C (Inter-Integrated Circuit)** protocol to communicate with external devices such as Arduino. I2C is a two-wire, serial communication protocol, where:

SCL (Serial Clock Line) carries the clock signal, generated by the master (Arduino).

SDA (Serial Data Line) carries the data, bidirectionally.

The **Arduino acts as the I2C master**, initiating communication. The **MPU6050 is the slave**, responding to requests at its default **7-bit I2C address:** `0x68` (can also be `0x69` depending on logic level at the AD0 pin).

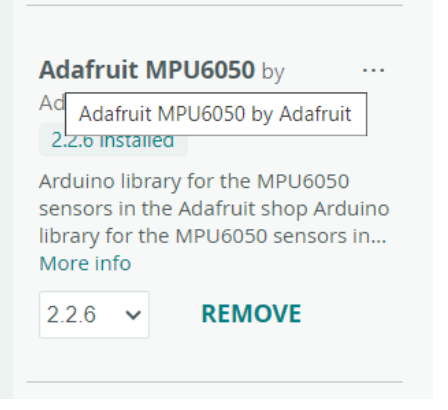
I2C Data Exchange

Communication occurs through a register-based system. Each internal value (like acceleration X, gyro Z, temperature, etc.) is stored in a **register** within the MPU6050. To read data the master writes the register address it wants to read from. The slave responds with the contents of that register (or a sequence of registers for multi-byte data).

This makes the sensor both **powerful** and **flexible**, but it also means developers must be comfortable with register maps and multi-byte data handling—something we will cover hands-on in this chapter.

Adafruit MPU6050 Library

The **Adafruit MPU6050 library** is a modern, user-friendly library developed by Adafruit Industries to simplify interfacing with the MPU6050 motion sensor. Built on top of the **Adafruit Sensor Unified Library**, it provides clean and consistent access to sensor data, including acceleration, gyroscopic rotation, and temperature, in **SI units** (m/s², rad/s, and °C respectively). This library abstracts away low-level register handling and raw data conversion, making it ideal for intermediate to advanced users who want to focus on application logic rather than communication protocol details. Installation is straightforward: open the **Arduino IDE**, go to **Sketch > Include Library > Manage Libraries**, then search for and install the following three libraries: **Adafruit MPU6050**, **Adafruit Unified Sensor**, and **Adafruit BusIO**. These dependencies work together to enable smooth communication over I2C and structured data handling for motion-based projects.



Basic MPU6050 Data Logger on Serial Monitor

This example shows how to initialize the MPU6050 and read acceleration, gyroscopic, and temperature data. It's the first step in becoming familiar with the sensor's capabilities.

```
//MPU6050
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>

Adafruit_MPU6050 mpu;
```

```
void setup() {
  Serial.begin(115200);
  while (!Serial)
    delay(10); // Wait for Serial Monitor

  Serial.println("MPU6050 Sensor Test");

  // Initialize MPU6050
  if (!mpu.begin()) {
    Serial.println("Failed to find MPU6050 chip");
    while (1) {
      delay(10);
    }
  }
  Serial.println("MPU6050 connected!");

  // Optional: Configure settings
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
}

void loop() {
  sensors_event_t accel, gyro, temp;

  // Get fresh sensor readings
  mpu.getEvent(&accel, &gyro, &temp);

  // Print acceleration
  Serial.print("Accel (m/s^2): ");
  Serial.print(accel.acceleration.x); Serial.print(", ");
  Serial.print(accel.acceleration.y); Serial.print(", ");
  Serial.print(accel.acceleration.z);

  // Print gyroscope
  Serial.print(" | Gyro (rad/s): ");
  Serial.print(gyro.gyro.x); Serial.print(", ");
  Serial.print(gyro.gyro.y); Serial.print(", ");
  Serial.print(gyro.gyro.z);

  // Print temperature
  Serial.print(" | Temp (°C): ");
  Serial.println(temp.temperature);

  delay(500); // Delay for readability
}
```

Units Matter: Unlike older libraries, this one provides data in meaningful units (no scaling needed).

Filter Bandwidth: Lower bandwidth filters out high-frequency noise but may respond slower to quick changes.

Acceleration in m/s²: Earth's gravity (~9.8 m/s²) will appear when stationary depending on

orientation.

Temperature sensor is internal and may not represent ambient room temperature accurately.

Line	What It Does
`#include` lines	Brings in required libraries: Wire (I2C), Adafruit MPU6050, and Adafruit Sensor API.
`Adafruit_MPU6050 mpu;`	Creates an object to interact with the sensor.
`mpu.begin()`	Initializes the MPU6050 over I2C. Returns `false` if the sensor is not found.
`mpu.setAccelerometerRange(...)`	Optional: Sets the sensitivity range of the accelerometer.
`getEvent(...)`	Reads the latest sensor values into the `accel`, `gyro`, and `temp` structures.
`Serial.print(...)`	Displays sensor readings on the Serial Monitor for testing and calibration.

now move on to **displaying MPU6050 sensor data on a 128x64 I2C OLED display (SSD1306)**. This is a powerful way for you to see real-time motion data visually on a hardware screen without needing a serial connection. Both of these devices use i2C communication, so the same bus lines will be used for both. The MikroDuino board however has separate headers for their connections.

We will use the **Adafruit SSD1306 library** along with the **Adafruit MPU6050 library**.

```
// MPU6050 with OLED
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET    -1

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
Adafruit_MPU6050 mpu;

void setup() {
  Serial.begin(115200);

  // Initialize OLED
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("SSD1306 allocation failed"));
    while (true);
  }
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);

  // Initialize MPU6050
  if (!mpu.begin()) {
```

```
    display.clearDisplay();
    display.setCursor(0, 0);
    display.println("MPU6050 not found");
    display.display();
    while (true);
  }

  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

  display.clearDisplay();
  display.setCursor(0, 0);
  display.println("MPU6050 Ready!");
  display.display();
  delay(1000);
}

void loop() {
  sensors_event_t accel, gyro, temp;
  mpu.getEvent(&accel, &gyro, &temp);

  // Clear display buffer
  display.clearDisplay();
  display.setCursor(0, 0);

  // Display acceleration
  display.println("Accel (m/s^2):");
  display.print("X: "); display.println(accel.acceleration.x, 1);
  display.print("Y: "); display.println(accel.acceleration.y, 1);
  display.print("Z: "); display.println(accel.acceleration.z, 1);

  // Display temperature
  display.print("Temp: "); display.print(temp.temperature, 1);
  display.println(" C");

  // Show buffer on screen
  display.display();
  delay(500);
}
```

The above example simply shows the MPU6050 data as text on the display. Now lets make it little bit graphical.

Graphical Acceleration Bars on OLED (SSD1306)

```
//MPU6050 OLED Graphics
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET    -1
```

```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
Adafruit_MPU6050 mpu;

void setup() {
  Serial.begin(115200);

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("SSD1306 allocation failed"));
    while (true);
  }

  if (!mpu.begin()) {
    display.clearDisplay();
    display.setCursor(0, 0);
    display.println("MPU6050 not found");
    display.display();
    while (true);
  }

  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

  display.clearDisplay();
  display.display();
  delay(1000);
}

void loop() {
  sensors_event_t a, g, t;
  mpu.getEvent(&a, &g, &t);

  display.clearDisplay();

  // Draw X bar
  drawAxisBar(0, "X", a.acceleration.x);

  // Draw Y bar
  drawAxisBar(22, "Y", a.acceleration.y);

  // Draw Z bar
  drawAxisBar(44, "Z", a.acceleration.z);

  display.display();
  delay(200);
}

void drawAxisBar(int y, const char* label, float value) {
  int centerX = SCREEN_WIDTH / 2;
  int barLength = map(constrain(value * 10, -80, 80), -80, 80, -50, 50); // ±8
m/s² ? ±50 pixels

  display.setCursor(0, y);
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
```

```
  display.print(label);
  display.print(":");

  // Draw central baseline
  display.drawLine(centerX, y + 10, centerX, y + 14, SSD1306_WHITE);

  // Draw bar to the right or left of center
  if (barLength >= 0) {
    display.fillRect(centerX + 1, y + 10, barLength, 4, SSD1306_WHITE);
  } else {
    display.fillRect(centerX + barLength, y + 10, -barLength, 4, SSD1306_WHITE);
  }
}
```

Each time the board is moved, Bars grow/shrink horizontally.

"X", "Y", and "Z" acceleration is immediately visualized.

You can tilt or shake the MPU6050 and observe **which direction changes the bars**.

CHAPTER 15

Humidity and Temperature | DHT11

In the world of embedded systems and DIY electronics, the ability to measure environmental conditions like temperature and humidity is essential for a wide range of applications—from smart homes and greenhouses to weather monitoring and personal health devices. Among the most popular sensors used by students and hobbyists for this purpose are the **DHT11** and **DHT22** temperature and humidity sensors.

These sensors are inexpensive, readily available, and easy to use with Arduino boards such as the **Arduino Nano**. Despite their simplicity, they offer a great opportunity to understand digital sensors, sensor data communication protocols, and real-time environmental monitoring.

In this chapter, we will explore:

- The theoretical background and working principle of DHT11 and DHT22 sensors.
- The key differences between DHT11 and DHT22, and when to choose one over the other.
- How to interface these sensors with an **Arduino Nano**.
- How to display the sensor readings on a **0.96" I2C OLED display**, making your project more interactive and user-friendly.
- Step-by-step coding examples and hands-on projects to solidify your understanding.

By the end of this chapter, you will be able to build your own Arduino-based weather station and gain a deeper appreciation of how digital sensors integrate with microcontrollers to create intelligent systems.

Theoretical Background

The **DHT11** and **DHT22** are digital sensors used for measuring **temperature** and **relative humidity**. These sensors are based on capacitive humidity sensing and a thermistor for temperature measurement, combined with a basic digital signal processing unit that outputs data in a digital format, making them easy to interface with microcontrollers like the Arduino Nano.

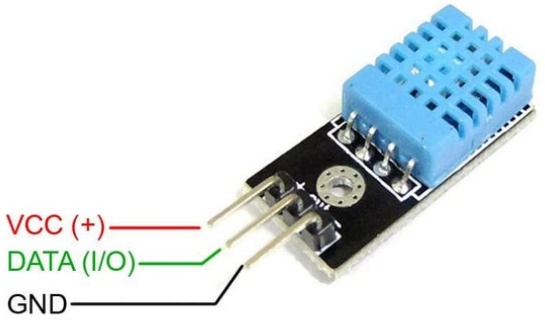
Working Principle

Humidity Sensing: Both sensors use a moisture-holding substrate (a polymer capacitor) whose capacitance changes with the relative humidity of the surrounding air. This change is measured and converted into a humidity reading.

Temperature Sensing: They include a **NTC thermistor**, whose resistance varies with temperature. An internal ADC (Analog-to-Digital Converter) digitizes this signal.

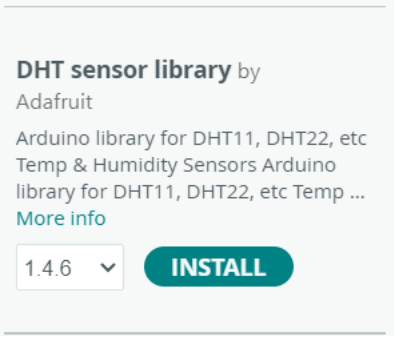
The sensor packages these readings and sends them to the microcontroller via a **single-wire digital communication protocol**.

Two modules are available in market, DHT11 and DHT22. Both of these are similar in interfacing but differ in accuracy and range of measurements. Generally speaking for ordinary usage DHT11 is sufficient, so we will be using this in our chapter.



Data Communication

Both sensors use a **proprietary single-wire protocol**. Communication involves a specific timing sequence where the microcontroller requests data, and the sensor responds with 40 bits: `16 bits` for humidity, `16 bits` for temperature, and `8 bits` for checksum. Since the communication is timing-sensitive, it's recommended to use existing libraries that handle the low-level protocol. You can use any digital GPIO line to connect this module to Arduino. MikroDuino board has a dedicated header for this, and has a tabbed jumper to connect the output pin to A1. When a tab is connected to this jumper the output pin get connected to pin A1 of Arduino nano. However you want to connect to some other pin, for your project, just remove the tab and connect the pin labelled DHT to any other GPIO pin of your choice using a jumper wire.



Additionally we will use the I2C SSD1306 OLED display to display the readings from this sensor. We have already installed the library for SSD1306 display, install the Adafruit DHT11 Library. It supports both DHT11 and DHT22. This library is widely used, well documented and regularly updated. It handles the sensor's timing specific single wire protocol internally and therefore user is free from bothering about low level signals.

Below is a **progressive series of example codes** demonstrating how to use the **DHT11/ DHT22** sensor and an **I2C OLED display** with an **Arduino Nano**. These examples not only teach basic functionality but also introduce important programming concepts and techniques.

Read Temperature and Humidity in Serial Monitor

This is the most basic program, that reads the temperature and humidity data from the sensor and display it on serial terminal. The program demonstrates how to use the DHT11 li-

```
// DHT11
#include <DHT.h>

#define DHTPIN A1         // Connect data pin of DHT to A1
#define DHTTYPE DHT11    // Change to DHT22 if you're using that
```

```
DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  float humidity = dht.readHumidity();
  float temperature =
dht.readTemperature();

  // Check if readings are valid
  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }

  Serial.print("Temp: ");
  Serial.print(temperature);
  Serial.print(" °C  |  Humidity: ");
  Serial.print(humidity);
  Serial.println(" %");

  delay(2000); // Wait 2 seconds between readings
}
```

```
Output  Serial Monitor ×
Message (Enter to send message to 'Arduino Nano' on 'COM14')
Temp: 29.80 °C  |  Humidity: 68.00 %
Temp: 29.80 °C  |  Humidity: 68.00 %
Temp: 29.80 °C  |  Humidity: 68.00 %
Temp: 29.80 °C  |  Humidity: 68.00 %
Temp: 29.80 °C  |  Humidity: 69.00 %
Temp: 29.80 °C  |  Humidity: 70.00 %
Temp: 30.20 °C  |  Humidity: 69.00 %
Temp: 30.20 °C  |  Humidity: 70.00 %
Temp: 30.20 °C  |  Humidity: 70.00 %
Temp: 30.80 °C  |  Humidity: 70.00 %
```

Display Readings on OLED

Now lets progress and include the OLED display and show text data on the OLED display.

```
// DHT11 on OLED
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

#define DHTPIN A1
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("SSD1306 allocation failed");
    for (;;);
  }
```

```
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
}

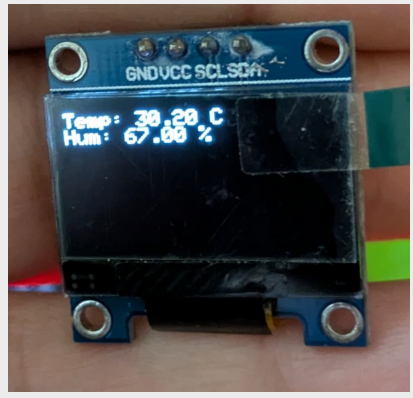
void loop() {
  float h = dht.readHumidity();
  float t = dht.readTemperature();

  if (isnan(h) || isnan(t)) {
    Serial.println("Sensor error");
    return;
  }

  display.clearDisplay();
  display.setCursor(0, 0);
  display.print("Temp: ");
  display.print(t);
  display.println(" C");

  display.print("Hum: ");
  display.print(h);
  display.println(" %");

  display.display();
  delay(2000);
}
```



DHT11 Basic Data Logger

Now lets combine the DHT11 with EEPROM on microcontroller to store readings. The readings will be available even when power is off. The program periodically gets DHT11 data and stores in EEPROM. The EEPROM can be queried by pressing D2 button, the log will be shown on terminal.

```
//DHT11 data logger store data in EEPROM Display on terminal when Button D2 is
pressed
#include <EEPROM.h>
#include <DHT.h>

#define DHTPIN A1
#define DHTTYPE DHT11
#define BUTTON_PIN 2

DHT dht(DHTPIN, DHTTYPE);

// Each entry will take 4 bytes: 2 for temperature, 2 for humidity (float values
stored as int * 100)
const int RECORD_SIZE = 4;
const int MAX_RECORDS = 100; // EEPROM on Nano has 1024 bytes. 100 records = 400
bytes max.
int recordIndex = 0;
```

```
void setup() {
  Serial.begin(9600);
  dht.begin();
  pinMode(BUTTON_PIN, INPUT);
  Serial.println("EEPROM Data Logger Ready");
}

void loop() {
  static unsigned long lastLogTime = 0;

  // Log data every 10 seconds
  if (millis() - lastLogTime > 10000 && recordIndex < MAX_RECORDS) {
    float temp = dht.readTemperature();
    float hum = dht.readHumidity();

    if (!isnan(temp) && !isnan(hum)) {
      saveToEEPROM(recordIndex, temp, hum);
      Serial.print("Saved Record #");
      Serial.print(recordIndex);
      Serial.print(" -> Temp: ");
      Serial.print(temp);
      Serial.print(" C, Humidity: ");
      Serial.print(hum);
      Serial.println(" %");

      recordIndex++;
    } else {
      Serial.println("Sensor error, skipping logging.");
    }

    lastLogTime = millis();
  }

  // If button pressed, print stored data
  if (digitalRead(BUTTON_PIN) == LOW) {
    Serial.println("\n--- EEPROM Logged Data ---");
    for (int i = 0; i < recordIndex; i++) {
      float temp, hum;
      readFromEEPROM(i, temp, hum);
      Serial.print("Record ");
      Serial.print(i);
      Serial.print(": Temp = ");
      Serial.print(temp);
      Serial.print(" C, Humidity = ");
      Serial.print(hum);
      Serial.println(" %");
    }
    Serial.println("----------------------------\n");
    delay(1000); // debounce and avoid repeat reads
  }
}

// Save a reading to EEPROM at given index
void saveToEEPROM(int index, float temp, float hum) {
```

```
  int addr = index * RECORD_SIZE;
  int t = int(temp * 100);
  int h = int(hum * 100);

  EEPROM.put(addr, t);
  EEPROM.put(addr + 2, h);
}

// Read a reading from EEPROM
void readFromEEPROM(int index, float &temp, float &hum) {
  int addr = index * RECORD_SIZE;
  int t, h;
  EEPROM.get(addr, t);
  EEPROM.get(addr + 2, h);

  temp = t / 100.0;
  hum = h / 100.0;
}
```

```
Record 0: Temp = 29.80 C, Humidity = 68.00 %
---------------------------

Saved Record #1 -> Temp: 29.80 C, Humidity: 68.00 %

--- EEPROM Logged Data ---
Record 0: Temp = 29.80 C, Humidity = 68.00 %
Record 1: Temp = 29.80 C, Humidity = 68.00 %
---------------------------
```

Key Concepts Introduced:

Using `EEPROM.put()` and `EEPROM.get()` to handle multibyte values (like `int`)

Memory address management using index * record size

Storing scaled `float` values as `int` to save EEPROM space

Triggering data read using a digital input (button)

Notes:

EEPROM has limited write cycles (around 100,000 per cell). Avoid writing too often in real-world applications. If needed, you can extend this to use external EEPROM or SD card for longer-term or frequent logging.

CHAPTER 16

Infra Red Remote Control

In the world of modern electronics, *wireless control* is not a luxury—it's an expectation. From adjusting the volume of your TV to switching off air conditioners and managing smart lighting systems, **infrared (IR) remote controls** have become a ubiquitous part of our daily lives. Despite the growing popularity of RF and Bluetooth-based communication, IR remote control systems continue to be widely used in **consumer electronics**, **home automation**, and even in certain **industrial environments** due to their simplicity, reliability, and low cost.

In this chapter, we will dive into the fascinating world of **IR remote control systems** and learn how to harness their potential using **Arduino**. You'll be able to build projects where you can remotely control lights, appliances, or any other Arduino-based setup—using a common TV remote or a custom IR remote.

One of the major uses of this system is that on microcontroller side it needs only one GPIO line, and the remote control we use has a number of buttons, so practically we get many buttons for our project using only a single GPIO line.

Why Infrared Remote Control?

Infrared (IR) technology offers a simple yet effective wireless communication solution. Unlike radio frequency systems, IR systems work on line-of-sight communication, which, in many applications, is not a limitation but an advantage. It helps avoid interference and ensures that control is confined to a specific area.

Key Advantages of IR Remote Systems:

- **Inexpensive hardware** (IR LEDs and receivers are cheap and widely available)
- **Low power consumption**
- **Minimal interference** from other wireless systems
- **Well-established protocols** like NEC, Sony SIRC, RC5, etc.
- **Easy integration** with microcontrollers like Arduino

IR communication is widely used in:

Consumer Electronics: TVs, audio systems, air conditioners, fans, projectors

Security Systems: Entry gates, alarms, and door locks

Industrial Applications: IR sensors for object detection and conveyor belt controls

Roadmap of This Chapter

This chapter will guide you step by step in understanding and applying IR remote control in Arduino projects. Here's what we'll cover:

Understanding Infrared Light and IR Communication

- Basics of infrared radiation
- What is an IR LED and how it transmits data
- How IR receivers detect and interpret signals
- Importance of modulation (specifically 38kHz)

How IR Remotes Work

- Signal structure of remote control systems
- Common IR protocols: NEC, Sony, RC5

Interfacing IR Receiver with Arduino

- Connecting an IR receiver module
- Installing and using the IR remote library
- Decoding and analyzing signals from a TV remote

Creating Your Own IR-Controlled Applications

- Using remote buttons to control LEDs
- Building a menu-based control system
- Creating a universal receiver for different remotes

Advanced Concepts

- Building a custom IR remote transmitter using Arduino
- Designing an IR-based communication system between two Arduino boards

Troubleshooting and Practical Tips

- Dealing with environmental interference
- Ensuring proper alignment and range
- Understanding limitations of IR communication

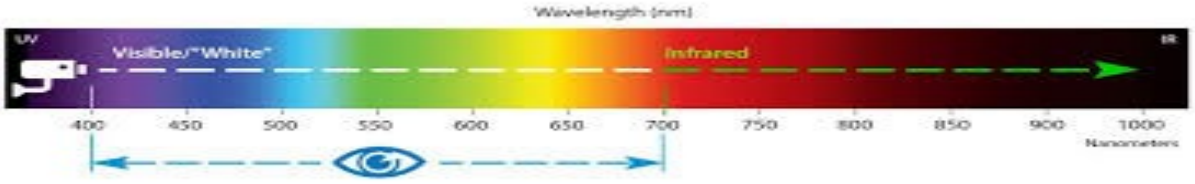
Fundamentals of Infrared Light and 38kHz Modulation

Before we delve into Arduino programming, let's understand the science behind infrared communication.

What is Infrared Light?

Infrared (IR) light is a type of electromagnetic radiation with a wavelength longer than visi-

ble light, typically in the range of **700 nm to 1 mm**. It is invisible to the human eye but can be detected by electronic components.



IR LED

An IR LED is a diode that emits infrared light when current flows through it. It is used in remote controls to send out bursts of modulated light representing data.



IR Sensor

Just like an IR LED there is an IR sensor, this is also a diode like device that is sensitive to IR light usually in the same range of wavelength as is emitted by the IR LED. Together the IR LED and sensor are used to detect the obstacles, and detect lines for line following robots. When the IR sensor receives the IR light it lowers its resistance.



IR Remote Receiver

The IR Remote control, has the same IR LED as mentioned above, but it modulates the signal on a square wave of 38KHz. This helps isolate the communication from surrounding IR noise.

An IR receiver module (such as **TSOP382**, **VS1838B**, or **HS0038**) is designed to detect IR signals modulated at a specific frequency—typically **38kHz**. It demodulates this signal and provides a clean digital output to the microcontroller.

Why 38kHz Modulation?

IR remotes do not send continuous light pulses. Instead, they use **modulation**, typically at **38kHz**, to distinguish the signal from ambient IR noise (like sunlight or incandescent bulbs). Modulation means the IR LED rapidly turns on and off 38,000 times per second to represent a logical “1” or “0”.



This technique allows the IR receiver to filter out any IR noise that is not modulated at the same frequency, significantly improving **signal integrity and reliability**.

How IR Remote Works

Infrared remote controls may look simple on the outside, but under the hood, they use a well-structured communication protocol to send digital data via invisible light pulses. Under-

standing how these remotes transmit data is crucial for decoding their signals and integrating them with Arduino projects.

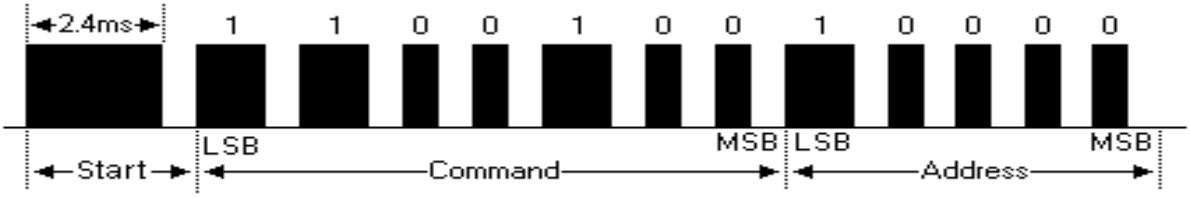
The Structure of an IR Signal

When you press a button on an IR remote, the IR LED doesn't just send out plain light—it sends out a **series of modulated bursts of infrared light** that represent binary data. These bursts are modulated at a specific frequency—typically **38kHz**—to make them distinguishable from other IR sources in the environment.

In the IR protocols, the duration of a burst of 38KHz signal indicates a logic '0' or logic '1'.

Here's what happens in a typical transmission:

1. **Start Signal / Header**: The transmission begins with a distinct signal that indicates the start of a message. This helps the receiver synchronize with the incoming data.
2. **Data Bits (On-Off Pulses)**: After the header, the data is transmitted as a series of **"marks" (bursts of 38kHz signal)** and **"spaces" (no signal)**. The duration of these marks and spaces determines whether the bit is a `0` or a `1`.
3. **End Signal**: Some protocols include a stop bit or a delay after data transmission to indicate the message has ended.



Binary Encoding Using Modulated Pulses

Most IR remotes use a form of **Pulse Distance Modulation (PDM)**:

A logical **'1'** is represented by a longer space after a fixed-length mark. A logical **'0'** is represented by a shorter space after the same mark. For example, in the **NEC protocol**: Mark (burst): 562 microseconds (for both 0 and 1) Space for '0': 562 microseconds Space for '1': 1,687 microseconds.

Common IR Remote Control Protocols

Various manufacturers use different IR communication protocols, but most follow similar principles with differences in timing, bit length, and format.

Here are some of the most common ones:

NEC Protocol

- Widely used in consumer electronics.
- Uses a 38kHz carrier.
- Data is 32 bits:
- 8-bit Address
- 8-bit Logical Inverse of Address

- 8-bit Command
- 8-bit Logical Inverse of Command
- Uses pulse distance encoding.

Example Use: TV remotes, DVD players.

Sony SIRC Protocol

- Uses variable-length bit encoding.
- Uses 12, 15, or 20 bits depending on device.
- No inverse bits.
- Each bit is a fixed-length burst followed by a variable-length space.
- Bit 1: 1.2ms pulse
- Bit 0: 0.6ms pulse
- **Example Use**: Sony TVs and audio equipment.

Philips RC-5 Protocol

- Uses **bi-phase modulation** (Manchester coding).
- 14-bit message:
- Start bit
- Control bit
- 5-bit address
- 6-bit command
- No carrier burst — toggles between on/off states in a balanced way.

Example Use: Older Philips and European audio-visual devices.

Custom and Raw Protocols

Some inexpensive remotes use **non-standard timing** or raw pulse trains without following any published protocol. These can still be decoded by capturing the raw timings using Arduino and libraries like `IRremote`.

Why Protocols Matter

Knowing which protocol your remote uses helps you:

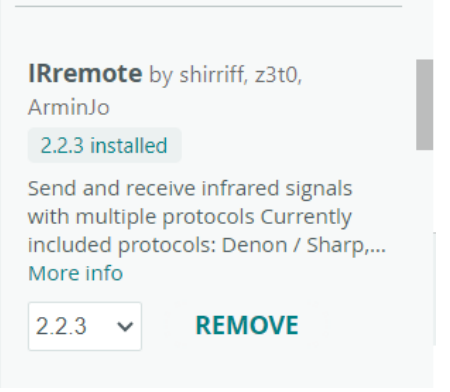
- Interpret button presses correctly
- Build custom transmitters that mimic real remotes
- Ensure compatibility when using universal remotes

The good news is that the **IRremote library** for Arduino handles the complexity of decoding and encoding these protocols, allowing you to focus on using the decoded values to control your project.

Interfacing IR Receiver with Arduino

After understanding how infrared remote controls work, it's time to get hands-on. In this section, we'll connect an **IR receiver module** to an **Arduino**, install the required **IRremote library**, and decode the signals sent from a **TV remote** or any standard IR remote control.

There are many libraries for Arduino available, they all do basically the same job, however they differ in their command structure and the number of different protocols they support. We will use an Arduino library by **shirriff**, now maintained officially under **z3t0**. Follow these steps to install it: It has many revisions and versions, the latest version at the time of this writing is version 4 I will however use an older version 2.23 of this library because of its simplicity and good documentation available. You can try other versions or even other libraries after reading their documentation.



The IR remote module needs only one GPIO pin, that you can connect to Arduino. The MikroDuino board has a dedicated header and a jumper tab for this module. By default it will connect the IR remote to A0 pin of Arduino. Though generally speaking A0 is an analog pin, nevertheless it can be used as digital as well. In case you want to use A0 for analog circuitry in your project, just remove the connecting tab and use a jumper wire to connect the IR remote module to any other pin of your choice.

Getting Raw data from IR remote Control

When you connect the module and read its data using the above library, you get two types of data from it. One is a long hexadecimal number, that corresponds to the key pressed, and second parameter is the type of protocol detected.

```
#include <IRremote.h>
IRrecv receiver(A0);
// IRsend send;
decode_results results;
void setup() {
  Serial.begin(9600);
  receiver.enableIRIn();
  Serial.println("IR Remote");
}

void loop() {
  if (receiver.decode(&results)) {
    Serial.println(results.value, HEX);
    receiver.resume();
  }
}
```

```
IR Remote
FF6897
FFFFFFFF
FF9867
FFFFFFFF
FFB04F
FFFFFFFF
```

When a button is pressed the code is sent, when the button is pressed for long time, after the initial code, FFFFFFFF is sent indicating that last sent code is being repeated.

You create an instance of the receiver object and pass on the pin module is connected as pa-

rameter. Then there is a complex variable type (actually a structure) that holds the results of decoded data. Decode_results is the data type defined by the library to hold the results of decoding. `receiver.enableIRIn();`

This command enables the receiver co start monitoring the IO pin.

`receiver.decode(&results)`

This function is passed the address of results variable, and it will return the data if any.

The results structure variable will then contain all the required data. fter receiving every command we have to reset the receiver by giving resume command.

`receiver.resume();`

Transmitting IR Signals

With the use of an IR LED and current-limiting resistor, you can **transmit** signals from Arduino using:

`irsend.sendNEC(code, bits);`
Sends an NEC protocol signal.

`irsend.sendSony(code, bits);`
Sends a Sony protocol signal.

Other available functions:

`sendRC5()`

`sendRC6()`

`sendPanasonic()`

`sendJVC()`

`sendSamsung()`

Decode Results Details

The `decode_results` object returned by `irrecv.decode(&results)` contains:

`results.value` – Decoded value (e.g., `0xFFA25D`)

`results.protocol` – Protocol name

`results.bits` – Number of bits received

`results.address` – Address part of the IR code (if available)

`results.command` – Command part (for some protocols)

Project: Control 3 LEDs with IR Remote

Here's a **simple demonstration project** using an **IR remote to control multiple LEDs** with an Arduino. This project teaches how to:

Read IR signals using the **IRremote** library

Decode remote buttons

Use them to toggle or control different outputs (like LEDs)

Component	Connect To
IR Receiver VCC	5V
IR Receiver GND	GND
IR Receiver OUT	Pin A0 (default in examples)
Red LED Anode	Pin 6 (through 220Ω resistor)
Green LED Anode	Pin 7
Blue LED Anode	Pin 8
All LED Cathodes	GND

```
#include <IRremote.h>

const int RECV_PIN = A0;

const int redLED = 6;
const int greenLED = 7;
const int blueLED = 8;

// Replace these with your remote's button codes (use Serial Monitor to find
them)
#define RED_BUTTON   0xFF6897
#define GREEN_BUTTON 0xFF9867
#define BLUE_BUTTON  0xFFB04F

IRrecv irrecv(RECV_PIN);
decode_results results;

bool redState = false;
bool greenState = false;
bool blueState = false;

void setup() {
  Serial.begin(9600);
  irrecv.enableIRIn(); // Start the receiver

  pinMode(redLED, OUTPUT);
  pinMode(greenLED, OUTPUT);
  pinMode(blueLED, OUTPUT);
}

void loop() {
  if (irrecv.decode(&results)) {
    Serial.print("Received: 0x");
    Serial.println(results.value, HEX);
```

```
    switch (results.value) {
      case RED_BUTTON:
        redState = !redState;
        digitalWrite(redLED, redState);
        break;
      case GREEN_BUTTON:
        greenState = !greenState;
        digitalWrite(greenLED, greenState);
        break;
      case BLUE_BUTTON:
        blueState = !blueState;
        digitalWrite(blueLED, blueState);
        break;
      case 0xFFFFFFFF: // Repeat code
        // Optionally ignore or repeat last command
        break;
    }

    irrecv.resume(); // Prepare to receive the next value
  }
}
```

Assignments:

1. Make an LED Dimmer to increase or decrease the brightness using a remote control
2. Make a Remote control copier. Read the values of a remote control, and then make a transmitter to transmit those values.

CHAPTER 17

Interfacing TM1637 | TM1638

One of the reasons for choosing microcontrollers for application development is to reduce the external hardware dependence and get the equivalent functionality using software. Though this is a great idea, this not only reduces the size of product but also cost. Driving seven segment LED displays, has always been traditionally done through microcontrollers using digital IO lines and multiplexing more digits to display the numbers. This technique though good, and applicable in many situations, but has few drawbacks. The LEDs are current driven devices, when they are scanned for multiplexing the average amount of power delivered is reduced and therefore digits become dull to display. Secondly if scanning is interrupted as for reading some sensor etc, the display tends to flicker. Special integrated circuits that can drive and scan the LEDs are developed to help these matters. We have seen previously that MAX7219 is one of them. Today we are going to explore yet another display driver IC, TM1637 and its elder brother TM1638.

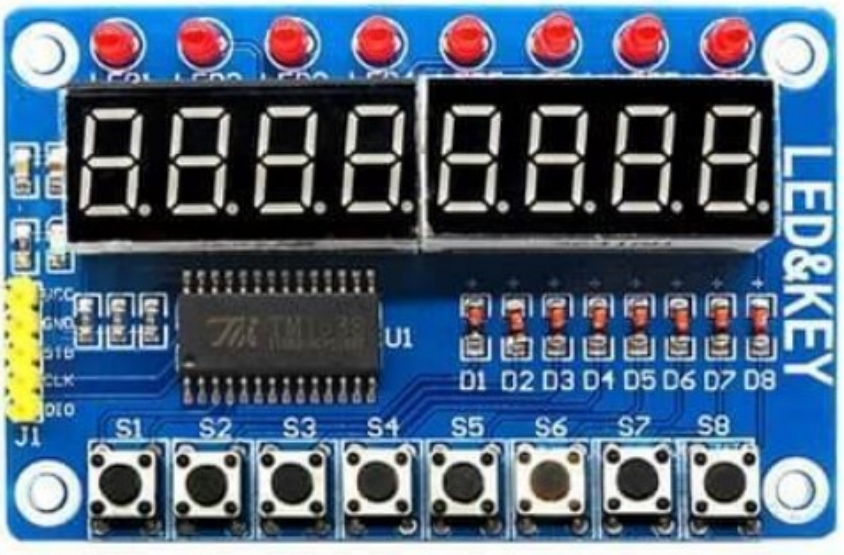


The **TM1637** and **TM1638** are versatile driver ICs widely used in Arduino-based projects for controlling LED displays and interfacing with input devices. These chips are popular in the maker community due to their simplicity, efficient communication protocols, and ease of integration into a wide range of microcontroller-based applications. Both ICs are produced by Titan Micro Electronics and are designed specifically to handle multiple 7-segment LED displays and other components like LEDs and push buttons with minimal wiring.

The **TM1637** is primarily known for its role in controlling 4-digit 7-segment displays. It uses a two-wire serial communication interface, similar in nature to I²C but with a custom protocol, which makes it extremely convenient for use with microcontrollers that have limited I/O capabilities. This IC handles the multiplexing and current limiting internally, which offloads much of the work from the microcontroller and simplifies the hardware design. With only two pins required for communication—usually referred to as CLK (clock) and DIO (data input/output)—the TM1637 allows efficient control of display modules while freeing up other microcontroller pins for additional tasks. It is often used in digital clocks, timers, and temperature displays.

On the other hand, the **TM1638** is a more powerful and feature-rich driver IC, typically

used in modules that combine multiple 7-segment displays, a row of LEDs, and push buttons in a compact layout. It communicates over a 3-wire serial interface and is capable of controlling up to 8 digits of 7-segment displays, 8 individual LEDs, and up to 24 key inputs. This makes it ideal for applications where both output display and user input are needed in a



single module, such as in control panels, counters, or digital dashboards. Like the TM1637, the TM1638 offloads the complexity of multiplexing and scanning, allowing for smooth and flicker-free display updates and reliable button detection.

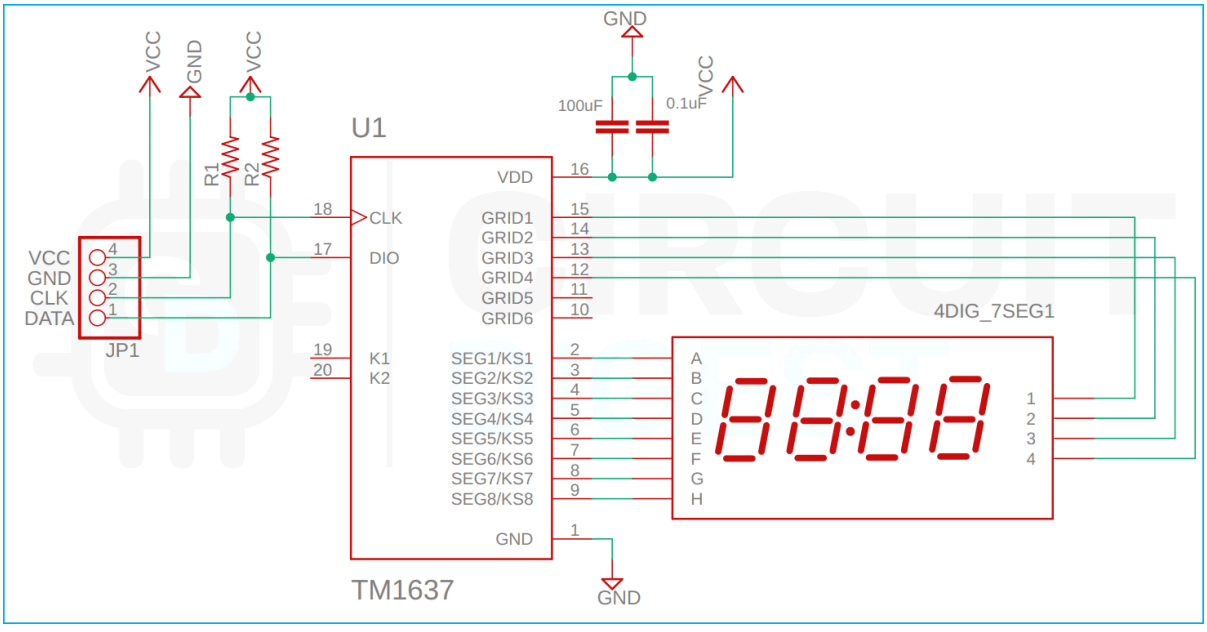
Both TM1637 and TM1638 play an important role in making LED display and input control simple and efficient in Arduino projects. They allow developers to build visually interactive systems with minimal hardware complexity and reduced I/O pin usage, which is especially valuable when working with microcontrollers that have limited resources. Their widespread availability and well-documented Arduino libraries make them an essential topic for anyone learning or teaching embedded system programming using Arduino.

TM1637

Interfacing the **TM1637** with an Arduino is straightforward and requires only two GPIO pins from the microcontroller. These are typically connected to the **CLK (clock)** and **DIO (data input/output)** pins of the TM1637-based display module. Unlike standard I²C or SPI protocols, the TM1637 uses a **custom 2-wire serial communication protocol**, which is synchronous but does not follow I²C addressing or handshaking conventions. This custom protocol consists of start and stop conditions, followed by a sequence of commands and data bytes, all managed through software-controlled timing (bit-banging). Due to its simplicity, many ready-made Arduino libraries—such as `TM1637Display.h`—exist to abstract the low-level communication, allowing users to display numbers, characters, or scrolling text with minimal code.

The communication begins when the Arduino sends a **start condition** by pulling both CLK and DIO low, followed by shifting out command or data bytes bit by bit while toggling the clock line. After data transmission, a **stop condition** is sent by setting the clock high while

the data line is also released to high. Though the TM1637 is not a fully open-drain bus like I²C, it expects **pull-up resistors** on both CLK and DIO lines—typically 10kΩ—to ensure



reliable signal levels. Some modules already include these resistors onboard.

From an electrical perspective, the TM1637 operates at **5V DC**, making it directly compatible with standard Arduino boards such as the Uno, Nano, or Mega. It can source a limited amount of current to drive **common cathode 7-segment displays**, with each segment capable of sinking up to **40mA**, though typical usage is much lower. To protect the IC and ensure consistent brightness, **internal current-limiting resistors** are often included in the display module itself. If not, external resistors should be added.

The TM1637 is generally used with **4-digit 7-segment displays**, where it performs automatic multiplexing to reduce power consumption and simplify circuit design. This means it turns on each digit sequentially in rapid succession, giving the illusion of a continuous display to the human eye. Due to current limitations and multiplexing speed, the TM1637 is best suited for small- to medium-sized displays—typically **0.36" to 0.56"** digit height. Larger displays with higher current demands may not be suitable without additional driver circuitry.

Pins K1 and K2 on TM1637

The pins labeled **K1** and **K2** on the TM1637 are internal **keyboard scan output lines**. The TM1637 chip includes **basic key scanning functionality**, intended for detecting simple keypresses. It supports scanning a matrix of keys by providing up to **2 output lines (K1, K2)** and using the **segment lines (SEG1 to SEG6)** as return paths, allowing detection of up to **12 keys** (6 columns × 2 rows). However, in most low-cost 7-segment display modules, this key-scanning feature is not utilized or even exposed, as the focus is on display control rather than input handling. Still, if you are using the bare IC or a custom PCB, these pins could be used to implement simple keypad input alongside display output without needing extra microcontroller I/O pins. We will talk more about these keypad scanning keys in

TM1638. The idea and concept is same.

Driving up to 6 Digits

Although many ready-made modules use the TM1637 with a **4-digit display**, the IC itself is capable of **driving up to 6 digits**. It features:

- **6 segment output pins (SEG1–SEG6)**
- **8 grid (digit select) pins (GRID1–GRID6 and others)**

Each digit typically uses 8 segments (7 segments plus decimal point), and the TM1637 handles the **multiplexing** of up to 6 digits internally. So, if you're designing your own display module or using a larger one that exposes more connections, the TM1637 can indeed be configured to handle a 6-digit 7-segment display. However, this comes at the cost of more current draw and may require careful attention to brightness and power considerations.

Modules with 6-digit displays are less common, partly due to size and power constraints, but are entirely possible if you're working with the IC directly or custom hardware.

Connect DIO to pin 11 and CLK to 12. You can choose any two GPIO pins. Install the TM1637 library by Avishay Orpaz , and you are ready to go.

The example below displays a value of 1234 all commands of the library are self explanatory. First you create an object of the TM1637Display type, you set the brighness of display




```
#include <TM1637Display.h>

// Define connections (CLK and DIO)
#define CLK 12
#define DIO 11

// Create display object
TM1637Display display(CLK, DIO);

void setup() {
  // Set display brightness (0 to 7)
  display.setBrightness(5);  // Medium brightness

  // Display a number
  int value = 1234;
  display.showNumberDec(value);  // Display 1234 in decimal
}

void loop() {
  // Nothing to do here
}
```

and display the value.

- `setBrightness(5)` sets brightness level. You can try values from `0` (dim) to `7` (brightest).
- `showNumberDec()` displays a decimal number up to 4 digits. You can also add options like leading zero padding if needed.
- You can use `display.clear();` to turn off the display.

Let’s extend the demo to showcase more of the **TM1637Display** library features, including:

1. **Counting numbers** (incrementing every second)
2. **Leading zero padding**
3. **Displaying hexadecimal numbers**
4. **Turning display on/off**
5. **Scrolling custom segments (for letters or animations)**

```
#include <TM1637Display.h>

// Define TM1637 connection pins
#define CLK 12
#define DIO 11

// Create display object
TM1637Display display(CLK, DIO);

int counter = 0; // Number to display

void setup() {
  display.setBrightness(7);  // Max brightness

  // Display a welcome message briefly
  display.showNumberDec(8888);  // All segments on
  delay(1000);
  display.clear();
}

void loop() {
  // Show counting number
  display.showNumberDec(counter);

  // Wait 1 second
  delay(1000);

  counter++;

  // After 20 counts, show other display modes as a demo
  if (counter == 20) {
    display.clear();
    delay(1000);

    // Show number with leading zeros
    display.showNumberDec(42, true);  // Displays 0042
    delay(2000);

    // Show hexadecimal value (e.g., 0x1A3F)
    display.showNumberHexEx(0x1A3F);
    delay(2000);

    // Turn off display
    display.setBrightness(0, true); // Brightness 0, display off
```

```
    delay(1000);

    // Turn display back on
    display.setBrightness(7, true); // Brightness 7, display on
    delay(1000);


    delay(2000);
    display.clear();
    counter = 0;  // Reset counter
  }
}
```

Library and Pin Setup

```
#include <TM1637Display.h>

#define CLK 3

#define DIO 2

TM1637Display display(CLK, DIO);
```

This sets up communication between the Arduino and the TM1637 using digital pins 3 (CLK) and 2 (DIO). The library handles the underlying communication protocol.

Leading Zeros

```
display.showNumberDec(i, true);
```

The second argument `true` forces the display of leading zeros, so "42" becomes "0042".

Displaying Text and Graphics on TM1637 7-Segment Display

The **TM1637 driver IC** is designed to control 4-digit 7-segment displays using only **two GPIO pins** (CLK and DIO). While it's primarily used to show **numeric values**, it can also display limited **text characters** and **simple graphical patterns** by directly controlling the LED segments.

```
// Define segment bitmaps for H, E, L, P
const uint8_t HELP[] = {
  SEG_B | SEG_C | SEG_E | SEG_F | SEG_G,          // H
  SEG_A | SEG_D | SEG_E | SEG_F | SEG_G,          // E
  SEG_D | SEG_E | SEG_F,                          // L
  SEG_A | SEG_B | SEG_E | SEG_F | SEG_G           // P
};
```

Displaying Text (Letters)

Since a 7-segment display can only form a limited set of letters (like A, b, C, d, E, F, H, L, P, etc.), you'll need to **manually define each character** using segment codes.

```
#include <TM1637Display.h>

#define CLK 12
#define DIO 11

TM1637Display display(CLK, DIO);

// Define segment bitmaps for H, E, L, P
const uint8_t HELP[] = {
  SEG_B | SEG_C | SEG_E | SEG_F | SEG_G,          // H
  SEG_A | SEG_D | SEG_E | SEG_F | SEG_G,          // E
  SEG_D | SEG_E | SEG_F,                          // L
  SEG_A | SEG_B | SEG_E | SEG_F | SEG_G           // P
};

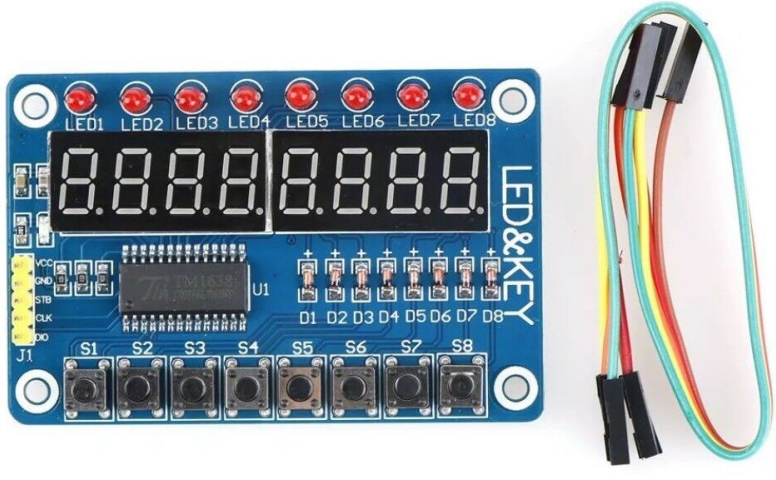
void setup() {
  display.setBrightness(7);           // Max brightness
  display.setSegments(HELP);          // Display HELP
}

void loop() {
  // Nothing in loop, static display
}
```



TM1638 Module

TM1638 is the big brother of TM1637 and can handle more digits, and scan inputs. It has a built-in constant current source for deriving the LEDs.



This module contains a TM1638 chip, 8 digits of seven segment display, 8 individual LEDs and 8 push buttons. The module uses a custom three wire protocol to communicate with microcontroller. This combination makes it an ideal component for **Arduino-based control**

panels, **debug consoles**, **educational kits**, and **DIY embedded systems**, offering a simple and effective way to interact with the user through both visual output and physical input.

TM1638 uses three pins to communicate, a CLK, DIO and STB (strobe or latch). The DIO is bidirectional, it can send data to the chip as well as read from the chip.

Key Features

- **Segment and LED control:** Each of the 8 digits has 8 bits (7 segments + decimal point), and the 8 LEDs are individually addressable.
- **Button scan matrix:** All 8 buttons are scanned and their states can be read with a single command.
- **Integrated current control:** TM1638 includes constant current drivers, removing the need for external resistors.
- **Simplified microcontroller interface:** Only 3 pins are required, making it suitable for microcontrollers with limited I/O.

How It Works

The TM1638 handles all display and input logic internally. The Arduino sends data serially to:

Update the LED segments and individual LEDs.

Query the state of buttons.

This offloads much of the work from the microcontroller and allows the Arduino to interact with the module using a few simple commands, without the need to continuously scan buttons or multiplex displays manually.

Data transfer is handled through:

STB (Strobe): Latches commands and data.

CLK (Clock): Clocks bits in/out.

DIO (Data I/O): Bidirectional data line for both sending and receiving data.

The TM1638 supports **multiple addressing modes** and **automatic address increment**, which means you can send a stream of data to fill all displays or LEDs in sequence, or target specific segments or LEDs as needed.

Let's proceed with the **first demo program** to interface the TM1638 module with an Arduino. This basic example will:

Initialize the TM1638 module

Display a number on the 7-segment display

Turn on LEDs one by one

Read button inputs and show their states

We'll use the popular and easy-to-use **TM1638 library** by rjbatista, which simplifies all interactions. This library is not available through Arduino library manager, instead you can

download the .zip file from here:

https://github.com/rjbatista/tm1638-library

Then in Arduino library manager, use .zip library installer to install this library.

Connect the STB pin to Arduino pin 7, DIO to 8 and CLK to 9. You can use any digital pins.

```
#include <TM1638.h>

#define DIO 8
#define CLK 9
#define STB 7

TM1638 module(DIO, CLK, STB);

void setup() {
  // Show a welcome message on the segments
  module.setDisplayToString("HELLO");

  delay(2000);

  // Display a number
  module.setDisplayToDecNumber(12345678, 0, false);

  // Turn on LEDs one by one
  for (int i = 0; i < 8; i++) {
    module.setLED(TM1638_COLOR_RED, i); // Red LED on at position i
    delay(200);
  }
}

void loop() {
  // Read buttons and reflect their state on LEDs
  byte buttons = module.getButtons();

  for (int i = 0; i < 8; i++) {
    bool pressed = buttons & (1 << i);
    module.setLED(pressed ? TM1638_COLOR_RED : TM1638_COLOR_NONE, i);
  }

  // Display the value of pressed buttons (as binary pattern)
  module.setDisplayToDecNumber(buttons, 0, false);
  delay(200);
}
```

Explanation

`module.setDisplayToString("HELLO");`

Displays the string **HELLO** on the segments. Only characters supported by 7-segment encoding will appear correctly (E, L, H, etc.).

`module.setDisplayToDecNumber(12345678, 0, false);`

Displays the number 12345678. The second parameter is the dots (decimal points), which we are skipping for now.

`module.setLED(TM1638_COLOR_RED, i);`
Lights up red LEDs one after another on positions 0 to 7.

`module.getButtons();`
Reads the 8 push buttons. Each bit of the returned byte represents one button (bit 0 = button 0, bit 1 = button 1, etc.).

`module.setLED(...);` `in loop()`
Updates each LED depending on whether its corresponding button is pressed.

`module.setDisplayToDecNumber(buttons, 0, false);`
Displays the raw binary value of the pressed buttons as a decimal number.

Let's now explore how to **display custom text** and **scroll messages** on the TM1638 module, and also understand how to format text when dealing with the limitations of 7-segment displays.

Scrolling Text and Custom Display on TM1638

Display Capabilities and Limitations

The TM1638 module supports **8 seven-segment digits**. That means:

You can show **numbers** easily: `0-9`

You can show **limited characters**: Letters that fit within a 7-segment shape (like `A`, `b`, `C`, `d`, `E`, `F`, `H`, `L`, `P`, etc.)

Curved or complex letters like `M`, `Q`, `W`, or `X` aren't fully displayable

So for longer words or unsupported characters, we use **scrolling** to show them partially.